

ماجراجویی با پایتون

سفری پروژه محور به دنیای برنامه نویسی

نویسنده: محمد مهدی قائمی

نام و لوگوی انتشارات

سرشناسه
عنوان قراردادی
عنوان و نام پدیدآور
مشخصات نشر
مشخصات ظاهری
شابک
وضعیت
فهرست نویسی
یادداشت
موضوع
موضوع
رده بندی کنگره
رده بندی دیوبی
شماره کتاب شناسی
ملی

ماجرای جویی با پایتون

نویسنده: محمد مهدی قائمی

ناشر: انتشارات

شمارگان: نسخه

نوبت چاپ: اول، ۱۴۰۴

قیمت: تومان

شابک: ۹۷۸-۶۰۰- - -

سخن ناشر

مقدمه

به جایی که غیرممکن‌ها ممکن می‌شن خوش اومدی!

تصور کن یک چوب جادویی داری که می‌تونی با اون، هر ایده‌ای که توی ذهنت می‌گذره را به واقعیت تبدیل کنی. چه حس فوق‌العاده‌ای داره، مگه نه؟ بشر همیشه عاشق این بوده که رویاهش رو بسازه و زندگی‌ش رو راحت تر کنه؛ از روزی که اولین ابزارهای ساده رو ساخت تا امروز که کامپیوترها دقیق‌ترین و سریع‌ترین دستیارهای ما هستن، هدف همیشه یک چیز بوده: زندگی راحت‌تر و ایده‌هایی بدون مرز.

حالا تو درست در نقطه‌ای ایستادی که می‌تونی پادشاه این دنیای دیجیتال باشی. برای صحبت با این دنیای شگفت‌انگیز و فرمان دادن به ماشین‌ها، ما به یک زبان مشترک نیاز داریم و چه انتخابی بهتر از پایتون؟

پایتون زبانی که هم‌زمان هم ساده‌ست و هم بی‌نهایت قدرتمند. یاد گرفتنش مثل آب خوردنه، اما نتیجه‌اش شگفت‌انگیزه. این همون کلید طلاییه که دروازه‌های هوش مصنوعی، تحلیل داده‌ها و خلق نرم‌افزارهای بزرگ رو به روی تو باز می‌کنه. میلیون‌ها نفر در سراسر جهان با همین زبان، آینده‌ای روشن و هوشمند رو رقم زدن و حالا نوبت درخشش توست.

در این کتاب، ما هم سفر هم هستیم. قرار است در کنار هم، صفحه‌به‌صفحه لذت ببریم، با هم بازی بسازیم، اپلیکیشن خلق کنیم و پروژه‌هایی رو اجرا کنیم که دیدن نتیجه‌شون حس غرور و شادی بهت میده. اینجا هر خط کد، یک قدم به سمت حرفه‌ای شدن و هر پروژه، پلی به سوی موفقیت توست.

آماده‌ای تا با هم این مسیر هیجان‌انگیز رو شروع کنیم و آینده‌ای که دوست داری رو با دست‌های خودت بسازی؟ جادوی واقعی درست از همین لحظه شروع می‌شه.

بزن بریم به سوی بی‌نهایت!

عهدنامه ماجراجویان (چجوری این کتاب رو بخونیم؟)

ماجراجوی عزیز، خواندن کتاب‌های برنامه‌نویسی مدل خاص خودشو داره. برای اینکه بتونی حداکثر استفاده رو بکنی، پیشنهاد میشه از قانون سه مرحله‌ای پیروی کنی:

۱. مرحله مشاهده: ابتدا فصل رو با دقت بخون و مثال‌ها را نگاه و تحلیل کن. در این مرحله فقط داری با شکل و شمایل طلسم آشنا می‌شوی و یه ایده کلی از نحوه کارشون پیدا می‌کنی.

۲. مرحله رونویسی: وقتی به پروژه پایان فصل رسیدیم، کد نهایی را جلویت بگذار و خط به خط از روش تایپ کن.

نکته مهم: کپی/پیست کردن ممنوع است! جادوگر باید حرکت دست و املای کلمات را حس کند. در حین نوشتن، سعی کن تحلیل کنی که هر خط دقیقاً چه کار می‌کند.

۳. مرحله اجرای کور: این مهم‌ترین مرحله است حالا کتاب را ببند. سعی کن همان پروژه را از ذهن خودت و بدون نگاه کردن بنویسی.

۴. ماموریت‌های جانبی: در انتهای هر بخش، چالش‌هایی تحت عنوان «ماموریت جانبی» برای طراحی شده. برخلاف پروژه‌های اصلی، برای این ماموریت‌ها باید سعی کنی خودت از ابتدا و بدون نگاه کردن به کد حلشون کنی و اگه موفق نشدی جواب‌ها رو بخونی! تو باید با استفاده از دانشی که تا اون لحظه به دست آوردی و خلاقیت خودت، راه حل رو پیدا کنی. اینجاست که یادگیری کامل میشه!

قانون طلایی: تا زمانی که نتونستی پروژه پایان فصل رو بدون نگاه کردن به کتاب بنویسی و ماموریت‌های جانبی با موفقیت پشت سر بگذاری، یعنی هنوز طلسم هارو کامل یاد نگرفتی و نباید به فصل بعد بری! (اینجا بیشتر تمرین کن چون فصل‌های بعد این پیش‌فرض رو دارن که تو مباحث فصل قبل رو یاد گرفتی).

فهرست مطالب

بخش اول: شروع ماجراجویی! س

فصل ۱: اولین قدم: آماده‌سازی محیط توسعه ۱۶

۱. چرا پایتون ؟ ۱۷

۲. آماده کردن کوله‌پشتی: نصب پایتون یا استفاده از آزمایشگاه آنلاین ۱۸

مسیر اول: ساخت کارگاه شخصی (نصب پایتون روی کامپیوتر و لپ‌تاپ) ۱۸

قدم اول: نصب پایتون ۱۸

۱. برای کاربران ویندوز ۱۸

۲. برای کاربران مک ۱۹

۳. برای کاربران لینوکس ۱۹

قدم دوم: آماده کردن دفترچه جادو (محیط توسعه VS Code) ۲۰

قدم سوم: افزونه شناسایی پایتون در دفترچه جادویی (VsCode) ۲۰

مسیر دوم: استفاده از آزمایشگاه جادویی آنلاین (Google Colab) ۲۰

۳. اولین طلسم جادویی: اجرای `print("Hello, World!")` و دیدن نتیجه! ۲۱

۴. پروژه: ساخت کارت ویزیت دیجیتال (The Digital ID Card) - 100 xp ۲۳

فصل ۲: تبدیل شدن به کارآگاه کد ۲۷

۱. ذهنیت یک ماجراجو: برنامه نویسی یعنی حل مسئله، نه حفظ کردن کد! ۲۸

۲. جعبه ابزار کارآگاه ۲۸

۱. Stack Overflow: کتابخانه بزرگ برای تمام جادوگران دنیا ۲۸

۲. دستیارهای هوشمند (AI): چطور از غول‌های چراغ جادو کمک بگیریم؟ ۲۹

۳. پروژه: شکار باگ در کد نفرین شده (Bug Hunting) - 100 xp ۲۹

فصل ۳: جعبه‌های جادویی: ماشین حساب جادویی ۳۳

۱. جعبه‌های جادویی (متغیرها) ۳۴

۳۵	۲. آشنایی با انواع داده‌های پایه (Primitive Types)
۳۶	اعداد صحیح (Integer)
۳۶	اعداد اعشاری (Float)
۳۶	رشته (String)
۳۶	بولین (Boolean)
۳۷	۳. جادوی ریاضی: کار با عملگرها
۳۸	۴. پروژه: ساخت ماشین حساب جادویی (The Magic Calculator) 100 xp

فصل ۴: منطق و شرطها؛ ساخت بازی حدس عدد ۴۳

۴۴	۱. صحبت با ماجراجو: گرفتن ورودی از کاربر با طلسم جادویی input()
۴۴	۲. تله‌ی کیمیاگری (تبدیل متن به عدد)
۴۵	۳. دو راهی سرنوشت (if و else)
۴۶	۴. مسیرهای چندگانه (elif)
۴۶	۵. تکرار تا پیروزی (حلقه while)
۴۷	۶. کنترل قدرتمند حلقه: طلسم‌های break و continue
۱۰۰	۷. پروژه: بازی بزرگ حدس عدد (The Great Number Guessing Game) 100 xp

۴۸

مأموریت‌های جانبی: تالار تمرین ۵۳

۵۴	مأموریت‌ها
۵۷	کتابخانه سایه‌ها (نتیجه مأموریت‌ها)

بخش دوم: مدیریت کوله پشتی ماجراجو ۶۱

فصل ۵: کوله‌پشتی جادویی؛ مدیریت غنائم ماجراجو ۶۲

۶۳	۱. کوله پشتی جادویی (Lists)
۶۳	۲. شروع ماجراجویی با کوله‌پشتی پُر (Initialized List)
۶۳	۳. پر کردن کوله پشتی (اضافه کردن، دیدن و حذف کردن)
۶۴	طلسم append(): اضافه کردن آیتم
۶۴	خواندن محتوای آیتم‌ها با «ایندکس» (Index)
۶۵	طلسم remove(): حذف کردن آیتم
۶۵	۴. بازرسی کوله پشتی (طلسم‌های len و in)

- ۶۵ طلسم len(): طلسم شمارش‌گر
- ۶۶ طلسم in: طلسم جستجوگر
۵. سرشماری غنائم (حلقه for) ۶۶
۶. پروژه: ساخت سیستم مدیریت کوله‌پشتی (Backpack Manager) 100 xp – ۶۷

فصل ۶: رازهای کوله‌پشتی حرفه‌ای: تابلوی افتخارات ماجراجویان

- ۷۲ ۱. هنر برش زدن (Slicing)
- ۷۳ ۲. کیسه‌ی مهر و موم شده (Tuples)
- ۷۴ ۳. جادوی باز کردن تاپل (Tuple Unpacking)
- ۷۵ ۴. ترکیب جادوها: لیستی از تاپل‌ها
- ۷۵ ۵. جادوی نظم‌دهی (مرتب کردن لیست)
- ۷۶ ۶. پروژه: تابلوی افتخارات (The High Score Board) 100 xp – ۷۷

فصل ۷: کتاب جادویی: ساخت مترجم کلمات

- ۸۱ ۱. دیکشنری‌ها: (Dictionaries) کتابچه راهنمای هوشمند
- ۸۲ ۲. جادوی دیکشنری: افزودن، تغییر و حذف
- ۸۳ اضافه کردن یا تغییر دادن یک «کلمه»
- ۸۳ حذف کردن یک «کلید»
- ۸۴ ۳. تله‌ی کلید گمشده (The KeyError)
- ۸۴ ۴. طلسم‌های ایمنی (in و get)
- ۸۴ طلسم اول: بررسی با in
- ۸۴ طلسم دوم (حرفه‌ای): متد get()
- ۸۵ ۵. سرشماری در کتاب جادویی (Looping)
- ۸۵ ۶. هشدار ماجراجو: طلسم نمایش حروف فارسی
- ۸۶ ۷. پروژه: ساخت مترجم هوشمند کلمات (The Translator) 100 xp – ۸۷

فصل ۸: ساخت جادوهای شخصی: هنر ساخت توابع

- ۹۱ ۱. "یک بار بنویس، صد بار استفاده کن": معرفی توابع
- ۹۲ ۲. «تعریف» (Define) و «صدا زدن» (Call) یک تابع
- ۹۳ ۳. اولین تابع ما: خداحافظی با تکرار
- ۹۴ ۴. ورودی و خروجی توابع (return و Arguments)
- ۹۴ ۴

- ۹۵ return طلسم نهایی: ۵
 ۹۷ ۵. تنظیمات کارخانه (ورودی‌های پیش‌فرض)
 ۹۸ ۶. جادوی دقیق: برچسب زدن به مواد اولیه
 ۹۹ ۷. پروژه: پاکسازی کد نفرین شده (The Clean-Up Code) 100 xp

۱۰۳ فصل ۹: طلسم محافظتی: ساخت رمز عبور امن

- ۱۰۴ ۱. قدرتهای مخفی پایتون: کتابخانه‌ها و Import
 ۱۰۴ ۲. جعبه ابزار random: بهترین دوست ما برای کارهای تصادفی
 ۱۰۵ ۳. جادوی شمارش: تکرار با for i in range()
 ۱۰۶ ۴. پروژه: ساخت قفل رمزنگاری شده (Password Generator) 100 xp

۱۱۰ مأموریت‌های جانبی: تالار تمرین

- ۱۱۱ مأموریت‌ها
 ۱۱۴ کتابخانه سایه‌ها (نتیجه مأموریت‌ها)

۱۱۷ بخش سوم: ورود به بعد چهارم

۱۱۸ فصل ۱۰: رمزگشایی از طومارهای باستانی: تحلیلگر متن

- ۱۱۹ ۱. کار حرفه‌ای با متن‌ها (طلسم‌های String)
 طلسم len() ۱۱۹
 ۱۱۹ طلسم‌های lower() و upper()
 ۱۲۰ طلسم split(): شمشیر سامورایی رشته‌ها
 ۱۲۱ طلسم‌های نگهبان startswith() و endswith()
 ۱۲۱ طلسم کیمیاگری: تبدیل مس به طلا (replace)
 ۱۲۲ ۲. هنر نمایش: سه طلسم جادویی برای قالب‌بندی (Formatting)
 ۱۲۲ طلسم اول: جادوی باستانی (قالب‌بندی با %)
 ۱۲۳ طلسم دوم: جادوی مدرن (متد format())
 ۱۲۴ طلسم سوم: جادوی استاد (f-strings)
 ۱۲۶ ۳. رمزهای کلمات: راز استفاده از ' و " و ""
 ۱۲۷ ۴. پروژه: تحلیلگر طومار باستانی (The Scroll Analyzer) 100 xp

۱۳۱ فصل ۱۱: طلسم ضد فراموشی: کار با فایل‌ها

- ۱۳۲ ۱. چطور از شر "کرش" کردن خلاص شویم؟ (try و except)

۲. کار با فایل‌ها: کتیبه‌های ماندگار (txt) ۱۳۳
- طلسم بازکردن دروازه (with open(...)) : ۱۳۳
۳. مشکل بزرگ: فایل‌ها فقط «متن» می‌فهمند! ۱۳۵
۵. طلسم زبان‌های باستانی: جادوی encoding (مخصوص فارسی) ۱۳۹
۶. پروژه: کتاب هیولاشناسی (The Bestiary) 100 xp – ۱۴۰

فصل ۱۲: دمیدن روح در کد: خلق موجودات با شیء‌گرایی ۱۴۴

۱. فراتر از لیست و دیکشنری ۱۴۵
۲. نقشه ساخت (Class): دستورالعمل ۱۴۵
۳. طلسم زنده کردن (Object): ساخت موجود واقعی ۱۴۶
۴. ویژگی موجودات (Attributes) ۱۴۶
۵. تغییر ویژگی‌ها: کیمیاگری داده‌ها ۱۴۷
۶. رفتارها (Methods): یاد دادن رفتار ۱۴۷
۷. راز بزرگ کلمه self من کی هستم؟ ۱۴۸
۸. طلسم خلقت (__init__): جادوی اتوماتیک ۱۴۹
۹. پروژه: ساخت مغازه جادوگری (The Magic Shop) 100 xp – ۱۵۰

مأموریت‌های جانبی: تالار پیشرفته ۱۵۴

- مأموریت‌ها ۱۵۵
- کتابخانه سایه‌ها (نتیجه مأموریت‌ها) ۱۵۸

بخش چهارم: آیین‌های جادوگر ارشد ۱۶۲

فصل ۱۳: طلسم تفکیک: هر کد سر جای خودش ۱۶۳

۱. چرا یک فایل کافی نیست؟ (اصل تفکیک مسئولیت‌ها) ۱۶۴
۲. طلسم import شخصی (ساخت اولین ماژول) ۱۶۴
۳. احضار مستقیم: جادوی from ... import ۱۶۶
۴. پروژه: معماری «مغازه جادوگری» 100 xp – ۱۶۷

فصل ۱۴: کتاب جادویی زمان ۱۷۱

۱. احضار datetime ۱۷۲
۲. طلسم now() (گرفتن لحظه حال) ۱۷۲

۱۷۳	۳. قالب‌بندی زمان (strptime) (طلسم)
۱۷۳	۴. سفر در زمان (timedelta) (طلسم)
۱۷۴	۵. پروژه: اعلان‌گر ددلاین (The Deadline Notifier)

فصل ۱۵: آیین قبیله پایتون هنرکدنویسی پایتونیک ۱۷۹

۱۸۰	۱. دُن پایتون (The Zen of Python)
۱۸۰	۲. طلسم enumerate (شمارش پایتونیک)
۱۸۱	۳. طلسم zip (جفت کردن)
۱۸۲	۴. طلسم تک‌خطی (List Comprehensions)
۱۸۲	۵. جادوی ناشناس (Lambda)
۱۸۳	۶. پروژه: کیمیاگری لیست 100 xp – (List Alchemy)

مأموریت‌های جانبی: محفل معماران زمان ۱۸۷

۱۸۸	مأموریت‌ها
۱۹۱	کتابخانه سایه‌ها (نتیجه مأموریت‌ها)

بخش پنجم: نبرد نهایی (پروژه بزرگ ماجراجو) ۱۹۴

فصل ۱۶: نبرد نهایی: ساخت دفترچه مأموریت ماجراجو ۱۹۵

۱۹۶	۱. نگاهی به گذشته (Gathering Your Spells)
۱۹۶	۲. توضیحات چالش (The Adventurer's Challenge)
۱۹۶	۳. میز معمار (Building Step-by-Step)

مأموریت‌های جانبی: نسخه بعدی (Expansion Pack) ۲۰۳

۲۰۴	مأموریت‌ها
۲۰۶	کتابخانه سایه‌ها (نتیجه مأموریت‌ها)

بخش ششم: ورود به تالار پیشگویان (پیش‌نیاز هوش مصنوعی) ۲۱۱

فصل ۱۷: دروازه احضار و آزمایشگاه جادویی (Pip & Venv) ۲۱۲

۲۱۳	۱. آزمایشگاه ایزوله (venv)
۲۱۳	۲. فعال‌سازی آزمایشگاه (ورود به حباب)
۲۱۴	۳. دروازه احضار (pip)
۲۱۴	۴. فایل نیازمندی‌ها (requirements.txt)

۵. پروژه: تابلوی اعلانات جادویی - 100 xp ۲۱۵

فصل ۱۸: طلسم‌های NumPy (جادوی محاسبات فوق سریع)

۱. احضار NumPy ۲۱۸

۲. ndarray چیست؟ (ارتش منظم) ۲۱۸

۳. جادوی Vectorization (حذف حلقه‌ها) ۲۱۸

۴. جادوی ابعاد: ماتریس‌ها ۲۱۹

۵. جعبه ابزار جادویی: ۱۰ طلسم پرکاربرد ۲۲۰

۶. مسابقه بزرگ: مفهوم زمان‌سنجی ۲۲۱

۷. پروژه: چالش ۱,۰۰۰,۰۰۰ ماجراجو (The Great Speed Race) - 100 xp .. ۲۲۲

فصل ۱۹: کتابخانه بزرگ‌پانداز (کار با داده‌های جدولی)

۱. احضار Pandas ۲۲۷

۲. DataFrame چیست؟ (جدول جادویی) ۲۲۷

۳. خواندن فایل اکسل (فایل‌های CSV, xlsx) ۲۲۸

۴. ذره‌بین جادویی: فیلتر کردن داده‌ها ۲۲۸

۵. جعبه ابزار پانداس: ۱۰ طلسم پرکاربرد ۲۲۹

۶. پروژه: تحلیل تابلوی امتیازات (The Grand Guild Analysis) - 100 xp ۲۳۰

فصل ۲۰: نمودار جادویی (Matplotlib)

۱. احضار نقاش سلطنتی (Matplotlib) ۲۳۶

۲. اولین قلم‌مو (plot) - نمودار خطی ۲۳۶

۳. ستون‌های مقایسه (bar) - نمودار میله‌ای ۲۳۶

۴. زیبایی‌شناسی جادویی (شخصی‌سازی) ۲۳۷

۵. جعبه ابزار نقاش: ۱۰ طلسم پرکاربرد ۲۳۷

۶. پروژه: گزارش تصویری انجمن (The Visual Report) - 100 xp ۲۳۸

مأموریت‌های جانبی: تالار پیشگویان

مأموریت‌ها ۲۴۴

کتابخانه سایه‌ها (نتیجه مأموریت‌ها) ۲۵۰

بخش هفتم: ماجراجویی ادامه دارد... ۲۵۶

فصل ۲۱: افق‌های بی‌پایان (نقشه‌های گنج آینده) ۲۵۷

۱. قدم‌های بعدی (The Essential Next Steps) ۲۵۸

۲. مسیر معماران دیجیتال (Programming & Web) ۲۵۸

۳. مسیر پیشگویان داده (AI & Data) ۲۵۹

۴. مسیر محافظان و مدیران سیستم (SysAdmin & Security) ۲۵۹

سخن پایانی: پایان آغاز ۲۶۲

منابع ۲۶۳

بخش اول: شروع ماجراجویی!

در این بخش، مشعل رو روشن می‌کنیم، با دنیای کد آشنا میشیم و یاد می‌گیریم چطور مثل یک ماجراجو، اولین معماها را حل کنیم.

فصل ۱: اولین قدم: آماده سازی محیط توسعه

مأموریت فصل: کارت معرفی دیجیتال برای ورود به انجمن ماجراجویان

نقشه راه:

- چرا پایتون؟
- آماده کردن کوله پشتی: نصب پایتون یا استفاده از آزمایشگاه آنلاین پایتون
- اولین طلسم جادویی: اجرای `print("Hello, World!")` و دیدن نتیجه!
- پروژه: ساخت کارت معرفی دیجیتال مخصوص ماجراجویان با اسم خودت.

۱. چرا پایتون؟

صدها زبان برنامه‌نویسی وجود دارد، پس چرا همه دارن از پایتون حرف می‌زنن و چرا ما این ماجراجویی رو با پایتون شروع می‌کنیم؟

۱. یادگیری مثل چت کردنه، خیلی راحت!

بزرگ‌ترین فرق پایتون با زبان‌هایی مثل C++ یا Java اینه که Syntax (قواعد نوشتاری) خیلی ساده و روانی داره. انگار داری به زبان انگلیسی دستور می‌دی. به جای اینکه درگیر کلی علامت عجیب و غریب و قوانین سخت بشی، مستقیم میری سراصل مطلب و راه حل رو پیاده می‌کنی.

۲. با یک تیر، سه تا نشونه رو هدف می‌گیری!

پایتون فقط برای شروع کردن عالی نیست؛ اونقدر قدرتمنده که بزرگترین شرکت‌های دنیا دارن ارزش برای ساختن آینده استفاده می‌کنن. با یادگیری پایتون، در واقع کلید ورود به سه تا از خفن‌ترین دنیاهای تکنولوژی رو به دست میاری:

- **هوش مصنوعی (AI):** تا حالا فکر کردی هوش مصنوعی‌هایی مثل ChatGPT چطور یاد می‌گیرن باهات حرف بزنن، یا ماشین‌های خودران چطور رانندگی رو یاد می‌گیرن؟ طلسم اصلی برای آموزش دادن به این هوش‌های مصنوعی ها، پایتونه. یعنی جادوگرهای دنیای تکنولوژی، با استفاده از پایتون به این ماشین‌ها یاد میدن که چطور فکر کنن، الگوها رو تشخیص بدن و تصمیم بگیرن. در واقع، پایتون زبان اصلی برای ساختن و تربیت کردن مغز متفکریه که پشت این تکنولوژی‌های شگفت‌انگیز قرار داره. (البته زبان‌های دیگه ای هم هستن ولی پایتون برای آموزش هوش مصنوعی محبوب‌ترین)
- **توسعه وب (Web Development):** اون بخش‌های مغز متفکر سایت‌ها و اپلیکیشن‌های معروفی مثل اینستاگرام، یوتیوب و کافه بازار با پایتون ساخته شده. (البته سایت‌های بزرگ از زبان‌های دیگه هم استفاده کردن و فقط پایون به تنهایی نیست)
- **علم داده (Data Science):** امروز، داده‌ها مثل گنج‌های پنهان هستن. پایتون بهترین بیل و کلنگ برای کشف این گنج‌هاست. باهاش می‌تونی حجم بالایی از اطلاعات رو تحلیل کنی، الگوهای عجیب و غریب پیدا کنی و آینده رو پیش‌بینی کنی!

و البته خیلی کارای جذاب دیگه میشه با پایتون میشه انجام داد ...

انتخاب پایتون مثل انتخاب یک چاقوی سوئیسی برای شروع ماجراجویییه. ، تو با یک ابزار ساده، قدرتمند و همه کاره شروع می کنی که هم راه رو برات باز می کنه و هم تو رو به هر مقصدی که بخوای می رسونه.

پس **کوله پشتی** رو آماده کن، چون قراره با این زبان جادویی، اولین قدم ها رو برای تبدیل شدن به یک ماجراجوی واقعی در دنیای تکنولوژی برداری!

۲. آماده کردن کوله پشتی: نصب پایتون یا استفاده از آزمایشگاه آنلاین

هر ماجراجویی به ابزار نیاز داره. ابزار اصلی ما، خود زبان پایتونه. برای استفاده از قدرت پایتون، دو راه اصلی پیش روی ماست: یکی اینکه **کارگاه شخصی** خودمون رو بسازیم (نصب پایتون روی کامپیوتر یا لپ تاپ خودمون) و دومی اینکه از یک **آزمایشگاه آنلاین** جادویی و آماده استفاده کنیم (پایتونی که به صورت آنلاین ارائه میشه مناسب بچه هایی هست که سیستم قوی ندارن و یا از گوشی و تبلت استفاده میکنن).

انتخاب با توئه ماجراجو، تمام کد ها برای ۲ مسیر در نظر گرفته شده پس با خیال راحت هر کدام رو که میخوای انتخاب کن!

مسیر اول: ساخت کارگاه شخصی (نصب پایتون روی کامپیوتر و لپ تاپ)

این مسیر برای کسانی که دوست دارن همه چیز تحت کنترل خودشون باشه و ابزارهاشون رو روی سیستم شخصی شون داشته باشن.

قدم اول: نصب پایتون

۱. برای کاربران ویندوز

به وبسایت رسمی پایتون python.org برو. اونجا آخرین نسخه پایتون منتظرته. روی دکمه "Downloads" کلیک کن و فایل نصبی (installer) رو برای ویندوز دانلود کن.

مهم ترین نکته نصب: فایل نصبی رو که اجرا کردی، به یک صفحه نصب می رسی. خیلی خیلی مهمه که در همون صفحه اول، تیک گزینه **Add Python to PATH** رو بزنی. این کار مثل اینه که به

ویندوز بگی "هی، من یه ابزار جدید به اسم پایتون دارم، حواست بهش باشه. (اگه این کار رو انجام ندی زمانی که کد های پایتون رو اجرا کنی بهت ارور اینکه پایتون رو نمیشناسم میده و اگه یادت رفت این تیک رو بزنی دوباره مراحل رو از اول برو و تیک رو بزنی)

نصب نهایی: حالا روی Install Now کلیک کن و منتظر بمون تا نصب کامل بشه. تمام! کارگاه شخصی تو آماده‌ست.

۲. برای کاربران مک

کاربران مک هم میتونن به سادگی از نصب‌کننده رسمی پایتون استفاده کنن.

سفر به منبع اصلی: به وب‌سایت رسمی پایتون python.org برو و وارد بخش "Downloads" شو. وب‌سایت به طور هوشمند سیستم عامل مک تو را تشخیص می‌دهد.

دریافت ابزار: روی دکمه دانلود آخرین نسخه برای macOS کلیک کن. یک فایل با پسوند pkg یا dmg دانلود میشه.

اجرای جادوی نصب: فایل نصب کننده را باز کن. این کار یک پنجره نصب استاندارد مک را باز می‌کند. مراحل بسیار ساده است؛ فقط کافیه روی دکمه‌های Continue, Agree و در نهایت Install کلیک کنی. در این مرحله، مک از تو رمز عبور سیستم را می‌خواهد تا اجازه نصب را صادر کند.

پایان نصب: پس از چند لحظه، نصب‌کننده کارش را تمام می‌کند و پایتون به صورت کامل روی سیستم تو نصب می‌شود. حالا کارگاه شخصی تو روی مک آماده است!

۳. برای کاربران لینوکس

لینوکس، سرزمین موردعلاقه برنامه نویس هاست و پایتون معمولاً جزئی از خون این سیستم‌عامله. اما باید مطمئن بشیم که جدیدترین نسخه رو داریم.

چک کردن نسخه: ترمینال رو باز کن و این دستور رو تایپ کن:

```
python3 --version
```

اگه نسخه‌ای که نشون می‌ده جدید (مثلاً ۳.۶ به بالا)، کارت راحته!

آپدیت کردن ابزار: اگه نسخه‌ت قدیمیه یا اصلاً نصب نیست، با توجه به توزیع لینوکست (مثل اوبونتو، فدورا و...)، با یک دستور ساده آپدیتش کن. برای اکثر سیستم‌های مبتنی بر دبیان (مثل اوبونتو)، این دستور کافیه:

```
sudo apt update
```

```
sudo apt install python3
```

تمام! تو آماده‌ای.

قدم دوم: آماده کردن دفترچه جادو (محیط توسعه VS Code)

حالا که پایتون را نصب کردیم، به یک "دفترچه جادویی" برای نوشتن طلسم‌هایمان (کدها) نیاز داریم. بهترین انتخاب برای این کار، Visual Studio Code (یا به اختصار VS Code) است. فکر کن VS Code یک دفترچه خفنه که کلمات جادویی رو برات رنگی می‌کنه، اگه دستوری رو اشتباه بنویسی زیرش خط می‌کشه و کمک می‌کنه کدهات رو مثل یک ماجراجوی حرفه‌ای، تمیز و مرتب نگه داری.

دانلود دفترچه جادو: به وب‌سایت رسمی code.visualstudio.com برو. سایت به طور خودکار سیستم‌عامل تو (ویندوز، مک یا لینوکس) را تشخیص می‌دهد. روی دکمه بزرگ دانلود کلیک کن. **نصب ساده:** فایل دانلود شده را اجرا کن و مراحل نصب را که بسیار ساده است، دنبال کن.

قدم سوم: افزونه شناسایی پایتون در دفترچه جادویی (VsCode)

برای اینکه دفترچه جادویی ما زبان پایتون را به بهترین شکل بفهمد، باید یک افزونه (Extension) به آن اضافه کنیم:

VS Code را باز کن. از نوار کناری سمت چپ، روی آیکونی که شبیه ۴ مربع است کلیک کن (بخش Extensions). در کادر جستجو، کلمه Python را تایپ کن.

اولین گزینه‌ای که توسط Microsoft ارائه شده را پیدا کن و روی دکمه آبی رنگ Install کلیک کن. تبریک! حالا کارگاه شخصی تو با بهترین ابزارها آماده‌ی ماجراجویی است.

مسیر دوم: استفاده از آزمایشگاه جادویی آنلاین (Google Colab)

اگه حوصله نصب و درگیری با سیستم‌عامل رو نداری، یا با گوشی و تبلت این ماجراجویی رو دنبال می‌کنی، یا سیستمت خیلی قوی نیست، این مسیر برای تو ساخته شده!

گوگل کولب (Google Colab) یک دفترچه یادداشت آنلاین و هوشمند که پایتون از قبل روش نصب شده و فقط منتظر دستورات جادویی توئه.

چرا خفنه؟

بدون نیاز به نصب: فقط با یک مرورگر و اکانت گوگل (رایگان) کارت راه میفته.

همیشه در دسترس: از هر جایی (کافه، کتابخانه، اتوبوس) و با هر وسیله‌ای (لپ‌تاپ، تبلت، گوشی) به کدها دسترسی داری.

قدرت ابری: انگار داری با کامپیوترهای قدرتمند گوگل کار می‌کنی، پس نگران ضعیف بودن سیستم نباش. (در عمل کدها روی سرورهای گوگل اجرا میشه)

چطور شروع کنیم؟

فقط کافیه به وبسایت `colab.research.google.com` بری، با اکانت گوگل وارد بشی و روی `New notebook` کلیک کنی. یک صفحه جدید باز می‌شه که می‌تونی اولین کدهای پایتون رو همونجا بنویسی و اجرا کنی. به همین سادگی و کاملاً رایگان (البته محدودیت‌هایی داره مثل تعداد ساعت و سخت‌افزار که برای افزایش اونا هزینه داره ولی ما اصلاً در این کتاب نیازی به اکانت پروی گوگل کولب نداریم و با اکانت رایگان میتونید تا انتهای کتاب رو بیاید)!

خب ماجراجو، کوله‌پشتیت رو انتخاب کردی؟ ابزارت رو آماده کن که می‌خوایم اولین طلسم جادویی رو اجرا کنیم!

۳. اولین طلسم جادویی: اجرای `print("Hello, World!")` و دیدن نتیجه!

تبریک! کوله‌پشتی‌ات آماده است و ابزارهایت را انتخاب کرده‌ای. اکنون زمان اینه که اولین طلسم جادویی خود را یاد بگیری و اون رو اجرا کنی. این لحظه، لحظه‌ای فراموش‌نشده‌ی و خیلی خاصه؛ لحظه‌ای که برای اولین بار به ماشین دستور می‌دهی و او به تو پاسخ می‌دهد. این طلسم جادویی، `print` نام دارد. کارش خیلی ساده است: هر چیزی که تو داخل پرانتز و بین دو علامت نقل قول "" به اون بده، روی صفحه نمایش نشون میده. مثل `print("Hi")`

بیایید با هم این طلسم را اجرا کنیم!

اگر از مسیر اول (کارگاه شخصی و VS Code) می‌آیی: (برای بچه‌های `colab` بخش جدایی رو در ادامه توضیح میدم)

برای اینکه ماجراجویی هامون مرتب باشه، اول یک پوشه (Folder) برای پروژمون می‌سازیم.

ساخت پوشه پروژه: روی دسکتاپ یا هر جای دیگری که دوست داری، یک پوشه (Folder) جدید بساز و اسم آن را my_project بزار. تمام طلسم‌ها، گنج‌ها و نقشه‌های ما در این پوشه نگهداری خواهند شد.

باز کردن دفترچه جادو (VS Code): برنامه VS Code را که قبلاً نصب کردی، باز کن. **باز کردن پوشه پروژه در vs code:** از منوی بالا، روی File کلیک کرده و گزینه‌ی Open Folder را انتخاب کن. حالا به همان پوشه‌ی پروژه که ساختی برو و آن را انتخاب کن. با این کار، VS Code تمام تمرکزش را روی این پوشه و پروژه تو میزارد.

ساخت اولین فایل پایتون: در سمت چپ VS Code، جایی که اسم پوشه‌ی پروژه را می‌بینی، یک آیکون کوچک شبیه به یک برگه با علامت + وجود دارد. روی آن کلیک کن و اسم فایل جدید را project.py بگذار. پسوند py به Vs code می‌گه که این یک فایل پایتون است! (حتماً پسوند فایل رو py. بزار این خیلی مهمه! و اگه نزاری دیگه vs code فایلت رو درست نمیشناسه و کلاً این قاعده وجود داره که پسوند فایل مثل png، txt، py. به ما می‌گن که محتوای این فایل قراره چی باشه)

نوشتن طلسم جادویی: حالا در صفحه‌ی سفیدی که باز شده، اولین طلسم جادویی خود را با دقت تایپ کن (خیلی به علامت‌های " و فاصله‌ها و ... دقت کن چون توی برنامه نویسی اگه حتی یک نقطه رو اشتباه بزاری کدت کار نمی‌کنه (اعدادی که در کنار کد ها توی این کتاب میبینی صرفاً دارن می‌گن خط چنده و جزوی از کد نیستن و اون هارو نباید بنویسی):

```
1. print("Hello, World!")
```

اجرای طلسم! وقت هیجان‌انگیز ماجرا فرا رسیده. باید "ترمینال" یا همان اتاق فرمان را باز کنیم تا طلسم خود را در آن بخوانیم.

از منوی بالای VS Code، روی Terminal کلیک کن و New Terminal را انتخاب کن. **راه میان‌بر و سریع‌تر:** می‌توانی دکمه‌های ترکیبی Ctrl + ~ (کلید ~ معمولاً کنار عدد ۱ روی کیبورد است) را فشار دهی تا ترمینال فوراً باز شود. در مک، این میان‌بر Cmd + ~ است.

خواندن دستور نهایی: در ترمینالی که پایین صفحه باز شده، این دستور را تایپ کن و کلید Enter را بزن (به فاصله و املای کلمات دقت کن و همچنین اگه اسم فایل پایتونت رو چیز دیگه ای گذاشتی اون رو اینجا بنویس و به صورت کامل با پسوند py.):

```
python project.py
```

نکته: برای ماجراجویان مک و لینوکس: بجای دستور بالا python3 project.py را امتحان کنید.
و ناگهان... جادو اتفاق می‌افتد!
درست در خط بعدی ترمینال، این پیام ظاهر می‌شود:

```
Hello, World!
```

تبریک! تو همین الان اولین کد خود را اجرا کردی. تو به کامپیوتر دستور دادی و او از تو اطاعت کرد. این اولین قدم تو برای تبدیل شدن به یک جادوگر قدرتمند در دنیای کدنویسی است. این لحظه را جشن بگیر!

اگر از مسیر دوم (آزمایشگاه جادویی Google Colab) می‌آیی:

کار تو بسیار ساده‌تر است! آزمایشگاه آنلاین همه چیز را برایت آماده کرده.
ورود به آزمایشگاه: در همان نوت بوک جدیدی که در Colab باز کردی، یک سلول کد خاکستری رنگ با یک دکمه "اجرا" (▶) در کنارش می‌بینی. (به این می‌گن سلول کد و میتونی داخلشون کد خودت رو بنویسی و با دکمه پلی کنارش کدت اجرا میشه و اگه نبود میتونی از دکمه بالا گزینه Code رو بزنی تا یه سلول کد جدید ایجاد بشه)

نوشتن طلسم جادویی: داخل این سلول، طلسم جادویی را تایپ کن (اعدادی که در کنار کد ها توی این کتاب میبینی صرفاً دارن می‌گن خط چنده و جزوی از کد نیستن و اون هارو نباید بنویسی):

```
1. print("Hello, World!")
```

اجرای طلسم: حالا کافیه روی همان دکمه "اجرا" (▶) کلیک کنی یا کلیدهای Shift + Enter را با هم فشار دهی.
بلافاصله زیر سلول کد، نتیجه‌ی جادوی خود را خواهی دید:

```
Hello, World!
```

فوق‌العاده بود، نه؟ تو اولین کد خودت رو روی سرور گوگل اجرا کردی!
ماجراجویی رسماً آغاز شد!

۴. پروژه: ساخت کارت ویزیت دیجیتال (The Digital ID Card) – 100 xp

ماجراجوی عزیز، به انجمن خوش آمدی! تو اولین طلسم جادویی خودت را یاد گرفتی. حالا وقت آن است که با استفاده از همین قدرت، هویت خودت را رسماً به «انجمن ماجراجویان پایتون» اعلام کنی. ما در این پروژه یک **کارت ویزیت دیجیتال** خواهیم ساخت تا همه بدانند با چه ماجراجوی جدیدی آشنا شده‌اند.

تصویر نهایی: این کارت چه شکلی است؟

قبل از شروع، بیا نتیجه را ببینیم. وقتی برنامه را اجرا کنی، قرار نیست یک کاغذ از پرینتر بیرون بیاید! بلکه در صفحه سیاه «ترمینال» «یا خروجی» Colab، مشخصات تو خط به خط و خیلی مرتب ظاهر می‌شود. چیزی شبیه به این:

```
1. Name: Mahdi Ghaemi
2. Title: Python World Explorer
3. Contact: adventurer@example.com
4. Motto: Ready to solve the first puzzle!
5. =====
```

نقشه و طرح اولیه

در اولین پروژه، ما باید ابزارهایمان را بشناسیم. این‌ها آچار و پیچ‌گوشتی‌های تو برای این مأموریت هستند:

۱. **طلمس چاپگر print()** و **راز پرانتزها** فکر کن print یک فرمانده است که به کامپیوتر دستور می‌دهد: نمایش بده. اما کامپیوتر می‌پرسد: چه چیزی را؟. اینجا است که پرانتزها () وارد می‌شوند. پرانتزها مثل یک **ظرف** یا سینی هستند. هر چیزی که داخل این ظرف بگذاری، تحویل پرینتر داده می‌شود.

- print دستور (انجام بده)
- () ظرف ورودی (روی چه چیزی انجام بده؟)

۲. **مرزهای جادویی نقل قول‌ها ("")** چرا متنی که می‌نویسیم را داخل "" می‌گذاریم؟ پایتون باید بفهمد کجا «دستور» تمام شده و کجا «پیام متنی» شروع شده است. به مثال زیر دقت کن:

```
1. print("print print")
```

- print اول: دستور پایتون (اجرا می‌شود).
- "print print" دوم: فقط یک متن ساده است (چاپ می‌شود). اگر "نباشد، پایتون گیج می‌شود و فکر می‌کند کلمه دوم هم یک دستور است!

۳. **قانون آبشار (جریان اجرا)** پایتون کدها را مثل یک آبشار می‌خواند؛ از بالا به پایین، خط به خط. یعنی اول خط ۱ را کامل اجرا می‌کند، بعد می‌رود سراغ خط ۲. پایتون هرگز نمی‌پرد و هرگز دو خط را همزمان اجرا نمی‌کند. این ترتیب به ما اجازه می‌دهد داستانمان را مرحله به مرحله تعریف کنیم.

۴. **یادداشت‌های نامرئی: # کامنت‌ها** ماجراجویان باهوش همیشه یادداشت برمی‌دارند. در پایتون، اگر اول خط علامت # بگذاری، آن خط **نامرئی** می‌شود! یعنی پایتون آن خط را نمی‌بیند

و اجرا نمی‌کند. پس فایده‌اش چیست؟ این یادداشت‌ها برای تو و بقیه انسان‌هاست تا یادتان نرود کد چه کاری انجام می‌دهد.

چالش‌های فکری

قبل از نوشتن کد اصلی، به این سوال‌ها فکر کن:

چالش ۱: ظرف خالی؟ اگر بنویسیم `print()` و داخل پرانتز چیزی نگذاریم، چه اتفاقی می‌افتد؟
(پاسخ: یک خط خالی چاپ می‌شود - مثل این است که یک کاغذ سفید به پرینتر بدهی)

چالش ۲: شکستن مرز اگر بنویسیم `print("Hello")` (یعنی کوتیشن دوم را فراموش کنیم)، پایتون چه واکنشی نشان می‌دهد؟ (پاسخ: خطای *Syntax Error* می‌دهد، چون منتظر است ببیند متن کجا تمام می‌شود).

کد نهایی پروژه (The Spell)

حالا وقت خلق کردن است. کد زیر را در فایل `project.py` بنویس .

مهم: اطلاعات داخل کد (مثل اسم من) را پاک کن و مشخصات واقعی خودت را جایگزین کن و به کامنت‌های انگلیسی دقت کن تا ببینی چطور از `#` استفاده کرده‌ایم.

```
1. # --- Adventurer's ID Card ---
2.
3. # Line 1: Displays the adventurer's name
4. # (Replace [Your Name] with your real name)
5. print("Name: Mahdi Ghaemi")
6.
7. # Line 2: Your chosen title in this journey
8. print("Title: Python World Explorer")
9.
10. # Line 3: A way for others to contact you
11. print("Contact: mahdighaemi123@gmail.com")
12.
13. # Line 4: Your personal motto (Slogan)
14. print("Motto: Don't touch the working code!")
15.
16. # Line 5: A border to make it look cool
17. print("=====")
```

اجرا کن و لذت ببر! دستور اجرا را بزن دکمه ▶ یا `python project.py` در ترمینال؟ پایتون دقیقاً به همان ترتیبی که گفتی، کارت تو را ساخت.

جشن پیروزی و تحلیل

تبریک می‌گوییم! تو نه تنها اولین کد خود را اجرا کردی، بلکه اولین **محصول نرم‌افزاری** خودت را ساختی. این کارت معرفی، مدرک ورود تو به دنیای شگفت‌انگیز برنامه‌نویسی است.

ماموریت جانبی: از خروجی کارنت عکس بگیر و در شبکه‌های اجتماعی با هشتگ `#codewithmahdi` و `#pythonadventure` به اشتراک بگذار. با این کار ما می‌فهمیم که یک ماجراجوی جدید و بااستعداد به جمع ما اضافه شده است!

توی فصل‌های بعد باهم یاد می‌گیریم کارهای باحال‌تری انجام بدیم و چیزای بیشتری بسازیم.

🌟 ارتقاء لول: ۲ → ۱ 🌟

مهارت‌های جدید:

◆ دستور `Print`

با نام باستانی: آوای آفرینش

توضیح: سکوت جهان دیجیتال شکسته شد. اکنون کلمات تو

بر صفحه سیاه هستی نقش می‌بندند.

◆ سطح جدید: `H`

توضیح: تبریک به سطح

جدیدی رسید.

فصل ۲: تبدیل شدن به کارآگاه کد

مأموریت فصل: شکار اولین "باگ"

نقشه راه:

- ذهنیت یک ماجراجو: برنامه نویسی یعنی حل مسئله، نه حفظ کردن کد!
- جعبه ابزار کارآگاه:
 - Stack Overflow: کتابخانه بزرگ برای تمام جادوگران دنیا.
 - دستیارهای هوشمند (AI): چگونه از غول های چراغ جادو کمک بگیریم؟
- هنر پرسیدن سوال خوب.
- **پروژه:** یک کد نفرین شده به شما داده می شود. با ابزارهای جدیدتان، خطای اون رو پیدا و آن را فیکس کنید.

۱. ذهنیت یک ماجراجو: برنامه نویسی یعنی حل مسئله، نه حفظ کردن کد!

اولین و مهم‌ترین قانونی که هر ماجراجوی کهنه‌کاری می‌دونه اینه:
برنامه نویسی یعنی حل مسئله، نه حفظ کردن کد!

بسیاری از کسانی که تازه شروع میکنند فکر می‌کنند که باید صدها دستور و طلسم جادویی مانند print را «حفظ» کنند. اما اصل ماجرا این نیست. اصل ماجرا در «تفکر منطقی» توست.

وقتی کد تو کار نمی‌کنه یا با یک پیام خطای ترسناک قرمز رنگ (که به زودی با آن‌ها آشنا می‌شویم) مواجه می‌شوی، نترس! این یک معما است. کامپیوتر، برخلاف انسان‌ها، موجودی کاملاً منطقی است. او دقیقاً کاری را می‌کند که به او می‌گویی. اگر نتیجه اشتباه است، یعنی ما دستور را به شکلی نامفهوم یا اشتباه به او داده‌ایم.

به این دستورات اشتباه، «باگ» یا «طلسم معیوب» می‌گیم. و مأموریت ما در این فصل، «رفع نفرین» (Debugging) کردن آن‌هاست.

پس ذهنیت خود را آماده کن؛ تو یک کارآگاه هستی. هر خطا (Error) یک سرخ است. هر باگ، یک چالش جذاب است که منتظر توست تا آن را حل کنی.

۲. جعبه ابزار کارآگاه

هر کارآگاهی به ابزار نیاز دارد. برای شکار باگ‌ها، لازم نیست همه چیز را از صفر اختراع کنیم. ما دو ابزار فوق‌العاده قدرتمند در «کوله پشتی» خود داریم که باید طرز استفاده از آن‌ها را یاد بگیریم.

۱. Stack Overflow: کتابخانه بزرگ برای تمام جادوگران دنیا

تصور کن یک کتابخانه جادویی بی‌انتهای وجود دارد که در آن، هر جادوگر و ماجراجویی که تا به حال با یک کد نفرین شده روبرو شده، مشکل و راه‌حلش را در آن ثبت کرده است.

آن کتابخانه، Stack Overflow است.

این یک وب‌سایت پرسش و پاسخ برای برنامه‌نویسان است. به احتمال ۹۹٪، هر مشکلی که تو در این ماجراجویی با آن روبرو می‌شوی، قبلاً صدها نفر دیگر هم با آن روبرو شده‌اند و بهترین جادوگران دنیا به آن پاسخ داده‌اند. یادگیری «جستجو کردن» در این کتابخانه، اغلب مهم‌تر از

حفظ کردن هر طلسمی است. (برای استفاده کافی هست پیغام خطایی که با آن مواجه شدی رو در گوگل سرچ کنی و حتماً از بین نتیجه جستجو Stack Overflow رو میبینی)

۲. دستیارهای هوشمند (AI): چطور از غول های چراغ جادو کمک بگیریم؟

حالا تصور کن غول هایی وجود دارند که می توانند به ما در نوشتن طلسم های بهتر کمک کنند، کدهای ما را توضیح دهند و حتی باگ های آن را پیدا کنند! ابزارهایی مثل Gemini و ChatGPT دقیقاً همین کار را می کنند. اونا دستیارهای هوشمند تو هستن. میتونی کد باگ دار خودت رو بهشون نشون بدی و پرسی: «کجای این کد اشتباست؟» اونا هم با صبر و حوصله به تو کمک میکنن که مشکل رو پیدا کنی.

هنر پرسیدن سوال خوب

داشتن جعبه ابزار کافی نیست؛ باید بلد باشی چطور از آن استفاده کنی. چه در Stack Overflow و چه از دستیار هوشمند، «خوب پرسیدن» یک هنر است. یک ماجراجوی تازه کار میگه که: «کمک! کد من کار نمی کنه!» و هیچ کس نمیتونه به اون کمک کنه. اما یک کارآگاه حرفه ای می گه: «من در حال اجرای یک طلسم print در فایل project.py هستم. انتظار داشتم کارت ویزیت را چاپ کنم، اما در عوض با این پیام خطای قرمز مواجه شدم: SyntaxError: EOL while scanning string literal. این هم کدی هست که نوشتم. به نظرتون مشکل از کجاست؟»

همیشه سه چیز را در سوالت بیاور:

۱. چه کار می خواستی بکنی؟ (نتیجه مورد انتظار؟)
 ۲. چه مشکلی پیش آمد؟ (پیام خطا دقیقاً چه بود؟ آن را کپی کن.)
 ۳. کدی که نوشتی چه بود؟ (کد خودت را نشان بده.)
- با این سه مورد، هر کسی می تواند به تو در شکار باگ کمک کند.

۳. پروژه: شکار باگ در کد نفرین شده (Bug Hunting) – 100 xp

آماده ای اولین شکار را انجام دهی، کارآگاه؟ در این مأموریت، ما قرار نیست چیزی را از صفر بسازیم. در عوض، یک تکه کد پایتون داریم که قرار بود «کارت ویزیت» یک ماجراجو را چاپ کند، اما توسط یک جادگر خبیث نفرین شده و کار نمی کند. پایتون با کسی شوخی ندارد؛ اگر قوانین

نگارش (Syntax) را رعایت نکنی، به تو خطای قرمز نشان می‌دهد. مأموریت تو این است که کد را «دییگ» کنی (منظور از باگ مشکل و خطا هست پس دییگ رو می‌گیم رفع خطا و مشکل).

تصویر نهایی: کارت ویزیت سالم

قبل از اینکه وارد صحنه جرم شویم، باید بدانیم نتیجه سالم چه شکلی است. اگر کد را درست کنی، خروجی باید دقیقاً این شکلی، تمیز و بدون هیچ نوشته قرمزی در ترمینال چاپ شود:

```
Name: Mahdi ghaemi
Title: World Ender
Motto: Brave and Bold!
=====
```

نقشه کارآگاه (چگونه باگ شکار کنیم؟)

یک برنامه‌نویس حرفه‌ای وقتی ارور (Error) می‌بیند، نمی‌ترسد؛ بلکه کلاه کارآگاهی‌اش را سر می‌کند. برای رفع طلسم، باید این ۳ مرحله را طی کنی:

۱. **صحنه جرم را ببین (اجرا کن)** : کد خراب را کپی و اجرا کن. پیام خطای قرمز رنگی که پایتون در پایین صفحه (ترمینال) به تو نشان می‌دهد، دشمن تو نیست؛ **سرنخ** توست!

۲. **آدرس دقیق را پیدا کن** : پایتون معمولاً در پیام خطا به تو می‌گوید که در کدام خط (Line x) گیج شده است. مستقیم به همان خط برو.

۳. **بررسی مظنون‌ها (قوانین نگارش)** : حالا باید مثل یک معلم سخت‌گیر، گرامر کد را چک کنی:

- **مرزهای جادویی** : آیا هر متنی که شروع شده (") ، پایان هم دارد (") ؟
- **ظروف جادویی** : آیا هر پرانتزی که باز شده (، بسته هم شده ؟)
- **رنگ کد** : در محیط‌های کدنویسی (IDE) ، رنگ متون باید متفاوت از کدها باشد. اگر رنگ‌ها قاطی شده‌اند، یعنی یک جای کار می‌لنگد.

چالش‌های فکری

کد طلسم شده زیر را در فایل project.py کپی کن (برای بچه‌های colab داخل یک سلول کپی کنید). این کد پراز اشکال است! قبل از دیدن جواب، سعی کن با ذره‌بین خودت پیدایشان کنی:

```
1. # --- Cursed Code (DO NOT RUN... yet!) ---
2. print("Name: Mahdi ghaemi)
3. print("Title: World Ender"
4. print("Motto: "Brave and Bold!")
5. print(=====)
```

چالش ۱: خط دوم (فرار متن) در خط ۲، متن `Name: ...` شروع شده اما مرز پایانی ندارد. پایتون از کجا بفهمد متن تمام شده؟ (راهنمایی: کوتیشن "گم شده")

چالش ۲: خط سوم (ظرف باز) در خط ۳، ما دستور `print` را باز کردیم اما درش را نبستیم! (راهنمایی: پرانتز گم شده)

چالش ۳: خط پنجم (اشتباه هویت) در خط آخر، ما می‌خواستیم یک خط جداکننده `===` چاپ کنیم. اما چون دورش "نگذاشتیم، پایتون فکر می‌کند این‌ها دستورات ریاضی هستند و گیج می‌شود!

جعبه ابزار کمکی: اگر روزی با خطایی روبرو شدی که نفهمیدی (مثلاً `SyntaxError`)، متن خطا را در گوگل کپی کن و کلمه **Stack Overflow** را ته آن بنویس. آنجا کتابخانه بزرگ جادوگران است که هزاران نفر قبلاً مشکل تو را داشته‌اند و راه حلش را پیدا کرده‌اند.

کد نهایی و درمان شده (The Cure)

حالا وقت باطل کردن طلسم است. تغییرات را در کد اعمال کن تا به شکل زیر در بیاید:

```
1. # --- My ID Card (Cure applied) ---
2.
3. # 1. Fixed the missing quote at the end
4. print("Name: Mahdi ghaemi")
5.
6. # 2. Fixed the missing parenthesis
7. print("Title: World Ender")
8.
9. # 3. Fixed the messy quotes inside the string
10. print("Motto: Brave and Bold!")
11.
12. # 4. Added quotes to make it a string, not code
13. print("=====")
```

اجرا کن و نفس راحت بکش! الان باید کارت ویزیت به زیبایی چاپ شود. این حس خوبی که الان داری؟ به آن می‌گویند لذت دیباگ کردن (واسه من لذتش شبیه به اینکه یه سوزنی که رفته توی پام رو از پام دربیارم).

جشن پیروزی و تحلیل

تبریک می‌گویم کارآگاه! تو الان رسماً اولین باگ خودت را شکار کردی. یادت باشد، حتی بهترین برنامه‌نویسان دنیا هم هر روز با ارورهای قرمز مواجه می‌شوند. تفاوت آن‌ها با بقیه این است که ارور را «شکست» نمی‌بینند، بلکه آن را یک «معما» می‌بینند که باید حل شود.

کوله‌پشتی تو برای ماجراجویی بعدی سنگین‌تر و ارزشمندتر شد. در فصل بعد، ابزارهای جادویی جدیدی برای کار با اعداد خواهیم ساخت!

🌟 ارتقاء لول: ۲ → ۳ 🌟

مهارت‌های جدید:

◆ مهارت Debugging

با نام باستانی: شکار سایه‌ها
توضیح: هیچ طلسمی بی‌نقص نیست. هنر تو یافتن شکاف‌ها و ترمیم آن‌هاست.

◆ جستجو در StackOverflow

با نام باستانی: احضار خرد جمعی
توضیح: اتصال به ذهن هزاران جادوگر دیگر برای یافتن پاسخ.

◆ سطح جدید: G

توضیح: تبریک به سطح جدیدی رسید.

فصل ۳: جعبه های جادویی:

ماشین حساب جادویی

مأموریت فصل: ساخت یک ماشین حساب جادویی برای محاسبه و تقسیم غنائم بین اعضای گروه

نقشه راه

- جعبه های جادویی (متغیرها): چطور غنائم و اطلاعات را در حافظه نگه داریم؟
- آشنایی با غنائم: انواع داده های پایه (Integers, Floats, Strings, Booleans).
- جادوی ریاضی: کار با عملگرهای + ، - ، * و / روی اعداد.
- جادوی متن: چطور طلسم + برای متن ها (Strings) متفاوت عمل می کند؟
- **پروژه:** ساخت یک ماشین حساب که غنائم (اعداد ثابت) را محاسبه و نتایج را چاپ می کند.

۱. جعبه های جادویی (متغیرها)

وقتی در ماجراجویی خود یک «سکه طلا» یا یک «معجون سلامتی» پیدا می کنی، آن را روی زمین رها نمی کنی تا گم شود! تو آن را در یک «جعبه» امن می گذاری تا بعداً از آن استفاده کنی. در برنامه نویسی، به این جعبه های جادویی «متغیر» (Variable) می گوئیم. متغیرها، مکان هایی در حافظه رم (RAM) کامپیوتر هستند که ما داده هایمان را در آن ها ذخیره می کنیم. این کار باعث می شود بتوانیم بعداً به آن داده ها دسترسی داشته باشیم. این داده ها موقتی هستند و وقتی برنامه تمام شود (یا کامپیوتر خاموش شود)، از بین می روند. (در فصل های بعد با اینکه چجوری به صورت دائمی اطلاعات رو ذخیره کنیم بیشتر آشنا میشیم) ساختن یک متغیر مثل برجسب زدن به یک جعبه است. ما از علامت = (که به آن عملگر تخصیص می گوئیم) برای ریختن چیزی در جعبه استفاده می کنیم (یادت نره وقتی # رو اول یه خطی میزاریم یعنی اون خط توضیحه و پایتون اون رو اجرا نمیکنه):

```
1. # "Mahdi Ghaemi" is the content inside the box
2. # adventurer_name is a label
3. adventurer_name = "Mahdi Ghaemi"
4.
5. # gold_coins is a label
6. # 50 is the content inside the box
7. gold_coins = 50
```

حالا، هر وقت که بخوایم، میتونیم محتوای داخل جعبه رو ببینیم. از print و نام اون جعبه استفاده میکنیم (دقت کن که نام متغیر نباید داخل " باشه وگرنه با متن اشتباه گرفته میشه):

```
1. # Let's make some magic boxes
2. adventurer_name = " Mahdi Ghaemi"
3. gold_coins = 50
4. health_points = 100
5.
6. # Now let's print the content of the boxes
7. print("Adventurer's Name:")
8. print(adventurer_name) # Python prints the box content, not its name
9.
10. print("Gold Coins:")
11. print(gold_coins)
12.
13. print("Health Points:")
14. print(health_points)
```

اجرا کن و نتیجه را ببین! وقتی این کد را اجرا کنی، کامپیوتر به جای چاپ کلمه "gold_coins"، می رود و محتوای داخل آن جعبه (یعنی ۵۰) را چاپ می کند.

قدرت متغیرها: تغییرپذیری

جادوی واقعی متغیرها این است که محتوای آن‌ها می‌تواند تغییر کند.

```
1. # At the start of the adventure, we have 10 coins
2. gold_coins = 10
3. print("Current gold coins:")
4. print(gold_coins)
5.
6. # We defeat a giant and get 20 new coins
7. # We replace the old content of the box with the new value
8. gold_coins = 20
9. print("Gold coins after defeating the giant:")
10. print(gold_coins)
11.
12. # We can even do calculations on the box itself
13. # We spend 5 coins
14. gold_coins = gold_coins - 5
15. print("Remaining gold coins:")
16. print(gold_coins) # Output will be 15
```

این قابلیت برای ماجراجویی ما خیلی مهمه!

۲. آشنایی با انواع داده‌های پایه (Primitive Types)

متغیرها مثل جعبه‌های جادویی هستن که می‌تونن چیزهای مختلفی رو داخل خودشون نگه دارن.

اما توی این ماجراجویی یه قانون خیلی مهم وجود داره:

نوع چیزی که داخل جعبه می‌ذاری، مشخص می‌کنه باهش چه کارهایی می‌تونی انجام بدی.

مثلاً توی دنیای واقعی نمی‌تونی یک شمشیر رو با سکه طلا جمع بزنی!

شمشیر شبیه «متنه» و سکه طلا شبیه «عدد»ه. هر کدوم جادوی مخصوص خودشون رو دارن.

به این نوع داده‌ها می‌گیم داده‌های پایه یا **Primitive Types**

این‌ها ساده‌ترین و اصلی‌ترین نوع داده‌ها هستن و از همون اول داخل خودِ پایتون ساخته شدن. پایتون خودش می‌فهمه که:

- عدد چیه
- متن چیه

و لازم نیست تو این چیزها رو بهش یاد بدی.

در ماجراجویی پایتون، ما با چهار نوع داده اصلی سروکار داریم:

۱. اعداد صحیح (Integer)

۲. اعداد اعشاری (Float)

۳. رشته (String)

۴. بولین (Boolean)

اعداد صحیح (Integer)

این تیرها در واقع اعداد کامل و صحیح هستند. می توانند مثبت، منفی یا صفر باشند. هر وقت چیزی را می شماری (تعداد تیر، تعداد دشمن، تعداد قدم) از این نوع استفاده می کنی.

```
1. age = 21
2. score = -20
3. count = 0
```

اعداد اعشاری (Float)

فلوت ها در واقع اعداد اعشاری هستند؛ هر وقت به «دقت» نیاز داری (مثل محاسبه میزان دما یا درصد سلامتی) از این نوع استفاده می کنی.

```
1. pi = 3.14
2. temp = -273.0
3. percent = 99.0
```

نکته: عدد 22 عدد صحیح حساب میشه و 22.0 عدد اعشاری حساب میشه. پایتون اعدادی که اعشار دارن رو به صورت خودکار تبدیل نمیکنه به صحیح و همونجوری که هست نگهشون میداره

رشته (String)

رشته ها در واقع «متن» هستند. هر چیزی که داخل علامت نقل قول (" یا ") قرار بگیرد، یک متن یا String است. نام ها، پیام ها و ... همه از این نوع هستند.

```
1. name = "mahdi"
2. message = "Oppsi"
```

بولین (Boolean)

بولین ها فقط دو حالت دارند: True (درست) یا False (نادرست). این بولین ها برای تصمیم گیری استفاده می شوند. (آیا در باز است؟ True. آیا من زنده هستم؟ False). ما در فصل بعد خیلی بیشتر از این بولین ها استفاده میکنیم.

```
1. is_up = True
2. is_down = False
```

۳. جادوی ریاضی: کار با عملگرها

حالا که «عدد صحیح» (Integers) و «عدد اعشاری» (Floats) را می‌شناسیم، بیا با آن‌ها محاسبات انجام دهیم. پایتون چهار عملگر اصلی ریاضی را بلد است:

+ (جمع)

- (تفریق)

* (ضرب)

/ (تقسیم)

بیا اینا را با متغیرها امتحان کنیم:

ما ۲ صندوقچه پیدا کردیم با ۵۰ و ۷۵ سکه می‌خواهیم این سکه هارو بین ۴ نفرمون تقسیم کنیم به هر نفر چقدر میرسه؟

```
1. # We found the loot
2. gold_coins_chest1 = 50
3. gold_coins_chest2 = 75
4. adventurers_count = 4
5.
6. # --- Performing the magic of math ---
7.
8. # 1. Add the loot from both chests
9. total_gold = gold_coins_chest1 + gold_coins_chest2
10.
11. # 2. Divide the loot among the party members
12. gold_per_adventurer = total_gold / adventurers_count
13.
14. # --- Displaying the results ---
15. print("Total gold found:")
16. print(total_gold) # Output: 125
17.
18. print("Share per adventurer:")
19. print(gold_per_adventurer) # Output: 31.25
```

نکته مهم: دیدی که حاصل تقسیم ۱۲۵ بر ۴ شد ۳۱٫۲۵؟ در پایتون، عملگر / همیشه نتیجه را به صورت «عدد اعشاری» (Float) برمی‌گرداند تا هیچ بخشی از غنائم در تقسیم از بین نرود!

۴. جادوی متن: چسباندن متن‌ها بهم

یک نکته بسیار مهم! عملگر + یک جادوی دوگانه دارد.

وقتی با اعداد (Integers/Floats) استفاده شود، کار «جمع ریاضی» را انجام می‌دهد.

وقتی با متن‌ها (Strings) استفاده شود، کار «چسباندن» (Concatenation) را انجام می‌دهد.

این دو مثال را مقایسه کن:

```

1. # The magic of math (adding numbers)
2. num1 = 20
3. num2 = 5
4. result_math = num1 + num2
5. print("Result of math magic:")
6. print(result_math) # Output: 25
7.
8. # The magic of text (concatenating strings)
9. text1 = "20" # This is a string, not a number
10. text2 = "5" # This is also a string
11. result_text = text1 + text2
12. print("Result of text magic:")
13. print(result_text) # Output: "205"

```

این تفاوت بسیار مهم است و در فصل های آینده که می خواهیم ورودی بگیریم، دوباره با آن روبرو خواهیم شد. فعلاً فقط یادت باشد: پایتون بین «سکه» ۲۰ و «رشته» "۲۰" تفاوت قائل می شود! (اولی عدد هست و دومی رشتست و در زمان جمع رفتار متفاوتی دارن)

۴. پروژه: ساخت ماشین حساب جادویی (The Magic Calculator) – 100

xp

ما جراجو، وقت آن است که طلسم های پراکنده ای که امروز یاد گرفتیم (Variables, Numbers, Math Operators) را با هم ترکیب کنیم. ما می خواهیم اولین ابزار محاسباتی خودمان را بسازیم. در دنیای برنامه نویسی، ما همیشه ماشین حساب دم دست نداریم؛ ما خودمان ماشین حساب را می سازیم!

مأموریت: محاسبه غنائم

فرض کن ما دو صندوقچه گنج پیدا کرده ایم (دو عدد). هدف این است که برنامه ای بنویسیم که این دو عدد را بگیرد و بلافاصله تمام محاسبات حیاتی (جمع، تفریق، ضرب و تقسیم) را روی آن ها انجام دهد و یک گزارش زیبا چاپ کند.

تصویر نهایی: این برنامه قراره چیکار کنه؟

قبل از کدنویسی، نتیجه را تصور کن. برنامه ما مثل یک لوح هوشمند عمل می کند. به محض اجرا شدن:

۱. به تو خوش آمد می گوید

۲. دو عددی که در دل کد مخفی کرده ایم را به نمایش می گذارد.

۳. و بعد ... **بوم!** نتایج جمع، تفریق، ضرب و تقسیم آن ها را زیر هم و خیلی مرتب لیست می کند. دیگر نیازی نیست چهار بار جداگانه حساب کنی، پایتون همه را یکجا انجام می دهد.

نقشه و طرح اولیه

بیا منطق این ابزار را مهندسی کنیم. ما سه مرحله اصلی داریم:

۱. **تعریف مواد اولیه: (Variables)** ریاضیات بدون عدد معنا ندارد. ما باید دو «جعبه» (متغیر) بسازیم و عددهایمان را داخل آن‌ها بریزیم.

- منطق: ساخت متغیر num1 و num2 و مقداردهی به آن‌ها (مثلاً ۱۵۰.۵ و ۲۵.۰).

۲. **انجام جادوی ریاضی: (The Math Logic)** حالا باید موتور محاسبات را روشن کنیم.

- منطق: چهار متغیر جدید می‌سازیم تا نتیجه‌ی جمع (+)، تفریق (-)، ضرب (*) و تقسیم (/) را در خودشان نگه دارند.

۳. **نمایش گزارش: (The Display)** محاسبات که تمام شد، باید آن‌ها را به کاربر نشان دهیم.

- منطق: استفاده از دستور print برای چاپ کردن خط‌کشی‌های زیبا و نتایج نهایی.

چالش‌های فکری

قبل از اینکه کد را ببینی، سعی کن این معماها را حل کنی:

چالش ۱: نمادهای ممنوعه! در ریاضیات روی کاغذ، برای ضرب از «×» استفاده می‌کنیم. اما اگر این را در پایتون بنویسی، خطا می‌گیری! یادت هست نماد ضرب در دنیای کامپیوتر چه شکلی بود؟
(راهنمایی: *) :

چالش ۲: ذخیره یا چاپ؟ آیا بهتر است محاسبات را مستقیم داخل پرانتزِ print انجام دهیم، یا اول آن‌ها را در یک متغیر ذخیره کنیم مثلاً sum_result و بعد چاپ کنیم؟ (راهنمایی: ذخیره کردن در متغیر بهتر است، چون کد را تمیزتر و خواناتر می‌کند).

کد نهایی پروژه (The Spell)

حالا وقت نوشتن است. به فایل project.py برو و این کد را با دقت بنویس. سعی کن کامنت‌های انگلیسی را بخوانی تا بفهمی در هر خط چه اتفاقی می‌افتد.

```
1. # --- The Magic Calculator ---
2.
3. print("Welcome to the Magic Calculator!")
4. print("Today's numbers for calculation...")
5.
6. # --- Step 1: Define the loot (Variables) ---
7. # We define our numbers here. Try changing them later!
8. num1 = 150.5 # First treasure
9. num2 = 25.0 # Second treasure
```

```

10.
11. # Show the chosen numbers to the user
12. print("First number:")
13. print(num1)
14. print("Second number:")
15. print(num2)
16.
17. # --- Step 2: Perform Math Magic ---
18. # We store the results in new variables
19. sum_result = num1 + num2      # Addition
20. subtract_result = num1 - num2 # Subtraction
21. multiply_result = num1 * num2 # Multiplication
22. divide_result = num1 / num2   # Division
23.
24. # --- Step 3: Display the Results ---
25. print("=====")
26. print("Your magical calculation results:")
27.
28. print("Sum (+): ")
29. print(sum_result)
30.
31. print("Subtraction (-): ")
32. print(subtract_result)
33.
34. print("Multiplication (*): ")
35. print(multiply_result)
36.
37. print("Division (/): ")
38. print(divide_result)
39.
40. print("=====")

```

اجرا کن و نتیجه را ببین! خروجی تو باید دقیقاً مثل یک گزارش رسمی و زیبا باشد. بعد از اینکه اجرا کردی، برگرد و اعداد num1 و num2 را تغییر بده (مثلاً ۱۰۰ و ۱۰ بگذار) و دوباره اجرا کن. می بینی؟ ماشین حساب تو همیشه درست کار می کند!

جشن پیروزی و تحلیل

آفرین! شاید ساده به نظر برسد، اما تو الان اولین نرم افزار محاسباتی خودت را نوشتی. تو یاد گرفتی که:

۱. چطور داده ها را در **متغیرها** ذخیره کنی.

۲. چطور از پایتون به عنوان یک **ماشین حساب** قدرتمند استفاده کنی.

۳. و چطور خروجی ها را **مرتب و خوانا** به کاربر نمایش دهی.

این پایه و اساس تمام برنامه های مالی و مهندسی بزرگ دنیاست. ماجراجویی ادامه دارد...

🌟 ارتقاء لول: ۵ → ۳ 🌟

مهارت های جدید:

◆ مفهوم Variable

با نام باستانی: جعبه های جادویی
توضیح: قدرت نام گذاری بر اشیاء. هر
داده ای که نامی داشته باشد در تسخیر توست.

◆ عملگرهای ریاضی

با نام باستانی: محاسبات کیهانی
توضیح: جهان بر پایه اعداد بنا شده و
تو اکنون معمار آنی.

فصل ۴: منطق و شرط‌ها:

ساخت بازی حدس عدد

مأموریت فصل: ساخت بازی "حدس عدد" و به چالش کشیدن کامپیوتر.

نقشه راه

- صحبت با ماجراجو: گرفتن ورودی از کاربر با `input()`.
- تله‌ی کیمیاگری (تبدیل متن به عدد)
- دو راهی سرنوشت (`if` و `else`)
- مسیرهای چنگانه (`elif`)
- تکرار تا پیروزی (حلقه `while`)
- ۶. کنترل قدرتمند حلقه: `break` و `continue`
- **پروژه:** پیاده‌سازی بازی حدس عدد.

۱. صحبت با ماجراجو: گرفتن ورودی از کاربر با `input()`

تا الآن ما فقط با دستور `print` حرف می‌زدیم (اطلاعات می‌دادیم). حالا می‌خواهیم گوش کنیم (اطلاعات بگیریم). طلسم جادویی برای شنیدن، `input()` است. وقتی پایتون به این خط می‌رسد، توقف می‌کند و منتظر می‌ماند تا شما چیزی تایپ کنید و دکمه Enter را بزنید.

```
1. print("what is your name?")
2. name = input()
3.
4. print("hi " + name + "weelcome")
```

نکته: طلسم `input` به قابلیت باحال داره

می‌تونی پیام رو همون داخل پرانتزش بنویسی؛ این‌طوری کدت کوتاه‌تر و تمیزتر می‌شه. وقتی برنامه اجرا می‌شه، پایتون خودش اون پیام رو نشون می‌ده؛ دقیقاً انگار از `print` استفاده کردی. مثلاً:

```
1. name = input("what is your name?")
2. print("hi " + name + "weelcome")
```

اینجا اول پیام سمت چپ نمایش داده می‌شه،

بعد چیزی که کاربر می‌نویسه داخل متغیر `name` ذخیره می‌شه و در آخر بهش خوش‌آمد می‌گیم

۲. تله‌ی کیمیاگری (تبدیل متن به عدد)

یک قانون خیلی مهم در پایتون وجود دارد: دستور `input` همیشه همه‌چیز را به صورت "رشته" (String) می‌گیره. حتی اگر شما عدد تایپ کنید!

اگر این کد را بنویسید، نتیجه فاجعه‌بار است:

```
1. # wrong code
2. num1 = input("num1:") # user enter -> 10
3. num2 = input("num2:") # user enter -> 20
4.
5. # python will "10" + "20" = "1020"
6. print(num1 + num2)
```

راه حل (کیمیاگری): ما باید متن را به عدد تبدیل کنیم تا پایتون به جای گذاشتن این ۲ عدد در کنار هم آن هارا با هم جمع کند . طلسم جادویی برای تبدیل کردن رشته به عدد صحیح، `int()` است.

```
1. # fixed code
2. text1 = input("num1:") # user enter -> 10
3. num1 = int(text1) # python will cast "10" to 10
4.
5. text2 = input("num2:") # user enter -> 20
6. num2 = int(text2) # python will cast "20" to 20
7.
8. print(num1 + num2) # 10 + 20 = 30
```

۳. دو راهی سرنوشت (if و else)

اینجا مهم ترین بخش منطق برنامه نویسی است. می خواهیم بگوییم: "اگر رمز درست بود، در را باز کن، وگرنه آژیر بکش." ما این کار را با `if` (اگر) و `else` (در غیر این صورت) انجام می دهیم.

قانون طلایی تورفتگی (Indentation)

پایتون چطور می فهمد کدام خط ها مربوط به "باز کردن در" است و کدام خط ها مربوط به "آژیر کشیدن"؟ جواب: با فاصله از سر خط. هر وقت جلوی خطی علامت **دو نقطه**: گذاشتید (مثل آخر خط `if`)، خط های بعدی باید کمی جلوتر (معمولاً ۴ تا فاصله یا یک دکمه `Tab` و بهتر هست از `Tab` استفاده کنید) شروع شوند. این فاصله به پایتون می گن که: "این خط ها متعلق به شرط بالایی هستند." بیایید در عمل ببینیم:

```
1. password = "123"
2. guess = input("Enter password: ")
3.
4. if guess == password:
5.     # We have an indentation (Tab) here.
6.     # This block runs ONLY if the condition is True
7.     print("Access Granted: Open the door")
8.
9. else:
10.    # We have an indentation (Tab) here too.
11.    # This block runs if the condition is False (Wrong password).
12.    print("Access Denied: Enable alarm")
13.
14. # This line has NO indentation.
15. # It is outside the if/else blocks and always runs.
16. print("Continue program...")
```

تصور کن با استفاده از `if` و `else` داری ۲ تا جعبه میسازی که روی جعبه اول نوشته شده فقط در صورتی که این شرط رمز درست بود منو باز کن و روی دیگری برعکس اینو نوشته پایتون میاد و بر اساس اون شرط شما جعبه متناسب درش رو باز میکنه و اجرا میکنه و اون یکی جعبه رو ایگنور میکنه کامل و رد میشه و میره بعد از شرط و ادامه میده

۴. مسیره‌های چندگانه (`elif`)

اگر بخواهیم بیشتر از دو حالت داشته باشیم چه؟ مثلاً در بازی حدس عدد (بازی که نفر اول یه عدد انتخاب میکنه مثلاً بین ۱ تا ۱۰ و یکی دیگه باید حدس بزنه مثلاً ۵ و نفر اول میگه که عدد بالاتره یا پایینتر یا درسته) ، فقط "درست" و "غلط" نداریم. سه حالت داریم:

- عدد درست است (برنده).
- عدد شما کوچک است (راهنمایی کن).
- عدد شما بزرگ است (راهنمایی کن).

برای این کار از `elif` (مخفف `Else If`) استفاده می‌کنیم.

```
1. secret_number = 7
2. user_guess = int(input("Guess a number: "))
3.
4. if user_guess == secret_number:
5.     print("Great! You guessed correctly.")
6.
7. elif user_guess < secret_number:
8.     print("Your number is too small!")
9.
10. else:
11.     # If it is not equal and not smaller, it must be larger.
12.     print("Your number is too big!")
```

نکته: پایتون به ترتیب از بالا `if` چک می‌کند. به محض اینکه یکی از شرطها درست باشد، آن را اجرا می‌کند و بقیه `elif` و `else` را نادیده می‌گیرد.

۵. تکرار تا پیروزی (حلقه `while`)

مشکل کد بالا این است که فقط یک بار اجازه حدس زدن می‌دهد. ما می‌خواهیم بازی آنقدر تکرار شود تا بازیکن برنده شود.

طلسم جادویی `while` (تا زمانی که) دقیقاً همین کار را می‌کند. ساختار آن دقیقاً مثل `if` است (نیاز به دو نقطه : و تورفتگی دارد)، با این تفاوت که وقتی کدهای داخلش تمام شد، دوباره به بالا برمی‌گردد و شرط را چک می‌کند.

```
1. # Create a variable to control the game state
2. game_over = False
3.
4. while game_over == False:
5.     # The lines below repeat constantly because they are indented
6.     print("The game is running...")
7.
8.     answer = input("Do you want to end the game? (yes/no) ")
9.
10.    if answer == "yes":
11.        game_over = True # This makes the loop condition False, stopping the loop
12.
13. print("Goodbye!")
```

نکته : وقتی داخل حلقه هستیم کدهای بعد اون اجرا نمیشن و حلقه دائم ادامه پیدا میکنه تا زمانی که این حلقه بلاخره تموم بشه و بعد کدهای بعدش شروع میکنن به اجرا شدن.

۶. کنترل قدرتمند حلقه: طلسم های `break` و `continue`

گاهی اوقات وسط اجرای یک طلسم تکرارشونده، اتفاقی می‌افتد که می‌خواهیم فوراً حلقه را متوقف کنیم یا بخشی از آن را نادیده بگیریم. برای این کار، پایتون دو طلسم جادویی قدرتمند در اختیار ما می‌گذارد:

الف) دستور `break` : خروج اضطراری

تصور کنید در حال مبارزه در حلقه هستید و ناگهان جان شما کم می‌شود. دیگر مهم نیست شرط حلقه چیست، شما باید همان لحظه فرار کنید! دستور `break` دقیقاً مثل شکستن شیشه اضطراری است؛ به محض اجرا شدن، حلقه بلافاصله نابود می‌شود و برنامه به خط بعد از حلقه می‌رود.

```
1. # Infinite loop seeking magic phrase
2. while True:
3.     text = input("Say the magic word: ")
4.
5.     if text == "Abra Kadabra":
6.         print("Spell broken! You are free.")
7.         break # The loop ends right here!
8.
9.     print("Wrong word! Try again.")
```

ب) دستور `continue`: جا خالی دادن

گاهی نمی‌خواهید از حلقه خارج شوید، بلکه فقط می‌خواهید از روی یک مرحله خاص بپرسید و به دور بعدی بروید. دستور `continue` به پایتون می‌گوید: بی‌خیال بقیه کدهای این دور شو و سریع برو سراغ دور بعدی!

مثلاً فرض کنید می‌خواهیم از ۱ تا ۱۰ بشماریم، اما عدد ۳ را چاپ نکنیم:

```
1. number = 0
2. while number < 5:
3.     number = number + 1
4.
5.     if number == 3:
6.         print("Skipping the unlucky number!")
7.         continue # Jumps back to line 2 (start of loop)
8.
9.     print("Number is:", number)
```

خروجی: اعداد ۱، ۲، ۴ و ۵ چاپ می‌شوند (۳ چاپ نمی‌شود چون قبل از `print` دستور `continue` اجرا شد).

نکته جادوگری: استفاده از `break` و `while True` یک الگوی بسیار رایج در بین برنامه‌نویسان حرفه‌ای است، چون کنترل کاملی به شما می‌دهد که دقیقاً «کی» و «کجا» بازی را تمام کنید.

۷. پروژه: بازی بزرگ حدس عدد (The Great Number Guessing Game) – 100 xp

ماجراجو، تبریک می‌گوییم! تو ابزارهای قدرتمندی مثل `input` (گوش شنوا)، `int` (تبدیل‌گر)، `if/else` (تصمیم‌گیرنده) و `while` (حلقه زمان) را در اختیار داری.

حالا وقت آن است که تمام این طلسم‌ها را با هم ترکیب کنیم و یک **بازی هوشمند** بسازیم.

مأموریت: قفل رمزی دروازه

هدف ما ساختن یک نگهبان دیجیتالی است. ما یک «عدد محرمانه» (مثلاً ۱۴) را در دل برنامه مخفی می‌کنیم. کاربر باید حدس بزند این عدد چیست. اما نگهبان ما ساکت نمی‌ماند؛ او راهنمایی می‌کند که عدد کاربر «بزرگتر» است یا «کوچکتر» تا بالاخره رمز درست پیدا شود.

تصویر نهایی: این بازی چطور کار می‌کند؟

قبل از اینکه وارد کد شویم، بیا جریان بازی را تصور کنیم.

۱. برنامه اجرا می‌شود و می‌گوید: «من یک عدد بین ۱ تا ۲۰ انتخاب کرده‌ام.»

۲. برنامه منتظر می‌ماند تا تو عددی را وارد کنی.

۳. اگر گفتی ۵: برنامه می‌گوید: «خیلی کوچک است! بالاتر برو.» و دوباره منتظر می‌ماند.

۴. اگر گفتی ۲۰: برنامه می‌گوید: «خیلی بزرگ است! پایین‌تر بیا.» و دوباره منتظر می‌ماند.

۵. اگر گفتی ۱۴: برنامه می‌گوید: «آفرین! دروازه باز شد.» و بازی تمام می‌شود.

نقشه و طرح اولیه

بیا مثل یک مهندس نرم افزار، منطق پشت صحنه را طراحی کنیم. ما به ۳ بخش اصلی نیاز داریم:

۱. تنظیمات اولیه: (The Setup)

- باید یک عدد را به عنوان رمز در یک متغیر ذخیره کنیم (مثلاً `secret_number`).
- به یک «کلید» نیاز داریم تا وضعیت بازی را کنترل کند. اول بازی، هنوز کسی برنده نشده، پس متغیر `correct_guess` را روی `False` (نادرست) می گذاریم.

۲. حلقه اصلی: (The Loop)

- بازی نباید با یک حدس اشتباه تمام شود! ما می خواهیم تا زمانی که کاربر اشتباه می گوید، بازی ادامه پیدا کند.
- منطق: تا زمانی که `(while)` حدس درست نیست (`correct_guess == False`)، حلقه باید بچرخد.

۳. منطق مقایسه: (The Logic)

- در هر دور از حلقه، باید عدد کاربر را بگیریم و با عدد رمز مقایسه کنیم.
- اینجا به ۳ شاخه نیاز داریم: (`if, elif, else`)
 - آیا مساوی است؟ (برنده شدن و پایان حلقه).
 - آیا کوچکتر است؟ (راهنمایی: برو بالا).
 - آیا بزرگتر است؟ (راهنمایی: بیا پایین).

چالش های فکری

قبل از دیدن کد، سعی کن این گره های ذهنی را باز کنی:

چالش ۱: طلسم تبدیل (The Type Conversion)

وقتی با دستور `input` عددی را از کاربر می گیریم، پایتون آن را به شکل «متن» (`String`) می بیند (مثلاً "۱۴"). اما ما می خواهیم آن را با عدد ۱۴ مقایسه کنیم. با کدام دستور جادویی باید متن را به عدد صحیح تبدیل کنیم؟
(راهنمایی `int()`)

چالش ۲: شکستن حلقه (Breaking the Loop)

حلقه `while` ما طوری تنظیم شده که تا وقتی `correct_guess` برابر با `False` است، بچرخد.

وقتی کاربر عدد درست را حدس زد، چه تغییری باید در متغیر `correct_guess` ایجاد کنیم تا حلقه متوقف شود؟

(راهنمایی: باید تبدیل به `True` شود)

چالش ۳: ترتیب شرطها

آیا ترتیب `if` و `elif` مهم است؟ (بله خیلی مهمه اگه همزمان ۲ تا شرط `if` درست باشن برنامه همیشه اون اولی رو اجرا میکنه و ترتیب مهمه ولی توی این مسئله چون شرط ها همزمان نمیتونن درست باشن ترتیب اهمیتی نداره و نتیجه یکسانه)

کد نهایی پروژه (The Spell)

حالا وقت نوشتن است. کد زیر را با دقت در فایل `project.py` بنویس.

تمرین ویژه: سعی کن یک بار از روی این کد بنویسی، و بار دوم سعی کنی بدون نگاه کردن (یا فقط با نگاه کردن به نقشه بالا) آن را بازنویسی کنی.

```
1. # --- The Great Number Guessing Game ---
2.
3. # 1. Initial Setup
4. secret_number = 14          # The secret number creates the lock
5. correct_guess = False      # The gate is currently closed (False)
6.
7. print("Welcome to the Number Guessing Game!")
8. print("I have chosen a number between 1 and 20.")
9.
10. # 2. Start the Loop
11. # Repeat logic WHILE the guess is NOT correct yet
12. while correct_guess == False:
13.
14.     # Get input and VITAL STEP: convert text to integer
15.     guess_text = input("What is your guess? ")
16.     guess = int(guess_text)
17.
18.     # 3. Check Game Logic (The Brain)
19.
20.     # Scene 1: The guess is correct
21.     if guess == secret_number:
22.         print("Excellent! You guessed correctly. The gate opens!")
23.         correct_guess = True # We flip the switch to 'True' to END the
loop
24.
25.     # Scene 2: The guess is too low
26.     elif guess < secret_number:
27.         print("Your number is too small! Try higher.")
28.
29.     # Scene 3: The guess is too high (Logic: if not equal and not smaller,
it must be bigger)
30.     else:
```

```
31.         print("Your number is too big! Try lower.")
32.
33. # 4. End of Game
34. # This line is outside the while loop (no indentation)
35. print("Game Over. Thanks for trying!")
```

اجراکن و جادو را ببین!

برنامه را اجرا کن و سعی کن عدد مخفی را پیدا کنی.
بعداً می‌توانی `secret_number` را به هر عددی که دوست داری (مثلاً ۱۰۰۰) تغییر بدهی و دوستانت را به چالش بکشی!

جشن پیروزی و تحلیل

آفرین ماجراجو!

الان تو یکی از مهم‌ترین مهارت‌های برنامه‌نویسی تاریخ را یاد گرفتی: **تفکر منطقی**.
تو یاد گرفتی چطور با `if` و `else` به کامپیوتر «شعور» بدهی. تا قبل از این، کدهای ما فقط خط به خط پایین می‌رفتند، اما الان کد تو **تصمیم می‌گیرد** که کدام خط را اجرا کند و کدام را نادیده بگیرد.

تمام هوش مصنوعی‌ها، بازی‌های کامپیوتری و ربات‌ها، از همین منطق ساده شروع می‌شوند.
تو حالا از یک «کدنویس ساده» به یک **برنامه‌نویس متفکر** تبدیل شدی.
ماجرای جوی تازه شروع شده... ادامه بده!

🌟 ارتقاء لول: ۱۰ → ۵ 🌟

مهارت‌های جدید:

◆ ساختار If و Else

با نام باستانی: دوراهی سرنوشت
توضیح: دمیدن روح شعور در کالبد کد.
برنامه اکنون می‌تواند بیاندیشد و انتخاب کند.

◆ حلقه While

با نام باستانی: طلسم پایداری
توضیح: تکرار یک خواسته تا زمانی که
حقیقت محقق شود.

◆ سطح جدید: F

توضیح: تبریک به سطح
جدیدی رسید.

مأموریت‌های جانبی: تالار تمرین

ماجرای عزیز! قبل از اینکه وارد سرزمین‌های ناشناخته‌ی بخش دوم شوی، باید در «تالار تمرین» مهارت‌هایت را تقویت کنی. اینجا ۷ چالش پشت سرهم منتظر توست. قانون تالار ساده است: تا وقتی حداقل یکبار خودت تلاش نکردی، به پاسخ‌نامه نگاه نکن!

مأموریت‌ها

۱. مأموریت: نشانِ ماجراجو (The ID Badge) – 20 xp

اولین قدم برای ورود به انجمن، معرفی خودت است.

- **وظیفه:** برنامه‌ای بنویس که فقط یک خط کد داشته باشد و جمله‌ی "I am a Python Adventurer!" را روی صفحه چاپ کند.
- **هدف:** مطمئن شویم محیط کدنویسی تو درست کار می‌کند.

۲. مأموریت: دره پژواک (Echo Canyon) – 20 xp

در این دره جادویی، هر کلمه‌ای بگویی چند بار تکرار می‌شود.

- **وظیفه:** یک کلمه از کاربر بگیر. سپس یک عدد از کاربر بگیر. برنامه باید آن کلمه را به تعداد آن عدد تکرار کند و چاپ کند.
- **هدف:** یادگیری ضرب کردن رشته‌ها در پایتون (مثلاً 3 * "Hello")

۳. مأموریت: تبدیل‌گر طلا (The Gold Converter) – 20 xp

در بازار شهر، هر ۱ سکه طلا معادل ۱۰ سکه نقره است.

- **وظیفه:** برنامه‌ای بنویس که تعداد سکه‌های طلا را از کاربر بگیرد (ورودی)، آن را در ۱۰ ضرب کند و تعداد سکه‌های نقره را نشان دهد.
- **هدف:** تبدیل ورودی متنی به عدد (int) و انجام محاسبات ریاضی.

۴. مأموریت: نفس اژدها (The Dragon's Breath) – 20 xp

اژدهای نگهبان فقط دمای فارنهایت را می‌فهمد، اما دماسنج ما سانتی‌گراد است.

- **وظیفه:** دمای سانتی‌گراد را بگیر و به فارنهایت تبدیل کن. (فرمول: ضربدر ۱.۸، به علاوه ۳۲).
- **هدف:** کار با اعداد اعشاری (float) و فرمول‌نویسی.

۵. مأموریت: نگهبان دروازه (The Gatekeeper) – 20 xp

به یک در بسته رسیده‌ای که فقط با رمز عبور باز می‌شود. رمز صحیح عدد 1234 است.

- **وظیفه:** یک عدد به عنوان رمز از کاربر بگیر. اگر رمز برابر با 1234 بود، چاپ کن "Welcome!" و در غیر این صورت (else) چاپ کن "Access Denied!"
- **هدف:** درک شرط ساده if/else.

۶. مأموریت: چراغ راهنما (Traffic Light) – 20 xp

ربات‌های شهر باید قوانین راهنمایی را بلد باشند.

- **وظیفه:** رنگ چراغ را از کاربر بپرس (red, yellow, green) یا (green).
 - اگر red بود: چاپ کن. "Stop"
 - اگر yellow بود: چاپ کن. "Get Ready"
 - اگر green بود: چاپ کن. "Go"
 - برای هر چیز دیگر: چاپ کن. "Unknown Color"
- **هدف:** مدیریت چند شرط با elif.

۷. مأموریت: معمار اهرام (The Pyramid Builder) – 20 xp

برای رسیدن به قلعه‌ی آسمانی، باید پله‌هایی از جنس نور بسازی.

- **وظیفه:** یک عدد از کاربر بگیر (مثلاً ۵). برنامه‌ای بنویس که با استفاده از حلقه، به تعداد آن عدد، پله بسازد و چاپ کند.
 - پله اول: *
 - پله دوم: **
 - پله سوم: ***
 - والی آخر...

- **هدف:** استفاده از حلقه while و جادوی ضرب رشته‌ها (تکرار کاراکتر).

۸. مأموریت: پرتاب موشک (Rocket Launch) – 20 xp

وقت هیجان است! می‌خواهیم یک موشک به فضا بفرستیم.

- **وظیفه:** یک شمارش معکوس بساز. یک متغیر با مقدار ۱۰ بساز. با استفاده از حلقه (while)، تا زمانی که عدد بزرگتر از ۰ است، آن را چاپ کن و یکی از آن کم کن. وقتی حلقه تمام شد، چاپ کن. "Liftoff!"
- **هدف:** درک ساختار حلقه کاهش.

۹. مأموریت: صندوق‌دار بی‌نهایت (The Infinite Cashier) – 20 xp

شما مسئول صندوق فروشگاه هستید. مشتری‌ها تعداد نامشخصی کالا می‌خرند.

- **وظیفه:** یک برنامه بنویس که در یک حلقه همیشگی (True)، قیمت کالا را از کاربر بپرسد.

- قیمت‌ها را با هم جمع کن (یک متغیر برای مجموع بساز).
- اگر کاربر عدد 0 را وارد کرد، حلقه را بشکن (break) و جمع کل فاکتور را نمایش بده.

• **هدف:** استفاده از break و متغیر جمع‌کننده (Accumulator).

۱۰. مأموریت: باریستای هوشمند (The Smart Coffee Machine) – 20 xp

و اما چالش نهایی! یک دستگاه قهوه‌ساز بساز که هم قیمت را چک کند و هم بقیه پول را پس بدهد.

• **سناریو:** قیمت یک قهوه \$3.50 است.

• **وظیفه:**

۱. مبلغ پول را از کاربر بگیر (مثلاً ۵ دلار).
 ۲. اگر پول کمتر از ۳.۵۰ بود، بنویس: "Not enough money".
 ۳. اگر پول کافی بود، قهوه را بده و «بقیه پول (Change)» را حساب کن و به کاربر نمایش بده.
- **هدف:** یک پروژه کامل با ورودی، تبدیل نوع، شرط و ریاضی.

آیا سفرت را در این مرحله کامل کرده‌ای، ماجراجو؟ اگر بله، کتاب را ورق بزن؛ کتابخانه سایه‌ها (نتیجه مأموریت‌ها) در صفحه بعد انتظار تو را می‌کشد.

کتابخانه سایه‌ها (نتیجه مأموریت‌ها)

۱. مأموریت: نشانِ ماجراجو (The ID Badge)

```
1. # Just a simple print statement
2. print("I am a Python Adventurer!")
```

۲. مأموریت: دره پژواک (Echo Canyon)

نکته: در پایتون می‌توانید یک رشته را در یک عدد ضرب کنید تا تکرار شود.

```
1. word = input("Say something: ")
2. count = int(input("How many times? "))
3.
4. # Repeating the string
5. print(word * count)
```

۳. مأموریت: تبدیل گرطلا (The Gold Converter)

نکته: ورودی‌های input همیشه به صورت متن (String) هستند، پس برای انجام محاسبات ریاضی باید با `int()` آن‌ها را به عدد تبدیل کنید.

```
1. gold = int(input("Enter gold coins: "))
2. silver = gold * 10
3. print(f"You have {silver} silver coins.")
```

۴. مأموریت: نفس اژدها (The Dragon's Breath)

نکته: برای دما و پول معمولاً از اعداد اعشاری (float) استفاده می‌کنیم.

```
1. celsius = float(input("Enter Temperature in C: "))
2. fahrenheit = (celsius * 1.8) + 32
3.
4. print(f"Temperature in F: {fahrenheit}")
```

۵. مأموریت: نگهبان دروازه (The Gatekeeper)

نکته: در برنامه‌نویسی برای مقایسه کردن دو مقدار از دو علامت مساوی `==` استفاده می‌کنیم.

```
1. password = int(input("Enter password: "))
2.
3. if password == 1234:
4.     print("Welcome!")
5. else:
6.     print("Access Denied!")
```

۶. مأموریت: چراغ راهنما (Traffic Light)

نکته: زمانی که بیش از دو حالت شرطی داریم (مثل قرمز، زرد، سبز)، از دستور `elif` استفاده می‌کنیم.

```
1. color = input("Enter light color (red/yellow/green): ")
2.
3. if color == "red":
4.     print("Stop")
5. elif color == "yellow":
6.     print("Get Ready")
7. elif color == "green":
8.     print("Go")
9. else:
10.    print("Unknown Color")
```

۷. مأموریت: معمار اهرام (The Pyramid Builder)

نکته: در پایتون اگر یک رشته (مثل `*`) را در یک عدد ضرب کنید، به همان تعداد تکرار می‌شود.

```
1. height = int(input("Enter height: "))
2. step = 1
3.
4. while step <= height:
5.     print("*" * step)
6.     step = step + 1
```

۸. مأموریت: پرتاب موشک (Rocket Launch)

نکته: فراموش نکنید که داخل حلقه، حتماً از متغیر شمارنده کم کنید ($n = n - 1$)، وگرنه حلقه تا ابد ادامه می‌یابد!

```
1. n = 10
2.
3. while n > 0:
4.     print(n)
5.     n = n - 1
6.
7. print("Liftoff!")
```

۹. مأموریت: صندوق‌دار بی‌نهایت (The Infinite Cashier)

نکته: ما یک متغیر به نام `total` بیرون از حلقه نیاز داریم تا مجموع قیمت‌ها را در آن ذخیره کنیم.

```
1. print("--- Enter prices (Type 0 to finish) ---")
2. total = 0
3.
4. while True:
5.     price = float(input("Price: $"))
6.
7.     # If price is 0, stop the loop
8.     if price == 0:
```

```
9.         break
10.
11.     # Add price to total
12.     total = total + price
13.
14. print(f"Total Bill: ${total}")
```

۱۰. مأموریت: باریستای هوشمند (The Smart Coffee Machine)

نکته: این پروژه ترکیبی از ریاضی (تفریق هزینه از پول نقد) و شرط (بررسی کافی بودن پول) است.

```
1. print("--- Coffee Price: $3.50 ---")
2. cash = float(input("Insert money: $"))
3.
4. if cash < 3.50:
5.     print("Not enough money. Money returned.")
6. else:
7.     change = cash - 3.50
8.     print("Here is your coffee")
9.     print(f"Your change is: ${change}")
```


بخش دوم: مدیریت کوله

پشتی ماجراجو

عالی! ماجراجویی ما تا اینجا فوق العاده بوده .

تو در فصل قبل، به پایتون قدرت «تصمیم‌گیری» دادی و اولین بازی خودت را ساختی. تو حالا با `input`، `while`، `if` و کیمیاگری (`casting`) کاملاً آشنا هستی .

اما تا به حال، «جعبه‌های جادویی» (متغیرهای) ما فقط می‌توانستند یک غنیمت را در خود نگه دارند مثل `coin = 14`.

اگر در یک ماجراجویی، ناگهان یک صندوقچه پر از آیتم‌های مختلف پیدا کنی چه؟ یک شمشیر، یک سپر، ۱۰ معجون، و یک نقشه... آیا خواهی که برای هر کدام یک متغیر جدا بسازی؟ `item1 = "sword"`، `item2 = "shield"`... این که خیلی نامرتبه!

ما به یک «کوله پشتی» جادویی نیاز داریم. یک کوله پشتی بزرگ که بتواند همه‌ی غنائم دیگر را به صورت مرتب در خود نگه دارد.

وقت آن است که ماجراجویی جدیدی شروع کنیم.

فصل ۵: کوله‌پشتی جادویی:

مدیریت غنائم ماجراجو

مأموریت فصل: ساخت یک برنامه برای مدیریت لیست خرید.

نقشه راه

- کوله پشتی جادویی (Lists): قدرتمندترین ابزار برای نگهداری آیتم‌ها.
- شروع ماجراجویی با کوله‌پشتی پُر (Initialized List)
- پر کردن کوله پشتی: اضافه کردن، دیدن، و حذف کردن آیتم‌ها.
- بازرسی کوله پشتی: طلسم‌های `len()` و `in`
- سرشماری غنائم (حلقه `for`): ابزاری برای خواندن و مدیریت کوله پشتی.
- پروژه: ساخت برنامه مدیریت لیست خرید با قابلیت افزودن و حذف آیتم.

۱. کوله‌پشتی جادویی (Lists)

تا به حال ما با «داده‌های پایه» (Primitive Types) مثل `int` و `string` کار می‌کردیم. اما حالا با اولین «داده‌ی غیرپایه» (Non-Primitive Type) آشنا می‌شویم: **لیست (List)**. لیست، دقیقاً مثل یک «کوله‌پشتی» است: این یک لیست مرتب از آیتم‌هاست. (ترتیب آیتم‌ها مهم است) (تکرار آیتم هم مجازه). می‌تواند شامل انواع مختلف داده باشد (عدد، متن، یا حتی لیست‌های دیگر!). ساختن یک کوله‌پشتی (لیست) جدید، با استفاده از براکت `[]` انجام می‌شود:

```
1. # Our backpack is empty at the start of the adventure
2. # We can create an empty list like this:
3. backpack = []
4. print(backpack)
```

خروجی:

```
[]
```

۲. شروع ماجراجویی با کوله‌پشتی پُر (Initialized List)

گاهی اوقات، ما نمی‌خواهیم با دست خالی سفرمان را شروع کنیم! شاید از قبل مقداری غذا، نقشه یا سکه داشته باشیم. برای ساختن لیستی که از همان ابتدا دارای «مقدار» است، کافیست آیتم‌ها را درون براکت قرار دهیم و آن‌ها را با علامت **ویرگول** (,) از هم جدا کنیم. به یاد داشته باشید که در پایتون، انواع مختلف داده (مثل متن، عدد، لیست و...) می‌توانند دوستانه کنار هم در یک کوله‌پشتی بنشینند:

```
1. # Starting the adventure with a Map, a Flashlight, and 10 Gold Coins
2. backpack = ["Map", "Flashlight", 10]
3.
4. print(backpack)
```

خروجی:

```
['Map', 'Flashlight', 10]
```

نکته مهم: فراموش نکنید که بین هر آیتم یک **ویرگول** (comma) بگذارید تا پایتون بفهمد کجای کوله‌پشتی یک آیتم تمام شده و بعدی شروع می‌شود و آگه نزارید با خطا روبرو میشوید.

۳. پر کردن کوله‌پشتی (اضافه کردن، دیدن و حذف کردن)

یک کوله‌پشتی خالی به دردی نمی‌خورد. ما باید طلسم‌های جادویی مدیریت آن را یاد بگیریم.

طلسم append(): اضافه کردن آیتم

برای اضافه کردن یک آیتم به انتهای کوله‌پشتی، از طلسم append() استفاده می‌کنیم:

```
1. # Let's create our shopping list for the journey
2. shopping_list = []
3. print("List at start:")
4. print(shopping_list)
5.
6. # Now, let's add items using the .append() spell
7. shopping_list.append("sword") #
8. shopping_list.append("health_potion")
9. shopping_list.append("bread")
10.
11. print("List after adding items:")
12. print(shopping_list)
```

خروجی:

```
List at start:
[]
List after adding items:
['sword', 'health_potion', 'bread']
```

نکته: برای این طلسم خیلی مهمه اول اسم لیست بیاد و بعدش نقطه و بعدش اسم طلسم. (در آینده و در فصل شی گرای بیشتر اشنا میشیم چرا بعضی طلسم‌ها اینجوری نوشته میشن)

خواندن محتوای آیتم‌ها با «ایندکس» (Index)

چطور آیتم اول را ببینیم؟ ما از «ایندکس» (شماره موقعیت) استفاده می‌کنیم.

هشدار ماجراجو! در دنیای جادویی پایتون، شمارش از ۰ (صفر) شروع می‌شود، نه از ۱.

اولین آیتم در ایندکس [۰] است.

دومین آیتم در ایندکس [۱] است.

والی آخر...

```
1. shopping_list = ["sword", "health_potion", "bread"]
2.
3. # Access items by their index number
4. first_item = shopping_list[0] #
5. second_item = shopping_list[1]
6.
7. print("First item needed:")
8. print(first_item) # Output: sword
9. print("Second item needed:")
10. print(second_item) # Output: health_potion
11.
12. # What about the LAST item?
13. # You can use -1 to get the last item, -2 for the second-to-last, etc.
14. last_item = shopping_list[-1] #
15. print("Last item:")
16. print(last_item) # Output: bread
```


طلسم remove(): حذف کردن آیتم

اگر یک آیتم رو خریدیم و بخوایم اون رو از لیست خرید حذف کنیم چی؟ اگر نام آیتم را بدانیم، از طلسم remove() استفاده می‌کنیم.

```
1. shopping_list = ["sword", "health_potion", "bread"]
2. print("Original list:")
3. print(shopping_list)
4.
5. # We found a health potion! Let's remove it from the list
6. shopping_list.remove("health_potion")
7.
8. print("List after removing potion:")
9. print(shopping_list)
```

خروجی:

```
Original list:
['sword', 'health_potion', 'bread']
List after removing potion:
['sword', 'bread']
```

نکته مهم: اگر مقداری که ما می‌خواهیم آن را حذف کنیم وجود نداشته باشد. پایتون به ما ارور میدهد. (برای جلوگیری در جلوتر یاد می‌گیریم با in اول بررسی که مقدار وجود دارد و آگه وجود داشت اون رو پاک کنیم تا از خطا جلوگیری کنیم)

۴. بازرسی کوله پشتی (طلسم‌های len و in)

گاهی اوقات ما نمی‌خواهیم آیتمی را اضافه یا حذف کنیم، بلکه فقط به «اطلاعات» در مورد کوله پشتی‌مان نیاز داریم. مثلاً:

«کوله پشتی من چقدر سنگین شده؟» (چند آیتم داخل آن است؟)
 «آیا من قبلاً "شمشیر" را برداشته‌ام؟» (آیا آیتم x در لیست وجود دارد؟)
 برای این دو سوال حیاتی، ما دو طلسم جادویی جدید داریم:

طلسم len(): طلسم شمارش‌گر

اگر بخواهی بدانی دقیقاً چند آیتم در لیست وجود دارد، میتونی از طلسم len() (مخفف Length به معنی طول) استفاده کنی.

```
1. shopping_list = ["sword", "health_potion", "bread"]
2. empty_list = []
3.
4. # Get the number of items in the list
5. item_count = len(shopping_list)
6. empty_count = len(empty_list)
7.
8. print("Items in shopping list:")
9. print(item_count) # Output: 3
```

```

10.
11. print("Items in empty list:")
12. print(empty_count) # Output: 0

```

این طلسم بسیار کاربردی و ما ازش در ادامه زیاد استفاده میکنیم.

طلسم in : طلسم جستجوگر

این یکی از قدرتمندترین طلسم‌های پایتون است. اگر بخواهی چک کنی که آیا یک آیتم خاص (داخل) (in) لیستت هست یا نه، از این طلسم استفاده می‌کنی. نتیجه این طلسم یک «بولین» (Boolean) یعنی True یا False است.

```

1. shopping_list = ["sword", "health_potion", "bread"]
2.
3. # Check if 'sword' is in the list
4. has_sword = "sword" in shopping_list
5. print("Do I have a sword?")
6. print(has_sword) # Output: True
7.
8. # Check if 'shield' is in the list
9. has_shield = "shield" in shopping_list
10. print("Do I have a shield?")
11. print(has_shield) # Output: False

```

نکته کاربردی: اگر ماجراجو بخواهد آیتمی را حذف کند که در لیست نیست، پایتون ارور میدهد پس ما قبل از حذف، اول با if چک می‌کنیم و فقط اگر True بود، آن را remove می‌کنیم.

```

1. if item_to_remove in shopping_list:
2.     shopping.remove(item_to_remove)

```

۵. سرشماری غنائم (حلقه for)

ما از حلقه while برای تکرار کد استفاده کردیم. while عالی بود چون تا زمانی که یک شرط True بود ادامه می‌داد.

اما برای لیست، ما اغلب می‌خواهیم یک کار ساده انجام دهیم مثلاً: «به ازای هر آیتم در کوله پشتی، آن را به من نشان بده.»

انجام این کار با while کمی سخت است. اما ما یک طلسم جادویی جدید و بسیار ساده‌تر برای این کار داریم: حلقه for.

حلقه for برای گشتن (iterate) روی یک «دنباله» (sequence) مثل لیست ساخته شده است.

```

1. shopping_list = ["sword", "health_potion", "bread"]
2.
3. print("--- Shopping List (one by one) ---")
4.
5. # The 'for' loop:
6. # "item" is a temporary variable we create.
7. # Python will put 'sword' in 'item', run the code.

```

```
8. # Then put 'health_potion' in 'item', run the code.
9. # Then put 'bread' in 'item', run the code.
10. for item in shopping_list: #
11.     print("I need to buy a " + item)
12.
13. print("-----")
```

خروجی:

```
--- Shopping List (one by one) ---
I need to buy a sword
I need to buy a health_potion
I need to buy a bread
-----
```

این حلقه بسیار تمیزتر از while است!

۶. پروژه: ساخت سیستم مدیریت کوله‌پشتی (Backpack Manager) –

100 xp

ماجرای ماجراجو، تبریک می‌گوییم! تو آماده‌ای. در این مأموریت، قرار نیست ابزار جدیدی یاد بگیریم؛ در عوض، می‌خواهیم تمام قدرت‌هایی که در فصل‌های گذشته به دست آوردیم (مثل لیست‌ها، حلقه‌ها و شرط‌ها) را با هم ترکیب کنیم تا یک نرم‌افزار واقعی بسازیم.

این ابزار، یک دستیار هوشمند برای مدیریت تجهیزات و کوله‌پشتی ما در طول سفر خواهد بود.

تصویر نهایی: این برنامه قراره چیکار کنه؟

قبل از اینکه وارد کدهای پیچیده بشیم، بیا یک لحظه چشمانمان را ببندیم و نتیجه نهایی را

تصور کنیم. تا الان، برنامه‌های ما اجرا می‌شدند، یک کار انجام می‌دادند و سریع بسته

می‌شدند. اما این بار ما یک برنامه «تعاملی (Interactive)» و «همیشه بیدار» می‌سازیم.

تصور کن وسط یک جنگل تاریک ایستاده‌ای و این برنامه مثل یک روح نگهبان کنار توست و تا

وقتی نگی «خدا حافظ»، منتظر دستورات می‌ماند:

- اگر بگویی **add** (بردار): آماده می‌شود تا اسم آیتم (مثلاً «شمشیر») یا «معجون» را بگیرد و در کوله‌پشتی بگذارد.
- اگر بگویی **show** (نشان بده): لیست تمام چیزهایی که در کوله داری را مرتب نشان می‌دهد.
- اگر بگویی **remove** (دور بنداز): آیتمی که مصرف کردی یا نیاز نداری را از کوله بیرون می‌اندازد.
- **ونکته حیاتی:** تا وقتی خودت نگی **quit** (استراحت)، برنامه بسته نمی‌شود و دوباره می‌پرسد: **خب، دستور بعدی چیه؟**

نقشه و طرح اولیه

یک برنامه‌نویس حرفه‌ای، اول فکر می‌کند، بعد کد می‌زند. بیا منطق برنامه را مهندسی کنیم. ما برای ساخت این سیستم به ۴ ستون اصلی نیاز داریم:

۱. **ساخت حافظه (مکان امن برای آیتم‌ها):** اول از همه به جایی نیاز داریم که تجهیزات (تیرکمان، نقشه، ...) را در آن ذخیره کنیم.

- **منطق:** یک لیست خالی [] می‌سازیم.

- **نکته حیاتی (خطر فراموشی):** این لیست را باید قبل از شروع حلقه بسازیم.

- چر؟ اگر لیست را داخل حلقه بسازی، هر بار که برنامه می‌چرخد و سوال

می‌پرسد، یک کوله‌پشتی نو و خالی ساخته می‌شود و تمام آیتم‌های قبلی پاک می‌شوند! پس حافظه باید بیرون از چرخه باشد.

۲. **روشن کردن موتور (حلقه تکرار):** برنامه نباید بعد از یک دستور بخوابد! باید بیدار بماند.

- **منطق:** یک متغیر به نام `is_running` (آیا در حال اجراست؟) را روی `True` تنظیم

می‌کنیم و یک حلقه `while` می‌سازیم که تا وقتی این متغیر روشن است، بچرخد.

۳. **گوش دادن به فرمان (Input):** داخل حلقه، ربات باید مودبانه بپرسد چه می‌خواهی.

- **منطق:** یک دستور `input` می‌نویسیم تا فرمان اصلی (`add`, `show`, ...) را بگیرد.

۴. **مغز متفکر (تصمیم‌گیری):** حالا باید فرمان را بررسی کنیم (اینجاست که `if` و `elif` وارد می‌شوند):

- اگر `add` بود: بپرس چی؟ -> اضافه کن.

- اگر `show` بود: با حلقه `for` چاپ کن.

- اگر `remove` بود: بپرس چی؟ -> حذف کن.

- اگر `quit` بود: کلید موتور (`is_running`) را خاموش کن.

چالش‌های فکری

این پروژه کمی سخت است، پس بیا قبل از کدنویسی، گره‌های ذهنی را باز کنیم:

چالش ۱: برداشتن واقعی (Looting) وقتی کاربر دستور `add` را می‌دهد، کار تمام نشده!

برنامه تازه باید بپرسد «چه آیتمی؟». پس ما به یک `input` دوم نیاز داریم. بعد از گرفتن اسم

آیتم، با کدام دستور آن را به انتهای لیست می‌چسبانیم؟ (راهنمایی: دستور `append()`)

چالش ۲: نمایش زیبا (نه زشت!) اگر کاربر گفت `show`، ما نمی‌خواهیم لیست را مدل کامپیوتری

و زشت چاپ کنیم (مثل `['Sword', 'Potion']`) ما می‌خواهیم اقلام زیر هم و مرتب چاپ شوند

تا راحت خوانده شوند. کدام حلقه بود که روی تک تک آیتم‌های لیست قدم می‌زد؟ (راهنمایی: *حلقه for item in list*)

چالش ۳: تله‌ی حذف کردن (باگ احتمالی) فرض کن کاربر می‌گوید «سپرا حذف کن»، اما اصلاً سپری در کوله نیست! اگر همین‌طوری دستور حذف بدهیم، برنامه با صورت زمین می‌خورد (Error) می‌دهد و بسته می‌شود. (چطور می‌توانیم قبل از حذف، چک کنیم که «آیا» این کالا اصلاً وجود دارد؟ (راهنمایی: استفاده از کلمه کلیدی *in* داخل یک *if*)

چالش ۴: پایان بازی چطور حلقه بی‌نهایت را متوقف کنیم؟ کافیسست متغیر `is_running` را که مثل کلید برق عمل می‌کند، به `False` تغییر دهیم.

کد نهایی پروژه (The Final Spell)

وقتش رسیده که کلمات را به عمل تبدیل کنیم. کد زیر را با دقت بخوان و در فایل `project.py` بنویس. سعی کن به کامنت‌های انگلیسی دقت کنی تا بفهمی هر خط دقیقاً چه کار می‌کند.

```
1. # --- The Adventurer's Backpack Manager ---
2.
3. # 1. Setup the Memory (The Backpack)
4. # WARNING: We create the list OUTSIDE the loop.
5. # If we put it inside, it would be wiped clean every time the loop runs!
6. backpack = []
7. is_running = True
8.
9. print("Welcome to your Adventure Backpack!")
10. print("Commands: 'add', 'remove', 'show', 'quit'")
11.
12. # 2. Start the Engine (The Loop)
13. # The program stays alive as long as is_running is True
14. while is_running == True:
15.     print("-----")
16.
17.     # 3. Get command from user (Listening)
18.     command = input("What is your command? ")
19.
20.     # 4. Make decisions based on command (The Brain)
21.
22.     # --- ADD COMMAND (Pick up item) ---
23.     if command == "add":
24.         item = input("What item did you find? ")
25.         backpack.append(item)
26.         print(item + " added to backpack.")
27.
28.     # --- REMOVE COMMAND (Drop item) ---
29.     elif command == "remove":
30.         item = input("What item to drop? ")
31.
```

```

32.     # Safety Check: Is the item actually in the backpack?
33.     if item in backpack:
34.         backpack.remove(item)
35.         print(item + " dropped.")
36.     else:
37.         # If item is NOT in the list, tell the user instead of
crashing
38.         print("Error: You don't have " + item + " in your backpack!")
39.
40.     # --- SHOW COMMAND (Check Inventory) ---
41.     elif command == "show":
42.         print("--- Your Inventory ---")
43.
44.         # Check if backpack is empty first
45.         if len(backpack) == 0:
46.             print("Your backpack is empty.")
47.         else:
48.             # Loop through the list to show items nicely
49.             for item in backpack:
50.                 print("- " + item)
51.
52.     # --- QUIT COMMAND (Rest) ---
53.     elif command == "quit":
54.         print("Safe travels, Adventurer!")
55.         # This turns off the 'while' loop and stops the program
56.         is_running = False
57.
58.     # --- INVALID COMMAND ---
59.     else:
60.         # If the user types something like "dance" or "fly"
61.         print("I don't know that spell (command).")
62.
63. # 5. End of program
64. print("System Closed.")

```

اجرا کن و لذت ببر!

برنامه را اجرا کن. چند قلم جنس (شمشیر، سپر، نقشه گنج) اضافه کن، لیست را ببین، نقشه را حذف کن و سعی کن چیزی که در کوله نیست را حذف کنی تا ببینی برنامه چطور هوشمندانه خطا را مدیریت می‌کند.

تبریک می‌گویم! تو الان ساختار اصلی تمام نرم‌افزارهای دنیا (چرخه دریافت دستور و پردازش) را ساختی. این یک قدم بزرگ بود!

🌸 ارتقاء لول: ۱۵ → ۱۰ 🌸

مهارت‌های جدید:

◆ ساختار List

با نام باستانی: گنجینه بی‌انتهای داده
توضیح: دیگر نیازی به حمل تک‌تک داده‌ها
نیست. همه چیز در یک صف منظم است.

◆ متد Append

با نام باستانی: به خدمت‌گیری نیرو
توضیح: افزودن سربازی تازه به ارتش
داده‌های تو.

فصل ۶: رازهای کوله‌پشتی

حرفه‌ای: تابلوی افتخارات

ماجرای جویان

مأموریت فصل: ساخت یک تابلوی امتیازات (High Score Board) برای بازی حدس عددمان.

نقشه راه

- برش زدن لیست (Slicing): چگونه فقط چند آیتم خاص را برداریم؟
- کیسه‌ی مهر و موم شده (Tuples): آیتم‌های جادویی که تغییر نمی‌کنند.
- ترکیب جادوها: گشتن روی لیستی از تاپل‌ها.
- جادوی نظم‌دهی (مرتب کردن لیست)
- **پروژه:** ساخت تابلوی امتیازات و نمایش ۳ نفر برتر.

۱. هنر برش زدن (Slicing)

گاهی ما نمی‌خواهیم فقط یک آیتم را برداریم، بلکه می‌خواهیم بخشی از لیست را جدا کنیم (مثلاً سه نفر اول برای تیم حمله). به این کار **برش زدن (Slicing)** می‌گویند. ساختار کلی طلسم برش به این صورت است:

```
list_name[start : end : step]
```

- **start شروع:** از چه ایندکسی شروع کنم؟ (شامل خودش می‌شود)
- **end پایان:** تا کجا بروم؟ (شامل خودش نمی‌شود و یکی قبل از آن می‌ایستد).
- **step گام:** چند تا چند تا بروم؟ (اگر ننویسید، پیش فرض ۱ است).

بیایید چند برش جادویی را روی لیست ماجراجویان امتحان کنیم:

```
1. # Our list of adventurers
2. adventurers = ["Aria", "Kael", "Lyra", "Zane", "Finn", "Bora"]
3.
4. # 1. The Standard Slice (Start to End)
5. # Get from index 1 up to (but not including) 4
6. team_alpha = adventurers[1:4]
7. print("Team Alpha:", team_alpha)
8. # Output: ['Kael', 'Lyra', 'Zane']
9.
10. # 2. The Shortcut Magic (Leaving blank)
11. # Empty start = Start from 0
12. first_three = adventurers[:3]
13. print("First Three:", first_three)
14. # Output: ['Aria', 'Kael', 'Lyra']
15.
16. # Empty end = Go to the very end
17. from_third_onwards = adventurers[2:]
18. print("From Lyra to End:", from_third_onwards)
19. # Output: ['Lyra', 'Zane', 'Finn', 'Bora']
```

قدم‌های بلند (Step)

پارامتر سوم، یعنی step، به ما اجازه می‌دهد که از روی آیت‌ها بپریم. مثلاً می‌خواهیم «یکی در میان» انتخاب کنیم:

```
1. # 3. The Jump Spell (Step = 2)
2. # Start from 0, go to end, take every 2nd person
3. odd_team = adventurers[::2]
4. print("Odd Team:", odd_team)
5. # Output: ['Aria', 'Lyra', 'Finn'] (Index 0, 2, 4)
```

طلسم معکوس زمان (Negative Step)

این جذاب‌ترین بخش ماجراست! اگر برای step از عدد منفی استفاده کنیم، پایتون دنده عقب حرکت می‌کند.

- عدد -1 - یعنی: از آخر به اول، یکی‌یکی بیا.

```

1. # 4. The Time Reversal Spell (Reversing a list)
2. # Start from end, go to start, step -1
3. reversed_team = adventurers[::-1]
4.
5. print("Reversed Team:", reversed_team)
6. # Output: ['Bora', 'Finn', 'Zane', 'Lyra', 'Kael', 'Aria']

```

خلاصه سریع دستورات برش:

معنی جادویی	دستور
از ۱ تا ۲ (۳ و حساب نمی‌شود).	list[1:3]
از ۰ تا ۲ (۳ تای اول).	list[:3]
از ۲ تا آخر آخر.	list[2:]
کپی کامل از کل لیست.	list[:]
یکی در میان (پرش دوتایی).	list[::2]
کل لیست برعکس می‌شود.	list[::-1]

۲. کیسه‌ی مُهر و موم شده (Tuples)

لیست‌ها عالی هستند چون می‌توانیم آیتم‌ها را به آن‌ها append یا remove کنیم. آن‌ها «تغییرپذیر» (Mutable) هستند.

اما گاهی غنائمی داریم که هرگز نباید تغییر کنند، مثل مختصات جادویی یک گنج (X=10, Y=20)، یا یک رکورد امتیاز (که نام بازیکن و امتیازش به هم قفل شده‌اند).

برای این کار، ما از «تاپل» (Tuple) استفاده می‌کنیم. تاپل‌ها دقیقاً مثل لیست‌ها هستند، اما با پرانتز () ساخته می‌شوند و تغییرناپذیر (Immutable) هستند. (اگر تلاش به تغییر کنید با خطا مواجه میشوید)

```

1. # A list is mutable (can change)
2. mutable_list = [10, 20]
3. mutable_list.append(30) # This works!
4. print("Mutable list:")
5. print(mutable_list)
6.
7. # A tuple is immutable (cannot change)
8. # Tuples are defined using parentheses ()
9. immutable_tuple = (10, 20)
10. print("Immutable tuple:")
11. print(immutable_tuple)
12.

```

```
13. # If you try to change a tuple, you get an ERROR!  
14. # immutable_tuple.append(30) # This will crash the code! (AttributeError)
```

ما از تاپل‌ها برای داده‌هایی استفاده می‌کنیم که می‌خواهیم مطمئن باشیم به صورت تصادفی تغییر نمی‌کنند.

۳. جادوی باز کردن تاپل (Tuple Unpacking)

حالا که «تاپل» داریم، چطور به محتویات آن دسترسی پیدا کنیم؟ راه عادی (که از لیست‌ها بلدیم) استفاده از ایندکس است:

```
1. # A single high score record  
2. score_record = ("Aria", 100)  
3.  
4. # The "old" way using indexes  
5. name = score_record[0]  
6. score = score_record[1]  
7. print(name + " scored " + str(score))
```

این روش کار می‌کند، اما یک راه جادویی‌تر و تمیزتر هم وجود دارد. پایتون به ما اجازه می‌دهد تا یک تاپل را در یک خط «باز» کنیم و هر آیتم آن را مستقیماً داخل یک متغیر بریزیم. به این کار Tuple Unpacking می‌گویند.

```
1. # The "magic" way: Tuple Unpacking  
2. score_record = ("Aria", 100)  
3.  
4. # Python smartly puts "Aria" into 'name' and 100 into 'score'  
5. name, score = score_record  
6.  
7. print(name + " scored " + str(score))
```

هر دو کد بالا یک نتیجه می‌دهند، اما روش دوم بسیار خواناتر و «پایتونیک»تر (Pythonic) است. ما به جای دو خط کد برای دسترسی به آیت‌ها، در یک خط آن‌ها را «باز» کردیم.

نکته: همچنین می‌توانیم از این روش برای لیست هم استفاده کنیم

```
1. my_list = ["mahdi", 200]  
2. name, score = my_list
```

۴. ترکیب جادوها: لیستی از تاپل‌ها

خب، جادوی واقعی زمانی اتفاق می‌افتد که این دو را ترکیب کنیم. یک «تاپل» برای قفل کردن داده‌ها (مثل نام و امتیاز) عالی است. یک «لیست» برای نگهداری مجموعه‌ای از این تاپل‌ها عالی است!

این ساختار، ستون فقرات تابلوی امتیازات ما خواهد بود:

```

1. # Our High Score board
2. # It's a LIST (backpack) containing Tuples (sealed records)
3. high_scores = [
4.     ("Aria", 100),
5.     ("Kael", 80),
6.     ("Lyra", 75),
7.     ("Zane", 60),
8.     ("Finn", 30)
9. ]

```

حالا چطور روی این لیست سرشماری کنیم؟ با حلقه `for`!

اما این بار، ما می‌توانیم از جادوی «باز کردن تاپل» (که در بخش قبل یاد گرفتیم) مستقیماً در خود حلقه `for` استفاده کنیم:

```

1. print("--- All Scores (unpacked) ---")
2.
3. # Instead of: for entry in high_scores:
4. # We write:   for name, score in high_scores:
5. # Python unpacks each tuple (like '("Aria", 100)')
6. # into 'name' and 'score' in each loop.
7. for name, score in high_scores:
8.     # Now we use str() to join text and numbers (from Chapter 4)
9.     print(name + " scored " + str(score) + " points")

```

خروجی این کد بسیار زیباتر است:

```

--- All Scores (unpacked) ---
Aria scored 100 points
Kael scored 80 points
...

```

۵. جادوی نظم‌دهی (مرتب کردن لیست)

تا اینجا یاد گرفتیم چطور داده‌ها را در لیست ذخیره کنیم، اما گاهی اوقات کوله‌پشتی ما حسابی شلخته می‌شود! و ما نیاز به یک لیست مرتب شده از اعداد داریم.

پایتون یک طلسم جادویی آماده به نام `sort()` دارد که در یک چشم برهم زدن، همه چیز را منظم می‌کند.

الف) از کم به زیاد (حالت پیش‌فرض)

وقتی دستور `sort()` را صدا می‌زنید، پایتون اعداد را از کوچک‌ترین به بزرگ‌ترین می‌چیند.

```

1. # 1. A messy list of scores
2. scores = [50, 10, 100, 5, 20]
3.
4. # 2. The sorting spell
5. scores.sort()
6.
7. # 3. See the result
8. print("Sorted scores:", scores)

```

خروجی :

Sorted scores: [5, 10, 20, 50, 100]

(ب) از زیاد به کم (معکوس)

اما در بازی‌ها، معمولاً کسی که **بیشترین** امتیاز را دارد اول است. برای این کار، باید یک تنظیمات اضافی به دستورمان اضافه کنیم. ما به پایتون می‌گوییم `reverse=True` یعنی برعکسش کن.!

```
1. scores = [50, 10, 100, 5, 20]
2.
3. # Sort from biggest to smallest
4. scores.sort(reverse=True)
5.
6. print("Top scores:", scores)
```

خروجی:

Top scores: [100, 50, 20, 10, 5]

۶. پروژه: تابلوی افتخارات (The High Score Board) – 100 xp

ماجراجو، تو آماده‌ای! درانجمن ماجراجویان، هر روز صدها نفر مأموریت انجام می‌دهند، اما فقط بهترین‌ها روی دیوار افتخارات ثبت می‌شوند. در این مأموریت، ما می‌خواهیم با استفاده از جادوی **تاپل‌ها** (بسته‌های غیرقابل تغییر) و **برش زدن** (Slicing)، تابلوی امتیازات بازی‌مان را بسازیم و ۳ قهرمان برتر را جدا کنیم.

تصویر نهایی: این تابلو چه شکلی است؟

قبل از کدنویسی، خروجی را تصور کن. ما یک لیست طولانی از بازیکنان داریم. برنامه ما باید دو کار انجام دهد:

۱. اول لیست همه بازیکنان را نشان دهد (تا کسی ناراحت نشود!).
۲. سپس با یک خط جداکننده، فقط ۳ نفر اول را با شماره رتبه (۱، ۲، ۳) به عنوان برندگان نهایی اعلام کند.

خروجی باید چیزی شبیه به این باشد:

```
--- All Scores ---
Aria scored 100 points
Kael scored 80 points
...
--- TOP 3 CHAMPIONS ---
1. Aria scored 100 points
2. Kael scored 80 points
3. Lyra scored 75 points
```

نقشه و طرح اولیه

برای مدیریت این داده‌ها، ما به یک نقشه‌ی دقیق نیاز داریم:

۱. **ساخت مخزن داده (لیست تاپل‌ها):** ما باید نام بازیکن و امتیاز او را به هم بچسبانیم تا گم نشوند. بهترین راه چیست؟ تاپل ("Name", Score)! سپس همه این تاپل‌ها را داخل یک لیست می‌ریزیم.

- **منطق:** یک لیست می‌سازیم که داخلش ۵ تاپل وجود دارد. فرض می‌کنیم لیست از قبل مرتب شده است (درمورد مرتب کردن لیستی از تاپل‌ها بعداً صحبت می‌کنیم).

۲. **اعلام همگانی: (Unpacking)** می‌خواهیم لیست را چاپ کنیم. اما نمی‌خواهیم خروجی زشت باشد مثل ('Aria', 100)

- **منطق:** از یک حلقه for استفاده می‌کنیم. اما اینجا از تکنیک **باز کردن تاپل** استفاده می‌کنیم. یعنی در همان خط اول حلقه، اسم و امتیاز را از داخل تاپل بیرون می‌کشیم و در دو متغیر جدا می‌ریزیم.

۳. **جدا کردن قهرمانان: (Slicing)** ما فقط ۳ نفر اول را می‌خواهیم.

- **منطق:** لازم نیست حلقه جدیدی بنویسیم یا بشماریم. از قابلیت **برش زدن** استفاده می‌کنیم تا از خانه ۰ تا ۳ لیست را ببریم و در یک لیست جدید (top_3) بریزیم.

۴. **مراسم اهدای جوایز: (Rank Loop)** حالا لیست ۳ نفره را چاپ می‌کنیم، اما این بار یک شمارنده (Rank) هم کنارش می‌گذاریم تا معلوم شود نفر اول کیست.

چالش‌های فکری

قبل از اینکه کد نهایی را ببینی، سعی کن این معماها را حل کنی:

چالش ۱: تاپل یا لیست؟ چرا برای نگه داشتن (نام و امتیاز) یک نفر از **تاپل** () استفاده می‌کنیم و نه لیست []؟ (راهنمایی: چون نام و امتیاز یک بازیکن یک زوج جدا نشدنی هستند و نباید تغییر کنند)

چالش ۲: فرمول برش اگر بخواهیم ۳ نفر اول را برداریم، فرمول برش list[start:stop] چه می‌شود؟ (راهنمایی: از ۰ شروع کن و سر ۳ متوقف شو. پایتون عدد آخر (Stop) را شامل نمی‌شود پس در نتیجه داریم ایتیم ۰ و ایتیم ۱ و ایتیم ۲)

چالش ۳: شمارنده رتبه در حلقه for، ما به متغیرهای لیست دسترسی داریم، اما چطور بفهمیم نفر چندم است؟ (راهنمایی: باید یک متغیر مثل rank = 1 قبل از حلقه بسازی و در هر دور حلقه، یکی به آن اضافه کنی)

کد نهایی پروژه (The Spell)

وقت آن است که تابلوی افتخارات را بسازیم. کد زیر را در فایل `project.py` بنویس. به نحوه باز کردن تاپل‌ها در حلقه `for` خیلی دقت کن.

```
1. # --- The High Score Board ---
2.
3. # 1. Setup the data
4. # We use a LIST of TUPLES.
5. # Each tuple is (Name, Score) and creates a solid, unchangeable pair.
6. high_scores = [
7.     ("Aria", 100),
8.     ("Kael", 80),
9.     ("Lyra", 75),
10.    ("Zane", 60),
11.    ("Finn", 30)
12. ]
13.
14. print("--- All Scores ---")
15.
16. # 2. Display all scores using "Tuple Unpacking"
17. # Instead of getting one item, we get "name" and "score" directly!
18. for name, score in high_scores:
19.     # Remember to convert score (int) to string (str) for printing
20.     print(name + " scored " + str(score) + " points")
21.
22. print("") # An empty print() adds a blank line for better look
23. print("--- TOP 3 CHAMPIONS ---")
24.
25. # 3. Get the Top 3 using "Slicing"
26. # Logic: Start at index 0, Stop BEFORE index 3 (so we get 0, 1, 2)
27. top_3_list = high_scores[0:3]
28.
29. # 4. Display the Top 3 with ranking
30. rank = 1 # We start ranking from 1
31.
32. for name, score in top_3_list:
33.     # We print the Rank, Name, and Score all together
34.     print(str(rank) + ". " + name + " scored " + str(score) + " points")
35.
36.     # Increase rank for the next champion
37.     rank = rank + 1
```

اجرا کن و نتیجه را ببین! خروجی باید دقیقاً لیست کامل و سپس لیست ۳ نفر برتر را نشان دهد.

جشن پیروزی و تحلیل

آفرین! تو همین الان دو مورد از قدرتمندترین تکنیک‌های مدیریت داده در پایتون را یاد گرفتی:

۱. **Tuples**. یاد گرفتی چطور داده‌های مرتبط (مثل نام و امتیاز) را در بسته‌های امن نگه داری.

۲. **Slicing**. یاد گرفتی چطور مثل یک نینجا، بخشی از یک لیست بزرگ را برش بزنی و استخراج کنی.

کوله‌پشتی تو حالا واقعاً حرفه‌ای شده است. این تکنیک‌ها در آینده برای کار با داده‌های بزرگ بسیار حیاتی هستند!

🌟 ارتقاء لول: ۱۸ → ۱۵ 🌟

مهارت‌های جدید:

◆ تکنیک Slicing

با نام باستانی: هنر برش لیزری
توضیح: جدا کردن بخشی از کل. انتخاب
دقیق آنچه نیاز داری از دل انبوه اطلاعات.

◆ ساختار Tuple

با نام باستانی: لوح‌های سنگی
توضیح: داده‌هایی که حک شده‌اند و
تغییر ناپذیرند.

فصل ۷: کتاب جادویی: ساخت مترجم کلمات

مأموریت فصل: ساخت یک دیکشنری ساده برای ترجمه کلمات در سفرهای خارجی.

نقشه راه

- دیکشنری‌ها (Dictionaries): کتاب جادویی که داده‌ها را به صورت (کلید : مقدار) ذخیره می‌کند.
- جادوی دیکشنری: دسترسی، اضافه کردن، تغییر دادن و حذف کردن کلمه‌ها.
- تله‌ی کلید گمشده (The KeyError): چه اتفاقی می‌افتد اگر کلمه‌ای در کتاب نباشد؟
- طلسم‌های ایمنی (in و get): دو راه برای جلوگیری از ارور در زمان بازیابی.
- سرشماری در کتاب جادویی: چطور روی تمام کلمات و معانی آن‌ها حلقه بزنیم؟
- **پروژه:** ساخت مترجمی که چند کلمه فارسی را به انگلیسی برمی‌گرداند.

۱. دیکشنری‌ها: (Dictionaries) کتابچه راهنمای هوشمند

تا به حال، ما از لیست ([]) برای نگهداری آیتم‌ها استفاده می‌کردیم، درست مثل کوله‌پشتی‌ای که وسایل را پشت سرهم داخلش می‌انداختیم. در لیست، ترتیب مهم بود و برای پیدا کردن اولین وسیله باید می‌گفتیم: شماره ۰ را بده.

اما نوع جدیدی از ساختار داده وجود دارد به نام **دیکشنری (Dictionary)** که با آکولاد {} ساخته می‌شود. دیکشنری شبیه یک قفسه با برچسب‌هایی برای هر بخش هاست. اینجا دیگر شماره‌ها (۰, ۱, ۲) مهم نیستند، «برچسب‌ها» مهم‌اند.

در دیکشنری، اطلاعات به صورت جفت‌های **کلید : مقدار (Key : Value)** ذخیره می‌شوند:

- **کلید: (Key)** مثل «برچسب» روی یک خانه از قفسه است. کلمه‌ای که با آن دنبال چیزی می‌گردیم (باید منحصر به فرد و غیر تکراری باشد یعنی همیشه ۲ تا قفسه با برچسب یکسان داشت).
- **مقدار: (Value)** چیزی است که داخل آن قفسه قرار دارد (معنی یا داده‌ی اصلی) (قبایل تکراری و هر چیزی می‌تونه باشد).

مثال اول: مترجم جیبی (دیکشنری انگلیسی به فارسی)

بیایید ساده‌ترین کاربرد آن را ببینیم. دقیقاً مثل یک دیکشنری واقعی! ما «کلمه انگلیسی» را می‌دهیم و «معنی فارسی» را می‌گیریم. (با نوشتار کلید دو نقطه مقدار و آگه چند کلید مقدار داشتیم با , بینشون اون هارو جدا میکنیم)

```
1. # A simple English to Persian dictionary
2. my_translator = {
3.     "apple": "سیب",
4.     "water": "آب",
5.     "book": "کتاب",
6.     "friend": "دوست"
7. }
8.
9. # How to use it:
10. print(my_translator["apple"]) # Output: سیب
11. print(my_translator["book"]) # Output: کتاب
```

می‌بینید؟ ما نگفتیم ایندکس شماره ۰ یا ۱ را بده، بلکه گفتیم: «معنی apple چیست؟» و او جواب داد.

مثال دوم: کارت شناسایی ماجراجو

حالا بیایید این را در پروژه خودمان ببینیم. اگر بخواهیم مشخصات قهرمان داستان را ذخیره کنیم، استفاده از لیست گیج‌کننده است (یادمان می‌رود که اسم در خانه دوم بود یا سوم). اما با دیکشنری همه چیز روشن است:

```
1. # We use curly braces {} for dictionaries
2. # We use colons (:) to link a KEY to a VALUE
3. adventurer_stats = {
4.     "name": "Kael",
5.     "level": 5,
6.     "class": "Mage",
7.     "gold": 150
8. }
```

حالا جادوی واقعی : برای دسترسی به اطلاعات، ما دیگر کورکورانه از ایندکس [0] استفاده نمی‌کنیم. ما مستقیماً **کلید** را صدا می‌زنیم تا مقدارش ظاهر شود:

```
1. # Accessing data using the 'key'
2. print("Adventurer's Name:")
3. print(adventurer_stats["name"]) # Output: Kael
4.
5. print("Current Level:")
6. print(adventurer_stats["level"]) # Output: 5
```

این روش بسیار خواناتر و حرفه‌ای‌تر از حدس زدن شماره‌هاست! انگار به جای گشتن در جیب‌های کوله‌پشتی، دقیقاً می‌دانید هر چیزی کجاست.

۲. جادوی دیکشنری: افزودن، تغییر و حذف

قدرت دیکشنری‌ها در اضافه کردن، تغییر دادن خیلی راحت و جستجوی سریع است.

اضافه کردن یا تغییر دادن یک «کلمه»

برخلاف لیست‌ها که برای اضافه کردن نیاز به append داشتند، دیکشنری‌ها بسیار مستقیم عمل می‌کنند.

قانون طلایی این است: **کلید را صدا بزن و مقدار را به آن بده.**

این دستور هوشمندانه عمل می‌کند:

۱. اگر کلید جدید باشد، آن را به دیکشنری اضافه می‌کند.

۲. اگر کلید از قبل وجود داشته باشد، مقدار قبلی را پاک کرده و مقدار جدید را جایگزین می‌کند (Update).

بیایید این جادو را در عمل ببینیم:

```
1. # Our translator starts empty
2. translator = {}
3.
4. # Add a new word (key-value pair)
```

```

5. translator["hello"] = "salam"
6. translator["goodbye"] = "khodahafez"
7.
8. print("Translator after adding words:")
9. print(translator) # Output: {'hello': 'salam', 'goodbye': 'khodahafez'}
10.
11. # Change an existing word (update the value)
12. translator["hello"] = "dorood"
13. print("Translator after update:")
14. print(translator) # Output: {'hello': 'dorood', 'goodbye': 'khodahafez'}

```

حذف کردن یک «کلید»

اگر بخواهیم یک کلمه و معنی آن را کاملاً از کتاب جادویی پاک کنیم، از `del` (مخفف `delete`) استفاده می‌کنیم:

```

1. translator = {"hello": "salam", "goodbye": "khodahafez"}
2. print("Translator before delete:")
3. print(translator)
4.
5. # Delete the word 'hello'
6. del translator["hello"]
7.
8. print("Translator after delete:")
9. print(translator) # Output: {'goodbye': 'khodahafez'}

```

۳. تله‌ی کلید گمشده (The KeyError)

هشدار ماجراجو! یک تله‌ی بزرگ در دیکشنری‌ها پنهان شده است. چه اتفاقی می‌افتد اگر سعی کنی به کلمه‌ای دسترسی پیدا کنی که در کتاب جادویی تو وجود ندارد؟

```

1. translator = {"hello": "dorood"}
2. # Try to access a key that DOES NOT exist
3. print(translator["cat"])

```

پایتون ارور می‌دهد: `KeyError: 'cat'`. این یک «باگ» جدید است. پایتون می‌گه که: «من کلیدی به نام 'cat' در این کتاب پیدا نمی‌کنم!»

۴. طلسم‌های ایمنی (`get` و `in`)

برای جلوگیری از ارور اینکه کلید وجود نداشته ما ۲ تا طلسم رو میتونیم استفاده کنیم:

طلسم اول: بررسی با `in`

ما می‌توانیم از همان طلسم `in` (که برای لیست‌ها یاد گرفتیم) استفاده کنیم تا ببینیم آیا «کلید» (کلمه) در دیکشنری ما وجود دارد یا نه.

```

1. translator = {"hello": "dorood"}
2. word_to_find = "cat"

```

```

3.
4. # Check if the 'key' is in the dictionary
5. if word_to_find in translator:
6.     print("Translation is:")
7.     print(translator[word_to_find])
8. else:
9.     print("Sorry, that word was not found.") # This will run!

```

این امن‌ترین و رایج‌ترین راه است.

طلسم دوم (حرفه‌ای): متد get()

دیکشنری‌ها یک طلسم جادویی داخلی به نام get() دارند. این طلسم دو ورودی می‌گیرد:

۱. کلیدی که دنبالش می‌گردی.

۲. یک «مقدار پیش‌فرض» که اگر کلید پیدا نشد، آن را برگرداند. (این ورودی اختیاری هست و اگر

اون رو ندهیم پیش‌فرض معادل None و به معنی هیچی برمیگرده در زمان نبود مقدار)

این طلسم هرگز کرش نمی‌کند!

```

1. translator = {"hello": "dorood"}
2.
3. # Get the translation for 'hello'
4. translation_1 = translator.get("hello")
5. print(translation_1) # Output: dorood
6.
7. # Get the translation for 'cat'
8. translation_2 = translator.get("cat", "Word not found.")
9. print(translation_2) # Output: Word not found. (No crash!)

```

۵. سرشماری در کتاب جادویی (Looping)

ما قبلاً با حلقه for روی «لیست‌ها» گشتیم. چطور می‌توانیم روی «دیکشنری» حلقه بزنیم؟

حلقه زدن روی «کلیدها» (Keys)

اگر به سادگی روی دیکشنری for بزنی، پایتون به تو «کلیدها» را می‌دهد:

```

1. translator = {"hello": "dorood", "goodbye": "khodāhāfez"}
2.
3. print("--- Dictionary Keys ---")
4. for word in translator:
5.     print(word)

```

خروجی:

```

--- Dictionary Keys ---
hello
goodbye

```

حلقه زدن روی «مقدارها» (Values)

اگر فقط به «معانی» (مقدارها) نیاز داری، از طلسم values() استفاده کن:

```

1. print("--- Dictionary Values ---")

```

```
2. for meaning in translator.values():
3.     print(meaning)
```

خروجی:

```
--- Dictionary Values ---
dorood
khodāhāfez
```

حلقه زدن روی هر دو (Key & Value) – بهترین روش!

و اما قدرتمندترین جادو! اگر بخواهی هم «کلید» و هم «مقدار» را با هم داشته باشی، از طلسم items() استفاده کن. این طلسم لیستی از «تاپل‌ها» را برمی‌گرداند!

ما می‌توانیم از همان جادوی «باز کردن تاپل» (Tuple Unpacking) در حلقه for استفاده کنیم:

```
1. print("--- Full Translator ---")
2. # .items() gives us (key, value) pairs
3. for word, meaning in translator.items():
4.     print(word + " means " + meaning)
```

خروجی:

```
--- Full Translator ---
hello means dorood
goodbye means khodāhāfez
```

۶. هشدار ماجراجو: طلسم نمایش حروف فارسی

یک تله‌ی کوچک ممکن است در مسیر ماجراجویی تو وجود داشته باشد! گاهی اوقات، «اتاق فرمان» (ترمینال) در برخی سیستم‌ها (به خصوص نسخه‌های قدیمی‌تر ویندوز) نمی‌داند چطور با متن‌های فارسی کار کند.

اگر بعد از وارد کردن یک کلمه فارسی، در خروجی علامت سوال (?????) یا حروف عجیب و غریب دیدی، نترس! این یعنی ترمینال تو به یک طلسم جادویی نیاز دارد تا زبان فارسی را بفهمد.

اگر در ترمینال VS Code یا ویندوز به مشکل خوردی، قبل از اجرای برنامه، این دستور را یک بار در ترمینالت تایپ کن و Enter را بزن (به ازای هر بار بازکردن ترمینال نیاز هست مجدد این دستور رو اجرا کنی):

```
chcp 65001
```

این طلسم به ویندوز می‌گه که از «کتاب رمزگشایی» استاندارد جهانی (UTF-8) برای خواندن حروف استفاده کند. در اکثر سیستم‌های مدرن (مثل macOS، لینوکس و Windows Terminal جدید) این مشکل وجود ندارد، اما دانستن این طلسم، تو را برای هر چالشی آماده می‌کند!

نکته: اگه فارسی درست نمایش داده نمیشد حتی با اینکه دستور chcp 65001 رو داخل ترمینال قبل از اجرا کدت زدی ناراحت نشو طبیعیه کاملاً و فارسی توی ترمینال همیشه اذیت میکنه

۷. پروژه: ساخت مترجم هوشمند کلمات (The Translator) – 100 xp

ما جراجو، تو آماده‌ای!
ما در فصل‌های قبل یاد گرفتیم چطور داده‌ها را در «لیست» ذخیره کنیم. اما اگر بخواهیم معنی کلمات را پیدا کنیم، گشتن در یک لیست طولانی خیلی سخت و کند هست.
امروز با استفاده از ابزار جدید و قدرتمندمان، یعنی **دیکشنری**، یک مترجم انگلیسی به فارسی می‌سازیم.

تصویر نهایی: این برنامه قراره چیکار کنه؟

قبل از کدنویسی، بیا نحوه کار این مترجم را ببینیم.
ما می‌خواهیم برنامه‌ای بسازیم که مثل یک دیکشنری جیبی هوشمند عمل کند:
۱. از تو می‌پرسد: چه کلمه‌ای را ترجمه کنم؟
۲. اگر کلمه‌ای که بلد باشد مثلاً Hello را بنویسی: سریع می‌گوید: سلام.
۳. اگر کلمه‌ای که بلد نیست مثلاً Dragon را بنویسی: می‌گوید: شرمنده، این کلمه در دایره لغات من نیست
۴. و این کار را تکرار می‌کند تا زمانی که بنویسی **quit**.

نقشه و طرح اولیه

بیایید منطق این ابزار را مهندسی کنیم. ما به ۳ بخش اصلی نیاز داریم:

۱. حافظه: (The Memory)

ما به یک حافظه نیاز داریم که کلمات انگلیسی و معنی فارسی آن‌ها را جفت‌جفت نگه دارد.
• منطق: از یک **دیکشنری** {} استفاده می‌کنیم. کلمه انگلیسی می‌شود **کلید (Key)** و کلمه فارسی می‌شود **مقدار (Value)**.

۲. حلقه بی‌پایان: (The Loop)

مترجم نباید بعد از یک کلمه بسته شود.
• منطق: یک حلقه `while True` می‌سازیم که تا ابد بچرخد (مگر اینکه ما متوقفش کنیم).

۳. موتور جستجو: (The Search Engine)

وقتی کاربر کلمه‌ای را وارد کرد، باید در پایگاه داده بگردیم.

• منطق:

- اول چک کن کاربر quit نوشته یا نه (برای فرار از حلقه).
- دوم چک کن آیا کلمه در دیکشنری هست؟ (با دستور in)
- اگر بود: کلید را بده، مقدار را بگیر. (dict[key])
- اگر نبود: پیام خطا بده.

چالش‌های فکری

قبل از اینکه کد را ببینی، سعی کن به این سوالات پاسخ بدهی:

چالش ۱: ساختار دیکشنری

یادت هست فرق دیکشنری با لیست چه بود؟ لیست‌ها با [] و اعداد (ایندکس) کار می‌کردند. دیکشنری‌ها با چه علامتی ساخته می‌شدند؟
(راهنمایی: آکولاد {})

چالش ۲: کلید یا مقدار؟

ما می‌خواهیم کلمه انگلیسی را بگیریم و فارسی را تحویل دهیم. پس کلمه انگلیسی باید **Key** باشد یا **Value**؟
(راهنمایی: چیزی که جستجو می‌کنیم همیشه *Key* است).

چالش ۳: حساسیت به حروف!

پایتون خیلی دقیق است. از نظر پایتون "Hello" با "hello" فرق دارد. در این پروژه ساده، ما کلمات را دقیقاً همانطور که در دیکشنری نوشتیم وارد می‌کنیم (مثلاً با حرف بزرگ).

کد نهایی پروژه (The Spell)

وقت نوشتن است. کد زیر را در فایل project.py بنویس.

دقت کن که چطور کلمات انگلیسی (سمت چپ) و فارسی (سمت راست) با دو نقطه: به هم وصل شده‌اند.

```
1. # --- The English to Farsi Translator ---
2.
3. # 1. Setup the Dictionary (Database)
4. # Structure -> Key (English) : Value (Farsi)
5. english_to_farsi = {
```



```

6.     "Hello": "سلام",
7.     "Goodbye": "خداحافظ",
8.     "Cat": "گربه",
9.     "Dog": "سگ",
10.    "House": "خانه",
11.    "Coder": "نویس برنامه"
12. }
13.
14. print("Welcome to the Smart Translator!")
15. print("--- Type 'quit' to exit ---")
16.
17. # 2. Start the main loop (Infinite Loop)
18. # We use 'while True' to keep running forever until 'break'
19. while True:
20.     print("-----")
21.
22.     # Get input from the user
23.     word = input("Enter an English word to translate: ")
24.
25.     # 3. Check for exit command FIRST
26.     if word == "quit":
27.         print("Goodbye, Adventurer!")
28.         break # The spell to smash the loop and exit!
29.
30.     # 4. Search logic (Safety Spell)
31.     # check IF the word exists inside the dictionary keys
32.     if word in english_to_farsi:
33.         # Get the translation
34.         translation = english_to_farsi[word]
35.         print("Meaning: " + translation)
36.     else:
37.         # If the word is NOT in the dictionary
38.         print("Sorry, I don't know that word yet.")

```

اجرا کن و نتیجه را ببین!

۱. برنامه را اجرا کن.
۲. کلمه Cat را بنویس باید ترجمه را ببینی.
۳. کلمه dragon را بنویس. باید پیام «نمی دانم» را ببینی.
۴. حالا کلمه cat (با حروف کوچک) را امتحان کن. چه شد؟ "پیدا نشد!". (چون در دیکشنری ما Cat با C بزرگ ذخیره شده و پایتون به بزرگ و کوچک بودن حروف حساس است).
۵. در نهایت quit را بنویس و خارج شو.

جشن پیروزی و تحلیل

آفرین !

تو الان سریع ترین ابزار جستجوی خودت را ساختی.

تو یاد گرفتی که:

- لیست‌ها برای وقتی خوب هستند که «ترتیب» مهم است.
 - دیکشنری‌ها برای وقتی عالی هستند که می‌خواهیم چیزی را سریع «پیدا کنیم» (مثل دفتر تلفن یا لغت‌نامه).
- کوله‌پشتی تو حالا یک مترجم هوشمند دارد که در سرزمین‌های ناشناخته به کارت می‌آید!

🌸 ارتقاء لول: ۲۲ → ۱۸ 🌸

مهارت‌های جدید:

◆ ساختار Dictionary

با نام باستانی: کتاب‌الاسرار

توضیح: هر کلیدی، دری را باز می‌کند.

سریع‌ترین راه برای یافتن معنای هر چیز.

◆ متد Get

با نام باستانی: دست ایمن

توضیح: جستجو در کتاب بدون ترس از

خطا و سقوط.

فصل ۸: ساخت جادوهای

شخصی: هنر ساخت توابع

مأموریت فصل: بازسازی یک «کدی که هی تکرار شده» (Repetitive Code) و ساخت یک کارت معرفی جادویی و قابل استفاده مجدد با استفاده از توابع.

نقشه راه

- "یک بار بنویس، صد بار استفاده کن": معرفی توابع (Functions) و اصل DRY.
- چگونه یک بلوک جادویی را «تعریف» و آن را «صدا» بزنیم؟ طلسم‌های def و ().
- اولین طلسم ما: ساخت یک تابع ساده برای جلوگیری از تکرار.
- ورودی توابع (Arguments): چگونه به طلسم خود «داده» پاس بدهیم؟
- خروجی توابع (Return): چگونه از طلسم خود «نتیجه» پس بگیریم؟
- جادوی دقیق: برچسب زدن به مواد اولیه
- پروژه: «رفع طلسم» یک کد تکراری برای نمایش مشخصات ماجراجویان.

۱. "یک بار بنویس، صد بار استفاده کن": معرفی توابع

در ماجراجویی‌هایمان تا به حال، ما از طلسم‌های جادویی داخلی پایتون مثل `print()`, `input()`, `len()` و `int()` استفاده کرده‌ایم. این‌ها «توابع» (Functions) از پیش ساخته شده هستند. اما قدرت واقعی اونجاست که خودت تابع مخصوص خودت رو بنویسی. یک تابع (Function)، یک بلوک کد نام‌گذاری شده و قابل استفاده مجدد است که برای انجام یک کار خاص طراحی شده.

یک مثال ساده: فکر کن می‌خواهی یک ساندویچ درست کنی.

کدنویسی بدون تابع (کاری که تا الان می‌کردیم):

نان را بردار.

کره بمال.

پنیر بگذار.

نان دوم را بگذار.

(حالا می‌خواهی ساندویچ دوم را درست کنی...)

نان را بردار.

کره بمال.

پنیر بگذار.

نان دوم را بگذار.

کدنویسی با تابع (کاری که یاد می‌گیریم):

دستورالعمل (تابع) می‌سازیم: اسمش را می‌گذاریم `make_sandwich()`.

داخل این دستورالعمل، ۴ مرحله‌ی بالا را می‌نویسیم.

(حالا هر وقت گرسنه شدیم...)

`make_sandwich()` را صدا می‌زنیم. (ساندویچ اول)

`make_sandwich()` را دوباره صدا می‌زنیم. (ساندویچ دوم)

شعار ماجراجویان حرفه‌ای این است: **Don't Repeat Yourself (DRY)** یا «خودت را تکرار نکن». اگر کدی را دوبار کپی-پیست کردی، احتمالاً باید آن را به یک تابع تبدیل کنی.

۲. «تعریف» (Define) و «صدا زدن» (Call) یک تابع

ساختن یک جادوی شخصی دو مرحله دارد:

مرحله ۱: «تعریف» تابع با `def` اول، باید جادورا به پایتون «یاد بدهی». ما این کار را با `def` (مخفف Define) انجام می‌دهیم.

```
1. # --- 1. Defining the Spell (Writing the Scroll) ---
2. # We create a new function called 'greet_adventurer'
3. # The code INSIDE (indented) is the spell's logic.
4. def greet_adventurer():
5.     print("Greetings, mighty adventurer!")
6.     print("Welcome to the Python Guild.")
```

اگر این کد را به تنهایی اجرا کنی، هیچ اتفاقی نمی‌افتد! چرا؟ چون ما فقط «دستورالعمل» ساندویچ را نوشته‌ایم، هنوز ساندویچی درست نکرده‌ایم.

مرحله ۲: تعریف تابع باعث اجرا شدن آن نمی‌شود! برای اجرا شدن باید نام تابع رو به همراه پرانتز بنویسی مثل `my_func()` اینجوری تابع تو اجرا میشه: (این کار رو هر چند بار که دوست داشته باشی میتونی انجام بدی)

```
1. # --- 1. Defining the Spell ---
2. def greet_adventurer():
3.     print("Greetings, mighty adventurer!")
4.     print("Welcome to the Python Guild.")
5.
6. # --- 2. Calling the Spell (Making the sandwich) ---
7. print("The guild master steps forward...")
8. greet_adventurer() # <-- The magic happens here!
9.
10. print("The quest begins...")
11. greet_adventurer() # <-- We can call it again, easily!
```

اجرا کن و نتیجه را ببین!

```
The guild master steps forward...
Greetings, mighty adventurer!
Welcome to the Python Guild.
The quest begins...
Greetings, mighty adventurer!
Welcome to the Python Guild.
```

ما کد را یک بار نوشتیم و دو بار (یا صد بار!) از آن استفاده کردیم.

۳. اولین تابع ما: خداحافظی با تکرار

یادت هست در پروژه‌های قبلی مدام از `print("=====")` برای جداسازی استفاده می‌کردیم؟ بیایید این کد تکراری را تمیز کنیم.

```
1. # --- The Old, Repetitive Way ---
2. print("--- Adventurer 1 ---")
3. print("Name: Aria")
4. print("=====")
5. print("--- Adventurer 2 ---")
6. print("Name: Kael")
7. print("=====")
8.
9. # --- The New, Clean Way (with a Function) ---
10.
11. # 1. Define the spell once
12. def print_divider():
13.     print("=====")
14.
15. # 2. Call it whenever we need it
16. print("--- Adventurer 1 ---")
17. print("Name: Aria")
18. print_divider()
19.
20. print("--- Adventurer 2 ---")
21. print("Name: Kael")
22. print_divider()
```

کد جدید تمیزتر، خواناتر و حرفه‌ای‌تر است! (شاید بگی که یک خط کد رو منطقی نیست بکنیم تابع ۲ خطی حرفت هم کاملاً درسته ولی فکر کن اگه بخوایم این دیوایدر رو عوض کنیم توی یه خط تغییرش میدیم و همه جا اعمال میشه)

۴. ورودی و خروجی توابع (return و Arguments)

طلسم‌های ما خوب بودند، اما کمی خنگ بودند. `greet_adventurer` به همه‌ی ماجراجویان یک پیام تکراری می‌دهد و `print_divider` فقط یک کار ثابت انجام می‌دهد. چطور طلسم‌هایی بسازیم که بتوانند «داده» بگیرند و «نتیجه» (غنیمت) برگردانند؟

ورودی تابع:

بیایید تابع را شبیه به یک **دستگاه مخلوط کن جادویی** تصور کنیم. این دستگاه همیشه یک کار تکراری انجام نمی‌دهد؛ بلکه ما می‌توانیم به آن «مواد اولیه» بدهیم تا نتیجه‌ای متفاوت تولید کند.

در دنیای برنامه‌نویسی، به این ورودی‌ها **پارامتر (Parameter)** و **آرگومان (Argument)** می‌گوییم.

تفاوت ظریف: پارامتر یا آرگومان؟

شاید این دو کلمه شبیه به هم به نظر برسند، اما تفاوتشان مثل تفاوت «جای خالی» و «محتوا» است. بیایید یک مثال ساده بزنیم:

۱. **پارامتر: (Parameter)** مثل **جای باتری** در کنترل تلویزیون است. وقتی کارخانه کنترل را می‌سازد، آن جای خالی را طراحی می‌کند تا مشخص شود باتری باید کجا قرار بگیرد. (در زمان **ساختن** تابع).

۲. **آرگومان: (Argument)** مثل **خودِ باتری** است. آن چیزی است که شما واقعاً داخل دستگاه می‌گذارید تا کار کند. (در زمان **استفاده** از تابع)

به زبان **کد**: وقتی تابع را تعریف می‌کنید، اسم متغیر **پارامتر** است. وقتی تابع را صدا می‌زنید، داده واقعی **آرگومان** است.

بیایید در کد زیر ببینیم این جادو چگونه کار می‌کند:

```
1. # 'name' is a PARAMETER (a magic box inside the function)
2. def greet_by_name(name):
3.     # 'name' acts like a variable that exists ONLY inside this function
4.     print("Greetings, " + name + "!")
5.     print("Welcome to the guild.")
6.
7. # Now, when we call it, we provide an ARGUMENT (the loot)
8. # An argument is the *actual data* we pass in.
9. greet_by_name("Aria") # "Aria" is put into the 'name' box
10. greet_by_name("Kael") # "Kael" is put into the 'name' box
```

خروجی:

```
Greetings, Aria!
Welcome to the guild.
Greetings, Kael!
Welcome to the guild.
```

نتیجه نهایی: طلسم `return`

تابع در ساده‌ترین حالت، مثل یک دستگاه آبمیوه‌گیری است. ما میوه (ورودی) می‌دهیم و انتظار داریم آبمیوه (خروجی) تحویل بگیریم.

کلمه کلیدی `return` همان جایی است که آبمیوه از دستگاه خارج شده و به لیوان شما می‌ریزد. این فرمان تعیین می‌کند که حاصل نهایی کار تابع چه چیزی است. وظیفه اصلی `return`، مشخص کردن نتیجه نهایی و قابل استفاده تابع است.

تعریف تابع آبمیوه‌گیری:

```
1. def make_juice(fruit_name):
2.     return fruit_name + " Juice"
```

`return` دو کار حیاتی را همزمان انجام می‌دهد:

تحويل محصول: مقدار محاسبه شده را به بیرون می‌فرستد.
توقف فوری: اجرای تابع در همان لحظه متوقف می‌شود و پایتون فوراً به همان خطی که تابع در آن فراخوانی شده بود، برمی‌گردد.

مثال عملی: جریان اجرای برنامه

ببینید چطور برنامه در حین اجرا، موقتاً به تابع می‌رود و سپس دقیقاً به همان خط برمی‌گردد:

```
1. # --- Example: Execution Flow ---
2.
3. # 1. The program starts here
4. print("--- 1. Start Program ---")
5.
6. # The program stops at this line and enters the function.
7. apple_juice = make_juice("Apple") # The function is called here.
8.
9. # 2. The function executes, reaches 'return', stops immediately, and
   returns the value "Apple Juice".
10. # 3. Execution returns exactly to this spot, and the returned value is
   stored in the 'apple_juice' variable.
11.
12. print("--- 2. Return to Main Line ---")
13.
14. # Now we can use the final result.
15. print("My drink is:", apple_juice)
16.
17. # Output: My drink is: Apple Juice
```

شرح فرآیند: وقتی پایتون به خط `apple_juice = make_juice("Apple")` می‌رسد، می‌داند که باید فعلاً اجرای این خط را نگه دارد، به سراغ تابع `make_juice` برود، آن را اجرا کند و منتظر بماند تا آبمیوه نهایی (مقدار برگشتی) تحويل داده شود. `return` این تضمین را می‌دهد که آن آبمیوه (داده) تحويل داده شده و برنامه دقیقاً از همان جایی که منتظر مانده بود (خط `apple_juice = ...`) ادامه پیدا کند.

خلاصه شیرین: `return` یعنی «این نتیجه نهایی است. دستگاه خاموش. ادامه کار را دقیقاً از لحظه تحویل محصول، از سر بگیرید.»

۵. تنظیمات کارخانه (ورودی‌های پیش فرض)

گاهی اوقات ماجراجویان عجله دارند! آن‌ها نمی‌خواهند هر بار تمام جزئیات را به تابع بگویند. آن‌ها می‌خواهند تابع در حالت عادی کارش را انجام دهد، مگر اینکه دستور خاصی صادر شود. به این قابلیت آرگومان پیش فرض (Default Argument) می‌گوییم. درست مثل پیتزا فروشی؛ اگر نگویند چه پنیری می‌خواهید، آشپز به صورت پیش فرض پنیر معمولی می‌ریزد. اما اگر تاکید کنید «پنیر گودا»، آن وقت دستور عوض می‌شود. بیایید طلسم `print_divider` را ارتقا دهیم. ما می‌خواهیم معمولاً خط تیره = چاپ کند، اما اگر دلمان خواست، بتوانیم ستاره * یا \$ چاپ کنیم.

```
1. # 1. We set a default value inside the parenthesis: char="="
2. def print_divider(char "="):
3.     # This prints the character 20 times
4.     print(char * 20)
5.
6. # Scenario A: The Lazy Way (Using Default)
7. print("--- Standard Divider ---")
8. print_divider() # We gave NO argument, so Python uses "=" automatically.
9.
10. # Scenario B: The Custom Way (Overriding Default)
11. print("--- Magic Divider ---")
12. print_divider("***") # We gave an argument, so "=" is ignored and "*" is
    used.
13.
14. print("--- Rich Divider ---")
15. print_divider("$")
```

خروجی:

```
--- Standard Divider ---
=====
--- Magic Divider ---
*****
--- Rich Divider ---
$$$$$$$$$$$$$$$$$$$$
```

تحلیل جادویی: در خط تعریف تابع: `def print_divider(char="=")`، ما به پایتون گفتیم: «اگر کسی چیزی داخل پرانتز نگذاشت، خودت `char` را مساوی = در نظر بگیر. اما اگر چیزی نوشتند، حرف آن‌ها اولویت دارد.» این ترفند باعث می‌شود توابع شما هم ساده باشند (برای استفاده سریع) و هم قدرتمند (برای تنظیمات خاص)

نکته مخفی: جادوی تکثیر (String Multiplication)

شاید درکد بالا متوجه یک طلسم عجیب شده باشید `char * 20` مگر می‌شود کلمات را ضرب کرد؟ در ریاضیات معمولی نه، اما در پایتون بله! علامت ستاره `*` وقتی بین دو عدد باشد، عمل ضرب را انجام می‌دهد. ($2 * 3 = 6$) اما وقتی بین یک «رشته» و یک «عدد» قرار می‌گیرد، تبدیل به دستگاه فتوکپی می‌شود! این دستور، متن شما را به تعداد آن عدد تکرار می‌کند و به هم می‌چسباند.

مثال:

```
1. # 1. Repeating a laugh
2. print("Ha" * 3)
3. # Output: HaHaHa
4.
5. # 2. Making a long line (divider)
6. print("-" * 10)
7. # Output: -----
```

هشدار جادوگر:

یادتان باشد که رشته را فقط می‌توانید در عدد صحیح (Integer) ضرب کنید.

- `"Ali" * 3` درست: علی‌علی‌علی
- `"Ali" * "Ali"` غلط: کلمه در کلمه ضرب نمی‌شود!
- `"Ali" * 2.5` غلط: نمی‌توانیم دو و نیم بار علی داشته باشیم!

۶. جادوی دقیق: برچسب زدن به مواد اولیه

گاهی وقت‌ها، طلسم‌های ما پیچیده می‌شوند و چند تا ورودی مختلف می‌گیرند. اگر ترتیب مواد اولیه را اشتباه بریزیم، ممکن است به جای اینکه «جون» بگیریم، «سم» بخوریم! برای جلوگیری از این فاجعه، پایتون به ما اجازه می‌دهد روی هر آرگومان یک «برچسب» بزنیم. یعنی موقع صدا زدن تابع، دقیقاً بگوییم کدام ماده مال کدام پارامتر است. این طوری حتی اگر ترتیبشان را هم قاطی کنیم، پایتون گیج نمی‌شود.

به زبان کد:

```
1. # A spell to make a potion with specific ingredients
2. def brew_potion(herb, liquid):
3.     print("Mixing " + herb + " with " + liquid + "...")
4.
5. # Standard way (Order matters!):
6. brew_potion("Mint", "Water")
7.
8. # Keyword Arguments (Order doesn't matter, labels do!):
9. # We explicitly say which argument belongs to which parameter.
```

```
10. brew_potion(liquid="Dragon Blood", herb="Fire Flower")
```

می بینید؟ با اینکه جایشان را عوض کردیم، چون برچسب (اسم پارامتر) داشتند، معجون درست ترکیب شد!

نکته مهم: به صورت ترکیبی هم میتونیم استفاده کنیم اینجوری که چند ورودی اول رو به ترتیب بدیم ولی چند ورودی آخر رو با اسمشون مشخص کنیم (برعکس این ترتیب شدنی نیست و ارور میده)

۷. پروژه: پاکسازی کد نفرین شده (The Clean-Up Code) – 100 xp

ماجراجو، تو آماده ای! امروز (مأموریت) ما با همیشه فرق دارد. ما قرار نیست چیزی جدید بسازیم، بلکه می خواهیم یک کد «بد» و «تکراری» را که توسط یک جادوگر تنبیل نوشته شده، با جادوی جدیدمان (توابع) پاکسازی و درمان کنیم.

تصویر نهایی: مشکل چیست و راه حل چه شکلی است؟

اول بیایید به این کد نفرین شده (The Cursed Scroll) نگاه کنیم. هدف کد این است که مشخصات دو قهرمان را چاپ کند:

```
1. # --- The Bad, Repetitive Code ---
2. print("--- Adventurer ID Card ---")
3. print("Name: Aria")
4. print("Class: Ranger")
5. print("HP: 100")
6. print("=====")
7.
8. print("--- Adventurer ID Card ---")
9. print("Name: Kael")
10. print("Class: Mage")
11. print("HP: 80")
12. print("=====")
```

چرا این کد افتضاح است؟ ما ۵ خط کد را دقیقاً کپی-پیست کردیم! (فقط نام و مشخصات تغییر کرده)

- اگر بخواهیم ۱۰۰ ماجراجو داشته باشیم، باید ۵۰۰ خط کد بنویسیم؟
- اگر بخواهیم فرمت چاپ را عوض کنیم، باید ۱۰۰ جا را ویرایش کنیم؟ این یعنی فاجعه!

راه حل (The Vision): ما می خواهیم یک ماشین چاپ کارت بسازیم.

تصور کن به جای نوشتن دستی، یک دستگاه داریم که فقط به آن می گوییم: (آریا، جادوگر، ۱۰۰) و دستگاه خودش تمام خطوط زیبا و کادربندی را چاپ می کند. این دستگاه، همان تابع (Function) است.

نقشه و طرح اولیه

بیایید منطق این دستگاه را مهندسی کنیم. هر تابع مثل یک کارخانه کوچک است:

۱. **شناسایی الگوی تکراری (قالب):** چه چیزی ثابت است؟

- خط‌کشی‌ها --- و ===

- کلمات Name: و Class: و HP:

۲. **شناسایی تغییرات (ورودی‌ها):** چه چیزی در هر کارت عوض می‌شود؟

- اسم (name)

- شغل (job_class)

- جان (hp) این‌ها می‌شوند پارامترهای تابع ما (مواد اولیه‌ای که داخل دستگاه

می‌ریزیم).

۳. **ساخت دستگاه :** (def) یک بار برای همیشه دستورات چاپ را داخل یک بسته به نام

show_adventurer_stats می‌نویسیم.

۴. **استفاده از دستگاه :** (Call) در برنامه اصلی، فقط نام تابع را صدا می‌زنیم و مواد اولیه را به

آن می‌دهیم.

چالش‌های فکری

قبل از دیدن کد تمیز، سعی کن این معماها را حل کنی:

چالش ۱: تعریف تابع چطور به پایتون می‌فهمانی که داری یک «ابزار» می‌سازی و نه یک کد

معمولی؟ با کدام کلمه کلیدی؟ (راهنمایی: def)

چالش ۲: جعبه‌های ورودی تابع ما به ۳ تکه اطلاعات نیاز دارد تا کار کند. این ۳ متغیر را داخل

پرانتز تعریف تابع () با چه اسم‌هایی می‌گذاری؟ (مثال (name, job, hp))

چالش ۳: چسباندن متن و عدد (تله‌ی خطرناک!) در خط چاپ HP، ما یک متن ثابت ("HP: ")

داریم و یک عدد hp که مثلاً ۱۰۰ است. اگر بنویسی hp + "HP: " پایتون ارور می‌دهد (چون

نمی‌تواند عدد را به متن بچسباند). راه حل چیست؟ (راهنمایی: استفاده از کیمیاگری str(hp)

برای تبدیل عدد به متن)

کد نهایی پروژه (The Clean Spell)

وقت پاکسازی است! کد زیر را در فایل `project.py` بنویس. بین چقدر برنامه اصلی (پایین کد) کوتاه و تمیز شده است.

```
1. # --- The Cleaned-Up Spellbook (Using Functions) ---
2.
3. # 1. --- Define the Reusable Spell (The Machine) ---
4. # We create ONE function that takes 3 arguments (inputs)
5. def show_adventurer_stats(name, job_class, hp):
6.     print("--- Adventurer ID Card ---")
7.     print("Name: " + name)
8.     print("Class: " + job_class)
9.
10.    # CRITICAL: We must convert integer 'hp' to string 'str()' to join it!
11.    print("HP: " + str(hp))
12.
13.    print("=====")
14.
15.
16. # 2. --- Call the Spell (Using the Machine) ---
17. # Our main code is now simple, clean, and easy to read!
18.
19. # Creating Aria's card
20. show_adventurer_stats("Aria", "Ranger", 100)
21.
22. # Creating Kael's card
23. show_adventurer_stats("Kael", "Mage", 80)
24.
25. # Want to add a new one? It's just ONE line now!
26. show_adventurer_stats("Gimli", "Warrior", 150)
```

اجرا کن و لذت ببر! خروجی این کد تمیز، دقیقاً مشابه آن کدِ کثیف و تکراری است. اما حالا تو یک معمار هستی. اگر بخواهی طرح کارت‌ها را عوض کنی، فقط کافیست ۱ بار (داخل تابع) تغییر ایجاد کنی تا کارت تمام ۱۰۰۰ ماجراجو تغییر کند. این یعنی قدرت!

جشن پیروزی و تحلیل

آفرین تو همین الان از یک «کارآموز» که کدها را پشت سر هم می‌نوید، به یک برنامه‌نویس ساختاریافته تبدیل شدی. تو یاد گرفتی:

- چطور جلوی تکرار را بگیری قانون DRY: Don't Repeat Yourself
 - چطور کدهای طولانی را در بسته‌های کوچک و قابل مدیریت (def) دسته‌بندی کنی.
- ماجرایوبی‌های بزرگ با همین ابزارهای کوچک ساخته می‌شوند. کوله‌پشتی‌ات را بردار، فصل بعدی منتظر است!

🌸 ارتقاء لول: ۳۰ → ۲۲ 🌸

مهارت‌های جدید:

◆ دستور Def

با نام باستانی: خلق طلسم شخصی
توضیح: کارهایی که تکرار می‌شدند، اکنون
در یک کلمه خلاصه شده‌اند.

◆ دستور Return

با نام باستانی: بازگشت کیوتر نامه‌بر
توضیح: تابعی که دست خالی بر نمی‌گردد
و نتیجه را تقدیم می‌کند.

فصل ۹: طلسم محافظتی:

ساخت رمز عبور امن

مأموریت فصل: تولیدکننده رمز برای یک صندوقچه گنج.

نقشه راه

- استفاده از قدرت‌های مخفی پایتون: کتابخانه‌ها (Modules) و طلسم import.
- کتابخانه random: بهترین دوست ما برای کارهای تصادفی.
- جادوی شمارش: تکرار یک طلسم با `for i in range()`.
- پروژه: ساخت برنامه‌ای برای تولید رمزهای عبور قوی و تصادفی.

۱. قدرت‌های مخفی پایتون: کتابخانه‌ها و Import

تا الان هر کدی نوشتیم، دست‌رنج خودمان بود که در فایل `project.py` می‌نوشتیم. اما پایتون موقع نصب، هزاران خط کد آماده و حرفه‌ای را هم همراه خودش روی کامپیوتر شما نصب کرده است. به این مجموعه عظیم، کتابخانه استاندارد (**Standard Library**) می‌گوییم. این کدها در بسته‌هایی جداگانه به نام ماژول (**Module**) دسته‌بندی شده‌اند. تصور کن: فایل `project.py` مثل میز کار شماست. ماژول‌ها مثل جعبه ابزارهای تخصصی در انبار هستند (مثلاً جعبه ابزار ریاضی، جعبه ابزار شانس، و...). شما همه ابزارها را همیشه روی میز نمی‌ریزید! هر وقت به یکی نیاز داشتید، با دستور `import` آن را از انبار به روی میز می‌آورید.

```
1. # Go to the library and bring the 'random' toolbox
2. import random
```

با این خط ساده، به پایتون می‌گوییم: جعبه ابزار `random` را بیاور، من می‌خواهم از ابزارهای داخلش استفاده کنم.

۲. جعبه ابزار random: بهترین دوست ما برای کارهای تصادفی

حالا که جعبه ابزار روی میز است، چطور ابزار خاصی را از داخلش برداریم؟ ما از نقطه (.) استفاده می‌کنیم. این نقطه یعنی دسترسی به محتویات جعبه. فرمول آن این است

```
ToolboxName.ToolName
```

در اینجا ۳ ابزار بسیار پرکاربرد از جعبه `random` را بررسی می‌کنیم:

الف) ابزار `random.randint(a, b)` تاس دیجیتال

این ابزار یک عدد صحیح (**Integer**) تصادفی بین دو عددی که به او می‌دهی انتخاب می‌کند. نکته مهم: در پایتون، این ابزار برخلاف بقیه، شامل خودِ عدد آخری هم می‌شود. دقیقاً مثل تاس!

```
1. import random
2.
3. # Roll a simple 6-sided die
4. # This will pick a number: 1, 2, 3, 4, 5, or 6
5. die_roll = random.randint(1, 6)
6.
7. print("You rolled a:")
8. print(die_roll)
```

ب) ابزار `random.choice(sequence)`: انتخاب شانسی (Looting)

این محبوب‌ترین ابزار ما در بازی‌سازی است. تو یک لیست به او می‌دهی، و او چشمانش را می‌بندد و یک آیتم را شانسی بیرون می‌کشد.

```
1. import random
2.
```



```
3. # A chest of possible loot
4. loot_options = ["Gold Coin", "Health Potion", "Rusty Dagger", "Diamond"]
5.
6. # Use the 'choice' tool to pick ONE item blindly
7. found_item = random.choice(loot_options)
8.
9. print("You opened the chest and found a...")
10. print(found_item)
11. # Try running this multiple times!
```

ج) ابزار random.shuffle(list): بُر زدن کارتها

این ابزار یک لیست را می‌گیرد و آیتم‌های آن را درجا بُر می‌زند (ترتیبشان را به هم می‌ریزد).
هشدار: این ابزار خروجی جدیدی نمی‌سازد (Return نمی‌کند)، بلکه خود لیست اصلی را تغییر می‌دهد. پس ترتیب قبلی برای همیشه از بین می‌رود!

```
1. import random
2.
3. # A list of party members
4. party = ["Aria", "Kael", "Lyra", "Zane"]
5.
6. print("Original order:")
7. print(party)
8.
9. # Shuffle the list in-place (Changes the 'party' list directly)
10. random.shuffle(party)
11.
12. print("New battle order:")
13. print(party)
```

اجرا کن و نتیجه را ببین! هر بار که کد shuffle را اجرا کنی، اعضای گروه در ترتیب جدیدی قرار می‌دهد

۳. جادوی شمارش: تکرار با for i in range()

ما یاد گرفتیم که چطور با حلقه for تمام جیب‌های کوله‌پشتی (List) را بگردیم (مثلاً (for item in backpack) اما اگر لیستی در کار نباشد چه؟ اگر فقط بخواهیم یک کار خاص را دقیقاً ۱۰ بار تکرار کنیم (مثلاً ۱۰ بار به در ضربه بزنیم)؟
برای این کار، ما از یک ابزار داخلی قدرتمند به نام range() استفاده می‌کنیم.
دستور range(5) یک دنباله‌ی نامرئی از اعداد می‌سازد
نکته مهم: در دنیای کامپیوتر، شمارش همیشه از صفر شروع می‌شود. پس range(5) اعداد ۱ تا ۵ را نمی‌سازد، بلکه ۰ تا ۴ را می‌سازد (که در مجموع می‌شود ۵ عدد).
بیایید امتحان کنیم:

```
1. # The range(stop) tool
2. # We want to repeat an action 5 times
3. # This generates numbers: 0, 1, 2, 3, 4
```

```

4.
5. for i in range(5):
6.     print("This is loop number: " + str(i))

```

متغیر i چیست؟

در کد بالا، i (که مخفف Index است) مثل یک شمارنده در دست داور مسابقه است.

- در دور اول، مقدارش 0 است.
- در دور دوم، مقدارش 1 است.
- و همین طور ادامه می‌یابد تا به 4 برسد و حلقه تمام شود.

نکته: اسم این متغیر لازم نیست حتماً i باشد. شما می‌توانید بنویسید

for number in range(5) یا هر اسم دیگری، اما i در بین برنامه‌نویسان یک رسم قدیمی است.

۴. پروژه: ساخت قفل رمزنگاری شده (Password Generator) – 100 xp

ماجراجو، تو آماده‌ای!

ما بالاخره یک «صندوقچه گنج» پیدا کردیم و می‌خواهیم غنائم خود را در آن پنهان کنیم. اما یک مشکل بزرگ داریم: دزدهای امروزی باهوش هستند و رمزهای ساده مثل 1234 یا password را سریع حدس می‌زنند.

ما به یک طلسم نیاز داریم که یک رمز عبور ۱۲ حرفی، قوی و کاملاً تصادفی برای ما بسازد. رمزی که حتی خودمان هم نتوانیم حدس بزنیم!

تصویر نهایی: این ماشین رمزساز چطور کار می‌کند؟

قبل از کدنویسی، تصور کن یک دستگاه جادویی داری که شبیه «گردونه شانس» است.

۱. تو دکمه را فشار می‌دهی.

۲. دستگاه دستش را داخل یک کیسه بزرگ پر از حروف و اعداد می‌کند.

و ۱۲ بار، هر بار یک مهره را شانس بیرون می‌کشد و کنار هم می‌چیند.

نتیجه می‌شود چیزی مثل: KJ9#mP2\$aL5!

این رمز غیرقابل نفوذ است چون هیچ الگوی خاصی ندارد.

نقشه و طرح اولیه

بیایید مواد لازم برای این طلسم را بررسی کنیم. ما می‌خواهیم یک «معجون» بسازیم:

۱. وارد کردن جادوی خارجی: (Import)

پایتون خودش بلد نیست «تاس» بیندازد. ما باید یک کتابخانه کمکی به نام random را احضار کنیم.

۲. آماده‌سازی مواد اولیه: (The Ingredients)

یک رمز قوی باید مخلوطی از ۴ چیز باشد:

- حروف کوچک (a-z)
- حروف بزرگ (A-Z)
- اعداد (0-9)
- نمادهای عجیب (!@#\$)

۳. ترکیب مواد: (The Soup)

ما همه این ۴ گروه را با عملگر جمع + به هم می‌چسبانیم و داخل یک ظرف بزرگ (متغیر all_characters) می‌ریزیم.

۴. حلقه تکرار و انتخاب: (Loop & Pick)

حالا ۱۲ بار این کار را تکرار می‌کنیم:

- دست در ظرف کن : یک کاراکتر شانسی بردار (choice) : به رمز نهایی اضافه کن.

چالش‌های فکری

قبل از دیدن کد، سعی کن این گره‌های ذهنی را باز کنی:

چالش ۱: احضار کتابخانه

در خط اول برنامه، با چه دستوری به پایتون می‌گویی که می‌خواهی از ابزارهای تصادفی استفاده کنی؟

(راهنمایی: `import ...`)

چالش ۲: چسباندن رشته‌ها

ما ۴ متغیر متنی داریم. چطور همه را به هم وصل کنیم و در یک متغیر جدید بریزیم؟

(راهنمایی: در دنیای رشته‌ها، علامت + یعنی چسباندن.)

چالش ۳: حلقه شمارشی

ما دقیقاً ۱۲ بار تکرار می‌خواهیم. کدام نوع حلقه برای زمانی که «تعداد دقیق» را می‌دانیم

مناسب‌تر است؟ while یا for؟

(راهنمایی: `for` همراه با `range`)

چالش ۴: انتخاب شانسی

وقتی مواد را مخلوط کردیم، با کدام دستور از کتابخانه random می‌توانیم فقط یک دانه از حروف را به صورت شانسی بیرون بکشیم؟
(راهنمایی *random.choice*)

کد نهایی پروژه (The Spell)

وقت ساختن قفل است.

کد زیر را در فایل project.py بنویس. به نحوه استفاده از random و ساختن تدریجی رمز عبور دقت کن.

```
1. import random # We need this wizard to generate random things!
2.
3. # --- 1. The Ingredients (Manual Entry) ---
4. # We define the 4 types of characters we want in our password
5. lowercase_letters = "abcdefghijklmnopqrstuvwxyz"
6. uppercase_letters = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
7. numbers = "0123456789"
8. symbols = "!@#$%^&*()_+-=[]{}|;:,.<>?"
9.
10. # --- 2. Mixing the Soup ---
11. # Combine all ingredients into one big pool of characters
12. all_characters = lowercase_letters + uppercase_letters + numbers + symbols
13.
14. # --- 3. Magic Configuration ---
15. password_length = 12 # How long should the password be?
16. password = "" # We start with an empty string
17.
18. # --- 4. Casting the Spell (The Loop) ---
19. # We loop 12 times. inside the loop, we pick one char and add it.
20. for _ in range(password_length):
21.     # Pick one random character from the big pool
22.     random_char = random.choice(all_characters)
23.
24.     # Add it to our password variable
25.     password = password + random_char # OR: password += random_char
26.
27. # --- 5. Reveal the Secret ---
28. print("-----")
29. print(f"Your secure password is: {password}")
30. print("-----")
```

اجرا کن و نتیجه را ببین!

برنامه را چند بار پشت سر هم اجرا کن.

می‌بینی؟ هر بار یک رمز کاملاً متفاوت و عجیب ساخته می‌شود. حالا می‌توانی این رمز را برای صندوقچه گنج (یا ایمیل واقعی خودت!) استفاده کنی. (البته یادت باشه اگه رمزی رو برات ساخت و یادت رفت چی بود دیگه نمیتونی برنامه رو اجرا کنی و همون رمز رو بهت بده)

جشن پیروزی و تحلیل

تبریک !

تو الان امنیت را تضمین کردی.

در این پروژه یاد گرفتی:

۱. چطور از کتابخانه‌های خارجی (import) استفاده کنی.

۲. چطور با رشته‌ها مثل مهره‌های بازی رفتار کنی (بچسبانی و انتخاب کنی).

۳. و چطور شانس را وارد برنامه نویسی کنی.

حالا هیچ دزدی نمی‌تواند الگوی رمزهای تو را پیدا کند! ماجراجویی ادامه دارد...

ارتقاء لول: ۳۵ → ۳۰

مهارت‌های جدید:

◆ دستور Import

با نام باستانی: دروازه ابعاد

توضیح: اتصال به کتابخانه جهانی پایتون

و استفاده از قدرت دیگران.

◆ کتابخانه Random

با نام باستانی: تاس سرنوشت

توضیح: افزودن عنصر شانس و غیرقابل

پیش‌بینی بودن به جهان منظم.

◆ سطح جدید: E

توضیح: تبریک به سطح

جدیدی رسید.

مأموریت‌های جانبی: تالار تمرین

ماجرای عزیز! قبل از اینکه وارد سرزمین‌های ناشناخته‌ی بخش سوم شوی، باید در «تالار تمرین» مهارت‌هایت را در کار با لیست‌ها، دیکشنری‌ها و توابع تقویت کنی. اینجا ۱۰ چالش پشت سر هم منتظر توست.

قانون تالار ساده است: تا وقتی کد را خودت ننوشتی، به پاسخ‌نامه نگاه نکن!

مأموریت‌ها

۱. مأموریت: شبیه‌ساز تاس (Dice Roller) – 20 xp

در بازی‌های رومیزی، همیشه به تاس نیاز داریم.

- **وظیفه:** تابعی بنویس که وقتی اجرا می‌شود، یک عدد تصادفی بین ۱ تا ۶ را برگرداند و چاپ کند.
- **هدف:** تمرین کار با ماژول random و تعریف تابع ساده.

۲. مأموریت: جعبه شانس (Loot Box Generator) – 20 xp

پاداش‌های بازی‌ها معمولاً تصادفی هستند.

- **وظیفه:** یک لیست از آیتم‌ها (مثل "Gold", "Sword", "Potion", "Shield") بساز. برنامه باید به صورت تصادفی یکی از آن‌ها را انتخاب کرده و به عنوان جایزه به کاربر بدهد.
- **هدف:** استفاده از دستور choice از ماژول random.

۳. مأموریت: شمارشگر حروف صدا دار (Vowel Counter) – 20 xp

تحلیلگران متن نیاز دارند بدانند یک جمله چقدر خوش‌آهنگ است.

- **وظیفه:** یک کلمه یا جمله از کاربر بگیر. برنامه باید بشمارد که چند حرف صدا دار (a, e, i, o, u) در آن وجود دارد.
- **هدف:** استفاده از حلقه for روی رشته‌ها و شرط‌ها.

۴. مأموریت: آینه جادویی (Palindrome Checker) – 20 xp

برخی کلمات جادویی هستند و از هر دو طرف یکسان خوانده می‌شوند (مثل "radar" یا "level").

- **وظیفه:** تابعی بنویس که یک کلمه بگیرد. اگر کلمه با برعکس خودش برابر بود، بگوید "Normal Word" وگرنه بگوید "Magic Word!"
- **هدف:** کار با اسلایسینگ رشته‌ها [:-1] و معکوس کردن آن‌ها.

۵. مأموریت: کلاه گروه‌بندی (The Sorting Hat) – 20 xp

به مدرسه جادوگری هاگوارتز خوش آمدی!

- **وظیفه:** یک لیست از گروه‌ها (Gryffindor, Hufflepuff, Ravenclaw, Slytherin) داشته باش. اسم دانش‌آموز را بگیر و به صورت تصادفی او را در یکی از این گروه‌ها قرار بده.
- **هدف:** ترکیب ورودی گرفتن و انتخاب تصادفی.

۶. مأموریت: حذف تکراری‌ها (Duplicate Remover) – 20 xp

- کوله‌پشتی ما پر از آیتم‌های تکراری شده است.
- **وظیفه:** یک لیست با اعداد تکراری بساز (مثلاً [1, 2, 2, 3, 4, 4]). برنامه‌ای بنویس که تکراری‌ها را حذف کند و یک لیست تمیز (مثلاً [1, 2, 3, 4]) تحویل دهد.
- **هدف:** ساختن یک لیست جدید و بررسی وجود آیتم‌ها با شرط `if ... not in`.

۷. مأموریت: تبدیل گرایموجی (Emoji Converter) – 20 xp

- پیام‌ها خشک و بی‌روح هستند، بیایید به آن‌ها احساس بدهیم.
- **وظیفه:** یک دیکشنری بساز که کلمات را به نماد تبدیل کند (مثلاً "happy" به "😊" و "sad" به "😞"). جمله‌ای از کاربر بگیر و کلمات کلیدی را با نمادها جایگزین کن.
- **هدف:** کار با متد `split` در رشته‌ها و جستجو در دیکشنری.

۸. مأموریت: سنگ، کاغذ، قیچی (Rock, Paper, Scissors) – 20 xp

- بازی کلاسیک دو نفره، اما این بار حریف تو کامپیوتر است.
- **وظیفه:** انتخاب کاربر (سنگ/کاغذ/قیچی) را بگیر. کامپیوتر هم به صورت رندوم انتخاب می‌کند. سپس طبق قوانین بازی، برنده را مشخص کن.
- **هدف:** مدیریت شرط‌های تودرتو و منطق بازی.

۹. مأموریت: فاکتور خرید (Shopping Cart) – 20 xp

- صندوقدار فروشگاه میوه نیاز به کمک دارد.
- **وظیفه:** یک دیکشنری از قیمت‌ها داشته باش (مثلاً Apple: 10, Banana: 5). لیست خرید هم داری (مثلاً ["Apple", "Apple", "Banana"]). برنامه باید مجموع قیمت این لیست خرید را حساب کند.
- **هدف:** ترکیب پیمایش لیست (`for`) و خواندن مقدار از دیکشنری.

۱۰. مأموریت: خالق شخصیت (RPG Character Generator) – 20 xp

برای بازی نقش‌آفرینی جدیدمان نیاز به تولید انبوه دشمن و قهرمان داریم.

- **وظیفه:** تابعی بنویس که هیچ ورودی‌ای نگیرد، اما در خروجی یک دیکشنری برگرداند که شامل Name, Role, Health, Power باشد (مقادیر باید از لیست‌های از پیش آماده به صورت رندوم انتخاب شوند).
- **هدف:** ساخت ساختمان داده‌های پیچیده (دیکشنری) درون توابع.

آیا سفرت را در این مرحله کامل کرده‌ای، ماجراجو؟ اگر بله، کتاب را ورق بزن؛ کتابخانه سایه‌ها (نتیجه مأموریت‌ها) در صفحه بعد انتظار تو را می‌کشد .

کتابخانه سایه‌ها (نتیجه مأموریت‌ها)

۱. مأموریت: شبیه‌ساز تاس (Dice Roller)

نکته: برای تولید عدد صحیح تصادفی از randint استفاده می‌کنیم.

```
1. import random
2.
3. def roll_dice():
4.     number = random.randint(1, 6)
5.     print(f"You rolled a: {number}")
6.
7. # Call the function
8. roll_dice()
```

۲. مأموریت: جعبه شانس (Loot Box Generator)

نکته: متد choice بهترین ابزار برای انتخاب یک آیتم از لیست است.

```
1. import random
2.
3. items = ["Gold", "Sword", "Potion", "Shield"]
4. reward = random.choice(items)
5.
6. print(f"You opened the box and found: {reward}")
```

۳. مأموریت: شمارشگر حروف صدادار (Vowel Counter)

نکته: می‌توانیم چک کنیم آیا حرف در رشته "aeiou" وجود دارد یا خیر.

```
1. text = input("Enter a sentence: ")
2. vowels = "aeiou"
3. count = 0
4.
5. for char in text:
6.     if char.lower() in vowels:
7.         count += 1
8.
9. print(f"Number of vowels: {count}")
```

۴. مأموریت: آینه جادویی (Palindrome Checker)

نکته: در پایتون [::-1] کل رشته را برعکس می‌کند.

```
1. def check_palindrome(word):
2.     # Check if word equals its reverse
3.     if word == word[::-1]:
4.         print("Magic Word! (It is a palindrome)")
5.     else:
6.         print("Normal Word.")
7.
8. user_word = input("Enter a word: ")
9. check_palindrome(user_word)
```

۵. مأموریت: کلاه گروه‌بندی (The Sorting Hat)

نکته: با لیست‌ها می‌توانیم هر تعداد گروه که بخواهیم تعریف کنیم.

```
1. import random
2.
3. houses = ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]
4. name = input("Enter student name: ")
5.
6. assigned_house = random.choice(houses)
7. print(f"{name} belongs to... {assigned_house}!")
```

۶. مأموریت: حذف تکراری‌ها (Duplicate Remover)

نکته: ما یک لیست خالی می‌سازیم و فقط اعدادی که قبلاً ندیده‌ایم را به آن اضافه می‌کنیم.

```
1. numbers = [1, 2, 2, 3, 4, 4, 5]
2. unique_numbers = []
3.
4. for num in numbers:
5.     if num not in unique_numbers:
6.         unique_numbers.append(num)
7.
8. print(f"Clean list: {unique_numbers}")
```

۷. مأموریت: تبدیل گرایموجی (Emoji Converter)

نکته: متد `get` در دیکشنری باعث می‌شود اگر کلمه‌ای پیدا نشد، خود کلمه را بدون تغییر برگردانیم (جلوگیری از خطا).

```
1. message = input("Type your message: ")
2. words = message.split(" ")
3.
4. emojis = {
5.     "happy": "😊",
6.     "sad": "😞"
7. }
8.
9. output = ""
10. for word in words:
11.     output += emojis.get(word, word) + " "
12.
13. print(output)
```

۸. مأموریت: سنگ، کاغذ، قیچی (Rock, Paper, Scissors)

نکته: منطق بازی را با شرط‌های `if/elif` پیاده‌سازی می‌کنیم.

```
1. import random
2.
3. options = ["rock", "paper", "scissors"]
4. computer = random.choice(options)
```

```

5. user = input("Choose (rock, paper, scissors): ")
6.
7. print(f"Computer chose: {computer}")
8.
9. if user == computer:
10.     print("It's a tie!")
11. elif user == "rock" and computer == "scissors":
12.     print("You win!")
13. elif user == "paper" and computer == "rock":
14.     print("You win!")
15. elif user == "scissors" and computer == "paper":
16.     print("You win!")
17. else:
18.     print("You lose!")

```

۹. مأموریت: فاکتور خرید (Shopping Cart)

نکته: اینجا از لیست برای اقلام و از دیکشنری برای قیمت‌ها استفاده کرده‌ایم.

```

1. prices = {"Apple": 10, "Banana": 5, "Orange": 7}
2. cart = ["Apple", "Apple", "Banana"]
3.
4. total = 0
5. for item in cart:
6.     if item in prices:
7.         total += prices[item]
8.
9. print(f"Total Bill: ${total}")

```

۱۰. مأموریت: خالق شخصیت (RPG Character Generator)

نکته: دیکشنری خروجی می‌تواند مقادیر خود را از توابع دیگر یا انتخاب‌های تصادفی بگیرد.

```

1. import random
2.
3. def create_character():
4.     names = ["Arthur", "Merlin", "Lancelot"]
5.     roles = ["Warrior", "Mage", "Archer"]
6.
7.     character = {
8.         "Name": random.choice(names),
9.         "Role": random.choice(roles),
10.        "Health": random.randint(50, 100),
11.        "Power": random.randint(10, 20)
12.    }
13.    return character
14.
15. # Create a new hero
16. hero = create_character()
17. print(hero)

```

بخش سوم: ورود به بعد چهارم

آفرین ماجراجو! تو بخش دوم را با موفقیت کامل پشت سر گذاشتی. تو حالا یک استاد مدیریت «کوله پشتی» (List)، «کیسه‌ی مهر و موم شده» (Tuple)، و «کتاب جادویی» (Dictionary) هستی. تو فقط یک ماجراجوی تازه کار نیستی، بلکه با یادگیری «توابع» (Functions) به یک «خالق طلسم» تبدیل شدی و با import یاد گرفتی که از جادوی دیگران هم استفاده کنی.

اما یک مشکل بزرگ باقی مانده است. یک تله‌ی مرگبار به نام «فراموشی»! وقتی ۲۰ آیتم به «مدیر لیست خرید» خود اضافه می‌کنی و برنامه را می‌بندی... چه اتفاقی می‌افتد؟ همه‌ی آیت‌ها پاک می‌شوند! تمام متغیرهای ما تا امروز در حافظه‌ی موقت (RAM) زندگی می‌کردند. به محض بسته شدن برنامه همه چیز از بین می‌رود.

در این بخش، ما یاد می‌گیریم که چطور اطلاعات‌های خودمون رو روی فایل‌های متنی روی هارد دیسک حک کنیم. ما یاد می‌گیریم چطور داده‌ها را ذخیره (Save) و بازیابی (Load) کنیم تا ابزارهای ما واقعاً ماندگار شوند. ما یاد می‌گیریم که مثل یک مهندس نرم‌افزار فکر کنیم و کدهایمان را به موجوداتی زنده و دارای «روح» تبدیل کنیم. وقت آن است که وارد بخش سوم: ورود به بعد چهارم شویم

فصل ۱۰: رمزگشایی از طومارهای باستانی: تحلیلگر متن

مأموریت فصل: ساخت ابزاری که یک متن را تحلیل کرده و آمار آن را استخراج می‌کند.

نقشه راه

- کار حرفه‌ای با متن‌ها: طلسم‌های جدید برای «رشته‌ها» (Strings).
- هنر نمایش: سه طلسم جادویی برای قالب‌بندی (Formatting).
- رمزهای کلمات: راز استفاده از ' و " و ""
- پروژه: ساخت برنامه‌ای که تعداد کلمات و کاراکترهای یک متن را می‌شمارد.

۱. کار حرفه‌ای با متن‌ها (طلمس‌های String)

«رشته» (String) در پایتون، فقط یک متن ساده نیست؛ بلکه یک «شیء» (Object) قدرتمند با مجموعه‌ای از طلمس‌های داخلی (متدها) است. بیا چند مورد از پرکاربردترین آن‌ها را یاد بگیریم.

طلمس len()

تو این طلمس را قبلاً برای (List) به استفاده کردی. خبر خوب این است که len() روی رشته‌ها هم دقیقاً همان‌طور کار می‌کند و «تعداد کل کاراکترها» (شامل فاصله‌ها و علائم) را برمی‌گرداند.

```
1. scroll = "Hello World"
2. char_count = len(scroll)
3. print(char_count) # Output: 11
```

طلمس‌های lower() و upper()

در دنیای کامپیوترها، حروف بزرگ و کوچک با هم فرق دارند. برای کامپیوتر، حرف 'A' یک موجود کاملاً متفاوت از 'a' است. این ابزارها به شما کمک می‌کنند تا فرمت نوشته‌هایتان را یکدست کنید.

- ابزار lower(): تمام حروف را به کوچک تبدیل می‌کند.
- ابزار upper(): تمام حروف را به بزرگ تبدیل می‌کند.

```
1. scroll = "Aria the Ranger"
2.
3. # Convert everything to lowercase
4. print(scroll.lower())
5. # Output: "aria the ranger"
6.
7. # Convert everything to UPPERCASE
8. print(scroll.upper())
9. # Output: "ARIA THE RANGER"
```

نکته کاربردی: مقایسه بی‌خطر

بزرگترین کاربرد این ابزارها زمانی است که می‌خواهید ورودی کاربر را چک کنید. فرض کن در کافه هستی و کاربر می‌خواهد سوپ سفارش دهد. اگر او بنویسد "SOUP" ولی کد تو منتظر "soup" باشد، برنامه به او غذا نمی‌دهد!

راه حل: ما ورودی کاربر را (هر طوری که نوشته باشد) با lower() به حروف کوچک تبدیل می‌کنیم تا با شرط ما جور دربیايد.

```
1. # Cafe Menu: Soup, Bread
2.
3. order = input("What do you want to eat? ")
4.
5. # We convert input to lowercase immediately to be safe
6. # Now "Soup", "SOUP", and "soup" all work!
```

```

7. if order.lower() == "soup":
8.     print("Here is your hot soup!")
9. else:
10.    print("We don't have that.")

```

تحلیل: مهم نیست کاربر دکمه Caps Lock را روشن گذاشته باشد یا نه؛ با این ترفند، برنامه همیشه منظور او را می‌فهمیم

طلسم: `split()`. شمشیر سامورایی رشته‌ها

این طلسم، کلید اصلی مأموریت این فصل است. تا الان جملات برای ما فقط یک رشته متنی طولانی و یک‌تکه بودند. اما ابزار `split()` مثل یک شمشیر سامورایی عمل می‌کند؛ این ابزار رشته را تکه‌تکه می‌کند و نتیجه را در قالب یک لیست (**List**) به ما برمی‌گرداند.

حالت اول: برش خودکار (پیش‌فرض)

اگر داخل پرانتز () چیزی ننویسیم، این ابزار به صورت پیش‌فرض «فاصله‌های خالی (Space)» را نقطه برش در نظر می‌گیرد. یعنی هر جا فاصله دید، یک برش می‌زند.

```

1. sentence = "Welcome to the Python Guild"
2.
3. # Cuts the string at every SPACE
4. words = sentence.split()
5.
6. print(words)
7. # Output: ['Welcome', 'to', 'the', 'Python', 'Guild']

```

حالت دوم: برش دقیق (با ورودی خاص)

اما دنیای برنامه‌نویسی همیشه مرتب نیست! گاهی اوقات کلمات با فاصله از هم جدا نشده‌اند، بلکه با **ویرگول (,)**، **خط تیره (-)** یا علامت‌های دیگر به هم چسبیده‌اند. اینجاست که ما باید به شمشیرمان بگوییم دقیقاً کجا را ببر!

کافیست علامت جداکننده (Separator) را داخل پرانتز بگذاریم:

```

1. # A raw data string (items separated by commas)
2. raw_data = "Sword,Shield,Potion,Map"
3.
4. # We tell Python: "Cut the string wherever you see a COMMA"
5. inventory = raw_data.split(",")
6.
7. print(inventory)
8. # Output: ['Sword', 'Shield', 'Potion', 'Map']

```


طلسم‌های نگهبان `.startswith()` و `.endswith()`

گاهی لازم نیست کل متن رو بخونیم؛ فقط کافیه مهر و موم ابتدا یا انتهای آن را چک کنید. این دو ابزار مثل نگهبانان دروازه عمل می‌کنند و به شما می‌گویند آیا متن با کلمه خاصی شروع یا تمام می‌شود یا خیر. جواب آن‌ها همیشه `True` (بله) یا `False` (خیر) است.

- `.startswith(x)`: آیا با `x` شروع می‌شود؟ (مثلاً برای چک کردن پیشوند شماره تلفن یا آدرس سایت).

- `.endswith(x)`: آیا با `x` تمام می‌شود؟ (مثلاً برای چک کردن پسوند فایل‌ها مثل `.pdf` یا `.jpg`).

```
1. filename = "treasure_map.png"
2. url = "https://google.com"
3.
4. # Check the file type
5. if filename.endswith(".png"):
6.     print("It is an image!") # This prints
7.
8. # Check if site is secure
9. if url.startswith("https"):
10.    print("Secure site detected.") # This prints
```

طلسم کیمیاگری: تبدیل مس به طلا (`.replace()`)

فرض کنید کوله‌پشتی شما پر از تجهیزات زنگ‌زده (`Rusty`) است. حالا یک سنگ کیمیا پیدا کرده‌اید و می‌خواهید با یک حرکت، تمام وسایل زنگ‌زده را به وسایل طلایی (`Golden`) تبدیل کنید.

طلسم `.replace()` دقیقاً مثل این سنگ کیمیا عمل می‌کند. این دستور کل متن را می‌گردد و هر جا کلمه قدیمی را ببیند، نسخه جدید را جایگزین می‌کند.

```
1. # 1. Your inventory with weak items
2. inventory = "Rusty Sword, Rusty Shield, Rusty Armor"
3.
4. # 2. The Alchemy Spell: Change ALL "Rusty" to "Golden"
5. # Input 1: Old text (What to find)
6. # Input 2: New text (What to replace with)
7. upgraded_inventory = inventory.replace("Rusty", "Golden")
8.
9. print(upgraded_inventory)
10. # Output: "Golden Sword, Golden Shield, Golden Armor"
```

کاربرد واقعی: تصور کنید در کدی که نوشته‌اید، آدرس سایت شما ۱۰۰ بار تکرار شده و حالا آدرس سایت عوض شده است. به جای اینکه ۱۰۰ بار دستی آن را تغییر دهید، با یک `.replace()` همه را اصلاح می‌کنید!

نکته جادوگری: یادتان باشد که رشته‌ها در پایتون «تغییرناپذیر» هستند. یعنی `replace()` رشته اصلی را دستکاری نمی‌کند، بلکه یک نسخه جدید و اصلاح شده به شما می‌دهد که باید آن را در یک متغیر ذخیره کنید.

۲. هنر نمایش: سه طلسم جادویی برای قالب‌بندی (Formatting)

تا به حال، اگر می‌خواستیم متن و متغیر را با هم چاپ کنیم، کارمان کمی زشت و سخت بود. ما مجبور بودیم از + برای چسباندن استفاده کنیم و اعداد را با `str()` به متن تبدیل کنیم:

```
1. # The "Ugly Way" (from Chapter 8)
2. name = "Aria"
3. score = 100
4. print("Adventurer: " + name + " - Score: " + str(score))
```

این کد کار می‌کند، اما خوانا نیست و به راحتی باعث خطا می‌شود. برای حل این مشکل، جادوگران پایتون سه طلسم قدرتمند اختراع کرده‌اند:

طلسم اول: جادوی باستانی (قالب‌بندی با %)

این کهن‌ترین روش نوشتن در پایتون است. در این روش، تو داخل متن اصلی‌ات، نشان‌های باستانی (Placeholders) می‌گذاری تا جای خالی متغیرها را مشخص کنی. سپس در انتهای خط، متغیرها را به ترتیب به آن پاس می‌دهی.

نشان‌های مهم عبارتند از:

- `%s`: جای خالی برای متن (String)
- `%d`: جای خالی برای عدد صحیح (Decimal/Integer)
- `%f`: جای خالی برای عدد اعشاری (Float)

۱. مثال پایه (متن و عدد صحیح)

بیایید ببینیم چطور نام قهرمان و امتیاز او را در یک جمله جاگذاری می‌کنیم:

```
1. # The "Ancient Way" (%)
2. name = "Aria"
3. score = 100
4.
5. # We put placeholders inside the string
6. # Then we map variables to them at the end
7. print("Adventurer: %s - Score: %d" % (name, score))
8.
9. # Output: Adventurer: Aria - Score: 100
```

۲. مثال پیشرفته (کنترل اعشار)

یکی از محدود جاهایی که این روش هنوز کاربرد دارد، زمانی است که می‌خواهیم اعداد اعشاری طولانی را کوتاه کنیم. طلسم %f به صورت پیش‌فرض ۶ رقم اعشار چاپ می‌کند، اما ما می‌توانیم با نوشتن **%.2f** به پایتون دستور دهیم که فقط ۲ رقم را نگه دارد و عدد را گرد (Round) کند.

```
1. accuracy = 87.56789
2.
3. # 1. Standard Float (Too long and messy)
4. print("Shot Accuracy: %f" % accuracy)
5. # Output: Shot Accuracy: 87.567890
6.
7. # 2. Controlled Precision (Clean - only 2 decimals)
8. print("Shot Accuracy: %.2f" % accuracy)
9. # Output: Shot Accuracy: 87.57
```

چرا این روش منسوخ شده؟ (معایب) این طلسم هنوز در کدهای قدیمی دیده می‌شود، اما استفاده از آن توصیه نمی‌شود. **مشکل اینجاست**: اگر تعداد متغیرها زیاد شود (مثلاً ۵ متغیر)، خواندن کد بسیار سخت می‌شود. باید مدام چشم‌ت را بین متن و لیست آخر خط جابجا کنی تا ببینی کدام متغیر مال کدام %s است. این کار گیج‌کننده است و احتمال خطا را بالا می‌برد.

طلسم دوم: جادوی مدرن (متد format())

این طلسم، نسخه‌ی بسیار قدرتمندتر و انعطاف‌پذیرتر از روش قدیمی است. در این روش، تو از آکولاد {} به عنوان **جانه‌دار (Placeholder)** استفاده می‌کنی و سپس طلسم format() را در انتهای رشته صدا می‌زنی.

تفاوت بزرگ اینجاست که دیگر نیازی نیست نگران نوع متغیر (متن یا عدد) باشی؛ این طلسم خودش همه چیز را مدیریت می‌کند.

۱. استفاده ساده (جایگذاری ترتیبی)

در این حالت، اولین متغیر در اولین {}, و دومین متغیر در دومین {} می‌نشیند.

```
1. # The "Modern Way" (.format)
2. name = "Aria"
3. score = 100
4.
5. # The {} are placeholders waiting to be filled
6. print("Adventurer: {} - Score: {}".format(name, score))
7.
8. # Output: Adventurer: Aria - Score: 100
```

۲. قدرت پنهان: کنترل ترتیب (Indexing)

اینجاست که روش مدرن قدرت‌نمایی می‌کند! در روش قدیمی، اگر می‌خواستی یک کلمه را دو بار چاپ کنی، باید آن را دو بار در لیست آخر می‌نوشتی. اما اینجا می‌توانی داخل آکولادها عدد بگذاری ($\{0\}$, $\{1\}$) و بگویی کدام متغیر کجا بنشیند.

مثال: تکرار نام قهرمان

```
1. name = "Kael"
2.
3. # We use {0} to refer to the first variable ("Kael")
4. # We can use it multiple times!
5. print("Run {0}, run! {0} is in danger!".format(name))
6.
7. # Output: Run Kael, run! Kael is in danger!
```

مثال: تغییر ترتیب

```
1. # {0} is 'Speed', {1} is 'Power'
2. print("Class: {1} - Attribute: {0}".format("Speed", "Power"))
3.
4. # Output: Class: Power - Attribute: Speed
```

۳. کنترل اعشار در روش مدرن

اگر بخواهیم در این روش اعداد اعشاری را کنترل کنیم، از دستور نگارشی (colon) داخل آکولاد استفاده می‌کنیم. فرمول آن $:.2f$ است.

```
1. gold = 150.7894
2.
3. # {:.2f} means: Take the variable, keep only 2 decimal places
4. print("Your Gold: {:.2f}".format(gold))
5.
6. # Output: Your Gold: 150.79
```

طلسم سوم: جادوی استاد (f-strings)

این جدیدترین، تمیزترین و سریع‌ترین طلسمی است که در پایتون مدرن (Python 3.6) به بعد کشف شده است. در روش‌های قبلی، متغیرها بیرون از متن منتظر می‌ماندند تا جایگذاری شوند. اما در این روش، دیوار بین متن و کد برداشته می‌شود! کافی‌ست قبل از علامت نقل قول، " فقط یک حرف **f** (مخفف format) بگذاری. حالا، می‌توانی متغیرهایت را مستقیماً داخل متن و درون آکولاد $\{\}$ قرار دهی!

۱. استفاده ساده (تله‌پاتی مستقیم)

دیگر نیازی به کاما ، یا `format()` در انتهای خط نیست. متغیر دقیقاً همان جایی می‌نشیند که باید باشد.

```
1. # The "Wizard's Way" (f-string)
2. name = "Aria"
3. score = 100
4.
5. # Notice the 'f' at the start outside the quotes
6. # Variables go directly inside {}
7. print(f"Adventurer: {name} - Score: {score}")
8.
9. # Output: Adventurer: Aria - Score: 100
```

۲. قدرت مخفی: انجام محاسبات درون متن!

این ویژگی فقط مخصوص **f-strings** است. شما می‌توانید درون آکولاد {} نه تنها متغیر، بلکه عملیات ریاضی یا کد پایتون هم بنویسید!

```
1. damage = 50
2.
3. # We can do math directly inside the curly braces!
4. print(f"Critical Hit! Damage doubled: {damage * 2}")
5.
6. # We can even use string methods inside
7. hero = "kael"
8. print(f"Hero Name: {hero.upper()}")
9.
10. # Output:
11. # Critical Hit! Damage doubled: 100
12. # Hero Name: KAEI
```

۳. کنترل اعشار در جادوی استاد

در اینجا هم برای کنترل اعداد اعشاری، از همان قانون کولن: استفاده می‌کنیم. بسیار ساده و چسبیده به خود متغیر.

```
1. gold = 150.7894
2.
3. # Keep only 2 decimal places
4. print(f"Your Gold: {gold:.2f}")
5.
6. # Output: Your Gold: 150.79
```

کدام را استفاده کنیم؟

از این به بعد در تمام طول ماجراجویی‌مان، ما فقط و فقط از **f-strings** استفاده خواهیم کرد.

- چرا؟ چون کوتاه‌تر است، خواندنش راحت‌تر است.
- پس چرا بقیه را یاد گرفتیم؟ چون وقتی کدهای قدیمی جادوگران دیگر را در اینترنت می‌بینی، باید بتوانی زبان آن‌ها را هم بخوانی و بفهمی.

۳. مرزهای کلمات: راز استفاده از ' و " و ""

تا اینجا دیدیم که متن‌ها (String) را داخل کوتیشن (نقل قول) می‌گذاریم. اما شاید دیده باشی که جادوگران گاهی از ' (تک)، گاهی از " (جفت) و گاهی از "" (سه تایی) استفاده می‌کنند. آیا این‌ها با هم فرق دارند؟ بیایید راز این "محفظه‌های متن" را کشف کنیم.

الف) نبرد تک ' در برابر جفت (Single vs Double)

خبر خوب: در پایتون، این دو کاملاً برابرند! نوشتن 'Hello' هیچ فرقی با "Hello" ندارد. پس چرا دو تا داریم؟ برای اینکه در تله‌ی تداخل نیفتیم!

تله‌ی کلمات: فرض کن می‌خواهی بنویسی **It's a trap**: اگر از ' استفاده کنی، پایتون گیج می‌شود:

```
1. # ERROR! Python thinks the string ends at 'It'
2. print('It's a trap')
```

راه حل هوشمندانه: اگر در متن ' داری، کل متن را در " بگذار. اگر در متن " داری، کل متن را در ' بگذار.

```
1. # 1. Used double quotes because the text contains a single quote (')
2. print("It's a trap!")
3.
4. # 2. Used single quotes because the text contains double quotes (")
5. print('The guard shouted: "Stop right there!"')
```

ب) رشته‌های بلند: جادوی (Triple Quotes) """"

مشکل کوتیشن‌های معمولی ' و " این است که نمی‌توانند خط را بشکنند. اگر وسط نوشتن دکمه Enter را بزنی، برنامه خطا می‌دهد.

اما اگر بخواهیم یک نامه بلند، یک شعر یا یک منوی چندخطی بنویسیم چه؟ اینجا است که از سه کوتیشن """" یا ''' استفاده می‌کنیم. این محفظه به تو اجازه می‌دهد هر چقدر می‌خواهی Enter بزنی و متن را در چندین خط بنویسی.

```
1. # The "Grand Scroll" (Multi-line String)
2. # Perfect for letters, menus, or ASCII art
3.
4. letter_to_king = """
5. My Lord,
6. The dragon has been defeated.
7. We found:
8. - 500 Gold
9. - 2 Magic Swords
10. - 1 Very angry chicken
11.
12. Regards,
13. Sir Pytholot
```

```
14. ""  
15.  
16. print(letter_to_king)
```

نکته نهایی: هر فضایی (Space) یا خط جدیدی که داخل "" بگذاری، دقیقاً همان طور چاپ می‌شود. پس مراقب فاصله‌های اضافی باش!

۴. پروژه: تحلیلگر طومار باستانی (The Scroll Analyzer) – 100 xp

ما جراجو، تو آماده‌ای! ما یک طومار قدیمی پیدا کرده‌ایم. اما خواندن کلمه به کلمه آن وقت‌گیر است. ما می‌خواهیم با استفاده از طلسم‌های جدیدمان (`split()` و `f-strings` و طلسم قدیمی‌مان (`len`))، یک ابزار هوشمند بسازیم که در یک چشم برهم زدن، این متن را آنالیز کند و آمار دقیق آن را به ما بدهد.

تصویر نهایی: این ابزار چه گزارشی می‌دهد؟

قبل از کدنویسی، تصور کن این برنامه مثل یک «اسکنر جادویی» عمل می‌کند. ما متن را به آن می‌دهیم و برنامه بلافاصله یک گزارش آماری زیبا و دقیق چاپ می‌کند:

۱. **طول متن:** چند حرف (کاراکتر) در کل متن وجود دارد؟ (حتی فاصله‌ها را هم می‌شمارد).

۲. **تعداد کلمات:** چند کلمه معنادار در متن وجود دارد؟

خروجی نهایی چیزی شبیه به این خواهد بود:

```
--- Text Analysis Results ---  
Total Characters: 96  
Total Words: 17
```

نقشه و طرح اولیه

بیایید منطق این ابزار را مهندسی کنیم. ما یک مسیر ۳ مرحله‌ای داریم:

۱. **دریافت ورودی (Input):** ما یک متن طولانی داریم که در چند خط نوشته شده است.
 - **منطق:** برای ذخیره متن‌های چندخطی، از **سه علامت نقل قول** "" استفاده می‌کنیم.
۲. **شمارش حروف (Character Count):** می‌خواهیم بدانیم طول کل متن چقدر است.
 - **منطق:** طلسم `len()` را مستقیماً روی متن اجرا می‌کنیم. این دستور همه چیز (حروف، فاصله‌ها، نقطه‌ها) را می‌شمارد.

۳. **شمارش کلمات (Word Count - بخش تله):** اینجا جایی است که تازه کارها اشتباه می‌کنند! اگر `len()` را روی متن بزنیم، تعداد حروف را می‌دهد، نه کلمات.

• منطق:

۱. اول باید متن را «تکه تکه» کنیم تا کلمات جدا شوند. (استفاده از متد `.split()`)
 ۲. این دستور به ما یک لیست از کلمات می‌دهد.
 ۳. حالا `len()` را روی این لیست اجرا می‌کنیم تا تعداد کلمات را بشمارد.
 ۴. گزارش نهایی: **(Formatted Output)** نتایج را باید تمیز چاپ کنیم.
- منطق: از f-string استفاده می‌کنیم تا متغیرها را مستقیماً داخل متن بگذاریم.

چالش‌های فکری

قبل از اینکه کد را ببینی، سعی کن این گره‌های ذهنی را باز کنی:

چالش ۱: متن‌های معمولی را با " نشان می‌دهیم. اما اگر بخواهیم Enter بزنیم و به خط بعد برویم، باید از چه علامتی استفاده کنیم؟ (راهنمایی: `"""`)

چالش ۲: `len("Hello World")` چه عددی می‌دهد؟ (۱۱، چون فاصله هم یک کاراکتر است). اما اگر آن را اسپلیت کنیم `len(["Hello", "World"])` چه عددی می‌دهد؟ (۲، چون دو کلمه داریم).

چالش ۳: جایگذاری هوشمند قدیم‌ترها برای چاپ کردن متغیر و متن، از + استفاده می‌کردیم که سخت بود. حالا با گذاشتن حرف f قبل از کوتیشن، چطور متغیر را داخل متن جا می‌دهیم؟ (راهنمایی: داخل آکولاد `{}`)

کد نهایی پروژه (The Spell)

وقت آنالیز است! کد زیر را در فایل `project.py` بنویس. به نحوه تبدیل متن به لیست در خط ۲۰ دقت کن.

```
1. # --- The Ancient Scroll Analyzer ---
2.
3. # 1. The Input Scroll
4. # We use triple quotes """ for multi-line strings
5. scroll_text = """
6. The dragon sleeps in the mountain's heart.
7. We must retrieve the magic sword.
8. Only the brave will succeed.
9. """
10.
11. print("--- Analyzing Ancient Scroll ---")
```



```
12. # Let's print the raw text first to see it
13. print(scroll_text)
14. print("=====")
15.
16.
17. # 2. Challenge: Count Characters
18. # len() on a string counts EVERY character (letters, spaces, newlines)
19. char_count = len(scroll_text)
20.
21.
22. # 3. Challenge: Split into words (The Trick)
23. # .split() cuts the string at every space and puts words into a LIST
24. word_list = scroll_text.split()
25.
26. # OPTIONAL: Uncomment the line below to see what the list looks like!
27. # print(word_list)
28.
29.
30. # 4. Challenge: Count Words
31. # Now len() counts the number of items inside the list (the words)
32. word_count = len(word_list)
33.
34.
35. # 5. Challenge: Display results with f-strings
36. # We use f-strings to insert variables directly into the message
37. print("--- Text Analysis Results ---")
38. print(f"Total Characters: {char_count}")
39. print(f"Total Words: {word_count}")
40. print("=====")
```

اجرا کن و نتیجه را ببین! آفرین! تو همین الان یک طومار باستانی را «تحلیل» کردی. تو یاد گرفتی که:

- رشته‌ها (Strings) فقط متن ساده نیستند، طلسم‌های قدرتمندی مثل `.split()` دارند.

- فرق بین حروف و کلمات در برنامه‌نویسی چیست.
- و چطور با f-strings گزارش‌های حرفه‌ای چاپ کنی.

در فصل بعد، ما بزرگترین چالش خود را تا به امروز انجام خواهیم داد: به ابزارهای خود «حافظه‌ی دائمی» خواهیم داد!

ارتقاء لول: ۴۰ → ۳۵

مهارت‌های جدید:

◆ متد Split

با نام باستانی: شمشیر سامورایی
توضیح: خرد کردن جملات به کلمات. تبدیل
یک کوه سخت به سنگ‌ریزه‌های قابل حمل.

◆ اف-استرینگ f-string

با نام باستانی: هم‌نشینی کلام و عدد
توضیح: تزریق مستقیم متغیرها به قلب
متن بدون زحمت.

فصل ۱۱: طلسم ضد فراموشی:

کار با فایل‌ها

مأموریت فصل: ارتقاء برنامه «مدیر لیست خرید» تا با بستن برنامه، اطلاعات از بین نرود!

نقشه راه

- چطور از شر "کرش" کردن خلاص شویم؟ مدیریت خطا با try و except.
- کار با فایل‌ها: خواندن و نوشتن در فایل‌های متنی (.txt).
- زبان جادویی جهانی (JSON): یک روش استاندارد برای ذخیره داده‌های ساختاریافته.
- استفاده از JSON برای ذخیره لیست و دیکشنری در فایل
- **پروژه:** کتاب هیولاشناسی : برای ذخیره و بازیابی لیست در فایل JSON

۱. چطور از شر "کرش" کردن خلاص شویم؟ (except و try)

قبل از اینکه فایل‌ها را دست بزنیم، باید یک «طلسم محافظتی» حیاتی یاد بگیریم.

چرا کار با فایل‌ها خطرناک است؟

چه اتفاقی می‌افتد اگر فایلی را بخوایم بخوانیم که وجود ندارد؟ در این حالت برنامه کرش می‌کند (یعنی به طور ناگهانی با یک پیام خطای قرمز متوقف می‌شود).

چه اتفاقی می‌افتد اگر بخوایم در فایلی بنویسیم که اجازه دسترسی به آن را نداریم؟ باز هم برنامه کرش می‌کند.

برای جلوگیری از این فاجعه، ما از یک «سپر جادویی» به نام `try...except` استفاده می‌کنیم.

`try` (امتحان کن): ما کد خطرناک را در این بلوک قرار می‌دهیم.

`except` (مگر اینکه): ما به پایتون می‌گوییم «اگر در بلوک `try` مشکلی پیش آمد، به جای کرش کردن، این بلوک را اجرا کن». (اینجوری برنامه ادامه پیدا می‌کند و متوقف نمی‌شود)

مثال: اگه کاربر به جای عدد، کلمه "hello" را وارد کند، تابع `int` کرش می‌کند.

۱. روش ناامن (کرش می‌کند!)

```
1. print("Enter a number:")
2. user_input = input()
3. number = int(user_input) # This line will CRASH if user types "hello"
4. print("Your number is: " + str(number))
```

۲. روش امن (با `try...except`)

```
1. print("Enter a magic number:")
2. user_input = input()
3.
4. try:
5.     # We "try" to run the dangerous code
6.     number = int(user_input)
7.     print(f"Your magic number is: {number}")
8. except:
9.     # If the 'try' block fails, this code runs instead
10.    print("That wasn't a number, adventurer! Spell failed.")
```

اجرا کن و نتیجه را ببین! اگر "hello" را تایپ کنی، برنامه دیگر کرش نمی‌کند، بلکه به زیبایی پیام خطا را نمایش می‌دهد. ما برای خواندن فایل‌ها دقیقاً به همین سپر نیاز داریم.

۲. کار با فایل‌ها: کتیبه‌های ماندگار (txt).

ساده‌ترین راه برای ذخیره اطلاعات به صورت دائمی، نوشتن آن‌ها در یک فایل متنی (txt) است. چرا این کار لازم است؟

- **RAM حافظه موقت: مثل میز کار شماست.** وقتی متغیری می‌سازید، روی میز است. دسترسی به آن سریع است، اما اگر برق برود (کامپیوتر خاموش شود)، میز کار تمیز می‌شود و همه چیز از بین می‌رود (این خاصیت ذاتی رم هست سریع و با قطعی برق اطلاعاتش پاک می‌شه).
- **Hard Drive حافظه دائم: مثل انبار.** نوشتن در آن کندتر است، اما اگر برق بره، اطلاعات از بین نمی‌رود و اونجا باقی می‌مونه.

ما می‌خواهیم یاد بگیریم چطور اطلاعات مهم را از روی میز (RAM) برداریم و در کتابخانه (File) بایگانی کنیم تا بعداً هم بتوانیم این اطلاعات رو داشته باشیم

طلسم بازکردن دروازه with open(...):

برای کار با فایل، ما از ساختار with open استفاده می‌کنیم. این طلسم یک «محیط امن» برای دسترسی فایل باز می‌کند و تضمین می‌دهد که وقتی کارمان تمام شد، فایل به صورت خودکار بسته شود (حتی اگر برنامه خطا بدهد) (بستن فایل در انتهای کار خیلی مهمه چون اگه فایل رو نبندیم برنامه دیگه ای نمیتونه با اون فایل کار کنه) ساختار کلی طلسم به این شکل است

```
with open("filename", "mode") as variable:
```

بیا ببینیم اجزای این طلسم را کالبدشکافی کنیم:

۱. مودها (Modes)

ورودی دوم (mode) بسیار حیاتی است. این ورودی به پایتون می‌گوید «قصدت از باز کردن این فایل چیست؟»

- **"r" برای خواندن:**

- **کارکرد:** فقط می‌توانید متن را بخوانید. (برای نوشتن به ارور می‌خورید)
- **امنیت:** کاملاً امن. هیچ چیزی تغییر نمی‌کند.
- **اگر فایل نباشد:** ارور می‌دهد.

• - "w" برای نوشتن :

- کارکرد: نوشتن متن جدید.
- هشدار مهم: اگر فایلی با این نام از قبل وجود داشته باشد، آن را کاملاً پاک می‌کند و یک فایل سفید نو می‌سازد! (مثل فرمت کردن).
- اگر فایل نباشد: آن را می‌سازد.

• - "a" برای اضافه کردن :

- کارکرد: اضافه کردن متن به انتهای فایلی که وجود داشته
- امنیت: فایل قبلی را پاک نمی‌کند، بلکه نوشته‌های جدید را به ته آن می‌چسباند.
- اگر فایل نباشد: آن را می‌سازد.

۲. راز as f : دستگیره فایل

شاید بپرسید این as f ته خط چیست؟ وقتی شما فایلی را روی هارد دیسک باز می‌کنید، نمی‌توانید مستقیماً با هارد حرف بزنید. پایتون یک رابط «یا» دستگیره (Handle) به شما می‌دهد تا با فایل کار کنید.

- ما با دستور as f به پایتون می‌گوییم: «فایل را باز کن و رابط آن را در متغیری به نام f بریز».

- حالا f مثل ریموت کنترل فایل عمل می‌کند.

- می‌خواهید بنویسید؟ دستور می‌دهید f.write(...):
- می‌خواهید بخوانید؟ دستور می‌دهید f.read(...):

- نکته: اسم f دلخواه است. می‌توانستید بنویسید as my_file یا as scroll، اما f بین برنامه‌نویسان استاندارد است.

۳. کاراکتر نامرئی \n اینترزدن

متد write() برخلاف print() خیلی ساده است! این متد به صورت پیش‌فرض خط را عوض نمی‌کند و پشت سر هم متنی که بهش دادید رو مینویسه اگر بنویسید:

```
1. f.write("Hello")
2. f.write("World")
```

نتیجه در فایل می‌شود HelloWorld: چسبیده به هم

برای اینکه به سر خط بعد برویم، باید دستی دکمه Enter را بزنیم. در دنیای کدنویسی، دکمه Enter با علامت `\n` (Backslash n) نشان داده می‌شود.

```
1. f.write("Hello\n") # Writes Hello, then hits Enter
2. f.write("World")
```

نتیجه در فایل می‌شود:

```
Hello
World
```

کد کامل و یکپارچه (The Master Spell)

این کد تمام مفاهیم بالا را ترکیب می‌کند. آن را در `project.py` بنویس و اجرا کن. سپس برو و فایل `scroll.txt` که ساخته شده را باز کن و نتیجه رو ببین

```
1. # --- STEP 1: WRITE MODE (Create/Overwrite) ---
2. # We open 'scroll.txt' with "w".
3. # WARNING: If scroll.txt existed, it is now empty!
4. # 'f' is our magic pen to write in this file.
5. with open("scroll.txt", "w") as f:
6.     f.write("Chapter 1: The Beginning\n") # \n moves cursor to next line
7.     f.write("The hero woke up in a dark forest.\n")
8.
9. # --- STEP 2: APPEND MODE (Add to end) ---
10. # We open with "a". The old text stays safe.
11. # We add new lines to the bottom.
12. with open("scroll.txt", "a") as f:
13.     f.write("Chapter 2: The Journey\n")
14.     f.write("He picked up his rusty sword.")
15.
16. # --- STEP 3: READ MODE (Retrieve info) ---
17. # We open with "r". We can ONLY read.
18. print("--- Reading from the Ancient Scroll ---")
19. with open("scroll.txt", "r") as f:
20.     # f.read() pulls ALL text from the file into a variable
21.     full_text = f.read()
22.     print(full_text)
```

۳. مشکل بزرگ: فایل‌ها فقط «متن» می‌فهمند!

بگذارید با یک حقیقت ساده شروع کنیم: فایل‌های متنی (`.txt`)، فقط و فقط می‌توانند رشته (`String`) ذخیره کنند.

در پایتون، ما انواع مختلفی از داده داریم:

- عدد 120 (`Integer`)
- لیست (`List`): `["Ali", "Reza"]`
- دیکشنری (`Dictionary`): `{"name": "Sara"}`

این‌ها برای پایتون معنی‌دار هستند، اما برای یک فایل متنی یا شبکه اینترنت، این‌ها عجیب و غریب‌اند!

اگر سعی کنید یک لیست را مستقیماً در فایل بنویسید، پایتون خطا می‌دهد چون نمی‌داند چگونه یک «مفهوم لیست» را به «متن ساده» تبدیل کند.

```
1. my_list = [1, 2, 3]
2.
3. # ERROR! Files only accept Strings, not Lists.
4. # f.write(my_list) # TypeError: write() argument must be str
```

راه حل: استاندارد JSON جی‌سون

ما باید داده‌هایمان را به یک فرمت متنی استاندارد تبدیل کنیم که همه برنامه‌ها آن را بفهمند. این استاندارد JSON نام دارد.

JSON (جی‌سون) چیست؟

JSON مخفف (JavaScript Object Notation) است، اما اجازه ندهید اسمش شما را بترساند.

JSON فقط یک رشته متنی طولانی (String) است که ساختارش دقیقاً شبیه لیست‌ها و دیکشنری‌های پایتون نوشته شده است.

قوانین سخت‌گیرانه سرزمین JSON

اگرچه JSON خیلی شبیه پایتون است، اما قوانین خشک و سخت‌گیرانه‌ای دارد. وقتی پایتون داده‌های شما را به JSON تبدیل می‌کند، تغییرات زیر اتفاق می‌افتد:

۱. قانون کوتیشن‌ها (فقط جفت!)

- در پایتون: ما آزادی داریم! هم ' (تک) و هم " (جفت) قبول است.
- در JSON: دیکتاتوری است! فقط و فقط دابل کوتیشن " قبول است. تمام تک‌کوتیشن‌ها باید تبدیل شوند.

۲. قانون حقیقت (کوچک و بزرگ)

- در پایتون: مقادیر منطقی با حرف بزرگ شروع می‌شوند True و False.
- در JSON: همه چیز کوچک است true و false.

۳. قانون پوچی (None vs null)

- در پایتون: وقتی چیزی وجود ندارد، می‌گوییم None.
- در JSON: به جای آن از کلمه null استفاده می‌شود.

مثال از جیسون (دقت کنید که این یه رشته string هست):

```
{"id": 101, "is_active": true, "nickname": null, "skills": ["Code", "Eat"]}
```

۴. جعبه ابزار: JSON: چهار طلسم اصلی

پایتون یک کتابخانه داخلی دارد که کار تبدیل را برای ما انجام می دهد. برای استفاده از آن، باید دستور `import json` را بنویسیم.

```
1. import json
```

ما کلاً با ۴ دستور سروکار داریم. راز یادگیری آن ها در حرف s است:

- دستوراتی که آخرشان s دارند (مثل `dumps`, `loads`) برای کار با `String` متن در حافظه هستند.

- دستوراتی که s ندارند (مثل `dump`, `load`) برای کار مستقیم با فایل هستند.

بخش اول: کار در حافظه (تبدیل داده به متن و برعکس)

فرض کنید می خواهید لیست را فقط به متن تبدیل کنید تا در صفحه چاپ کنید (بدون ذخیره در فایل).

۱. طلسم `json.dumps` داده -> متن

این دستور مخفف "Dump String" است. داده پایتونی را می گیرد و خروجی متن می دهد. (طلسم `type(variable)` به ما نوع داده رو نشون میده)

```
1. import json
2.
3. # 1. A Python List (It is ALIVE, we can edit it)
4. my_list = ["Sword", "Shield", 100, True, None]
5.
6. # 2. Convert to JSON String (Freezing it into text)
7. # dumps = Dump String
8. json_string = json.dumps(my_list)
9.
10. print("--- JSON String ---")
11. print(json_string)
12. # Output: ["Sword", "Shield", 100, true, null]
13. # Notice: True -> true, None -> null, Quotes are double (")
14.
15. print(type(json_string))
16. # Output: <class 'str'> (It is just text now!)
```

۲. طلسم `json.loads` متن -> داده

این دستور مخفف "Load String" است. یک متن JSON را می گیرد و دوباره آن را به لیست یا دیکشنری پایتون تبدیل می کند.

```
1. # Imagine we received this text from the internet
2. received_data = '["Ali", "Reza", "Sina"]' # This is a String!
3.
```

```

4. # Convert the text back to a REAL Python List
5. # loads = Load String
6. real_list = json.loads(received_data)
7.
8. print(real_list[0]) # Output: Ali
9. # Now we can use it like a normal list (indexing, loops, etc.)

```

بخش دوم: کار با فایل (ذخیره و بازیابی دائمی)

حالا اگر بخواهیم این داده‌ها را مستقیماً در فایل بریزیم، از دستورات بدون `s` استفاده می‌کنیم. این دستورات کار «باز و بسته کردن پاکت» و «نوشتن / خواندن» را هم خودشان انجام می‌دهند.

۳. طلسم `json.dump` ذخیره در فایل

داده را می‌گیرد + فایل را می‌گیرد -> داده را به جیسون تبدیل کرده و در فایل می‌نویسد.

```

1. import json
2.
3. shopping_list = ["Apple", "Bread", "Milk"]
4.
5. print("Saving list to file...")
6.
7. # Open file in WRITE mode ("w")
8. with open("market.json", "w") as f:
9.     # Magic: Dump the list DIRECTLY into the file 'f'
10.    json.dump(shopping_list, f)
11.
12. print("Saved successfully!")

```

۴. طلسم `json.load` خواندن از فایل

فایل را می‌گیرد -> متن را می‌خواند -> به لیست پایتون یا دیکشنری تبدیل می‌کند و تحویل می‌دهد.

```

1. print("Loading list from file...")
2.
3. # Open file in READ mode ("r")
4. with open("market.json", "r") as f:
5.     # Magic: Read from file 'f' and convert back to Python List
6.     loaded_data = json.load(f)
7.
8. print("List restored:")
9. print(loaded_data)      # Output: ['Apple', 'Bread', 'Milk']
10. print(loaded_data[1])  # Output: Bread

```

خلاصه کلیدی

کاربرد	خروجی چیست؟	ورودی چیست؟	مخفف چیست؟	دستور
تبدیل به متن برای نمایش یا ارسال	متن (String)	داده پایتون	Dump String	<code>json.dumps</code>
تبدیل متنی که داریم به لیست / دیکشنری	داده پایتون	متن (String)	Load String	<code>json.loads</code>
ذخیره مستقیم در فایل	وشتن در فایل	داده + فایل	Dump (File)	<code>json.dump</code>
خواندن مستقیم از فایل	داده پایتون	فایل	Load (File)	<code>json.load</code>

۵. طلسم زبان‌های باستانی: جادوی encoding (مخصوص فارسی)

طلسم بالا یک مشکل پنهان دارد. کد ما برای متن انگلیسی عالی کار کرد. اما اگر بخواهیم کلمات فارسی را ذخیره کنیم چه؟

```
1. # --- Potential Trap! ---
2. with open("farsi_scroll.txt", "w") as f:
3.     f.write("سلام ماجراجو!")
```

ویندوز به صورت پیش فرض مثل یک آدم فقط انگلیسی زبان عمل می‌کند! یعنی اگر فقط حروف انگلیسی بهش بدی اوکیه، ولی اگر فارسی بنویسی، گیج می‌شه و یا بهت خطا می‌ده یا جاش علامت سوال (?) می‌ذاره.

برای حل این مشکل، باید به ویندوز بگیم که استاندارد جهانی (UTF-8) رو استفاده کنه تا همه‌ی حروف دنیا رو بفهمه. برای این کار، موقع باز کردن فایل، فقط کافیه این دستور رو اضافه کنیم:

```
encoding='utf-8'
```

با این کار، کامپیوتر دیگه مشکلی با فارسی نداره و همه چیز درست ذخیره می‌شه و درست خونده میشه.

```
1. # --- The "Safe Way" for all languages ---
2.
3. # Writing Farsi safely
4. with open("farsi_scroll.txt", "w", encoding="utf-8") as f:
5.     f.write("سلام ماجراجو!")
6.
7. # Reading Farsi safely
8. with open("farsi_scroll.txt", "r", encoding="utf-8") as f:
```

```

9.     content = f.read()
10.    print(content) # Output: سلام ماجراجو!

```

قانون ماجراجویی: از این به بعد، همیشه هنگام کار با فایل‌های متنی از "encoding="utf-8" استفاده کن تا مطمئن شوی طلسم تو برای همه‌ی زبان‌ها کار می‌کند.

۶ پروژه: کتاب هیولاشناسی (The Bestiary) – 100 xp

شکارچی هیولا، گوش کن! دنیای ما پر از موجودات خطرناک است: اژدها، زامبی، خون‌آشام و گرگینه. هر کدام از این هیولاها فقط یک «نقطه ضعف» خاص دارند. مثلاً خون‌آشام از "سیر" متنفر است، اما اگر جلوی اژدها "سیر" بگیری، تو را کباب می‌کند! مشکل اینجا است که تعداد هیولاها زیاد است و ماجراجویان معمولاً نقطه ضعف‌ها را فراموش می‌کنند.

امروز می‌خواهیم یک کتاب هیولاشناسی جادویی بسازیم. کتابی که هر وقت هیولای جدیدی دیدیم، نام و نقطه ضعفش را در آن می‌نویسیم و جادوی FILE و JSON آن را برای همیشه حفظ می‌کند. دفعه بعد که هیولا را دیدیم، فقط کافیسٹ اسمش را بگوییم تا کتاب راه شکست دادنش را به ما بگوید.

تصویر نهایی: این برنامه چطور کار می‌کند؟

قبل از کدنویسی، بیا ببیند چرخه حیات این کتاب جادویی را تصور کنیم:

۱. **شروع:** وقتی برنامه را باز می‌کنی، تمام دانش قبلی‌ات از فایل بارگذاری می‌شود.
۲. **یادگیری:** اگر هیولای جدیدی دیدی، اسم و نقطه ضعفش را وارد می‌کنی.
۳. **نبرد:** اگر هیولایی دیدی، اسمش را جستجو می‌کنی و کتاب بلافاصله به تو می‌گوید چطور آن را شکست دهی.
۴. **پایان:** وقتی کتاب را می‌بندی (Exit)، اطلاعات روی هارد دیسک به صورت فایل حک می‌شود.

نقشه و طرح اولیه

در پروژه‌های قبلی از «لیست» استفاده کردیم. اما اینجا لیست به درد نمی‌خورد! چرا؟ چون در لیست، فقط اقلام را پشت سر هم می‌نوشتیم. اما اینجا هر هیولا (کلید) یک نقطه ضعف (مقدار) دارد. پس بهترین ساختار، **دیکشنری (Dictionary)** است.

۱. طلسم بارگذاری: (The Load Spell)

- **کجا؟** اول برنامه.
- **هدف:** فایل bestiary.json را باز کن و دیکشنری را بخوان.

- **سپر محافظ :** اگر فایلی نبود (بار اول)، برنامه نباید خراب شود. در عوض باید یک دیکشنری خالی {} به ما بدهد.

۲. حلقه اصلی و منو: (The Loop)

- برنامه باید همیشه بیدار باشد و ۳ گزینه به کاربر بدهد:
 ۱. افزودن هیولا.
 ۲. جستجوی هیولا.
 ۳. خروج و ذخیره.

۳. طلسم ذخیره: (The Save Spell)

- کجا؟ فقط زمانی که کاربر گزینه ۳ (خروج) را انتخاب کرد.
- **هدف :** دیکشنری را تبدیل به JSON کن و در فایل ذخیره کن تا دانش ما از بین نرود.

چالش‌های فکری

قبل از اینکه کد بزنی، سعی کن این گره‌های ذهنی را باز کنی:

چالش ۱: ساختار دیکشنری چطور یک دیکشنری تعریف می‌کنی که خالی باشد؟ (راهنمایی: استفاده از آکولاد {})

چالش ۲: افزودن دانش اگر بخواهیم در دیکشنری bestiary، برای کلید "Zombie" مقدار "Fire" را ذخیره کنیم، دستور آن چیست؟ (راهنمایی: `bestiary["Zombie"] = "Fire"`)

چالش ۳: جستجوی سریع چطور چک کنیم که آیا یک هیولا در کتاب ما وجود دارد یا نه؟ (راهنمایی: استفاده از دستور `if monster in bestiary`)

چالش ۴: ذخیره دیکشنری آیا `json.dump` می‌تواند دیکشنری را هم مثل لیست ذخیره کند؟ (پاسخ: بله، `JSON` عاشق دیکشنری‌هاست چون ساختارشان خیلی شبیه هم است!)

کد نهایی پروژه (The Monster Hunter's Tool)

این کد را در فایل `bestiary.py` بنویس و اجرا کن .

```
1. import json # Import the magic library to handle data
2.
3. # --- The File Name ---
4. book_file = "bestiary.json"
5.
6. # --- Step 1: The Load Spell ---
7. # We try to open the existing book. If it doesn't exist, we start a new
one.
8. try:
9.     with open(book_file, "r") as f:
10.         bestiary = json.load(f) # Load the dictionary from the file
```

```

11.     print("Bestiary loaded! Ancient knowledge is ready.")
12. except:
13.     # This block runs if the file is missing (first time running)
14.     print("No book found. Starting a new blank Bestiary.")
15.     bestiary = {} # Create an empty dictionary
16.
17. # --- The Main Program Loop ---
18. while True:
19.     print("\n" + "*" * 30)
20.     print("Monster Hunter Menu:")
21.     print("1. Add a new Monster")
22.     print("2. Find Weakness (Search)")
23.     print("3. Save & Exit")
24.
25.     choice = input("Choose an option (1-3): ")
26.
27.     # --- Option 1: Learn a new monster ---
28.     if choice == "1":
29.         monster = input("Enter Monster Name: ")
30.         weakness = input(f"What is the weakness of {monster}? ")
31.
32.         # Save to dictionary (Key = Monster, Value = Weakness)
33.         bestiary[monster] = weakness
34.         print(f"Recorded: {monster} is weak against {weakness}.")
35.
36.     # --- Option 2: Search for battle ---
37.     elif choice == "2":
38.         monster = input("What monster are you fighting? ")
39.
40.         # Check if the monster exists in our dictionary keys
41.         if monster in bestiary:
42.             # Get the value (weakness) using the key (monster name)
43.             print(f"ADVICE: Use [{bestiary[monster]}] to defeat it!")
44.         else:
45.             print("DANGER! Unknown monster. Run away!")
46.
47.     # --- Option 3: The Save Spell ---
48.     elif choice == "3":
49.         print("Saving knowledge to file...")
50.
51.         # Open the file in 'write' mode ('w')
52.         with open(book_file, "w") as f:
53.             json.dump(bestiary, f)
54.
55.         print("Book closed. Goodbye Hunter!")
56.         break # Break the loop to stop the program
57.
58.     else:
59.         print("Invalid option. Try again.")

```

اجرا کن و معجزه را ببین!

۱. برنامه را اجرا کن. چون بار اول است، پیام Starting a new blank Bestiary را می‌بینی

۲. گزینه ۱ را انتخاب کن و بنویس Vampire: و نقطه ضعف Garlic:

۳. دوباره گزینه ۱ را بزن و بنویس Zombie: و نقطه ضعف Fire:

۴. حالا گزینه ۳ را بزن (Save & Exit)

۵. **معجزه:** دوباره برنامه را اجرا کن. گزینه ۲ (Search) را بزن و بنویس Vampire. برنامه به تو می‌گوید! Use [Garlic] to defeat it!

جشن پیروزی و تحلیل

آفرین! تو الان یک **حافظه دائمی** واقعی ساختی. تو دو مفهوم بسیار مهم مهندسی نرم‌افزار را یاد گرفتی:

۱. **Dictionary:** بهترین ابزار برای ذخیره اطلاعاتی که به هم ربط دارند (مثل اسم و نقطه ضعف).

۲. **Persistence ماندگاری:** (اینکه چطور دانش را از ذهن کامپیوتر (RAM) به هارد دیسک منتقل کنی تا هرگز پاک نشود).

حالا تو آماده‌ای که تمام هیولاهای دنیا را شکست دهی، چون حافظهات هرگز پاک نمی‌شود!

🌸 ارتقاء لول: ۴۸ → ۴۰ 🌸

مهارت‌های جدید:

◆ کار با File

با نام باستانی: کتیبه‌های ماندگار
توضیح: آنچه در RAM است می‌پرد، اما
آنچه در فایل است جاودانه می‌ماند.

◆ بلوک Try / Except

با نام باستانی: سپر محافظ
توضیح: برنامه تو دیگر با هر خطایی فرو
نمی‌ریزد. تو بحران‌ها را مدیریت می‌کنی.

فصل ۱۲: دمیدن روح در کد:

خلق موجودات با شیء‌گرایی

مأموریت فصل: طراحی یک سیستم ساده برای مدیریت محصولات یک مغازه جادوگری.

نقشه راه

- ترکیب داده و رفتار: چرا لیست‌ها و دیکشنری‌ها برای پروژه‌های بزرگ کافی نیستند؟
- نقشه ساخت (Class): طراحی دستورالعمل یک موجود روی کاغذ.
- طلسم زنده کردن (Object): تبدیل نقشه به یک موجود واقعی در حافظه.
- کوله پشتی موجودات (Attributes): اضافه کردن داده به شیء به روش دستی.
- تغییر ویژگی‌ها: چطور دارایی‌های یک موجود را در طول زمان تغییر دهیم؟
- رفتارها (Methods): یاد دادن کارهای جادویی و توابع به موجودات.
- راز کلمه self: کلید طلایی برای درک هویت شیء (من کی هستم؟).
- طلسم خلقت (__init__): جادوی اتوماتیک‌سازی برای شروع قدرتمند.
- پروژه: ساخت کلاس Potion و خلق چند معجون واقعی از روی آن برای مغازه.

۱. فراتر از لیست و دیکشنری

تا اینجا سفر، ما از ابزارهای آماده‌ی پایتون استفاده می‌کردیم:

- list مثل یک صف مرتب برای نگه داشتن آیتم‌ها.
- dict مثل یک قفسه با برچسب برای نگه داشتن اطلاعات.

این ابزارها عالی هستند، اما یک مشکل بزرگ دارند: آنها "هوشمند" نیستند. فرض کن می‌خواهی یک بازی بسازی که در آن صدها "هیولا" وجود دارد. هر هیولا نام، جان (HP)، قدرت حمله و سرعت دارد. اگر بخواهی از دیکشنری استفاده کنی، باید برای هر هیولا یک دیکشنری بسازی

```
1. monster1 = {'name': 'Orc', 'hp': 100, 'attack': 15}
2. monster2 = {'name': 'Undead', 'hp': 10, 'attack': 5}
...
```

مشکل کجاست؟

۱. اگر بخواهی هیولاها "حمله کنند" چه؟ دیکشنری‌ها نمی‌توانند حمله کنند؛ آن‌ها فقط داده نگه می‌دارند و احتمالاً نیاز به کلی توابع داری و به برنامه ای که حسابی پیچیده میشه. (مدل کردن نیاز ما که یکسری رفتار هست اینجا خیلی میتونه سخت باشه)

۲. اگر موقع نوشتن کلید 'hp' اشتباه کنی و بنویسی 'HP'، برنامه ارور می‌دهد و پیدا کردنش سخت است. (ساختار پایه ای وجود نداره که یکم نظم بده به همه چی)

اینجاست که شیءگرایی (OOP) مثل یک قهرمان وارد می‌شود. ما نوع داده‌ی مخصوص خودمان مثلاً Monster یا Potion را می‌سازیم که هم داده دارد و هم رفتار! یعنی هم می‌داند چقدر جان دارد و هم بلد است چطور حمله کند. (اصلاً یه چیز عجیب خوبیه)

۲. نقشه ساخت (Class): دستورالعمل

اولین قدم برای خلق کردن هر چیزی (مثلاً یک خانه یا یک ربات)، کشیدن نقشه‌ی آن است.

- **کلاس (Class):** دقیقاً همان "نقشه فنی" یا "دستورالعمل ساخت" است.
- **نکته مهم:** کلاس یه مفهومه و وجود خارجی ندارد و به تنهایی کاری انجام نمی‌دهد. فقط یک کاغذ است که می‌گویی: "اگر خواستی یک معجون بسازی، باید این شکلی باشه".

برای تعریف نقشه، از کلمه کلیدی class استفاده می‌کنیم. (طبق رسم جادوگران پایتون، نام کلاس‌ها را بر خلاف متغیرها، همیشه با حرف بزرگ شروع می‌کنیم.)

```
1. # 1. Defining the Blueprint (The Class)
2. # We tell Python: "I want to design a new TYPE of thing called Potion"
3. class Potion:
```

```
4. pass # 'pass' is a placeholder. It means: "The map is empty for now,
just register the name."
```

۳. طلسم زنده کردن (Object): ساخت موجود واقعی

نقشه به تنهایی کاری نمی‌کند. ما باید از روی نقشه، موجودات واقعی بسازیم.

- **شیء (Object):** موجودی است که از روی نقشه ساخته شده، زنده می‌شود و در دنیای کامپیوتر نفس می‌کشد.

قدرت اصلی کلاس اینجاست: ما می‌توانیم از روی یک **نقشه (Class)**، هزاران **شیء (Object)** متفاوت بسازیم. مثل کارخانه‌ای که از روی یک قالب، هزاران ماشین تولید می‌کند. برای ساختن شیء، نام کلاس را مثل یک تابع صدا می‌زنیم: `()`.

```
1. # 2. Creating Actual Beings (Objects)
2. # Magic happens here: We turn the blueprint into real objects
3.
4. red_potion = Potion() # Object 1: Created and stored in 'red_potion'
5. blue_potion = Potion() # Object 2: Created and stored in 'blue_potion'
6.
7. print(red_potion)
8. print(blue_potion)
```

چیزی که در خروجی می‌بینید معنیش اینه که ما دو آبجکت در آدرس‌های `0x10a2f3c40` ، `0x10a2f3c10` حافظه (RAM) داریم که از کلاس `Potion` ساخته شدن (اون `__main__` هم معنیش اینه که کلاس `Potion` در همین فایل اصلیمون قرار داره یجورایی لوکیشن کلاسه اگه فایل دیگه ای بود اسم اون فایل رو مینوشت)

```
<__main__.Potion object at 0x10a2f3c10>
<__main__.Potion object at 0x10a2f3c40>
```

۴. ویژگی موجودات (Attributes)

موجودات ما ساخته شدند، اما مثل روح سرگردان هستند؛ هیچ مشخصاتی ندارند. بیایید به آن‌ها **ویژگی (Attribute)** بدهیم. هر شیء در پایتون مثل یک "کیف جادویی" است. شما می‌توانید هر زمان که بخواهید، هر متغیری را در آن قرار دهید.

برای دسترسی به داخل شیء، از **نقطه (.)** استفاده می‌کنیم.

- فرمول: `نام-شیء.نام-ویژگی = مقدار`
- ترجمه فرمول: "برو داخل شیء، یه متغیر با این اسم بساز و این مقدار رو بریز توش".

```
1. # 3. Adding Data Manually
2. red_potion = Potion()
3. blue_potion = Potion()
4.
```

```
5. # Let's give them distinct features
6. # We open 'red_potion' and put "Red" in a variable called 'color'
7. red_potion.color = "Red"
8. red_potion.price = 50
9.
10. # We open 'blue_potion' and put "Blue" in a variable called 'color'
11. blue_potion.color = "Blue"
12. blue_potion.price = 100
13.
14. print(red_potion.color) # Output: Red
15. print(blue_potion.price) # Output: 100
```

می‌بینید؟ با اینکه هر دو از نوع Potion هستند، اما اطلاعاتشان کاملاً مستقل است. اگر قیمت معجون قرمز را عوض کنید، قیمت معجون آبی تغییر نمی‌کند.

۵. تغییر ویژگی‌ها: کیمیاگری داده‌ها

ویژگی‌ها ثابت نیستند. همانطور که در بازی وقتی ضربه می‌خورید جانتان (HP) کم می‌شود، مقادیر ویژگی‌های اشیاء هم در طول برنامه قابل تغییر است. کافیسست دوباره با همان علامت مساوی مقدار جدیدی را در ویژگی بریزید. مقدار قبلی پاک شده و مقدار جدید جایگزین می‌شود.

```
1. # 4. Changing Attributes
2. my_potion = Potion()
3. my_potion.level = 1 # Starting level
4.
5. print(f"Level before upgrade: {my_potion.level}")
6.
7. # Magic upgrade! Something happened in the game...
8. my_potion.level = 2 # We overwrite the old value
9. print(f"Level after upgrade: {my_potion.level}")
```

۶. رفتارها: یاد دادن رفتار (Methods)

ما تا اینجا به اشیاءمان "ویژگی (Attributes)" دادیم. اما آن‌ها هنوز بی‌حرکت هستند. بیا ببینیم به آن‌ها یاد بدهیم چطور حرف بزنند یا کاری انجام دهند. در دنیای برنامه‌نویسی، به تابعی که داخل بدنه یک کلاس نوشته می‌شود، **متد (Method)** می‌گوییم. متدها دقیقاً مثل توابع معمولی هستند که در فصل‌های قبل یاد گرفتیم، اما یک تفاوت حیاتی دارند: متدها روی اشیاء کار می‌کنند، پس همیشه باید بدانند **کدام شیء** آن‌ها را صدا زده است (اولین ورودی این متد‌ها همیشه `self` هست که در ادامه باهاش بیشتر آشنا می‌شیم به چه دردی می‌خوره)

```
1. class Potion:
2.     # A method is just a function defined INSIDE the class
```

```

3.     # It defines an action the object can DO.
4.     def drink(self):
5.         print("Gulp gulp... You drank the potion!")
6.
7. # Create object
8. p1 = Potion()
9.
10. # Call the behavior (method) using the dot (.)
11. p1.drink()
12. # Output: Gulp gulp... You drank the potion!

```

۷. راز بزرگ کلمه self من کی هستم؟

این مهم‌ترین و شاید گیج‌کننده‌ترین بخش برای تازه‌کارهاست. خوب دقت کن! چرا در تعریف متد بالا نوشتیم `def drink(self)` ولی موقع صدا زدن نوشتیم `p1.drink()` و چیزی داخل پرانتز نگذاشتیم؟

self چیست؟

- **self** یعنی "خودم"
- وقتی ما یک متد (مثلاً "رنگت را بگو") را طراحی می‌کنیم، نمی‌دانیم قرار است توسط چه شی‌ای صدا زده شود؟ معجون قرمز یا آبی.
- پایتون قانون دارد: "هر وقت متدی روی یک شیء صدا زده شد، خود آن شیء به عنوان اولین ورودی به متد ارسال می‌شود".

مثال: ضربه خوردن در بازی آنلاین

تصور کن در یک بازی آنلاین مثل *Call of Duty* یا *Dota* هستی و هم‌تیمی‌ات دقیقاً کنار تو ایستاده است. هر دوی شما از یک نوع سرباز (یک کلاس مشترک) ساخته شده‌اید.

وقتی دشمن به تو شلیک می‌کند، چرا از «جان» (HP) «هم‌تیمی‌ات کم نمی‌شود؟ چون متد «ضربه خوردن» این‌طوری نوشته شده `self.hp = self.hp - 10`

- وقتی تیر به تو می‌خورد، **self** تبدیل به کاراکتر تو می‌شود و جان تو کم می‌شود.
- وقتی تیر به هم‌تیمی‌ات می‌خورد، **self** تبدیل به کاراکتر او می‌شود و جان او کم می‌شود.

در واقع **self** مثل یک انگشت اشاره است که به پایتون می‌گوید: این تغییر را فقط روی همین کاراکتری که الان تیر خورده اعمال کن، نه بقیه!

```

1. class Potion:
2.     # We MUST pass 'self' as the first argument to access the object's own
   data
3.     def show_color(self):

```

```

4.         # self.color translates to: "Go inside ME and find the 'color'
variable"
5.         print(f"My color is {self.color}")
6.
7. # Setup objects
8. p1 = Potion()
9. p1.color = "Red"
10.
11. p2 = Potion()
12. p2.color = "Blue"
13.
14. # Magic of self
15. p1.show_color()
16. # Python secretly does this: show_color(p1) -> self becomes p1 -> prints
"Red"
17.
18. p2.show_color()
19. # Python secretly does this: show_color(p2) -> self becomes p2 -> prints
"Blue"

```

۸. طلسم خلقت: (__init__) جادوی اتوماتیک

حالا که self را فهمیدیم، یک مشکل کوچک داریم. در بخش ۴ دیدیم که برای ساختن و تنظیم یک معجون، مجبور بودیم ۳ خط کد بنویسیم (ساختن، تنظیم رنگ، تنظیم قیمت). اگر ۱۰۰ تا معجون داشته باشیم، ۳۰۰ خط کد تکراری لازم است!

راه حل چیست؟ **طلسم (__init__ (Constructor))**

این یک متد جادویی است که پایتون قول داده "به محض اینکه یک شیء جدید ساختی، من قبل از هر کاری این متد را برایت اجرا می‌کنم". ما از این فرصت طلایی استفاده می‌کنیم تا ویژگی‌های اولیه (مثل رنگ و قیمت) را همان لحظه اول تنظیم کنیم. خوبیش اینه که زمانی که داری شیء میسازي میتونی مقادیر پاس بدی بهش و همونجا ست کنی)

```

1. class Potion:
2.     # The Constructor Spell (Creating + Setting up)
3.     # We ask for color and price right at the beginning
4.     def __init__(self, color, price):
5.         # Instead of setting them manually later, we do it right here!
6.         self.color = color # Put the 'color' input into 'self.color'
7.         self.price = price # Put the 'price' input into 'self.price'
8.         print(f"A new {self.color} potion was created!")
9.
10.    def show_info(self):
11.        print(f"Details -> Color: {self.color} | Price: {self.price}")
12.
13. # --- OLD WAY (The Hard Way) ---
14. # p1 = Potion()
15. # p1.color = "Red"
16. # p1.price = 50
17.
18. # --- NEW WAY (The Professional Way) ---

```

```

19. # We give the data right when we create the object! One line!
20. p1 = Potion("Red", 50)
21. # Output: A new Red potion was created!
22.
23. p2 = Potion("Blue", 100)
24. # Output: A new Blue potion was created!
25.
26. # Checking if it worked
27. p1.show_info()

```

خلاصه نهایی:

۱. **کلاس (Class):** نقشه و طرح اولیه.
۲. **شیء (Object):** موجود واقعی ساخته شده از نقشه.
۳. **self:** اشاره‌گر به "خود" شیء (کلید دسترسی به حافظه داخلی شیء).
۴. **__init__:** متدی که لحظه تولد شیء اجرا می‌شود تا ویژگی‌های اجباری را بگیرد و ما را از تنظیم دستی نجات دهد.

۹. پروژه: ساخت مغازه جادوگری (The Magic Shop) – 100 xp

ماجراجو، تو آماده‌ای! امروز می‌خواهیم با استفاده از جادوی **کلاس (Class)**، موجوداتی به نام «معجون» خلق کنیم و بهترین مغازه جادویی خود را با آن‌ها پر کنیم.

تصویر نهایی: این برنامه چطور کار می‌کند؟

قبل از کدنویسی، باید یک مفهوم حیاتی را درک کنی: تفاوت **کلاس و شیء**.

۱. **کلاس (Class):** مثل «نقشه ساخت» است. به خودی خود چیزی نیست، فقط یک دستورالعمل است که می‌گوید معجون‌ها چه شکلی هستند.
۲. **شیء (Object):** آن چیزی است که از روی نقشه ساخته می‌شود. مثلاً «معجون سلامتی» یک شیء واقعی است که از روی کلاس معجون ساخته شده.

برنامه نهایی ما ۳ معجون مختلف را از روی یک نقشه می‌سازد و مشخصات آن‌ها را در خروجی چاپ می‌کند.

نقشه و طرح اولیه

بیایید منطق این سیستم را مهندسی کنیم. ما ۲ مرحله داریم: «طراحی نقشه» و «تولید محصول».

۱. **طراحی نقشه (The Blueprint - Class):** باید کلاسی به نام Potion بسازیم. این کلاس دو بخش دارد:

- **طلمس خلقت: (`__init__`)** این یک تابع خاص است که وقتی معجون جدیدی متولد می‌شود، اجرا می‌شود. اینجا مشخصات اولیه (نام، قیمت، اثر) را به معجون می‌دهیم.
- **رفتار: (`display_info`)** یک تابع دیگر که به معجون یاد می‌دهد چطور خودش را معرفی کند (مشخصاتش را چاپ کند).

۲. راه‌اندازی مغازه: (`The Shop`)

- یک لیست خالی (قفسه مغازه) می‌سازیم.
- از روی نقشه (`Class`)، سه معجون واقعی (`Object`) می‌سازیم.
- آن‌ها را در لیست می‌چینیم و با یک حلقه، مشخصات همه را نشان می‌دهیم.

چالش‌های فکری

این مبحث جدید است، پس با دقت به این معماها فکر کن:

چالش ۱: کلمه کلیدی جادویی چطور به پایتون می‌گوییم که می‌خواهی یک «نقشه ساخت» تعریف کنی؟ (راهنمایی: استفاده از کلمه `class`)

چالش ۲: مفهوم `self` ؟ من خودم! در داخل کلاس، کلمه‌ای به نام `self` زیاد دیده می‌شود. `self` یعنی همین شیء که الان داریم می‌سازیم. وقتی می‌نویسیم `self.name = name`، یعنی: نامِ خودِ این معجون را برابر با ورودی بگذار.

چالش ۳: متد `init` چرا اسم این متد عجیب است و دو تا زیرخط (`Underscore`) در دو طرفش دارد؟ (پاسخ: این قانون پایتون برای توابع ویژه است. این متد به طور "خودکار" هنگام ساختن شیء اجرا می‌شود).

چالش ۴: دسترسی به رفتار اگر یک معجون به نام `p1` ساخته باشیم، چطور به او دستور بدهیم که خودش را معرفی کند؟ (راهنمایی: استفاده از نقطه. مثل `p1.display_info()`)

کد نهایی پروژه (`The Magic Shop`)

وقت خلقت است! کد زیر را با دقت در فایل `project.py` بنویس. سعی کن رابطه بین «کلاس» (بالای کد) و «اشیاء» (پایین کد) را درک کنی.

```
1. # --- The Adventurer's Magic Shop ---
2.
3. # 1. --- The Blueprint (Class) ---
4. # This is not a potion yet. It's the RECIPE for making potions.
5. class Potion:
```

```

6.
7.     # 2. --- The Constructor (__init__) ---
8.     # This runs automatically when we create a new Potion()
9.     # 'self' means: "The specific object being created right now"
10.    def __init__(self, name, price, effect):
11.        # 3. --- The Attributes (Data) ---
12.        # We attach data to the object using 'self.'
13.        self.name = name
14.        self.price = price
15.        self.effect = effect
16.
17.    # 4. --- The Behavior (Method) ---
18.    # A function inside a class is called a "Method"
19.    def display_info(self):
20.        print(f"--- {self.name} ---")
21.        print(f"Effect: {self.effect}")
22.        print(f"Price: {self.price} gold")
23.        print("=====")
24.
25.
26. # 5. --- Create the Shop's Stock (List) ---
27. magic_shop_stock = []
28.
29. print("Brewing new potions for the shop...")
30.
31. # 6. --- Create Beings (Objects) ---
32. # Now we use the Class (Blueprint) to make REAL objects
33. health_potion = Potion("Health Potion", 50, "Heals 25 HP")
34. mana_potion = Potion("Mana Potion", 70, "Restores 50 MP")
35. invisibility_potion = Potion("Invisibility Elixir", 300, "Grants 30s
invisibility")
36.
37. # 7. --- Fill the Shelves ---
38. # Add our new objects to the list
39. magic_shop_stock.append(health_potion)
40. magic_shop_stock.append(mana_potion)
41. magic_shop_stock.append(invisibility_potion)
42.
43. print("")
44. print("WELCOME TO THE MAGIC SHOP!")
45.
46. # 8. --- Open the Shop (Loop) ---
47. for potion in magic_shop_stock:
48.     # We ask EACH potion object to display its own info
49.     potion.display_info()

```

اجرا کن و نتیجه را ببین! تو همین الان یک سیستم کامل ساختی. به خروجی نگاه کن. با اینکه ما فقط یک بار تابع `display_info` را نوشتیم، اما هر معجون اطلاعات منحصر به فرد خودش را چاپ می‌کند. چرا؟ چون هر «شیء» حافظه مخصوص به خودش را دارد.

جشن پیروزی و تحلیل

آفرین! تو همین الان به کد خود «روح» دمیدی و وارد دنیای قدرتمند شیءگرایی (OOP) شدی. تا قبل از این، کدهای تو مثل دستور پخت غذا بودند (یک سری دستور پشت سرهم). اما الان، کدهای تو مثل موجودات زنده هستند. تو موجودی به نام Potion خلق کردی که می‌داند قیمتش چقدر است و می‌داند چگونه خودش را معرفی کند. این همان روشی است که بازی‌های بزرگ کامپیوتری و نرم‌افزارهای پیچیده با آن ساخته می‌شوند.

ارتقاء لول: ۶۰ → ۴۸

مهارت‌های جدید:

◆ مفهوم Class

با نام باستانی: نقشه‌ی خلقت
توضیح: تو دیگر فقط کد نمی‌نویسی،
موجودات را طراحی می‌کنی.

◆ مفهوم Object

با نام باستانی: تولد موجود
توضیح: تبدیل نقشه ذهنی به موجودی
زنده که رفتار دارد.

◆ سطح جدید: D

توضیح: تبریک به سطح
جدیدی رسید.

مأموریت‌های جانبی: تالار پیشرفته

منطقه: دروازه ورود به بُعد چهارم (فایل‌ها و کلاس‌ها)

ماجراجو! تو اکنون قدرت‌های بزرگی داری: می‌توانی اطلاعات را روی (فایل‌ها) ذخیره کنی و موجودات زنده (کلاس‌ها) خلق کنی. در اینجا ۱۰ چالش دشوار وجود دارد که تمام مهارت‌های تو را آزمایش می‌کند.

قانون تالار: کدها را خودت بنویس، فایل‌های لازم را بساز و حتماً از کلاس‌های نمونه (Object) بساز!

مأموریت‌ها

۱. مأموریت: دفتر خاطرات ابدی (The Eternal Diary) – 20 xp

ماجرای جوان نباید خاطراتشان را فراموش کنند.

- **وظیفه:** برنامه‌ای بنویس که متنی را از کاربر بگیرد و به انتهای یک فایل متنی (diary.txt) اضافه کند. (Append)
- **چالش:** تاریخ و ساعت دقیق ثبت خاطره را هم به صورت خودکار بالای متن بنویس.

۲. مأموریت: جستجوگر گنج (The Word Hunter) – 20 xp

در میان طومارهای قدیمی به دنبال کلمات خاص بگرد.

- **وظیفه:** یک فایل متنی بساز و متنی داخلش بنویس. برنامه باید یک کلمه از کاربر بگیرد و بگوید آیا این کلمه در فایل وجود دارد یا خیر (فقط بگوید Found یا Not Found)

۳. مأموریت: کپی‌کار معکوس (Mirror Copier) – 20 xp

رمزنگاری به سبک لئوناردو داوینچی!

- **وظیفه:** محتویات یک فایل (source.txt) را بخوان. تمام متن را برعکس کن (از حرف آخر به اول) و نتیجه را در یک فایل جدید (mirror.txt) ذخیره کن.

۴. مأموریت: استخراج‌گر ایمیل (Email Extractor) – 20 xp

در یک متن شلوغ، آدرس‌های ارتباطی را پیدا کن.

- **وظیفه:** یک متن طولانی (شامل چند ایمیل) در فایل باشد. برنامه فایل را بخواند و تمام کلماتی که شبیه ایمیل هستند (دارای @ و نقطه را جدا کرده و در یک فایل جدید لیست کند)

۵. مأموریت: حیوان خانگی مجازی (Virtual Pet) – 20 xp

اولین موجود زنده خودت را خلق کن.

- **وظیفه:** کلاسی به نام Pet بساز با ویژگی‌های name, hunger, energy.
- **متد:** eat(): گرسنگی را کم کند.
- **متد:** sleep(): انرژی را زیاد کند.
- **متد:** play(): انرژی را کم و گرسنگی را زیاد کند.
- **چالش:** یک آبجکت از این حیوان بساز و با او بازی کن!

۶. مأموریت: تایمر بمب ساعتی (Countdown Timer) – 20 xp

زمان در حال گذر است... تیک تاک!

- **وظیفه:** کلاسی بساز که یک عدد شروع (مثلاً ۱۰) می‌گیرد. هر بار که متد tick() صدا زده شود، عدد را یکی کم کرده و چاپ کند. وقتی به صفر رسید، چاپ کند. "BOOM!"

۷. مأموریت: بانک کوتوله‌ها (Dwarf Bank Account) – 20 xp

سیستم بانکداری امن در اعماق کوهستان.

- **وظیفه:** کلاسی به نام BankAccount بساز با ویژگی balance موجودی.
- متد deposit(amount): پول واریز کند.
- متد withdraw(amount): پول برداشت کند (اگر موجودی کافی نبود، پیام خطا بدهد).
- **چالش:** یک حساب بساز، پول واریز کن و سپس برداشت کن.

۸. مأموریت: تولیدکننده گزارش (Report Generator) – 20 xp

لیست نمرات جادوگران جوان نیاز به بررسی دارد.

- **وظیفه:** کلاسی به نام Student (نام و نمره) بساز. یک لیست از چند دانش‌آموز (آبجکت) ایجاد کن. برنامه باید اسامی کسانی که نمره بالای ۱۰ دارند را در یک فایل متنی جدید (passed.txt) ذخیره کند.

۹. مأموریت: آزمون‌گیر از فایل (Quiz Loader) – 20 xp

برگزاری امتحان بدون دخالت انسان.

- **وظیفه:** سوالات و جواب‌ها را در یک فایل متنی ذخیره کن مثلاً: پایتون چیست؟ @ زبان برنامه نویسی برنامه باید خط به خط سوال را بخواند، از کاربر بپرسد و اگر جواب درست بود، امتیاز بدهد. (سوال و جواب با @ از هم جدا شدن)

۱۰. مأموریت: سیستم ذخیره بازی (Save Game System) – 20 xp

قهرمان خسته است و می‌خواهد استراحت کند.

- **وظیفه:** یک کلاس Hero بساز (با نام و مقدار جان hp)
- متد save_to_file(): وضعیت فعلی قهرمان را در فایل بنویسد.

- متد `load_from_file()` اطلاعات را از فایل بخواند و قهرمان را با همان وضعیت برگرداند.

آیا سفرت را در این مرحله کامل کرده‌ای، ماجراجو؟ اگر بله، کتاب را ورق بزن؛ کتابخانه سایه‌ها (نتیجه مأموریت‌ها) در صفحه بعد انتظار تو را می‌کشد .

کتابخانه ساید (نتیجه مأموریت‌ها)

۱. مأموریت: دفتر خاطرات ابدی

نکته: برای دسترسی به زمان از ماژول `datetime` و برای اضافه کردن به فایل از حالت `a` (`append`) استفاده می‌کنیم.

```
1. import datetime
2.
3. text = input("Write your memory: ")
4. current_time = str(datetime.datetime.now())
5.
6. with open("diary.txt", "a") as file:
7.     file.write(f"\nDate: {current_time}\n")
8.     file.write(f"{text}\n")
9.     print("Memory saved.")
```

۲. مأموریت: جستجوگر گنج

نکته: عملگر `in` می‌تواند وجود یک کلمه را در کل متن فایل بررسی کند.

```
1. search_word = input("Enter word to search: ")
2.
3. with open("my_file.txt", "r") as file:
4.     content = file.read()
5.
6.     if search_word in content:
7.         print("Found!")
8.     else:
9.         print("Not Found.")
```

۳. مأموریت: کپی‌کار معکوس

نکته: اسلایسینگ `[::-1]` کل متن را معکوس می‌کند.

```
1. # Read from source
2. with open("source.txt", "r") as reader:
3.     content = reader.read()
4.
5. reversed_content = content[::-1]
6.
7. # Write to mirror file
8. with open("mirror.txt", "w") as writer:
9.     writer.write(reversed_content)
10.    print("Mirror file created.")
```

۴. مأموریت: استخراج‌گر ایمیل

نکته: با `split` متن را به کلمات تقسیم می‌کنیم و شرط‌های ساده را چک می‌کنیم.

```
1. with open("data.txt", "r") as file:
2.     words = file.read().split()
```

```
3.
4. with open("emails.txt", "w") as output:
5.     for word in words:
6.         if "@" in word and "." in word:
7.             output.write(word + "\n")
8.
9. print("Emails extracted.")
```

۵. مأموریت: حیوان خانگی مجازی

نکته: بعد از تعریف کلاس، حتماً باید یک نمونه (آبجکت) از آن بسازیم تا کار کند.

```
1. class Pet:
2.     def __init__(self, name):
3.         self.name = name
4.         self.hunger = 50
5.         self.energy = 50
6.
7.     def eat(self):
8.         self.hunger -= 10
9.         print(f"{self.name} is eating...")
10.
11.    def play(self):
12.        self.energy -= 10
13.        print(f"{self.name} is playing...")
14.
15. # --- Object Creation ---
16. dog = Pet("Rex") # create object
17. dog.eat()        # call eat
18. dog.play()
```

۶. مأموریت: تایمر بمب ساعتی

نکته: آبجکت وضعیت خودش (self.time) را نگه می‌دارد.

```
1. class Bomb:
2.     def __init__(self, start_time):
3.         self.time = start_time
4.
5.     def tick(self):
6.         if self.time > 0:
7.             self.time -= 1
8.             print(f"Tick... {self.time}")
9.         if self.time == 0:
10.            print("BOOM!")
11.
12. # --- Object Creation ---
13. tnt = Bomb(3)
14. tnt.tick() # 2
15. tnt.tick() # 1
16. tnt.tick() # BOOM!
```

۷. مأموریت: بانک کوتوله‌ها

نکته: با ساختن چند آبجکت، می‌توانیم چند حساب بانکی جداگانه داشته باشیم.

```

1. class BankAccount:
2.     def __init__(self, owner, balance):
3.         self.owner = owner
4.         self.balance = balance
5.
6.     def deposit(self, amount):
7.         self.balance += amount
8.         print(f"New Balance: {self.balance}")
9.
10.    def withdraw(self, amount):
11.        if amount <= self.balance:
12.            self.balance -= amount
13.            print(f"Withdrawn. Remaining: {self.balance}")
14.        else:
15.            print("Error: Not enough gold!")
16.
17. # --- Object Creation ---
18. my_acc = BankAccount("Gimli", 100)
19. my_acc.deposit(50) # balance = 150
20. my_acc.withdraw(200) # error!

```

۸. مأموریت: تولیدکننده گزارش

نکته: اینجا لیستی از آبجکت‌ها داریم s در حلقه for، یک آبجکت است.

```

1. class Student:
2.     def __init__(self, name, grade):
3.         self.name = name
4.         self.grade = grade
5.
6. # --- List of Objects ---
7. students = [
8.     Student("Ali", 15),
9.     Student("Sara", 9),
10.    Student("Reza", 20)
11. ]
12.
13. with open("passed.txt", "w") as file:
14.     for s in students:
15.         if s.grade > 10:
16.             file.write(s.name + "\n")

```

۹. مأموریت: آزمون‌گیر از فایل

نکته: فرض می‌کنیم در فایل quiz.txt خطوط به صورت Question, Answer ذخیره شده‌اند.

```

1. score = 0
2. with open("quiz.txt", "r") as file:
3.     for line in file:
4.         data = line.strip().split(",")
5.         question = data[0]
6.         correct_answer = data[1]
7.

```


مأموریت‌های جانبی: تالار پیشرفته / ۱۶۱

```
8.         user_ans = input(f"{question} ")
9.         if user_ans == correct_answer:
10.             score += 1
11.
12. print(f"Final Score: {score}")
```

۱۰. مأموریت: سیستم ذخیره بازی

نکته: ببینید چطور متد save مقادیر self را در فایل می‌نویسد.

```
1. class Hero:
2.     def __init__(self, name, hp):
3.         self.name = name
4.         self.hp = hp
5.
6.     def save_to_file(self):
7.         with open("savefile.txt", "w") as f:
8.             f.write(f"{self.name},{self.hp}")
9.         print("Game Saved.")
10.
11. # --- Object Creation ---
12. player = Hero("Aragorn", 100)
13. player.save_to_file()
```

بخش چهارم: آیین‌های جادوگر ارشد

تبریک میگم! الان که با طلسم‌های پایه آشنا شدی، نوبت اونه که که وارد لول بالاتر بشی و هنر «معماری» رو یاد بگیری. در این مرحله، یاد می‌گیری چطور کدهایت را منظم کنی، زمان رو کنترل کنی و کدهایت را مانند یک استاد واقعی پایتون، شیک و حرفه‌ای بنویسی.

فصل ۱۳: طلسم تفکیک: هر کد سر جای خودش

مأموریت فصل: فایل `project.py` ما به یک کد درهم پیچیده تبدیل شده! باید یاد بگیریم چگونه
طلسم‌ها و نقشه‌های ساخت (کلاس‌ها) خودمون رو توی فایل‌های جداگانه مرتب کنیم.

نقشه راه:

- چرا یک فایل کافی نیست؟ (اصل تفکیک مسئولیت‌ها).
- طلسم `import` شخصی: چگونه فایل `spells.py` خودمان را بسازیم و آن را در `main.py` وارد (`import`) کنیم.
- احضار مستقیم: جادوی `from ... import`
- **پروژه:** معماری «مغازه جادوگری»

۱. چرا یک فایل کافی نیست؟ (اصل تفکیک مسئولیت‌ها)

ماجرای ما، تا الان تمام کدها، کلاس‌ها و پروژه‌هایمان را در یک فایل به نام `project.py` نوشته‌ایم. این کار برای شروع عالی بود، اما حالا که به یک جادوگر باتجربه تبدیل شده‌اید، با یک مشکل جدید روبرو می‌شوید: **درهم‌پیچیدگی**.

تصور کنید یک جادوگر تمام طلسم‌های خود را از جادوی درمان گرفته تا گلوله‌های آتشین و طلسم‌های پرواز را روی یک ورق بی‌انتهای هزارمتری بنویسد. چه اتفاقی می‌افتد؟ **پیدا کردن سخت می‌شود**: اگر فقط بخواهید طلسم «درمان زخم ساده» را پیدا کنید، باید ساعت‌ها جستجو کنید.

خطرناک است: هنگام ویرایش طلسم آتش، ممکن است تصادفاً جوهر روی طلسم پرواز بریزید و آن را تغییر دهید.

قابل اشتراک‌گذاری نیست: اگر جادوگر دیگری فقط به طلسم‌های درمان شما نیاز داشته باشد، شما نمی‌توانید آن بخش از ورق را به او بدهید؛ باید کل ورق آشفته را به او بدهید.

در برنامه‌نویسی، این اصل رو «**تفکیک مسئولیت‌ها**» (**Separation of Concerns**) می‌گیم. هر فایل (که ما آن را ماژول می‌نامیم) باید یک مسئولیت واحد و شفاف داشته باشد.

- یک فایل برای نقشه‌های ساخت (کلاس‌ها).
 - یک فایل برای طلسم‌های کمکی (توابع).
 - و یک فایل اصلی (`main.py`) که جادو را اجرا می‌کند.
- در این فصل، ما یاد می‌گیریم که چگونه مانند یک کتابدار، کتابخانه جادویی خود را سازماندهی کنیم.

۲. طلسم `import` شخصی (ساخت اولین ماژول)

ما قبلاً از طلسم `import` برای احضار کتابخانه‌های قدرتمند پایتون مانند `random` و `json` استفاده کرده‌ایم. خبر خوب این است: هر فایل `py` که خودتان می‌سازید، یک «ماژول» است و شما می‌توانید آن را `import` کنید!

بیایید امتحان کنیم:

فرض کنید ما یک طلسم کمکی داریم که بارها از آن استفاده می‌کنیم بیایید آن را به کتاب فایل جداگونه‌ای منتقل کنیم.

۱. **کتاب طلسم (`spells.py`)**: یک فایل جدید به نام `spells.py` بسازید و این کد را در آن بنویسید:

```
1. # spells.py
2. # This is our library for helper spells
3.
4. def print_divider(character="="):
5.     """
6.     Prints a neat divider.
7.     """
8.     print(character * 30)
9.
10. def greet_adventurer(name):
11.     """
12.     Welcomes a new adventurer.
13.     """
14.     print(f"Greetings, {name}! Welcome to the guild.")
```

برای بچه های colab و ژوپیتِر:

در ابتدای سلول کد بالا %%writefile spells.py رو اضافه کنید تا کد اجرا نشه و فقط داخل فایل spells.py ذخیره بشه

۲. فایل اصلی (main.py): حالا، یک فایل جدید دیگر به نام main.py در همان پوشه بسازید. برای استفاده از آن طلسم‌ها، ما آن‌ها را import می‌کنیم:

```
1. # main.py
2. # Our main program
3.
4. # We import the spells.py module (file) that we just created
5. import spells
6.
7. print("Main program started...")
8.
9. # To use the spells, we must reference the book (module) name
10. # Just like we did with random.choice()
11. spells.greet_adventurer("Aria")
12. spells.print_divider()
13. spells.greet_adventurer("Gandalf")
14. spells.print_divider("-")
```

اجرا کن و نتیجه را ببین! وقتی main.py را اجرا می‌کنید، خروجی این خواهد بود:

```
Main program started...
Greetings, Aria! Welcome to the guild.
=====
Greetings, Gandalf! Welcome to the guild.
-----
```

ما با موفقیت جادوهایمان را «تفکیک» کردیم! main.py ما اکنون تمیز و خواناست و منطق اصلی را نشان می‌دهد، در حالی که spells.py تمام جزئیات پیاده‌سازی را پنهان می‌کند.

به نظرت اینجوری تمیز تر نیست ؟ اگه جوابت بله هست سعی کن تا جای ممکن فایل های طولانی کدت رو به چندین فایل کوتاه تر که هر کدوم کار خاصی رو میکنه تقسیم کنی.

۳. احضار مستقیم: جادوی from ... import

در روش قبلی، ما کل کتابچه spells را همراهمان می آوردیم و هر بار که می خواستیم طلسمی بخوانیم، باید اشاره می کردیم که این طلسم از کدام کتاب است (مثلاً spells.print_divider) اما گاهی اوقات شما یک طلسم خاص را آنقدر زیاد استفاده می کنید که دوست ندارید هر بار نام کتاب را صدا بزنید. می خواهید آن طلسم را از کتاب بیرون بکشید و توی جیبتان بگذارید تا سریع تر به آن دسترسی داشته باشید!

برای این کار از دستور from ... import استفاده می کنیم. بیایید فایل main.py را تغییر دهیم تا ببینیم چطور کار می کند:

```
1. # main.py
2. # The Shortcut Way
3.
4. # Instead of importing the whole book, we pick specific spells
5. from spells import greet_adventurer, print_divider
6.
7. print("Main program started...")
8.
9. # Now we can use the spells DIRECTLY!
10. # No need to type 'spells.' anymore.
11. greet_adventurer("Lara Croft")
12.
13. print_divider("")
14.
15. greet_adventurer("Harry Potter")
```

تفاوت اصلی چیست؟

- در روش اول: (import spells) شما کتاب را دستتان می گیرید. برای استفاده باید بگویید: از کتاب spells، طلسم greet را بخوان.
- در روش دوم: (from spells import ...) شما طلسم را حفظ می کنید! حالا مستقیماً می گوئید: طلسم greet را بخوان.

هشدار جادوگر باهوش: استفاده از from ... import کد را کوتاه تر می کند، اما باید مراقب باشید! اگر در فایل اصلی خودتان تابعی به نام greet_adventurer داشته باشید و یکی هم از spells ایمپورت کنید، پایتون گیج می شود (تداخل نام ها). پس اگر تداخل و نام تکراری وجود داشت از روش اول بربید.

۴. پروژه: معماری «مغازه جادوگری» – 100 xp

مأموریت: هدف ما در این پروژه افتتاح یک مغازه واقعی است. ما نمی‌خواهیم فقط یک معجون بسازیم؛ ما می‌خواهیم یک لیست موجودی (Inventory) داشته باشیم که شامل چندین معجون مختلف (مثل معجون سلامتی، مانا و قدرت) باشد. برنامه ما باید بتواند به صورت خودکار تمام معجون‌های موجود در انبار را یکی یکی بردارد و مشخصات آن‌ها (نام، قیمت و اثر جادویی) را در خروجی چاپ کند تا مشتری بتواند انتخاب کند. اما چالش اصلی اینجاست: ما می‌خواهیم این کار را مثل یک معمار حرفه‌ای انجام دهیم و کدهایمان را دسته‌بندی کنیم نه اینکه همه رو تو یه فایل کد بزنیم!

تصویر نهایی:

وقتی برنامه اجرا شود چه می‌بینیم؟

قبل از اینکه کد بزنیم، بیایید ببینیم نتیجه نهایی باید چه شکلی باشد. وقتی فایل اصلی برنامه را اجرا می‌کنی، انگار کرکره مغازه را بالا داده‌ای. برنامه باید:

۱. به مشتری خوش‌آمد بگوید

۲. به انبار برود و لیست معجون‌ها را چک کند.

۳. ویتترین مغازه را بچیند؛ یعنی اطلاعات هر ۳ معجون را پشت سر هم و مرتب در صفحه (ترمینال) چاپ کند.

خروجی نهایی در ترمینال دقیقاً این شکلی خواهد بود:

```
--- Welcome to the Adventurer's Magic Shop! ---
Stocking the shelves...
--- Today's Wares ---
--- Health Potion ---
  Effect: +25 HP
  Price: 50 Gold
=====
--- Mana Potion ---
  Effect: +50 MP
  Price: 70 Gold
=====
--- Potion of Strength ---
  Effect: +5 Strength for 3 min
  Price: 150 Gold
=====
```

نقشه و طرح اولیه

ما باید کدمان را در دو فایل جداگانه مدیریت کنیم:

۱. فایل نقشه: (potions.py)

- این فایل فقط حاوی کلاس Potion است.
- هیچ دستوری اجرا نمی‌شود (پرینت نمی‌کند)، فقط تعریف می‌کند که معجون چیست.

۲. فایل مغازه: (shop.py)

- این فایل اصلی ماست.
- اول باید کلاس Potion را ایمپورت کند. (Import)
- سپس معجون‌ها را می‌سازد. (Object Creation)
- و در آخر ویتترین مغازه را نمایش می‌دهد.

چالش‌های فکری

قبل از اینکه کدها را ببینی، این گره‌های ذهنی را باز کن:

چالش ۱: تفکیک وظایف چرا کلاس را در یک فایل جدا می‌گذاریم؟ (پاسخ: تا اگر فردا خواستیم یک بازی جنگی بسازیم، بتوانیم همین فایل *potions.py* را آنجا هم استفاده کنیم بدون اینکه کدهای مغازه مزاحمان شوند)

چالش ۲: طلسم واردات (Import) در فایل مغازه، با چه دستوری می‌توانی به پایتون بگویی: «برو از فایل *potions*، کلاس *Potion* را بیاور!»؟ (راهنمایی: *from ... import ...*)

چالش ۳: اجرای اشتباه! اگر فایل *potions.py* را اجرا کنی چه اتفاقی می‌افتد؟ (پاسخ: هیچ! چون این فایل فقط یک تعریف است و دستوری برای اجرا ندارد. همیشه باید فایل اصلی (*shop.py*) اجرا شود.)

کد نهایی پروژه (The Architect's Blueprint)

فایل ۱: *potions.py* (کتابچه نقشه ساخت ما)

```
1. # potions.py
2. # This module only contains the blueprints (classes)
3. # for magical items.
4.
5. class Potion:
6.     """
7.     A blueprint for a magical potion in the shop.
8.     """
9.
10.    # 1. The Constructor Spell
11.    def __init__(self, name, price, effect):
```



```
12.         # 2. The Attributes
13.         self.name = name
14.         self.price = price
15.         self.effect = effect
16.
17.         # 3. The Behavior (Method)
18.         def display_info(self):
19.             """Prints the potion's stats neatly."""
20.             print(f"--- {self.name} ---")
21.             print(f"    Effect: {self.effect}")
22.             print(f"    Price: {self.price} Gold")
23.             print("=====")
```

برای بچه های colab و ژوپیتتر:

در ابتدای سلول کد بالا `%%writefile potions.py` رو اضافه کنید تا کد اجرا نشه و فقط داخل فایل `potions.py` ذخیره بشه

فایل ۲: `shop.py` (طلسم اصلی و ویتترین مغازه ما)

```
1. # shop.py
2. # The main application for our magic shop.
3.
4. # 1. Summon the blueprint from our other file
5. # We import the Potion class directly from the potions.py file
6. from potions import Potion
7.
8. print("--- Welcome to the Adventurer's Magic Shop! ---")
9. print("Stocking the shelves...")
10.
11. # 2. Create the shop's inventory (a list)
12. magic_shop_stock = []
13.
14. # 3. Create the beings (Objects) from the blueprint
15. health_potion = Potion(name="Health Potion", price=50, effect="+25 HP")
16. mana_potion = Potion(name="Mana Potion", price=70, effect="+50 MP")
17. strength_potion = Potion(name="Potion of Strength", price=150, effect="+5 Strength for 3 min")
18.
19. # 4. Fill the shelves
20. magic_shop_stock.append(health_potion)
21. magic_shop_stock.append(mana_potion)
22. magic_shop_stock.append(strength_potion)
23.
24. print("--- Today's Wares ---")
25.
26. # 5. Open the shop and display the items
27. for item in magic_shop_stock:
28.     # We call the display_info() method on EACH potion object
29.     item.display_info()
```

اجرا کن و نتیجه را ببین!

فایل shop.py را اجرا کن (نه potions.py). خواهی دید که برنامه اصلی ما به potions.py نگاه می‌کند، کلاس Potion را قرض می‌گیرد و مغازه را با موفقیت باز می‌کند. خروجی:

```
--- Welcome to the Adventurer's Magic Shop! ---
Stocking the shelves...
--- Today's Wares ---
--- Health Potion ---
    Effect: +25 HP
    Price: 50 Gold
=====
--- Mana Potion ---
    Effect: +50 MP
    Price: 70 Gold
=====
--- Potion of Strength ---
    Effect: +5 Strength for 3 min
    Price: 150 Gold
=====
```

جشن پیروزی و تحلیل

آفرین! تو همین الان از یک جادوگر ساده که همه چیز را در یک فایل می‌نویسد، به یک **معمار نرم‌افزار** تبدیل شدی. تو یاد گرفتی چطور کدهایت را سازماندهی کنی. حالا اگر بخواهی ۱۰۰ نوع آیتم جادویی مختلف (شمشیر، سپر، کلاه) داشته باشی، هر کدام را در فایل مخصوص خودش میزاری و فایل اصلی‌ات همیشه تمیز و خلوت باقی می‌ماند

🌸 ارتقاء لول: ۶۵ → ۶۰ 🌸

مهارت‌های جدید:

◆ ماژول‌سازی Modularization

با نام باستانی: معماری شهرها

توضیح: تقسیم یک قلعه بزرگ به ساختمان‌های

کوچک‌تر. نظم در اوج پیچیدگی.

فصل ۱۴: کتاب جادویی زمان

مأموریت فصل: تا به حال، ابزارهای ما هیچ درکی از «زمان» نداشته‌اند. در این فصل، یاد می‌گیریم چطور بفهمیم الان چه ساعت و تاریخیه؟، تاریخ‌ها را محاسبه کنیم و برای مأموریت‌هایمان «ددلاین» تعیین کنیم.

نقشه راه:

- احضار `datetime`: وارد کردن ماژول استاندارد `datetime`.
- طلسم `now()`: گرفتن تاریخ و زمان دقیق همین لحظه.
- قالب‌بندی زمان (`strftime`): تبدیل تاریخ به یک «متن» خوانا.
- سفر در زمان (`timedelta`): محاسبه زمان‌های آینده یا گذشته.
- پروژه: ساخت «اعلان‌گر ددلاین».

۱. احضار datetime

تا به حال، کدهای جادویی ما از زمان استفاده نکردن. هیچ‌کدام نمی‌دانند الان صبح است یا شب. برای حل این مشکل، باید یکی از قدرتمندترین کتاب‌های جادو در کتابخانه استاندارد پایتون را احضار کنیم: `datetime`.

مانند `random` و `json`، این کتاب طلسم از قبل با پایتون روی سیستم شما نصب شده. فقط کافیست آن را `import` کنید. این ماژول طلسم‌های قدرتمندی برای مدیریت تاریخ و زمان به ما می‌دهد.

```
1. # We summon the entire datetime spellbook
2. import datetime
```

حالا، کد ما از زمان و تاریخ الان آگاه است (ساعت و تاریخ رو از سیستم شما می‌گیره پس اگه توی ویندوز ساعت ۱۰ عه اینجام همونه دقیقا)!

۲. طلسم `now()` (گرفتن لحظه حال)

اولین طلسمی که هر جادوگر زمان یاد می‌گیرد این است: «الان ساعت چنده؟»
ماژول `datetime` (فایل) شامل یک کلاس قدرتمند هم‌نام `datetime` است. (میدونم عجیبه کلاس دقیقا هم اسم ماژوله ولی خب اینجوری ساختنش دیگه)
برای گرفتن لحظه دقیق، از متد `now()` آن استفاده می‌کنیم:

```
1. import datetime
2.
3. # Ask the datetime class for the current moment
4. current_moment = datetime.datetime.now()
5.
6. print("The magical clock reads:")
7. print(current_moment)
```

خروجی (برای شما متفاوت خواهد بود):

```
2066-11-16 06:24:08.123456
```

این `123456`... خوب نیست. این یک شیء از کلاس `datetime` است، نه یک متن خوانا برای اینکه خوانایی و اطلاعاتی از زمان که دوست داریم رو نمایش بدیم مثلا فقط سال و ماه نیاز داریم از قالب بندی استفاده کنیم و به این کلاس بگیم که توی یه قالب خاصی بھون این اطلاعات رو بده.

اگه براتون جالبه اینکه وقتی شی رو پرینت میکنیم یه حالت خاصی به خودش گرفته مال متد `__str__` داخل اون کلاس هست و خودتون هم میتونید کاستوم کنید وقتی یه شی از کلاس شما پرینت قراره بشه به حالت خاصی دربیاد)

۳. قالب‌بندی زمان (طلسم strftime)

اون متن ۲۰۶۶-۱۱-۱۶... یک شیء زمانه. ما باید اون رو به یک رشته (String) قالب‌بندی کنیم. برای این کار، از طلسم (String Format Time) `strftime` استفاده می‌کنیم. این طلسم یک «رشته قالب‌بندی» به عنوان ورودی می‌گیرد که به اون نیگه چجوری و با چه فرمتی زمان رو بنویسه، با استفاده از نماد های جادویی خاص (نماد هایی که با % شروع می‌شوند):

```
%Y: سال کامل (2066)
%m: شماره ماه (01-12)
%d: روز ماه (01-31)
%A: نام کامل روز هفته (Sunday)
%B: نام کامل ماه (November)
```

کد ما:

```
1. import datetime
2.
3. current_moment = datetime.datetime.now()
4.
5. # Let's format this object into a readable string
6.
7. # Simple format: YYYY-MM-DD like 2026-12-12
8. simple_date = current_moment.strftime("%Y-%m-%d")
9. print(f"Simple Date Scroll: {simple_date}")
10.
11. # A more beautiful, adventurous format
12. fancy_date = current_moment.strftime("Today is %A, the %dth of %B, %Y")
13. print(f"Fancy Date Scroll: {fancy_date}")
```

خروجی:

```
Simple Date Scroll: 2026-11-16
Fancy Date Scroll: Today is Sunday, the 16th of November, 2026
```

حالا این متن پیامی در خور یک پادشاه است!

۴. سفر در زمان (طلسم timedelta)

استاد ما یک مأموریت جدید به ما میده: (۷ روز وقت داری اینو انجام بدی!) چطور مهلت انجام پروژه رو محاسبه کنیم که تا کی وقت داریم؟ ما نمی‌توانیم فقط + ۷ را به شیء `current_moment` خود اضافه کنیم. این مانند اضافه کردن «عدد» به «رشته» است — اونا از یه جنس نیستن!

برای اضافه کردن زمان، ما به یک طلسم («مدت زمان») خاص نیاز داریم. این `timedelta` نامیده می‌شود. یک شیء `timedelta` یک بازه زمانی را نشان می‌دهد (مثلاً «۷ روز» یا «۳ ساعت»).

```
1. # We need both datetime and timedelta for this
2. import datetime
3.
4. # Get the current moment
5. now = datetime.datetime.now()
6. print(f"Quest accepted on: {now.strftime('%Y-%m-%d')}")
7.
8. # 1. Create a "duration" spell
9. quest_duration = datetime.timedelta(days=7)
10.
11. # 2. Add the duration to the current date
12. deadline = now + quest_duration
13.
14. print(f"Quest deadline is: {deadline.strftime('%Y-%m-%d')}")
15.
16. # You can also travel to the past!
17. three_hours_ago = now - datetime.timedelta(hours=3)
18. print(f"We started 3 hours ago, at: {three_hours_ago.strftime('%H:%M')}")
```

خروجی:

```
Quest accepted on: 2025-11-16 Quest deadline is: 2025-11-23
We started 3 hours ago, at: 03:24
```

۵. پروژه: اعلان‌گر ددلاین (The Deadline Notifier)

ماجراجو، زمان طلاست ! در دنیای ماجراجویی، هر مأموریت (Quest) یک مهلت خاص دارد. اگر «معجون جاودانگی» را سر وقت به پادشاه نرسانی، مأموریت شکست می‌خورد. امروز می‌خواهیم یک **تقویم هوشمند** بسازیم. ابزاری که تاریخ دقیق همین لحظه را پیدا کند و به ما بگوید دقیقاً ۷ روز دیگر (ددلاین) چه روزی خواهد بود.

تصویر نهایی: این ابزار چه کاری انجام می‌دهد؟

قبل از کدنویسی، عملکرد این ساعت شنی دیجیتال را تصور کن. این برنامه به محض اجرا شدن :

۱. به ساعت جهانی نگاه می‌کند و تاریخ **امروز** را پیدا می‌کند (مثلاً جمعه، ۸ آپریل).

۲. یک سفر در زمان انجام می‌دهد و **۷ روز** به جلو می‌رود.

۳. نتیجه را محاسبه می‌کند (مثلاً جمعه هفته بعد، ۱۵ آپریل).

۴. و در نهایت، هر دو تاریخ را خیلی شیک و خوانا (نه با اعداد عجیب و غریب کامپیوتری) به تو نشان می‌دهد.

نقشه و طرح اولیه

کار با زمان در پایتون مثل کیمیاگری است. ما ۳ مرحله اصلی داریم:

۱. **احضار زمان (Current Time)** کامپیوتر ساعت دارد، اما ما باید به برنامه‌مان یاد بدهیم چطور آن را بخواند.

- `datetime`: کتابخانه

- `منطق`: دستوری که می‌گوید «همین الان ساعت چند است؟».

۲. **ساخت ماشین زمان (Time Delta)** ما نمی‌توانیم بگوییم `today + 7` چون کامپیوتر نمی‌فهمد ۷ یعنی چه (۷ ثانیه؟ ۷ سال؟). ما باید یک «بسته زمانی» بسازیم که دقیقاً مفهوم «(۷ روز)» را داشته باشد.

- `timedelta`: به معنی اختلاف زمانی.

۳. **ریاضیات زمان (Time Travel)** حالا که «امروز» را داریم و «مدت سفر» (۷ روز) را هم داریم، کافیه آن‌ها را با هم جمع کنیم.

- `منطق`: `ددلاین = امروز + ۷ روز`.

۴. **زیباسازی (Formatting)** تاریخ خام در کامپیوتر زشت است مثلاً 2023-10-10

14:30:05.123456 ما می‌خواهیم آن را مثل انسان‌ها بخوانیم مثلاً Sunday, October 10

- `String Format Time`: متد جادویی `strftime` مخفف `String Format Time`.

چالش‌های فکری

قبل از اینکه کد نهایی را ببینی، سعی کن این معماها را حل کنی:

چالش ۱: وارد کردن ابزار کتابخانه مدیریت زمان در پایتون چه نام دارد؟ چطور آن را `import` می‌کنی؟ (راهنمایی `import datetime`)

چالش ۲: ثبت لحظه چطور تاریخ همین لحظه را می‌گیری و در متغیر `today` می‌ریزی؟ (راهنمایی `(datetime.datetime.now())`)

چالش ۳: تعریف ۷ روز چطور یک متغیر به نام `one_week` می‌سازی که ارزشش دقیقاً ۷ روز باشد؟ (راهنمایی `(datetime.timedelta(days=7))`)

چالش ۴: جمع بستن زمان اگر امروز شنبه باشد و `one_week` را به آن اضافه کنی، نتیجه چه می‌شود؟ پایتون چطور این جمع را انجام می‌دهد؟ (پاسخ: `پایتون هوشمند است و خودش تاریخ، ماه و حتی سال جدید را محاسبه می‌کند!`)

چالش ۵: برای استفاده از `strftime` باید نماد های خاصی را بلد باشی.

- `%A` : اسم روز مثلاً Sunday
- `%B` : اسم ماه مثلاً November
- `%d` : شماره روز (مثلاً ۲۵)

کد نهایی پروژه (The Time Spell)

وقت سفر در زمان است! کد زیر را در فایل `project.py` بنویس. به نحوه استفاده از `strftime` برای تمیز کردن خروجی دقت کن.

```
1. # --- The Quest Deadline Notifier ---
2.
3. # 1. Summon the magic (Library)
4. # We import the datetime module to handle dates
5. import datetime
6.
7. # 2. Get the current date (The "Now" Spell)
8. # datetime.datetime.now() gets the exact current moment
9. today = datetime.datetime.now()
10.
11. # 3. Create the duration (The "Time Travel Ticket")
12. # timedelta allows us to define a span of time (days, hours, etc.)
13. one_week = datetime.timedelta(days=7)
14.
15. # 4. Calculate the future (The Math)
16. # Just add the duration to the current date!
17. deadline = today + one_week
18.
19. # 5. Format the scrolls (Beautification)
20. # We use .strftime() to convert ugly computer time to readable text
21. # %A = Day Name, %B = Month Name, %d = Day Number
22. today_formatted = today.strftime("%A, %B %d")
23. deadline_formatted = deadline.strftime("%A, %B %d")
24.
25. # 6. Display the results
26. print("--- Quest Master's Log ---")
27. print(f"Quest assigned on: {today_formatted}")
28. print(f"Quest deadline is: {deadline_formatted}")
29. print("=====")
```

اجرا کن و نتیجه را ببین! خروجی باید دقیقاً تاریخ امروز واقعی و تاریخ ۷ روز بعد را نشان دهد:

```
--- Quest Master's Log ---
Quest assigned on: Tuesday, October 24
Quest deadline is: Tuesday, October 31
=====
```

جشن پیروزی و تحلیل

فوق العاده است! تو نه تنها زمان را خواندی، بلکه آن را خم کردی. تو یاد گرفتی:

- چطور با `datetime` لحظه حال را ثبت کنی.
 - چطور با `timedelta` بازه‌های زمانی (روز، هفته، ساعت) بسازی.
 - و چطور با `strftime` تاریخ را به زبان انسان ترجمه کنی.
- حالا دیگر هیچوقت ددلاین مأموریت‌هایت را فراموش نمی‌کنی. کوله‌پشتی جادویی تو اکنون بر زمان هم مسلط است!

ارتقاء لول: ۷۰ → ۶۵

مهارت‌های جدید:

♦ کتابخانه `Datetime`

با نام باستانی: کنترل زمان

توضیح: درک لحظه حال، سفر به آینده

و محاسبه زمان گذشته.

فصل ۱۵: آیین قبیله پایتون

هنرکدنویسی پایتونیک

مأموریت فصل: بازگشت به کدهای قبلی. حالا که باتجربه‌ایم، طلسم‌های قدیمی خود را بازنویسی می‌کنیم تا سریع‌تر، خواناتر و «پایتونیک» (Pythonic) شوند.

نقشه راه:

- **ذِن پایتون (The Zen of Python):** آشنایی با اصول و آیینی که جادوگران ارشد پایتون به آن متعهد هستند.
- **طلسم enumerate:** شمارش پایتونیک (جایگزین حلقه‌های `range(len())`).
- **طلسم zip:** جفت کردن (پیمایش همزمان دو لیست).
- **طلسم تک‌خطی (List Comprehensions):** جادوی ساختن لیست‌ها در یک خط نفس‌گیر.
- **جادوی ناشناس (Lambda):** ساخت توابع یکبار مصرف و بی‌نام.
- **پروژه:** کیمیاگری لیست (ترکیب جادوهای پایتونیک).

۱. دُن پایتون (The Zen of Python)

ماجرای عزیز، تا به حال یاد گرفته‌ایم که کدی بنویسیم که «کار کند». اما در قبیله پایتون، این کافی نیست. کد ما باید «زیبا»، «خوانا» و «ساده» باشد. جادوگران ارشد پایتون، فلسفه‌ای دارند که به آن «دُن پایتون» می‌گویند. این فلسفه در خود پایتون پنهان شده است. در یک اسکریپت پایتون یا سلول colab این طلسم را اجرا کن:

```
1. import this
```

پیامی ظاهر می‌شود که با این کلمات آغاز می‌شود:

The Zen of Python, by Tim Peters

Beautiful is better than ugly. (زیبا بهتر از زشت است)

Explicit is better than implicit. (آشکار بهتر از پنهان است)

Simple is better than complex. (ساده بهتر از پیچیده است)

Readability counts. (خوانایی اهمیت دارد)

در این فصل، ما یاد می‌گیریم که چگونه با رعایت این آیین، از یک «شاگرد» به یک «استاد پایتونیک» تبدیل شویم.

۲. طلسم enumerate (شمارش پایتونیک)

مشکل: فرض کنید می‌خواهیم در کوله‌پشتی (لیست) خود بگردیم و هر آیتم را همراه با شماره ردیف آن چاپ کنیم (مثلاً: ۱. شمشیر، ۲. سکه ...).

راه شاگرد (راه غیر پایتونیک): یک شاگرد تازه‌کار (که شاید از سرزمین‌های دیگر مثل C آمده باشد) ممکن است چنین طلسمی بنویسد. این کد کار می‌کند، اما زشت و غیر پایتونیک است:

```
1. # The "Clunky" Way
2. backpack = ["Sword", "Shield", "Health Potion"]
3. print("Backpack Inventory (Clunky):")
4.
5. i = 0
6. for item in backpack:
7.     print(f"{i}: {item}")
8.     i = i + 1
```

راه استاد (طلسم enumerate): یک استاد پایتونیک می‌داند که برای این کاریک طلسم داخلی و زیبا وجود دارد: enumerate. این طلسم، لیست شما را می‌گیرد و آن را به لیستی از «جفت‌ها» (ایندکس، آیتم) تبدیل می‌کند.

```
1. # The "Pythonic" Way
2. backpack = ["Sword", "Shield", "Health Potion"]
3. print("Backpack Inventory (Pythonic):")
4.
5. # enumerate gives us (index, item) pairs
6. # We use tuple unpacking to catch them!
7. for index, item in enumerate(backpack):
8.     print(f"{index}: {item}")
```

خروجی (هر دو):

```
0: Sword
1: Shield
2: Health Potion
```

کد دوم خواناتر، کوتاه‌تر و «ساده‌تر» است. (Simple is better than complex).

۳. طلسم zip (جفت کردن)

مشکل: فرض کنید دو لیست جداگانه داریم: یکی برای نام ماجراجویان و دیگری برای امتیازات آنها. چطور می‌توانیم آنها را با هم چاپ کنیم؟

```
1. adventurers = ["Aria", "Kael", "Lyra"]
2. scores = [100, 80, 75]
```

راه شاگرد: باز هم، شاگرد به سراغ ایندکس‌های عددی می‌رود. این کد شکننده است (اگر طول لیست‌ها متفاوت باشد چه؟) و خوانایی ندارد.

```
1. # The "Clunky" Way
2. print("Leaderboard (Clunky):")
3. for i in range(len(adventurers)):
4.     print(f"{adventurers[i]}: {scores[i]}")
```

راه استاد (طلسم zip): استاد از طلسم zip استفاده می‌کند. zip مانند یک زیپ لباس، دو لیست را می‌گیرد و آیتم‌های متناظر آنها را به صورت «جفت» به هم می‌دوزد.

```
1. # The "Pythonic" Way
2. print("Leaderboard (Pythonic):")
3.
4. # zip creates pairs: ("Aria", 100), ("Kael", 80), ...
5. for name, score in zip(adventurers, scores):
6.     print(f"{name}: {score}")
```

این کد «آشکار» (Explicit) و «زیبا» (Beautiful) است.

۴. طلسم تک خطی (List Comprehensions)

این یکی از قدرتمندترین جادوهای پایتونیک است.

مشکل: فرض کنید لیستی از قیمت غنائم داریم و می‌خواهیم لیستی جدید فقط از آیتم‌های گران قیمت (بالای ۶۰ سکه) بسازیم.

راه شاگرد: این روشی است که ما در فصل ۵ یاد گرفتیم. کاملاً درست است، اما ۴ خط طول می‌کشد.

```
1. # The "Old" Way (from Chapter 5)
2. loot_prices = [100, 50, 20, 400, 30]
3. expensive_loot = []
4.
5. for price in loot_prices:
6.     if price > 60:
7.         expensive_loot.append(price)
8.
9. print(f"Old Way: {expensive_loot}")
```

راه استاد (طلسم تک خطی): استاد این ۴ خط را در یک خط جادویی خلاصه می‌کند.

[(شرط تو) if (لیست) in (آیتم) for (اعمال: مثلاً ۲ برابر کردن)]

نتیجه:

```
1. # The "Pythonic" Way (List Comprehension)
2. loot_prices = [100, 50, 20, 400, 30]
3.
4. expensive_loot = [price for price in loot_prices if price > 60]
5.
6. print(f"Pythonic Way: {expensive_loot}")
```

این طلسم حتی می‌تواند داده‌ها را «تغییر» دهد. اگر بخواهیم قیمت همه آیتم‌ها را دو برابر کنیم (حتی بدون بخش if هم کار می‌کند):

```
1. # Transformation with List Comprehension
2. doubled_prices = [price * 2 for price in loot_prices]
3. print(f"Doubled Prices: {doubled_prices}")
```

۵. جادوی ناشناس (Lambda)

مشکل: گاهی ما به یک تابع ساده فقط برای یک بار استفاده نیاز داریم.

راه شاگرد: فرض کنید می‌خواهیم لیست امتیازات را بر اساس امتیاز (عدد دوم تاپل) مرتب کنیم.

```
1. # The "Old" Way (using a full function)
2. leaderboard = [("Kael", 80), ("Aria", 100), ("Lyra", 75)]
3.
4. # We need a helper function just for sorting
5. def get_score(entry):
6.     return entry[1] # Returns the score
7.
```

```
8. leaderboard.sort(key=get_score)
9. print(f"Sorted (Old): {leaderboard}")
```

راه استاد (طلمس lambda): استاد برای چنین تابع ساده‌ای، یک تابع کامل def نمی‌سازد. او از lambda (یک تابع ناشناس و یک خطی) استفاده می‌کند.

```
1. # The "Pythonic" Way (using lambda)
2. leaderboard = [("Kael", 80), ("Aria", 100), ("Lyra", 75)]
3.
4. # We create an anonymous function "on the fly"
5. leaderboard.sort(key=lambda entry: entry[1]) # Sort by the 2nd item (score)
6.
7. print(f"Sorted (Pythonic): {leaderboard}")
```

۶. پروژه: کیمیاگری لیست (List Alchemy) – 100 xp

کیمیاگر جوان، گوش کن ! درمانگاه قلعه شلوغ شده و سربازان زیادی زخمی شده‌اند. انباردار پیر ما، لیستی از تمام معجون‌های موجود را در اختیار دارد، اما این لیست درهم و برهم است. معجون‌های نامرئی‌کننده، مانا و سلامتی همه با هم مخلوط شده‌اند. ما وقت نداریم که دانه دانه قفسه‌ها را بگردیم. ما به یک «طلمس پالایش» نیاز داریم که در یک چشم برهم زدن، فقط معجون‌های «درمانی» را بیرون بکشد و نامشان را با صدای بلند (حروف بزرگ) فریاد بزند تا پرستاران سریع پیدایشان کنند.

تصویر نهایی: تفاوت دو روش

قبل از کدنویسی، تفاوت «شاگرد» و «استاد» را تصور کن:

۱. روش شاگرد (حلقه For معمولی): شاگرد جادوگر یک سبد خالی برمی‌دارد. به سمت قفسه می‌رود. اولین بطری را برمی‌دارد. نگاه می‌کند: «آیا درمانی است؟» اگر بله، اسمش را روی کاغذ با حروف درشت می‌نویسد و در سبد می‌اندازد. بعد سراغ بطری دوم می‌رود... این کار کند و خسته‌کننده است.

۲. روش استاد (List Comprehension): استاد جادوگر وسط اتاق می‌ایستد. عصایش را بالا می‌برد و یک جمله جادویی تک خطی می‌گوید. ناگهان تمام معجون‌های درمانی خودشان از قفسه پرواز می‌کنند، اسمشان در هوا درخشان و بزرگ می‌شود و داخل سبد قرار می‌گیرند. همه در یک لحظه!

نقشه و طرح اولیه

ما می‌خواهیم لیست potions را پالایش کنیم.

لیست اولیه (انبار درهم):

```
1. potions = [
2.     {"name": "Lesser Healing", "type": "Heal", "price": 50},
```

```

3.     {"name": "Mana Potion", "type": "Mana", "price": 70},
4.     {"name": "Greater Healing", "type": "Heal", "price": 150},
5.     {"name": "Invisibility", "type": "Stealth", "price": 200}
6. ]

```

فرمول طلسم استاد: در پایتون، ما می‌توانیم یک حلقه ۴ خطی را در ۱ خط فشرده کنیم. ساختار دستوری آن مثل دستور زبان انسان است

[تغییر-شکل بده | به ازای هر آیتیم | اگر شرایط داشت]

۱. **تغییر (Select/Transform):** چه چیزی می‌خواهی؟ (نام معجون با حروف بزرگ).
۲. **تکرار (From/Iteration):** از کجا؟ (از لیست معجون‌ها).
۳. **فیلتر (Where/Filter):** کدام‌ها؟ (آن‌هایی که تایپشان Heal است).

چالش‌های فکری

قبل از اینکه عصایت را تکان دهی، به این معماها فکر کن:

چالش ۱: شرط فیلتر چطور به پایتون می‌گویی «فقط آن‌هایی را انتخاب کن که کلید "type" آن‌ها برابر با "Heal" است»؟ (*راهنمایی: if potion["type"] == "Heal")

چالش ۲: تغییر شکل ما فقط نام معجون را می‌خواهیم، نه کل دیکشنری را. و آن را با حروف بزرگ می‌خواهیم. دستور تبدیل حروف کوچک به بزرگ در رشته‌ها چیست؟ (راهنمایی: `.upper()`)

چالش ۳: ترکیب جادویی چطور این قطعات را داخل یک [] جدید می‌چینی؟ یادت باشد در این جادو، اول «نتیجه نهایی» را می‌نویسیم، بعد «حلقه» و در آخر شرط.

کد نهایی پروژه (The Master's Spell)

وقت کیمیاگری است! کد زیر را در فایل `project.py` بنویس. من هر دو روش را نوشتم تا تفاوت قدرت آن‌ها را ببینی.

```

1. # --- The List Alchemy ---
2.
3. # 1. Our inventory of potions (The Raw Material)
4. potions = [
5.     {"name": "Lesser Healing", "type": "Heal", "price": 50},
6.     {"name": "Mana Potion", "type": "Mana", "price": 70},
7.     {"name": "Greater Healing", "type": "Heal", "price": 150},
8.     {"name": "Invisibility", "type": "Stealth", "price": 200}
9. ]
10.
11. # 2. --- The Apprentice's Way (Slow & Steady) ---
12. # He creates an empty list, walks to the shelf, checks one by one...
13. print("Apprentice's Result:")
14. healing_potion_names = []

```



```

15. for potion in potions:
16.     if potion["type"] == "Heal":
17.         name_upper = potion["name"].upper()
18.         healing_potion_names.append(name_upper)
19.
20. print(healing_potion_names)
21.
22. print("-" * 20)
23.
24. # 3. --- The Master's Way (List Comprehension) ---
25. # ONE powerful line to do everything at once!
26. print("Master's Result:")
27.
28. # Structure: [ TRANSFORM for ITEM in LIST if CONDITION ]
29. healing_potion_names = [
30.     potion["name"].upper()      # 1. What we want (The Transformation)
31.     for potion in potions      # 2. Where we look (The Iteration)
32.     if potion["type"] == "Heal" # 3. The Filter condition
33. ]
34.
35. print(healing_potion_names)

```

اجرا کن و نتیجه را ببین! خروجی هر دو روش یکسان است:

```
[ 'LESSER HEALING', 'GREATER HEALING' ]
```

اما کدی که به روش استاد نوشتی، تمیزتر، سریع‌تر و حرفه‌ای‌تر است.

جشن پیروزی و تحلیل

تبریک می‌گویم استاد! تو الان تکنیک **List Comprehension** را یاد گرفتی. این یکی از محبوب‌ترین ویژگی‌های پایتون است. هر وقت دیدی داری:

۱. یک لیست خالی می‌سازی

۲. یک حلقه می‌نویسی،

۳. و داخلش چیزی را `append` می‌کنی ... سریع به یاد این درس بیفت و از «طلسم تک خطی» استفاده کن. کوله‌پشتی تو حالا سبک‌تر و چابک‌تر شده است!

ارتقاء لول: ۷۵ $\rightarrow$ ۷۰

مهارت‌های جدید:

◆ تکنیک List Comprehension

با نام باستانی: کیمیاگری تک خطی
توضیح: انجام کارهای بزرگ در کمترین
فضای ممکن. زیبایی در عین قدرت.

◆ توابع Lambda

با نام باستانی: ارواح ناشناس
توضیح: توابعی سریع و بی‌نام برای
ماموریت‌های فوری.

◆ سطح جدید: C

توضیح: تبریک به سطح
جدیدی رسید.

مأموریت‌های جانبی: محفل معماران زمان

ما جراجو! تو اکنون ا وارد «محفل معماران» شده‌ای. در اینجا، قدرت فقط در "اجرا شدن کد" نیست؛ بلکه در **ظرافت، سرعت و نظم** است. تو یاد می‌گیری که زمان را کنترل کنی، کدهای کوتاه و پایتونیک (Pythonic) بنویسی و قدرت‌هایت را در فایل‌های جداگانه دسته‌بندی کنی. قانون محفل: **کدها باید کوتاه، تمیز و ساختاریافته باشند. زیبایی کد به اندازه عملکرد آن مهم است!**

مأموریت‌ها

۱. مأموریت: تقویت کریستال‌ها (Crystal Amplification) – 20 xp

جادوگران برای افزایش قدرت، کریستال‌های خود را تقویت می‌کنند.

- **داستان:** ما یک کیسه پر از کریستال‌های خام با قدرت‌های معمولی (۱ تا ۱۰) داریم. باید با یک طلسم سریع، قدرت همه آن‌ها را به توان ۲ برسانیم.
- **وظیفه:** یک لیست از اعداد [1, 2, ..., 10] بساز. با استفاده از جادوی **List Comprehension** (روش تک خطی)، لیست جدیدی تولید کن که شامل توان ۲ این اعداد باشد.
- **قانون:** استفاده از حلقه for معمولی (چند خطی) اکیداً ممنوع است!

۲. مأموریت: فیلتر کردن طومارها (Filtering Scrolls) – 20 xp

کتابخانه به هم ریخته و پر از طومارهای بی‌استفاده است.

- **داستان:** در میان انبوهی از طومارها، فقط آن‌هایی ارزش دارند که نامشان با حرف "F" شروع شود و نام طولانی (بیشتر از ۳ حرف) داشته باشند.
- **وظیفه:** یک لیست شامل کلمات "Fireball", "Ice", "Fly", "Arcane", "Freeze" تعریف کن. با یک خط کد (**List Comprehension**)، لیستی بساز که فقط شامل کلمات واجد شرایط باشد.

۳. مأموریت: تابلوی اعلانات (The Quest Board) – 20 xp

لیست مأموریت‌ها باید مرتب و شماره‌گذاری شده باشد تا قهرمانان گیج نشوند.

- **داستان:** پادشاه لیستی از کارها را داده است، "Save Village", "Find Sword": "Defeat Boss" اما این لیست شماره ندارد.
- **وظیفه:** با استفاده از حلقه for و دستور جادویی enumerate، لیست را طوری چاپ کن که شماره‌گذاری از عدد ۱ شروع شود (مثلاً 1. Save Village).

۴. مأموریت: روز سرنوشت (The Day of Destiny) – 20 xp

تقویم‌های باستانی را رمزگشایی کن.

- **داستان:** یک پیشگو تاریخ تولد تو (با یک تاریخ مهم دیگر) را می‌پرسد و باید بگوید آن تاریخ دقیقاً چه روزی از هفته بوده است.

- **وظیفه:** تاریخ مورد نظر را (سال، ماه، روز) با `datetime` تعریف کن. برنامه باید نام کامل روز هفته (مثلاً Monday) را چاپ کند.

۵. مأموریت: ساماندهی کتابخانه (Library Organization) – 20 xp

- یک جادوگر منظم، تمام طلسم‌ها را در یک فایل نمی‌ریزد.
- **داستان:** توابع ریاضی ما زیاد شده‌اند. باید آن‌ها را به یک "کتابچه جداگانه" منتقل کنیم.
 - **وظیفه:**

۱. یک فایل جدید به نام `magic_math.py` بساز. توابع `area_square(side)` و `area_circle(radius)` را در آن بنویس.
۲. یک فایل دیگر به نام `main.py` بساز. فایل ریاضی را `import` کن و مساحت یک مربع ۵ متری و دایره ۳ متری را حساب کن.

۶. مأموریت: تست سرعت طلسم (Spell Speed Test) – 20 xp

- چقدر طول میکشه اجرا بشه؟ زمان طلاست!
- **داستان:** می‌خواهیم بدانیم شمارش تا یک میلیون چقدر طول می‌کشد.
 - **وظیفه:**
۱. زمان سیستم را در متغیر `start` ذخیره کن.
 ۲. یک حلقه `for` بنویس که ۱,۰۰۰,۰۰۰ بار اجرا شود (بدون انجام کار خاصی، فقط `pass`).
 ۳. زمان دقیق سیستم را در متغیر `end` ذخیره کن.
 ۴. تفاضل این دو زمان را چاپ کن تا سرعت پردازشگر را ببینی.

۷. مأموریت: چرخه بازیابی مانا (Mana Regeneration) – 20 xp

- برای اجرای جادوهای سنگین، باید استراحت کنی.
- **داستان:** جادوگر باید ۲۵ دقیقه تمرکز کند تا مانا (انرژی جادویی) جمع کند و سپس ۵ دقیقه استراحت کند.
 - **وظیفه:** سیستمی بساز که:
۱. چاپ کند "Focusing..." و ۵ ثانیه (به جای ۲۵ دقیقه) صبر کند.
 ۲. چاپ کند "Resting..." و ۱۰ ثانیه (به جای ۵ دقیقه) صبر کند.
 ۳. این کار را با تابع `sleep` از ماژول `time` انجام بده.

۸. مأموریت: شمارش معکوس تا خورشیدگرفتگی (Eclipse Countdown) – 20 xp

رویداد بزرگ نزدیک است!

- داستان: منجمان می‌گویند در تاریخ ۱ ژانویه سال ۲۰۳۰ خورشیدگرفتگی رخ می‌دهد. چقدر زمان داریم؟
- وظیفه: زمان فعلی (now) را بگیر. یک زمان هدف (target) برای آینده بساز. تفاضل آن‌ها را حساب کن و به کاربر بگو دقیقاً چند روز و ساعت باقی مانده است.

۹. مأموریت: آهنگر کلیدها (The Key Smith) – 20 xp

دروازه‌های مخفی نیاز به کلیدهای منحصر به فرد دارند.

- داستان: برای ورود به سیاه‌چال، نیاز به یک کلید امنیتی ۸ رقمی داریم که از حروف و اعداد تصادفی ساخته شده باشد.
- وظیفه:

- یک فایل ماژول به نام security.py بساز.
- در آن تابعی بنویس که یک عدد (طول کلید) بگیرد و یک رشته تصادفی (شامل حروف و اعداد) برگرداند.
- در فایل اصلی main.py، از این ماژول استفاده کن تا یک کلید ۸ رقمی بسازی.

۱۰. مأموریت: چرخاندن نقشه (Rotating the Map) – 20 xp

نقشه گنج به صورت یک ماتریس است، اما باید آن را بچرخانی تا مسیر را ببینی.

- داستان: یک جدول ۳x۳ از اعداد داری. برای باز شدن قفل، جای سطرها و ستون‌ها باید عوض شود (بهش ترانپوز یا Transpose هم می‌گویند).
- وظیفه: ماتریس `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]` را تعریف کن. با استفاده از **Nested List Comprehension** (تکنیک پیشرفته تو در تو)، جای سطر و ستون را عوض کن تا خروجی `[[1, 4, 7], ...]` شود.

آیا سفر را در این مرحله کامل کرده‌ای، ماجراجو؟ اگر بله، کتاب را ورق بزن؛ کتابخانه سایه‌ها (نتیجه مأموریت‌ها) در صفحه بعد انتظار تو را می‌کشد.

کتابخانه سایه‌ها (نتیجه مأموریت‌ها)

۱. مأموریت: تقویت کریستال‌ها

نکته: سریع‌ترین راه برای تغییر تمام اعضای یک لیست بدون نوشتن حلقه طولانی.

```
1. crystals = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2.
3. # Square each number using List Comprehension logic
4. # Syntax: [expression for item in list]
5. power_crystals = [c ** 2 for c in crystals]
6.
7. print(power_crystals)
```

۲. مأموریت: فیلتر کردن طومارها

نکته: اضافه کردن شرط if به انتهای List Comprehension برای انتخاب گزینشی.

```
1. scrolls = ["Fireball", "Ice", "Fly", "Arcane", "Freeze"]
2.
3. # Filter: Starts with 'F' AND length > 3
4. valid Scrolls = [s for s in scrolls if s.startswith("F") and len(s) > 3]
5.
6. print(valid Scrolls)
```

۳. مأموریت: تابلوی اعلانات

نکته enumerate: به ما هم "اینده‌کس" را می‌دهد و هم "مقدار" را. عدد ۱ در ورودی دوم، یعنی شمارش از ۱ شروع شود.

```
1. quests = ["Save Village", "Find Sword", "Defeat Boss"]
2.
3. # Enumerate allows us to get the index and item together
4. for index, quest in enumerate(quests, 1):
5.     print(f"{index}. {quest}")
```

۴. مأموریت: روز سرنوشت

نکته: متد strftime با کد %A نام کامل روز هفته را از تاریخ استخراج می‌کند.

```
1. import datetime
2.
3. # Format: Year, Month, Day
4. my_date = datetime.date(2000, 5, 15)
5.
6. # Get the full English name of the weekday
7. print(my_date.strftime("%A"))
```

۵. مأموریت: ساماندهی کتابخانه

نکته: این کد شامل دو فایل مجزا است که باید در کنار هم (در یک پوشه) باشند.

```
1. # --- File: magic_math.py ---
2. def area_square(side):
3.     return side * side
4.
5. def area_circle(radius):
6.     return 3.14 * radius * radius
7.
8. # --- File: main.py ---
9. import magic_math
10.
11. # Using the imported functions
12. print(magic_math.area_square(5))
13. print(magic_math.area_circle(3))
```

۶. مأموریت: تست سرعت طلسم

نکته: تفریق دو زمان (datetime)، یک آبجکت timedelta (مدت زمان سپری شده) برمی‌گرداند.

```
1. import datetime
2.
3. # Capture start time
4. start = datetime.datetime.now()
5.
6. # A heavy loop simulation (1 million iterations)
7. for i in range(1000000):
8.     pass
9.
10. # Capture end time
11. end = datetime.datetime.now()
12.
13. print(f"Time taken: {end - start}")
```

۷. مأموریت: چرخه بازیابی مانا

نکته: time.sleep: برنامه را برای مدت مشخصی (به ثانیه) متوقف می‌کند.

```
1. import time
2.
3. print("Focus State! (Regenerating Mana...)")
4. time.sleep(3) # Simulating work time
5.
6. print("Rest State! (Cooldown...)")
7. time.sleep(1) # Simulating break time
8.
9. print("Cycle Complete!")
```

۸. مأموریت: شمارش معکوس تا خورشیدگرفتگی

نکته: می‌توانیم تاریخ‌های آینده را بسازیم و زمان حال را از آن کم کنیم.

```
1. import datetime
2.
3. now = datetime.datetime.now()
```



```
4. target_date = datetime.datetime(2030, 1, 1) # Set a future date
5.
6. # Calculate remaining time (Time Delta)
7. remaining = target_date - now
8. print(f"Time left: {remaining}")
```

۹. مأموریت: آهنگر کلیدها

نکته: استفاده از ماژول random و string برای تولید کدهای تصادفی بسیار کاربردی است.

```
1. # --- File: security.py ---
2. import random
3. import string
4.
5. def generate_key(length):
6.     # Combine letters and digits
7.     chars = string.ascii_letters + string.digits
8.     # Select random characters
9.     key = "".join(random.choices(chars, k=length))
10.    return key
11.
12. # --- File: main.py ---
13. import security
14.
15. my_key = security.generate_key(8)
16. print(f"Generated Key: {my_key}")
```

۱۰. مأموریت: چرخاندن نقشه

نکته: حلقه بیرونی روی ستون‌ها (range) و حلقه داخلی روی سطرها (row) حرکت می‌کند تا جای آن‌ها عوض شود.

```
1. map_matrix = [
2.     [1, 2, 3],
3.     [4, 5, 6],
4.     [7, 8, 9]
5. ]
6.
7. # Nested List Comprehension to swap rows and columns
8. rotated = [[row[i] for row in map_matrix] for i in
9. range(len(map_matrix[0]))]
10. print(rotated)
```

بخش پنجم: نبرد نهایی (پروژه بزرگ ماجراجو)

تو به انتهای آموزش نزدیک می‌شوی و زمان «آزمون نهایی» فرا رسیده است. در این بخش، ما تمام جادوهایی که از فصل‌های گذشته گرفته‌ایم را در یک پروژه بزرگ و کاربردی ترکیب می‌کنیم.

فصل ۱۶: نبرد نهایی: ساخت دفترچه مأموریت ماجراجو

مأموریت بزرگ : ساخت ابزار نهایی: یک دفترچه مأموریت (Quest Log) کامل. این پروژه، مدرک فارغ التحصیلی شما از این ماجراجویی است! (1000 xp)

آفرین ماجراجو! تو «نبرد نهایی» رسیده‌ای. این فصل، آزمون توست. ما قرار است تمام جادوهایی که در این ماجراجویی یاد گرفته‌ایم را با هم ترکیب کنیم تا قدرتمندترین ابزار خود را بسازیم.

۱. نگاهی به گذشته (Gathering Your Spells)

قبل از ورود به نبرد، یک ماجراجوی خردمند ابزارهایش را بررسی می‌کند. برای ساختن «دفترچه مأموریت»، ما به تمام جادوهای قدرتمندی که آموخته‌ایم نیاز داریم:

- **کلاس‌ها:** برای طراحی نقشه ساخت (Blueprint) یک مأموریت.
- **ماژول‌ها:** برای سازماندهی کد و جدا کردن «نقشه ساخت» از منطق اصلی.
- **Datetime:** برای ثبت خودکار «زمان ایجاد» هر مأموریت.
- **JSON:** برای دادن «حافظه ابدی» به مأموریت‌هایمان تا با بستن برنامه پاک نشوند.
- **توابع:** برای نوشتن طلسم‌های تمیز و قابل استفاده مجدد (مثل `add_quest`, `show quests`).
- **حلقه‌ها:** برای ساختن «منوی فرمان» تعاملی برنامه.

۲. توضیحات چالش (The Adventurer's Challenge)

اینجا جایی است که تو خودت را به چالش میکشی، ماجراجو!

مأموریت تو: به فایل `project.py` خود برو و سعی کن برنامه‌ای با این مشخصات بسازی:

۱. **نقشه ساخت:** به یک کلاس `Quest` نیاز داری که `title`, `description`, `status` و `created_at` (تاریخ ایجاد) را نگه دارد.

۲. **منوی فرمان:** برنامه باید یک منوی `while True` داشته باشد که ۴ گزینه ارائه دهد:

افزودن، نمایش همه، تکمیل کردن و خروج.

۳. **حافظه ابدی:** وقتی کاربر «خروج» را وارد میکند، کل لیست مأموریت‌ها باید در یک فایل `quest_log.json` ذخیره شود و برنامه تمام شود.

۴. **بارگذاری حافظه:** وقتی برنامه دوباره اجرا می‌شود، باید فایل `quest_log.json` را بررسی کند و تمام مأموریت‌های قبلی را بارگذاری کند.

سعی کن! ببین چقدر می‌توانی پیش بروی. اگر در هر بخشی گیر کردی، نگران نباش. میتونی از جستجو و دوباره خوندن فصل‌های قبل حتی `chatgpt` کمک بگیری تا کاررو پیش ببری.

۳. میز معمار (Building Step-by-Step)

آماده‌ای؟ بیا این شاهکار را با هم بسازیم. ما پروژه را به دو فایل تقسیم می‌کنیم

۱. `quest_model.py`: نقشه ساخت ما.

۲. `main.py`: مرکز فرماندهی ما.

مرحله ۱: نقشه ساخت (quest_model.py)

ابتدا، «نقشه ساخت» مأموریت را طراحی می‌کنیم. این کلاس مسئول نگهداری داده‌های یک مأموریت است.

یک فایل جدید به نام quest_model.py بسازید:

```

1. # quest_model.py
2. # This file contains the "blueprint" for our quests.
3. # We import the datetime magic (from Chapter 14).
4. import datetime
5.
6. class Quest:
7.     """
8.     Blueprint for a single quest in our Quest Log.
9.     Uses magic from Chapter 12 (Classes).
10.    """
11.
12.    def __init__(self, title, description):
13.        # 1. Attributes from the user
14.        self.title = title
15.        self.description = description
16.
17.        # 2. Automatic attributes (Chapter 14 magic)
18.        self.created_at = datetime.datetime.now()
19.
20.        # 3. Default status
21.        self.status = "Pending" # Other status: "Complete"
22.
23.    def display_info(self):
24.        """
25.        Prints the details of this quest in a neat format.
26.        Uses magic from Chapter 14 (strftime) and Chapter 10 (f-strings).
27.        """
28.        # Format the date into a readable string
29.        date_str = self.created_at.strftime("%Y-%m-%d %H:%M")
30.
31.        print(f" Title: {self.title}")
32.        print(f" Status: {self.status}")
33.        print(f" Created: {date_str}")
34.        print(f" Description: {self.description}")
35.
36.    def complete_quest(self):
37.        """
38.        Changes the status of the quest to "Complete".
39.        """
40.        self.status = "Complete"
41.        print(f"\n[Quest '{self.title}' marked as complete!]")

```

برای بچه‌های colab و ژوپیتِر:

در ابتدای سلول کد بالا %%writefile quest_model.py رو اضافه کنید تا کد اجرا نشه و

فقط داخل فایل quest_model.py ذخیره بشه

عالی! نقشه ساخت ما کامل شد.

مرحله ۲: مرکز فرماندهی (main.py)

حالا، فایل اصلی برنامه را می‌سازیم. این فایل منطق اصلی، منو و جادوهای ذخیره‌سازی را مدیریت می‌کند.

یک فایل جدید به نام main.py بسازید. ما آن را تکه به تکه خواهیم ساخت.

تکه ۱: احضار جادوها ابتدا، تمام ابزارهای مورد نیازمان را import می‌کنیم.

```
1. # main.py
2. # This is our main command center.
3.
4. # --- 1. SUMMONING OUR MAGIC ---
5. import json # For eternal memory (Chapter 11)
6. import datetime # For time magic (Chapter 14)
7. from quest_model import Quest # Our own blueprint! (Chapter 13)
8.
9. # Global variables (our magic scrolls)
10. quest_list = [] # This is our main "backpack" for quests
11. SAVE_FILE = "quest_log.json" # The name of our eternal memory file
```

تکه ۲: طلسم‌های حافظه ابدی این مهم‌ترین بخش است. json نمی‌تواند Quest را مستقیماً ذخیره کند. ما باید اشیاء خود را به «دیکشنری» که JSON می‌فهمد تبدیل کنیم و هنگام بارگذاری، دیکشنری‌ها را دوباره به اشیاء Quest تبدیل کنیم.

```
1. # --- 2. ETERNAL MEMORY SPELLS (Chapter 11 Magic) ---
2.
3. def load_questions():
4.     """
5.     Loads the quest list from the JSON file at the start.
6.     Uses try...except to handle the first run.
7.     """
8.     global quest_list
9.     try:
10.         with open(SAVE_FILE, "r") as f:
11.             # 1. Load the simple data (list of dictionaries)
12.             data_list = json.load(f)
13.
14.             # 2. Re-create the Quest objects
15.             quest_list = [] # Clear the list before loading
16.             for item in data_list:
17.                 # Create a new quest object
18.                 quest = Quest(item['title'], item['description'])
19.                 # Restore its saved data
20.                 quest.status = item['status']
21.                 quest.created_at =
22.                     datetime.datetime.fromisoformat(item['created_at'])
23.                 quest_list.append(quest)
24.
25.             print(f"Quest Log loaded. {len(quest_list)} quests found.")
26.     except FileNotFoundError:
27.         print("No save file found. Starting a new Quest Log.")
28.     except Exception as e:
```

```

28.         print(f"An error occurred while loading: {e}")
29.
30. def save_requests():
31.     """
32.     Saves the current quest_list to the JSON file before quitting.
33.     """
34.     # 1. We must convert our list of Objects into a list of Dictionaries
35.     data_list = []
36.     for quest in quest_list:
37.         # Convert datetime object to a string to save in JSON
38.         date_str = quest.created_at.isoformat()
39.
40.         data_list.append({
41.             "title": quest.title,
42.             "description": quest.description,
43.             "created_at": date_str,
44.             "status": quest.status
45.         })
46.
47.     # 2. Now, save the simple list of dictionaries
48.     try:
49.         with open(SAVE_FILE, "w") as f:
50.             json.dump(data_list, f, indent=4)
51.             print("Quests saved successfully!")
52.     except Exception as e:
53.         print(f"An error occurred while saving: {e}")

```

تکه ۳: طلسم‌های تعاملی حالا، توابع تمیزی می‌نویسیم که هر کدام یک کار در منوی ما انجام می‌دهند.

```

1. # --- 3. INTERACTIVE SPELLS (Chapter 8 Magic) ---
2.
3. def add_quest():
4.     """
5.     Asks the user for quest details and adds a new Quest to the list.
6.     """
7.     print("\n--- Add New Quest ---")
8.     title = input("Enter quest title: ")
9.     description = input("Enter quest description: ")
10.
11.     # Create a new Quest object (using our blueprint)
12.     new_quest = Quest(title, description)
13.
14.     # Add it to our main backpack
15.     quest_list.append(new_quest)
16.     print("\n[Quest added successfully!]")
17.
18. def show_all_requests():
19.     """
20.     Displays all quests in the quest_list.
21.     Uses magic from Chapter 15 (enumerate) and Chapter 12 (methods).
22.     """
23.     print("\n--- Your Current Quest Log ---")
24.     if not quest_list:
25.         print("Your quest log is empty. Time to find adventure!")
26.         return
27.

```

```

28.     # Use 'enumerate' to get a clean index (Chapter 15 magic)
29.     for index, quest in enumerate(quest_list):
30.         print(f"\n--- Quest {index + 1} ---")
31.         quest.display_info() # Call the method from our class!
32.         print("-----")
33.
34. def complete_quest():
35.     """
36.     Asks the user which quest to mark as complete.
37.     """
38.     # Show them the list first
39.     show_all_requests()
40.     if not quest_list:
41.         return # Nothing to complete
42.
43.     try:
44.         choice = input("Enter the number of the quest to complete: ")
45.         # Convert choice to a zero-based index
46.         index = int(choice) - 1
47.
48.         # Check if the index is valid
49.         if 0 <= index < len(quest_list):
50.             quest_list[index].complete_quest()
51.         else:
52.             print("Invalid quest number! No quest was completed.")
53.
54.     except ValueError:
55.         print("That's not a number! Spell failed.")

```

تکه ۴: حلقه اصلی برنامه در نهایت، حلقه while را می نویسیم تا همه چیز را به هم متصل کند.

```

1. # --- 4. THE MAIN SPELL LOOP (Chapter 4 Magic) ---
2.
3. def main():
4.     """
5.     The main entry point of our program.
6.     """
7.     # Load quests from our eternal memory first!
8.     load_requests()
9.
10.    while True:
11.        print("\n==== Adventurer's Quest Log =====")
12.        print("1. Add a new quest")
13.        print("2. Show all quests")
14.        print("3. Complete a quest")
15.        print("4. Save and Quit")
16.        print("=====")
17.        choice = input("Cast your spell (1-4): ")
18.
19.        if choice == '1':
20.            add_quest()
21.        elif choice == '2':
22.            show_all_requests()
23.        elif choice == '3':
24.            complete_quest()
25.        elif choice == '4':
26.            # Save before quitting!
27.            save_requests()

```



```
28.         print("\nQuest Log saved. Goodbye, adventurer!")
29.         break # Exit the while loop
30.     else:
31.         print("\nInvalid spell! Try again.")
32.
33. # --- 5. RUN THE PROGRAM ---
34. # This standard Python line ensures main() only runs
35. # when the script is executed directly.
36. if __name__ == "__main__":
37.     main()
```

۴. پیروزی نهایی!

ماجراجو، تو موفق شدی! حالا دو فایل در پوشه خود داری:

۱. quest_model.py نقشه ساخت

۲. main.py مرکز فرماندهی

فایل main.py را اجرا کن:

```
python main.py
```

خروجی اولین اجرا

```
No save file found. Starting a new Quest Log.

==== Adventurer's Quest Log ====
1. Add a new quest
2. Show all quests
3. Complete a quest
4. Save and Quit Cast your spell
=====
Cast your spell (1-4):
```

حالا می‌توانی:

۱. مأموریت‌ها را Add کنی.

۲. آن‌ها را Show کنی.

۳. یک مأموریت را Complete کنی.

۴. و در نهایت، Quit کنی.

وقتی خارج شدی، یک فایل جدید به نام quest_log.json در کنار برنامه‌ات ظاهر می‌شود. اگر

main.py را دوباره اجرا کنی، خواهی دید:

خروجی دومین اجرا

```
Quest Log loaded. 3 quests found ...
```

تمام مأموریت‌های تو به صورت جادویی بارگذاری شده‌اند!

آفرین! تو همین الان یک برنامه کامل، سازمان‌یافته، با حافظه دائمی و مبتنی بر کلاس ساختی.

تو تمام جادوهای اصلی را با هم ترکیب کردی و در «نبرد نهایی» پیروز شدی. تو رسماً یک جادوگر

پایتون هستی.

ارتقاء لول: ۸۰ $\mapsto$ ۷۵

دستاورد بزرگ:

♦ ساخت پروژه جامع
با نام باستانی: آزمون بزرگ
توضیح: تمام طلسم‌هایی که آموختی در یک
سمفونی واحد نواخته شدند.

لقب جدید:
برنامه نویس ارشد

♦ سطح جدید: B
توضیح: تبریک به سطح
جدیدی رسید.

مأموریت‌های جانبی: نسخه بعدی (Expansion Pack)

ما جراجو! تو اکنون نسخه اولیه (v1.0) دفترچه مأموریت خود را ساخته‌ای و فارغ‌التحصیل شده‌ای. اما کار یک برنامه‌نویس واقعی هرگز تمام نمی‌شود. نرم‌افزارها موجوداتی زنده هستند که باید رشد کنند. انجمن جادوگران ۱۰ قابلیت جدید برای تو لیست کرده است. این قابلیت‌ها را به کد پروژه اصلی اضافه کن تا دفترچه‌ات تبدیل به قدرتمندترین ابزار شود. قانون تالار: سعی کن اول خودت تلاش کنی دونه دونه این قابلیت‌ها را اضافه کنی و اگر شکست خوردی میتونی از کتابخانه سایه‌ها استفاده کنی.

مأموریت‌ها

۱. مأموریت: سپر محافظ (The Guardian Shield) – 20 xp

مشکل: اگر کاربر در عنوان مأموریت فقط اینتر بزند، یک مأموریت خالی ساخته می‌شود.
وظیفه: تابع `add_quest` را تغییر بده. تا زمانی که کاربر متنی وارد نکرده، برنامه نباید جلو برود (استفاده از حلقه `while` برای اجبار کاربر).

۲. مأموریت: طلسم نابودی (The Vanishing Spell) – 20 xp

مشکل: لیست پر از مأموریت‌های قدیمی شده و نمی‌توانیم حذفشان کنیم. **وظیفه:**
۱. یک تابع جدید `delete_quest` بساز که لیست را نشان دهد، شماره بگیرد و آن را حذف کند.
۲. گزینه 5 را به منوی اصلی اضافه کن.

۳. مأموریت: عینک حقیقت‌بین (The Focus Lens) – 20 xp

مشکل: دیدن کارهای انجام‌شده (Complete) در میان کارهای باقی‌مانده (Pending) تمرکز را به هم می‌زند. **وظیفه:** تابع `show_all_requests` را بازنویسی کن. قبل از نمایش، پرس `Show: Pending? (1) All or (2) Pending?` اگر کاربر 2 را زد، کارهای تمام شده را چاپ نکن.

۴. مأموریت: قلم ویرایش (The Magic Quill) – 20 xp

مشکل: اگر در توضیحات غلط املایی داشته باشیم، باید مأموریت را پاک کنیم و دوباره بسازیم! **وظیفه:** تابع `edit_quest` را بساز. شماره مأموریت را بگیرد. عنوان جدید را بپرسد (اگر کاربر اینتر خالی زد، عنوان قبلی بماند). توضیحات جدید را بپرسد (اگر خالی زد، قبلی بماند).

۵. مأموریت: ساعت شنی (The Hourglass) – 20 xp

مشکل: نمی‌دانیم هر مأموریت تا کی مهلت دارد. **وظیفه:**
۱. در `quest_model.py`، متد `__init__` را تغییر بده تا `deadline` را هم بگیرد.
۲. در `main.py`، توابع `add_quest` و `save_requests` و `load_requests` را آپدیت کن تا این تاریخ را ذخیره و بازیابی کنند.

۶. مأموریت: نشان پادشاهی (The Priority Seal) – 20 xp

مشکل: کارهای مهم و معمولی قاطی شده‌اند. **وظیفه:** به کلاس Quest ویژگی priority اضافه کن. در main.py هنگام ساخت مأموریت بپرس (High/Low) در نمایش لیست، اگر High بود، یک ایموجی 🔥 کنارش بگذار.

۷. مأموریت: سیستم تجربه (Level Up System) – 20 xp
ایده: انجام کارها باید لذت‌بخش باشد. **وظیفه:** یک متغیر XP (تجربه) به برنامه اضافه کن. هر وقت مأموریتی Complete شد، امتیاز زیاد شود. نکته مهم: امتیاز باید در فایل JSON ذخیره شود تا با بستن برنامه نپرد!

۸. مأموریت: گوی پیشگو (The Oracle Sphere) – 20 xp
مشکل: در یک لیست ۵۰ تایی، پیدا کردن "خرید نان" سخت است. **وظیفه:** تابع search_quest را اضافه کن. یک کلمه بگیرد و فقط مأموریت‌هایی را چاپ کند که آن کلمه در عنوانشان باشد.

۹. مأموریت: بازگشت به صفر (The Void Spell) – 20 xp
مشکل: گاهی می‌خواهیم همه چیز را پاک کنیم. (Reset Factory) **وظیفه:** گزینه‌ای مخفی یا انتهایی اضافه کن که با تأیید کاربر (Are you sure?)، لیست مأموریت‌ها و فایل ذخیره را کامل پاک کند.

۱۰. مأموریت: کاتب اتوماتیک (The Auto-Scribe) – 20 xp
مشکل: می‌خواهم لیستم را روی کاغذ پرینت کنم، اما فایل JSON ناخواناست. **وظیفه:** تابعی بنویس که لیست را به صورت یک فایل متنی تمیز (.txt) ذخیره کند. (Export)

آیا سفر را در این مرحله کامل کرده‌ای، ماجراجو؟ اگر بله، کتاب را ورق بزن؛ کتابخانه سایه‌ها (نتیجه مأموریت‌ها) در صفحه بعد انتظار تو را می‌کشد.

کتابخانه سایه‌ها (نتیجه مأموریت‌ها)

۱. مأموریت: سپر محافظ (اصلاح add_quest در main.py)

این کد را جایگزین تابع قبلی کن. ما از یک حلقه while استفاده می‌کنیم تا کاربر مجبور شود چیزی بنویسد.

```
1. def add_quest():
2.     print("\n--- Add New Quest ---")
3.
4.     # Input Validation Loop for Title
5.     while True:
6.         title = input("Enter quest title: ").strip()
7.         if title: # If title is not empty
8.             break
9.         print("Error: Title cannot be empty! Try again.")
10.
11.     description = input("Enter quest description: ").strip()
12.
13.     # Create and add
14.     new_quest = Quest(title, description)
15.     quest_list.append(new_quest)
16.     print("\n[Quest added successfully!]")
```

۲. مأموریت: طلسم نابودی (افزودن delete_quest به main.py)

این تابع جدید را اضافه کن و فراموش نکن که در حلقه while اصلی، گزینه آن را هم بگذاری.

```
1. def delete_quest():
2.     show_all_questions() # Show list so user knows the numbers
3.
4.     if not quest_list:
5.         return
6.
7.     try:
8.         choice = int(input("Enter number to DELETE: ")) - 1
9.
10.        if 0 <= choice < len(quest_list):
11.            removed = quest_list.pop(choice) # Removes item from list
12.            print(f"\n[Quest '{removed.title}' has been deleted
forever!]")
13.        else:
14.            print("Invalid number.")
15.
16.    except ValueError:
17.        print("Please enter a valid number.")
```

۳. مأموریت: عینک حقیقت‌بین (اصلاح show_all_questions در main.py)

این کد هوشمندتر است و قابلیت فیلتر کردن دارد.

```
1. def show_all_questions():
```

```

2.     print("\n--- Your Quest Log ---")
3.     if not quest_list:
4.         print("Log is empty.")
5.         return
6.
7.     # Ask for filter mode
8.     print("1. Show All")
9.     print("2. Show Pending Only")
10.    mode = input("Choose view mode (1/2): ")
11.
12.    for index, quest in enumerate(quest_list):
13.        # Filter Logic
14.        if mode == '2' and quest.status == "Complete":
15.            continue # Skip this iteration if quest is complete
16.
17.        print(f"\n--- Quest {index + 1} ---")
18.        quest.display_info()
19.        print("-----")

```

۴. مأموریت: قلم ویرایش (افزودن edit_quest به main.py)

این تابع اجازه می‌دهد اطلاعات را تغییر دهیم بدون اینکه مأموریت را حذف کنیم.

```

1. def edit_quest():
2.     show_all_quests()
3.     if not quest_list: return
4.
5.     try:
6.         index = int(input("Enter number to EDIT: ")) - 1
7.         if 0 <= index < len(quest_list):
8.             quest = quest_list[index]
9.
10.            print(f"Editing '{quest.title}'...")
11.
12.            # Get new data (leave empty to keep old)
13.            new_title = input(f"New Title [{quest.title}]: ").strip()
14.            new_desc = input(f"New Description [{quest.description}]: ").strip()
15.
16.            if new_title:
17.                quest.title = new_title
18.            if new_desc:
19.                quest.description = new_desc
20.
21.            print("[Quest updated!]")
22.        else:
23.            print("Invalid number.")
24.    except ValueError:
25.        print("Invalid input.")

```

۵ و ۶. مأموریت: ساعت شنی و نشان پادشاهی (تغییر quest_model.py)

این تغییرات باید در فایل `quest_model.py` (کلاس `quest_model`) اعمال شوند. متد `__init__` و `display_info` تغییر می‌کنند.

```

1. # In quest_model.py
2.
3. class Quest:
4.     def __init__(self, title, description, deadline="None",
priority="Normal"):
5.         self.title = title
6.         self.description = description
7.         self.status = "Pending"
8.         self.created_at = datetime.datetime.now()
9.
10.        # --- NEW ATTRIBUTES ---
11.        self.deadline = deadline
12.        self.priority = priority
13.
14.    def display_info(self):
15.        # Add icon for High Priority
16.        icon = "🔴" if self.priority == "High" else "🟢"
17.
18.        print(f"{icon} Title: {self.title}")
19.        print(f"    Priority: {self.priority}")
20.        print(f"    Deadline: {self.deadline}")
21.        print(f"    Status: {self.status}")
22.        # ... rest of the code ...

```

نکته: در `main.py` هم باید هنگام `dd_quest` این ورودی‌های جدید را از کاربرگیری و به کلاس بدهی

۷. مأموریت: سیستم تجربه (اصلاح `save` و `load` در `main.py`)

ما باید XP را هم کنار لیست مأموریت‌ها ذخیره کنیم. ساختار JSON کمی تغییر می‌کند.

```

1. # In main.py
2. total_xp = 0 # Global variable
3.
4. def save_requests():
5.     data = {
6.         "xp": total_xp,
7.         "quests": []
8.     }
9.
10.    # Add requests to data
11.    for q in request_list:
12.        data["quests"].append({
13.            "title": q.title,
14.            "description": q.description,
15.            "status": q.status,
16.            # Add other fields like deadline/priority here too
17.        })
18.
19.    with open(SAVE_FILE, "w") as f:
20.        json.dump(data, f, indent=4)

```



```

21.
22. def load QUESTS():
23.     global quest_list, total_xp
24.     try:
25.         with open(SAVE_FILE, "r") as f:
26.             data = json.load(f)
27.
28.             # Load XP (if it exists, otherwise 0)
29.             total_xp = data.get("xp", 0)
30.
31.             # Load Quests
32.             quest_data = data.get("quests", [])
33.             for item in quest_data:
34.                 # Re-create objects...
35.                 pass
36.     except FileNotFoundError:
37.         print("No save file.")

```

۸. مأموریت: گوی پیشگو (افزودن search_quest به main.py)

```

1. def search_quest():
2.     keyword = input("Search for: ").lower()
3.     found = False
4.
5.     print(f"\n--- Search Results for '{keyword}' ---")
6.     for q in quest_list:
7.         if keyword in q.title.lower():
8.             q.display_info()
9.             print("----")
10.            found = True
11.
12.     if not found:
13.         print("No quests found.")

```

۹. مأموریت: بازگشت به صفر (افزودن reset_data به main.py)

```

1. def reset_data():
2.     confirm = input("WARNING: This will delete ALL quests and XP. Are you
3.     sure? (yes/no): ")
4.
5.     if confirm.lower() == "yes":
6.         global quest_list, total_xp
7.         quest_list = []
8.         total_xp = 0
9.
10.        # Overwrite file with empty data
11.        with open(SAVE_FILE, "w") as f:
12.            f.write("{}")
13.
14.        print("[Memory Wiped. Fresh Start!]")
15.    else:
16.        print("[Reset cancelled.]")

```

۱۰. مأموریت: کاتب اتوماتیک (افزودن export_to_txt به main.py)

```
1. def export_to_txt():
2.     filename = "my_quest_log.txt"
3.
4.     with open(filename, "w", encoding="utf-8") as f:
5.         f.write(f"ADVENTURER QUEST LOG - XP: {total_xp}\n")
6.         f.write("="*30 + "\n")
7.
8.         for i, q in enumerate(quest_list, 1):
9.             f.write(f"{i}. {q.title} [{q.status}]\n")
10.            f.write(f"    Desc: {q.description}\n")
11.            f.write("-" * 20 + "\n")
12.
13.     print(f"[Exported successfully to {filename}]")
```

بخش ششم: ورود به تالار پیشگویان (پیش نیاز هوش مصنوعی)

تبریک می‌گم تا الان تبدیل به یه برنامه نویس پایتون شدی.

در ادامه کتاب چند تا کتابخونه خیلی باحال رو بهتون می‌زنم کتابخونه‌هایی که بشدت کاربردین مخصوصا برای بچه‌هایی که میخوان در آینده هوش مصنوعی کارکنن و این کتابخونه هارو بشدت نیاز دارن و پیشنهاد میکنم حتی اگه هوش مصنوعی نمیخواید کار کنید این کتابخونه هارو یاد بگیرید چون استفاده‌های دیگه‌ای هم دارن.

فصل ۱۷: دروازه احضار و آزمایشگاه جادویی (Pip & (Venv

مأموریت فصل: یاد می‌گیریم چطور آزمایشگاه جادویی خود را ایزوله کنیم (venv) و سپس طلسم‌های جدیدی که دیگران نوشته‌اند را با pip احضار کنیم.

نقشه راه:

- آزمایشگاه ایزوله (venv): چرا نباید کتابخانه‌ها را در محیط اصلی سیستم نصب کرد؟
- فعال‌سازی آزمایشگاه: طلسم ورود به حباب جادویی.
- دروازه احضار (pip): نصب اولین کتابخانه خارجی.
- فایل نیازمندی‌ها: ساخت requirements.txt برای به اشتراک گذاشتن جادوها.
- **پروژه:** ساخت «تابلوی اعلانات جادویی» با یک کتابخانه خارجی.

۱. آزمایشگاه ایزوله (venv)

ماجرای عزیز، به تالار پیشگویان خوش آمدی. در اینجا ما با کتابخانه‌های عظیمی مثل pandas و numpy و matplotlib کار خواهیم کرد. اما قبل از دست زدن به این قدرت‌ها، باید یک قانون مهم امنیتی را یاد بگیریم.

خطر آلودگی جادویی: تصور کن تمام معجون‌های دنیا را در یک دیگ بزرگ بریزی. چه می‌شود؟ انفجار! پایتون نصب شده روی کامپیوتر تو (System Python)، همان «دیگ بزرگ» است. اگر برای هر پروژه، کتابخانه‌های مختلف را مستقیم روی آن نصب کنی، به زودی تداخل ایجاد می‌شود و همه چیز از کار می‌افتد.

راه حل: حباب جادویی (Virtual Environment) ما برای هر پروژه جدید، یک «آزمایشگاه ایزوله» می‌سازیم. این یک نسخه کپی شده از پایتون است که مخصوص همان پروژه است. اگر این آزمایشگاه منفجر شود، هیچ آسیبی به بقیه سیستم نمی‌رسد.

طلسم ساخت آزمایشگاه: ترمینال CMD یا PowerShell یا Terminal را باز کن، به پوشه پروژه‌ات برو و این دستور را بنویس:

```
# Windows
python -m venv my_magic_lab

# Mac / Linux
python3 -m venv my_magic_lab
```

با این دستور، یک پوشه جدید به نام my_magic_lab ساخته می‌شود. این آزمایشگاه شخصی توست.

۲. فعال سازی آزمایشگاه (ورود به حباب)

ساختن آزمایشگاه کافی نیست؛ باید وارد آن شویم. وقتی وارد شوی، نام آزمایشگاه را در کنار خط فرمان خود خواهی دید (مثلاً my_magic_lab). **طلسم ورود (بسته به سیستم عامل):**

- برای ویندوز

```
my_magic_lab\Scripts\activate
```

- برای مک و لینوکس

```
source my_magic_lab/bin/activate
```

اگر موفق شده باشی، ابتدای خط فرمان تو تغییر می‌کند و چیزی شبیه به این می‌بینی

```
(my_magic_lab) C:\Users\Adventurer\Project>
```

حالا تو درایمنی کامل هستی! هر چه نصب کنی، فقط همین جا می ماند.

نکته مهم: یادت باشه وقتی ترمینال رو ببندی و دوباره باز کنی نیاز هست دوباره این دستور فعال سازی رو بزنی و وارد آزمایشگاه بشی.

۳. دروازه احضار (pip)

حالا که در آزمایشگاه هستیم، زمان احضار جادوهای دیگران است. در دنیای پایتون، میلیون ها جادوگر کدهای خود را در مخزنی به نام **PyPI** (Python Package Index) گذاشته اند. (منظور از مخزن محلیه که یه عالمه مازول و کد آماده داخل اون وجود داره و ما ازش چیزی که لازم داریم رو دانلود و نصب میکنیم)

ابزار ما برای آوردن این بسته ها، **pip** نام دارد.

بیایید اولین کتابخانه خارجی را نصب کنیم: ما می خواهیم کتابخانه ای به نام **art** را نصب کنیم که برای چاپ متن های هنری و زیبا (ASCII Art) در ترمینال استفاده می شود.

در همان ترمینال (که آزمایشگاهش فعال است) بنویس:

```
pip install art
```

ترمینال شروع به دانلود و نصب می کند. وقتی تمام شد، می گوید

```
Successfully installed art....
```

تبریک! تو اولین جادوی خارجی خود را احضار کردی.

۴. فایل نیازمندی ها (requirements.txt)

فرض کن می خواهی کد خود را به یک دوست بدهی. او از کجا بداند که باید کتابخانه **art** را نصب کند تا کد تو کار کند؟

ما لیست تمام مواد اولیه (کتابخانه ها) را در یک فایل متنی به نام **requirements.txt** ذخیره می کنیم. (اسم دیگه ای هم میشه برای این فایل گذاشت ولی چیزی که رسمه همین **requirements.txt** هست)

طلسم ثبت مواد اولیه :

```
pip freeze > requirements.txt
```

این دستور، لیست تمام چیزهایی که در آزمایشگاهت نصب کرده ای را در فایل می نویسد.

طلسم نصب مواد اولیه (برای دوستت): وقتی دوستت کد تو را گرفت، فقط کافیه بنویسد:

```
pip install -r requirements.txt
```



```
-----  
Your magic lab is ready.  
-----
```

آفرین! تو حالا نه تنها کد می‌نویسی، بلکه می‌دانی چطور محیط کار حرفه‌ای بسازی و از قدرت هزاران برنامه‌نویس دیگر در کدهای خودت استفاده کنی. آزمایشگاه تو آماده است.

🌟 ارتقاء لول: ۸۵ → ۸۰ 🌟

مهارت‌های جدید:

◆ ابزار Venv

با نام باستانی: جباب ایزوله

توضیح: ساخت جهان‌های موازی برای هر

پروژه تا تداخلی در قوانین آن‌ها پیش نیاید.

فصل ۱۸: طلسم های NumPy

(جادوی محاسبات فوق

سریع)

مأموریت فصل : لیست‌های پایتون برای چند صد آیتم عالی‌اند، اما با میلیون‌ها داده‌دهنده می‌شوند. ما به ابزاری نیاز داریم که محاسبات را با سرعت نور انجام دهد. در این فصل کدی می‌نویسیم و مقایسه می‌کنیم تا ببینیم استفاده از کتابخانه نام‌پای (NumPy) چه تأثیر عظیمی روی سرعت اجرای محاسبات ما دارد.

نقشه راه:

- احضار NumPy: نصب و وارد کردن کتابخانه.
- ndarray چیست؟ (تفاوت ارتش منظم با گروه ماجراجویان).
- جادوی Vectorization: حذف حلقه‌های for.
- جادوی ابعاد: ساخت ماتریس‌ها و ضرب آن‌ها.
- جعبه ابزار جادویی: ۱۰ طلسم پرکاربرد NumPy با مثال.
- مسابقه بزرگ: مفهوم زمان‌سنجی و تفاوت سرعت.
- پروژه: چالش ۱,۰۰۰,۰۰۰ ماجراجو (برگزاری مسابقه سرعت).

۱. احضار NumPy

ترمینال را باز کن و این کتابخانه را احضار کن (اگه خواستی میتونی وارد آزمایشگاه ایزوله (venv) که ساختی هم بشی):

```
pip install numpy
```

حالا در فایل پایتون خود، آن را وارد می‌کنیم. جادوگران معمولاً برای NumPy یک نام مستعار کوتاه (np) انتخاب می‌کنند تا نوشتن کد سریع‌تر شود. (این np as np)

```
1. import numpy as np
```

۲. ndarray چیست؟ (ارتش منظم)

در فصل‌های قبل، ما از لیست‌ها (List) استفاده می‌کردیم. لیست‌ها مثل یک «کوله پشتی» هستند که می‌توانند هر چیزی را در خود نگه دارند: یک عدد، یک رشته، یک شیء Potion و... این انعطاف‌پذیری عالی است، اما هزینه دارد: سرعت پایین.

آرایه جادویی NumPy (یا ndarray): این آرایه مثل یک «ارتش منظم» است. **قانون سخت‌گیرانه:** تمام سربازان باید یک نوع باشند (مثلاً فقط عدد صحیح). **پاداش:** نام‌پای بسیار سریع‌تره، چون محاسبات رو خودش به عهده می‌گیره و در پس‌زمینه از زبان C و تکنیک‌های پیشرفته استفاده می‌کند. **طلسم ساخت آرایه:**

```
1. import numpy as np
2.
3. # 1. Python List (The Backpack)
4. my_list = [1, 2, 3, 4, 5]
5.
6. # 2. NumPy Array (The Army)
7. my_array = np.array([1, 2, 3, 4, 5])
8.
9. print(f"List: {my_list}")
10. print(f"Array: {my_array}")
11. print(type(my_array)) # <class 'numpy.ndarray'>
```

۳. جادوی Vectorization (حذف حلقه‌ها)

اینجا جایی است که جادو اتفاق می‌افتد. فرض کنید می‌خواهیم تمام اعداد لیست را در ۲ ضرب کنیم.

روش قدیمی (پایتون خالص): باید یک حلقه for بنویسیم و دانه دانه با اعداد حرف بزنیم:

```
1. # Old way
2. numbers = [1, 2, 3]
3. doubled = []
4. for x in numbers:
5.     doubled.append(x * 2)
```

روش NumPy (Vectorization): ما به کل ارتش یک فرمان واحد می دهیم و همه همزمان اطاعت می کنند! نیازی به حلقه نیست.

```
1. # NumPy way
2. arr = np.array([1, 2, 3])
3. doubled = arr * 2 # Magic! No loop needed.
4.
5. print(doubled) # Output: [2 4 6]
```

شما می توانید جمع، تفریق، ضرب، تقسیم و توان را روی کل آرایه با یک دستور انجام دهید. (و این خیلی سریعتره)

۴. جادوی ابعاد: ماتریس ها

در علم داده و هوش مصنوعی، ما اغلب با جداول اعداد (ردیف و ستون) کار می کنیم. به این ها ماتریس (Matrix) می گویند. NumPy استاد ساخت ماتریس است.

ساخت ماتریس (آرایه ۲ بعدی):

```
1. # Creating a 2x3 Matrix (2 rows, 3 columns)
2. matrix_a = np.array([
3.     [1, 2, 3],
4.     [4, 5, 6]
5. ])
6.
7. print("--- Matrix A ---")
8. print(matrix_a)
9. print(f"Shape: {matrix_a.shape}") # Output: (2, 3)
```

ضرب ماتریسی (The Dot Product): این یکی از مهم ترین طلسم ها در هوش مصنوعی است. ضرب ماتریسی با ضرب معمولی فرق دارد. در NumPy، ما از طلسم dot یا عملگر @ استفاده می کنیم.

```
1. matrix_b = np.array([
2.     [10, 20],
3.     [30, 40],
4.     [50, 60]
5. ])
6.
7. # Multiplying Matrix A (2x3) by Matrix B (3x2)
8. # Result will be a 2x2 Matrix
9. result = np.dot(matrix_a, matrix_b)
10. # Or simply: result = matrix_a @ matrix_b
11.
```

```
12. print("--- Matrix Multiplication Result ---")
13. print(result)
```

۵. جعبه ابزار جادویی: ۱۰ طلسم پرکاربرد

یک دانشمند داده همیشه به ابزارهای سریع نیاز دارد. در اینجا ۱۰ مورد از پرکاربردترین طلسم های NumPy را برای لیست کرده ایم که در هر مأموریتی به کارت می آیند.

۱. **np.arange** (ساخت دنباله اعداد): مثل range در پایتون است، اما آرایه می سازد.

```
1. # Create numbers from 0 to 9
2. seq = np.arange(10)
3. print(seq) # [0 1 2 3 4 5 6 7 8 9]
```

۲. **np.zeros** و **np.ones** (ساخت بوم خالی): ساخت آرایه ای پر از صفر یا یک (برای شروع محاسبات).

```
1. # Create a 3x3 matrix of zeros
2. zeros = np.zeros((3, 3))
3. print(zeros)
4.
```

۳. **np.random.randint** (تاس جادویی): تولید اعداد تصادفی (بسیار مهم برای شبیه سازی).

```
1. # 5 random numbers between 1 and 100
2. lucky_numbers = np.random.randint(1, 100, 5)
3. print(lucky_numbers)
```

۴. **np.reshape** (تغییر آرایش ارتش): تغییر شکل آرایه (مثلاً تبدیل یک صف طولانی به یک مستطیل).

```
1. arr = np.arange(12) # 0 to 11 (12 items)
2. # Change to 3 rows, 4 columns
3. matrix = arr.reshape(3, 4)
```

۵. **np.max**, **np.min**, **np.sum** (گزارش میدان): گرفتن آمار سریع از کل آرایه.

```
1. scores = np.array([10, 50, 90, 20])
2. print(f"Max Score: {np.max(scores)}") # 90
3. print(f"Total Score: {np.sum(scores)}") # 170
```

۶. **np.mean** (میانگین): مهم ترین طلسم آماری.

```
1. print(f"Average Score: {np.mean(scores)}")
```

۷. **np.sort** (مرتب سازی): مرتب کردن اعداد از کم به زیاد.

```
1. sorted_scores = np.sort(scores)
2. print(sorted_scores) # [10 20 50 90]
```

۸. **np.unique** (حذف تکراری‌ها): پیدا کردن آیتم‌های یکتا.

```
1. items = np.array(["Sword", "Shield", "Sword", "Potion"])
2. print(np.unique(items)) # ['Potion' 'Shield' 'Sword']
```

۹. **فیلتر کردن شرطی (Boolean Masking)**: پیدا کردن اعدادی که شرط خاصی دارند (جادوی خالص!).

```
1. # Find all scores greater than 40
2. high_scores = scores[scores > 40]
3. print(high_scores) # [50 90]
```

۱۰. **np.concatenate** (اتحاد ارتش‌ها): چسباندن دو آرایه به هم.

```
1. a = np.array([1, 2])
2. b = np.array([3, 4])
3. c = np.concatenate((a, b))
4. print(c) # [1 2 3 4]
```

۶. مسابقه بزرگ: مفهوم زمان سنجی

ما ادعا کردیم که NumPy سریع است. اما یک دانشمند داده به «ادعا» اعتماد نمی‌کند؛ او به «داده» نیاز دارد. ما باید راهی برای اندازه‌گیری زمان اجرای کد پیدا کنیم. برای این کار، ماژول استاندارد time را احضار می‌کنیم. منطق ساده است: زمان شروع را ثبت کن (start). طلسم را اجرا کن. زمان پایان را ثبت کن (end). مدت زمان = پایان - شروع.

```
1. import time
2.
3. # Record the start time
4. start_time = time.time()
5.
6. # ... The spell runs here ...
7. # (Example: Count to 1,000,000)
8. x = 0
9. for i in range(1000000):
10.     x += 1
11.
12. # Record the end time
13. end_time = time.time()
```

```

14.
15. # Calculate duration
16. duration = end_time - start_time
17. print(f"The spell took {duration} seconds.")

```

حالا که ابزار زمان سنجی را داریم، آماده ی برگزاری بزرگترین مسابقه تاریخ انجمن هستیم.

۷. پروژه: چالش ۱,۰۰۰,۰۰۰ ماجراجو (The Great Speed Race) – 100 xp

ماجراجو، انجمن ما به تازگی ۱,۰۰۰,۰۰۰ عضو جدید جذب کرده است! ما لیستی از امتیازات تمام این اعضا (اعداد بین ۰ تا ۱۰۰) را داریم. شورای عالی جادوگران از ما خواسته تا «میانگین» امتیازات کل اعضا را محاسبه کنیم. ما دو داوطلب برای انجام این محاسبه داریم:

۱. **حلقه پایتون (Python Loop)** روش سنتی، دقیق اما قدم به قدم

۲. **نامپای (NumPy)** جادوی مدرن، برداری و بی نهایت سریع.

شما باید کدی بنویسید که این دو را در یک مسابقه عادلانه شرکت دهد و برنده را اعلام کند.

تصور نهایی: این مسابقه چطور برگزار می شود؟

قبل از کدنویسی، صحنه مسابقه را تصور کن. داور مسابقه (ماژول time کورنومتر دقیق در دست دارد).

- **رانند اول:** به پایتون می گوئیم: «شروع کن!» پایتون تک تک ۱ میلیون عدد را برمی دارد، جمع می کند و در نهایت تقسیم می کند. این کار طول می کشد. (روش لیست)
- **رانند دوم:** به نامپای می گوئیم: «شروع کن!» نامپای با یک طلسم میانگین کل اعداد را یکجا پردازش می کند.
- **پایان:** داور زمان ها را مقایسه می کند و می گوید نامپای چند برابر سریع تر بوده است.

نقشه و طرح اولیه

برای برگزاری این مسابقه بزرگ، به یک طرح دقیق نیاز داریم:

۱. **تولید ارتش داده ها (Data Generation):** ما نمی توانیم ۱ میلیون عدد را دستی تایپ کنیم!

- **منطق:** از جادوی np.random.randint استفاده می کنیم تا ۱ میلیون عدد تصادفی تولید کنیم.

۲. **آماده سازی زمین مسابقه (Fairness):** برای اینکه مسابقه عادلانه باشد، باید داده ها را به

زبان هر شرکت کننده ترجمه کنیم.

- نامپای داده ها رو از جنس **Array** میسازه.
- پایتون معمولی با **List** کار می کند. پس باید داده های تولید شده با نامپای رو به لیست معمولی تبدیل کنیم.

۳. داور زمان: (The Stopwatch) چطور بفهمیم چقدر طول کشیده؟

- منطق: از کتابخانه time استفاده می کنیم.
 - لحظه شروع (start) را ثبت می کنیم.
 - عملیات را انجام می دهیم.
 - لحظه پایان (end) را ثبت می کنیم.
 - مدت زمان = پایان - شروع.

۴. محاسبه شتاب: (The Verdict) در آخر، زمان پایتون را بر زمان نامپای تقسیم می کنیم تا بفهمیم نامپای چند برابر سریع تر است.

چالش های فکری

قبل از دیدن کد، به این سوالات تکنیکی فکر کن:

چالش ۱: تولید انبوه چطور با یک خط کد، یک میلیون عدد بین ۰ تا ۱۰۰ می سازی؟ (راهنمایی: `np.random.randint(0, 100, 1000000)`)

چالش ۲: تبدیل آرایه به لیست خروجی نامپای یک Array است. چطور آن را به یک List معمولی تبدیل می کنی تا پایتون بتواند با روش سنتی (حلقه for روی آن کار کند؟) (راهنمایی: استفاده از تابع `list()`)

چالش ۳: میانگین گیری در یک خط پایتون برای میانگین گیری باید حلقه بنویسد. نامپای چه تابعی دارد که میانگین کل آرایه را یکجا می دهد؟ (راهنمایی: `np.mean()`)

کد نهایی پروژه (The Speed Test)

این کد را در فایل `speed_test.py` بنویس و اجرا کن. آماده باش که شگفت زده شوی!

```
1. # speed_test.py
2. # The Great Race: Python Lists vs. NumPy Arrays
3.
4. import numpy as np
5. import time
6.
7. print("--- Generating Data for 1,000,000 Adventurers ---")
8.
9. # 1. Create the NumPy Array (The Modern Army)
10. # Generates 1 million random numbers between 0 and 100
11. numpy_array = np.random.randint(0, 100, 1_000_000)
12.
13. # 2. Create the Python List (The Traditional Backpack)
```

```

14. # We convert the array to a list so Python takes the "hard way" (for
    fairness)
15. python_list = list(numpy_array)
16.
17. print("Data ready! Starting the race...\n")
18.
19. # --- RACE 1: The Old Way (Python Loop) ---
20. # We start the stopwatch
21. start_time = time.time()
22.
23. total_sum = 0
24. for score in python_list:
25.     total_sum += score
26.
27. average_python = total_sum / len(python_list)
28.
29. # We stop the stopwatch
30. end_time = time.time()
31. python_duration = end_time - start_time
32.
33. print(f"Python Loop Result: {average_python}")
34. print(f"Python Time: {python_duration} seconds\n")
35.
36.
37. # --- RACE 2: The Modern Way (NumPy) ---
38. # We start the stopwatch again
39. start_time = time.time()
40.
41. # NumPy calculates mean in ONE ultra-fast step (Vectorization)
42. average_numpy = np.mean(numpy_array)
43.
44. # We stop the stopwatch
45. end_time = time.time()
46. numpy_duration = end_time - start_time
47.
48. print(f"NumPy Result: {average_numpy}")
49. print(f"NumPy Time: {numpy_duration} seconds\n")
50.
51.
52. # --- The Verdict ---
53. # Calculate how many times faster NumPy was
54. speedup = python_duration / numpy_duration
55.
56. print("-----")
57. print(f"CONCLUSION: NumPy was {speedup} times faster!")
58. print("-----")

```

اجرا کن و نتیجه را ببین! وقتی کد را اجرا کنی، احتمالاً می بینی که پایتون معمولی حدود ۰.۱ تا ۰.۲ ثانیه طول می کشد، اما نامپای آنقدر سریع است (مثلاً ۰.۰۰۱ ثانیه) که انگار زمان متوقف شده است. در قسمت CONCLUSION خواهی دید که نامپای معمولاً بین ۵۰ تا ۲۰۰ برابر سریع تر است!

جشن پیروزی و تحلیل

شگفت‌انگیز است، نه؟ تو الان قدرت واقعی **علم داده** را لمس کردی. چرا نامپای اینقدر سریع‌تر است؟ چون پایتون معمولی (حلقه for مجبور است اعداد را دانه دانه بررسی کند، اما نامپای به زبان C) نوشته شده و عملیات را به صورت بهینه و سریع انجام می‌دهد. این همان دلیلی است که دانشمندان داده، مهندسان هوش مصنوعی و تحلیلگران در سراسر جهان از **NumPy** استفاده می‌کنند. ابزار درست، غیرممکن را ممکن می‌کند.

🌟 ارتقاء لول: ۹۰ → ۸۵ 🌟

مهارت‌های جدید:

♦ آرایه‌های NumPy

با نام باستانی: ماتریس‌های قدرتمند
توضیح: انجام محاسبات میلیونی در کسری
از ثانیه. سرعتی فراتر از درک انسان.

فصل ۱۹: کتابخانه بزرگ پانداز

(کار با داده‌های جدولی)

مأموریت فصل: آرایه‌های NumPy برای اعداد عالی بودند، اما اگر داده‌های ما ترکیبی از «نام»، «تاریخ» و «عدد» باشند چه؟ (مثل یک فایل اکسل). ما به ابزاری برای مدیریت جداول عظیم داده نیاز داریم. به کتابخانه بزرگ Pandas خوش آمدید. ما یک فایل اکسل را می‌خوانیم و تحلیل تابلوی امتیازات (خواندن، فیلتر کردن و ذخیره گزارش) را انجام می‌دهیم.

نقشه راه:

- احضار Pandas: نصب و وارد کردن کتابخانه.
- DataFrame چیست؟ (جدول جادویی).
- خواندن فایل اکسل: بارگذاری فایل‌های CSV و XLSX.
- ذره‌بین جادویی: بازرسی و آمارگیری سریع داده‌ها.
- جعبه ابزار پانداس: ۱۰ طلسم پرکاربرد با مثال.
- **پروژه:** تحلیل تابلوی امتیازات (خواندن، فیلتر کردن و ذخیره گزارش).

۱. احضار Pandas

ترمینال را باز کن و این کتابخانه را احضار کن (اگر خواستی میتونی وارد آزمایشگاه ایزوله (venv) که ساختی هم بشی):

```
pip install pandas
```

در کد پایتون، جادوگران همیشه برای Pandas از نام مستعار pd استفاده می‌کنند تا راحت تر و کوتاه تر بشه اسمش برای استفاده. (این as pd)

```
1. import pandas as pd
```

۲. DataFrame چیست؟ (جدول جادویی)

در NumPy با «اعداد» کار می‌کردیم، اما در Pandas با «جدول‌ها» سرو کار داریم.

۲ دسته دیتایی که با پانداز با اون کار داریم:

- Series (سری): یک ستون از داده‌هاست. (یه ستون از اکسل رو تصور کنید)
 - DataFrame (دیتافریم): یک جدول کامل است که از سطرها و ستون‌ها تشکیل شده؛ هر ستون نام دارد و هر سطر یک شماره (Index). (یک اکسل کامل رو تصور کنید)
- طلسم ساخت یک جدول دستی: فرض کنید می‌خواهیم لیست موجودی مغازه جادوگری را به یک جدول تبدیل کنیم.

```
1. import pandas as pd
2.
3. # 1. Prepare data (Dictionary of Lists)
4. data = {
5.     "Item": ["Health Potion", "Mana Potion", "Iron Sword"],
6.     "Price": [50, 70, 150],
7.     "Stock": [10, 5, 2]
8. }
9.
10. # 2. Create the DataFrame
11. df = pd.DataFrame(data)
12.
13. print("--- My Magic Shop Inventory ---")
14. print(df)
```

خروجی:

```
--- My Magic Shop Inventory ---
   Item  Price  Stock
0  Health Potion    50    10
1   Mana Potion    70     5
2   Iron Sword   150     2
```

می‌بینید؟ پایتون حالا داده‌ها را به صورت یک جدول مرتب و زیبا می‌فهمد!

۳. خواندن فایل اکسل (فایل‌های CSV, xlsx)

برای کار با داده‌های واقعی، به جای تایپ دستی، اطلاعات را مستقیماً از فایل‌ها بارگذاری می‌کنیم. فایل‌های اکسل (xlsx) : اگر داده‌های شما در نرم‌افزار اکسل ذخیره شده است. با دستور `pd.read_excel` می‌توانید فایل‌های xlsx را با همان سرعت و سهولت بخوانید و در محیط پایتون استفاده کنید.

```
# Reading a scroll named 'adventurers.xlsx'
df = pd.read_excel("adventurers.xlsx")
```

فایل‌های CSV: فرمت CSV که مخفف مقادیر جدا شده با کاما (Comma-Separated Values) هست، ساده‌ترین روش برای ذخیره داده‌های جدولی به صورت متنی است. در این فایل‌ها، هر سطر (مثلاً اطلاعات یک نفر) است و ستون‌های مختلف با یک علامت کاما (,) از هم جدا می‌شوند.

برای مثال `users.csv` می‌تونه همچین شکلی داشته باشه (خط اول اسم ستون هاست همیشه و از خط دوم دیتا ها شروع میشه)

```
name,age,city
Alex,25,Tehran
Sara,30,Isfahan
```

به زبان ساده، این فایل‌ها مثل یک اکسل هستند که تمام فرمت‌بندی‌ها (مثل رنگ و کادر) حذف شده و فقط متن‌ها باقی مانده‌اند. چون ساختار بسیار ساده‌ای دارند، تقریباً تمام نرم‌افزارها و زبان‌های برنامه‌نویسی (از جمله پایتون) قادر به خواندن آن‌ها هستند. شما می‌توانید فایل اکسل خود را از منوی `Save As` به این فرمت تبدیل کنید تا حجمش کم شده و پردازش آن آسان‌تر گردد. (این فرمت بسیار متداول هست برای کار با دیتا ها)

برای خواندن فایل csv با کمک پانداز از کد زیر استفاده میکنیم

```
# Reading a scroll named 'adventurers.csv'
df = pd.read_csv("adventurers.csv")
```

۴. ذره‌بین جادویی: فیلتر کردن داده‌ها

قدرت واقعی Pandas در «پرس و جو» (Query) است. مثلاً: «فقط آیتم‌هایی را به من نشان بده که قیمتشان زیر ۶۰ سکه است».

در پایتون معمولی، این کار نیاز به حلقه for و if داشت. در Pandas، این کار با یک خط انجام می‌شود:

```
1. # Filter: Find items cheaper than 60 gold
2. cheap_items = df[df["Price"] < 60]
3.
4. print("\n--- Bargain Bin (< 60 Gold) ---")
5. print(cheap_items)
```

خروجی:

```
1.           Item  Price  Stock
2. 0  Health Potion     50     10
3. 3  Wooden Shield     30     15
```

توضیح کد: در اینجا از (Boolean Indexing) استفاده می‌کنیم. عبارت `df["Price"] < 60` روی تمام سطرهاى دیتافریم اجرا می‌شود و برای هر آیتیم بررسی می‌کند که آیا قیمت آن کمتر از ۶۰ است یا خیر. خروجی این دستور یک سری (Series) از مقادیر True (درست) و False (غلط) است. سپس پانداس (Pandas) از این نقشه راه استفاده می‌کند و فقط سطرهایی که مقدار آن‌ها True است را انتخاب کرده و در متغیر `cheap_items` ذخیره می‌کند. این روش بسیار سریع‌تر و خواناتر از نوشتن حلقه‌های for و شرط‌های if است.

۵. جعبه ابزار پانداس: ۱۰ طلسم پرکاربرد

یک دانشمند داده بدون این ۱۰ طلسم نمی‌تواند زندگی کند. فرض کنید متغیر `df` همان جدول مغازه ماست.

۱. `df.head(n)`: n سطر اول جدول را نشان می‌دهد. عالی برای وقتی که فایل خیلی بزرگ است.

```
1. print(df.head(2)) # Show first 2 rows
```

۲. `df.info()`: اطلاعات فنی جدول (جنس ستون‌ها، تعداد داده‌های خالی و...) را می‌گوید.

```
1. df.info()
```

۳. `df.describe()` (آمار سریع): مهم‌ترین طلسم آماری! میانگین، ماکزیمم، مینیمم و انحراف معیار ستون‌های عددی را یکجا می‌دهد.

```
1. print(df.describe())
```

۴. `df.columns` (لیست ستون‌ها): اسامی تمام ستون‌های جدول را برمی‌گرداند.

```
1. print(df.columns) # Index(['Item', 'Price', 'Stock'], dtype='object')
```

۵. `df.shape` (ابعاد): می‌گوید جدول چند سطر و چند ستون دارد. (مثل NumPy).

```
1. print(df.shape) # (4, 3) -> 4 rows, 3 columns
```

۶. `df.sort_values()` (مرتب‌سازی): جدول را بر اساس یک ستون خاص مرتب می‌کند.

```
1. # Sort by Price (High to Low)
2. sorted_df = df.sort_values(by="Price", ascending=False)
```

۷. `df["Column"].mean()` (میانگین ستونی): میانگین یک ستون خاص را حساب می‌کند.

```
1. avg_price = df["Price"].mean()
```

۸. `df.to_csv()` (ذخیره طلسم): جدول تغییر یافته را در یک فایل جدید ذخیره می‌کند.

```
1. df.to_csv("new_inventory.csv", index=False)
```

۹. `df.iloc` (انتخاب با آدرس): دسترسی به یک سلول خاص با شماره سطر و ستون.

```
1. # Row 0, Column 0
2. item_name = df.iloc[0, 0] # "Health Potion"
```

۱۰. `df["NewCol"] = ...` (خلق ستون جدید): ساخت یک ستون جدید بر اساس محاسبات ستون‌های قبلی.

```
1. # Calculate total value (Price * Stock)
2. df["Total Value"] = df["Price"] * df["Stock"]
```

۶. پروژه: تحلیل تابلوی امتیازات (The Grand Guild Analysis) – 100 xp

ماجرابو، انجمن ما بزرگ شده است! ما اکنون اطلاعات ۱۰۰ ماجرابو (نام، کلاس و امتیاز) را در یک طومار طولانی و خسته‌کننده (فایل CSV) داریم. شورای عالی از ما خواسته تا ۳ کار مهم انجام دهیم:

۱. میانگین قدرت کل انجمن را حساب کنیم.

۲. توزیع کلاس‌ها را ببینیم (چند جنگجو، چند جادوگر؟).

۳. لیست «نخبگان» (کسانی که امتیاز بالای ۸۵ دارند) را جدا کنیم و در یک طومار طلایی ذخیره کنیم.

انجام دستی این کار ساعت‌ها طول می‌کشد، اما با کتابخانه **Pandas**، این کار فقط چند ثانیه زمان می‌برد.

تصویر نهایی: پانداس چگونه کار می‌کند؟

قبل از کدنویسی، تصور کن که **Pandas** یک اکسل (Excel) فوق‌العاده قدرتمند است که داخل مغز پایتون زندگی می‌کند.

۱. **بارگذاری**: فایل متنی CSV را می‌گیرد و تبدیل به یک جدول منظم به نام **DataFrame** می‌کند.

۲. **تحلیل**: مثل یک ماشین حساب هوشمند، میانگین ستون‌ها را می‌گیرد.

۳. **فیلتر**: مثل یک الک، ماجراییان ضعیف را جدا می‌کند و فقط قوی‌ها را نگه می‌دارد.

۴. **ذخیره**: جدول نهایی را دوباره به یک فایل تبدیل می‌کند.

گام صفر: خلق داده‌های تمرینی (The Generator)

چون تو هنوز فایل `adventurers.csv` را نداری، اول باید آن را با جادو خلق کنیم. یک فایل جدید به نام `create_data.py` بساز و کد زیر را در آن اجرا کن تا فایل CSV برایت ساخته شود. (بچه‌های کولب و ژوپیتر میتونید در یک سلول جداگانه اول ران کنید و اکیه)

```
1. # create_data.py
2. import pandas as pd
3. import numpy as np
4.
5. # 1. Create dummy data using NumPy magic
6. # Create 100 names like "Adventurer_1", "Adventurer_2"...
7. names = [f"Adventurer_{i}" for i in range(1, 101)]
8.
9. # Randomly choose classes for 100 people
10. classes = np.random.choice(["Mage", "Warrior", "Rogue", "Healer"], 100)
11.
12. # Generate random scores between 50 and 100
13. scores = np.random.randint(50, 100, 100)
14.
15. # 2. Create DataFrame (The Table)
16. df = pd.DataFrame({
17.     "Name": names,
18.     "Class": classes,
19.     "Score": scores
20. })
21.
22. # 3. Save to CSV
23. df.to_csv("adventurers.csv", index=False)
24. print("File 'adventurers.csv' created successfully!")
```

نقشه و طرح اولیه

حالا که فایل `adventurers.csv` را داریم، نوبت پروژه اصلی است. بیایید فایل `analyze_scores.py` را مهندسی کنیم:

۱. **احضار طومار (Read):** باید فایل CSV را بخوانیم و در متغیری به نام `df` مخفف `DataFrame` بریزیم (استفاده از `df` به رسم قدیمیه میتونید هر اسمی دوست دارید بزارید).
• `pd.read_csv()` / `بِزار` :

۲. **جادوی آماری (Statistics):**

- برای میانگین: ستون امتیاز را انتخاب می‌کنیم و متد `mean()` را صدا می‌زنیم.
 - برای شمارش کلاس‌ها: ستون کلاس را انتخاب و `value_counts()` را صدا می‌زنیم.
۳. **الک کردن نخبگان (Filtering):** این جذاب‌ترین بخش است! ما به پانداس می‌گوییم: «کل جدول را نگه دار، اما فقط ردیف‌هایی که امتیازشان بالای ۸۵ است».
- منطق `df [ df["Score"] > 85 ]`

۴. **ثبت تاریخ (Save):** جدول نخبگان را مرتب می‌کنیم و در فایل `elites.csv` ذخیره می‌کنیم.

چالش‌های فکری

قبل از دیدن کد تحلیلگر، به این سوالات فکر کن:

چالش ۱: خواندن فایل با چه دستوری فایل `adventurers.csv` را وارد برنامه می‌کنی؟
(راهنمایی: `pd.read_csv("filename")`)

چالش ۲: انتخاب ستون اگر بخواهیم فقط ستون `"Score"` را داشته باشیم، چطور آن را از `df` بیرون می‌کشیم؟ (راهنمایی: مثل دیکشنری رفتار کن `df["Score"]`)

چالش ۳: شرط گذاشتن چطور به پایتون می‌گوییم: «آنهايي که Score بزرگتر از ۸۵ دارند»؟
(راهنمایی: `df["Score"] > 85`)

کد نهایی پروژه (The Pandas Analyzer)

این کد اصلی ماست. آن را در فایل `analyze_scores.py` بنویس و اجرا کن.

```
1. # analyze_scores.py
2. # The Pandas Analysis Tool
3.
4. import pandas as pd
5.
6. print("--- Loading the Ancient Scroll (CSV) ---")
7.
```



```
8. # 1. Read the file (Import Data)
9. # This loads the CSV into a DataFrame called 'df'
10. df = pd.read_csv("adventurers.csv")
11.
12. # Show the first 5 rows just to peek at the data
13. print("First 5 adventurers:")
14. print(df.head())
15. print("-" * 40)
16.
17.
18. # 2. Statistical Magic
19. # Calculate the average of the "Score" column
20. average_score = df["Score"].mean()
21. print(f"Average Guild Score: {average_score:.2f}")
22.
23. # Count how many adventurers are in each Class
24. print("\nClass Distribution:")
25. print(df["Class"].value_counts())
26.
27.
28. # 3. Filtering the Elites
29. # We want only rows where Score is greater than 85
30. print("\n--- Searching for Elites (Score > 85) ---")
31.
32. elites = df[df["Score"] > 85]
33.
34. # Sort them by Score (Highest score on top)
35. elites_sorted = elites.sort_values(by="Score", ascending=False)
36.
37. print(f"Found {len(elites_sorted)} elite adventurers.")
38. print(elites_sorted.head()) # Show top 5 elites
39.
40.
41. # 4. Saving the Result
42. print("\n--- Saving the Elite List ---")
43. # index=False means we don't save the row numbers (0, 1, 2...)
44. elites_sorted.to_csv("elites.csv", index=False)
45. print("Saved to 'elites.csv'. Mission Complete!")
```

اجراکن و لذت ببر!

۱. فایل را اجرا کن.

۲. آمار دقیق انجمن را در خروجی ببین.

۳. حالا به پوشه پروژه‌ات برو. یک فایل جدید به نام elites.csv می‌بینی. آن را باز کن. می‌بینی؟ فقط بهترین جنگجویان آنجا هستند و ضعیف‌ها حذف شده‌اند.

جشن پیروزی و تحلیل

تبریک می‌گوییم! تو الان رسماً وارد دنیای **Data Science** شدی. کاری که الان کردی (خواندن داده، محاسبه آمار، فیلتر کردن و ذخیره گزارش)، دقیقاً همان کاری است که تحلیلگران داده در

شرکت‌های بزرگ مثل گوگل و آمازون هر روز انجام می‌دهند. تو یاد گرفتی چطور داده‌های با ارزشمند استخراج کنی.

ارتقاء لول: ۹۵ → ۹۰ 🌟

مهارت‌های جدید:

◆ ساختار DataFrame

با نام باستانی: روح داده

توضیح: رام کردن اطلاعات خام و تبدیل

آنها به دانشی قابل فهم و منظم.

فصل ۲۰: نمودار جادویی

(Matplotlib)

مأموریت فصل: یک پیشگو (Data Scientist) می‌داند که پادشاهان و فرماندهان وقت ندارند هزاران سطر عدد را در جدول‌ها بخوانند. آن‌ها «نمودار» می‌خواهند. در این فصل، یاد می‌گیریم چطور داده‌ها را به نمودار بصری زیبا تبدیل کنیم تا داستان پنهان پشت اعداد آشکار شود. به دنیای Matplotlib خوش آمدید.

نقشه راه:

- احضار نقاش سلطنتی (Matplotlib): نصب و راه‌اندازی.
- اولین قلم‌مو (plot): رسم نمودار خطی برای دیدن «روند».
- ستون‌های مقایسه (bar): رسم نمودار میله‌ای.
- زیبایی‌شناسی جادویی: اضافه کردن عنوان، برچسب و رنگ.
- جعبه ابزار نقاش: ۱۰ طلسم پرکاربرد با مثال.
- پروژه: گزارش تصویری انجمن (ترکیب Matplotlib و Pandas).

۱. احضار نقاش سلطنتی (Matplotlib)

ما برای رسم نمودار به یک کتابخانه قدرتمند نیاز داریم. مشهورترین آن‌ها matplotlib است. ابتدا آن را نصب می‌کنیم (اگر خواستی میتونی وارد آزمایشگاه ایزوله (venv) که ساختی هم بشی):

```
pip install matplotlib
```

در کد پایتون، ما زیرمجموعه pyplot را احضار می‌کنیم و طبق سنت قدیمی جادوگران، به آن نام مستعار plt می‌دهیم.

```
1. import matplotlib.pyplot as plt
```

۲. اولین قلم‌مو (plot) - نمودار خطی

نمودار خطی (Line Plot) برای نشان دادن تغییرات در طول زمان (Time Series) عالی است. مثلاً: «قدرت جادویی ما در ۱۰ روز گذشته چقدر رشد کرده است؟»

```
1. import matplotlib.pyplot as plt
2.
3. # 1. Prepare the data
4. days = [1, 2, 3, 4, 5]
5. power_levels = [10, 15, 18, 25, 40]
6.
7. # 2. Cast the plot spell
8. # plt.plot(x_axis, y_axis)
9. plt.plot(days, power_levels)
10.
11. # 3. Reveal the canvas
12. print("Opening the magical canvas...")
13. plt.show()
```

وقتی این کد را اجرا کنید، پنجره‌ای باز می‌شود و خط رشد قدرت را به شما نشان می‌دهد. جادو مرئی شد!

برای بچه‌های colab و ژوپیتِر:

شما نیازی نیست متد plt.show رو صدا بزنید و خودکار زیر سلول کد شما نمودار به نمایش در میاد.

۳. ستون‌های مقایسه (bar) - نمودار میله‌ای

اگر بخواهیم چیزهای مختلف را با هم مقایسه کنیم (مثلاً قدرت شمشیر در برابر قدرت تبر)، از نمودار میله‌ای (Bar Chart) استفاده می‌کنیم.

```

1. # Data
2. weapons = ["Sword", "Axe", "Bow", "Staff"]
3. damage = [50, 65, 40, 80]
4.
5. # Cast the bar spell
6. plt.bar(weapons, damage)
7. plt.show()

```

۴. زیبایی‌شناسی جادویی (شخصی‌سازی)

یک نقشه گنج بدون راهنما و اسم، بی‌ارزش است. ما باید به نمودارمان «عنوان»، «برچسب محورها» و «رنگ» اضافه کنیم.

```

1. days = [1, 2, 3, 4, 5]
2. gold = [100, 80, 50, 120, 200]
3.
4. # Customizing the magic
5. plt.plot(days, gold, color='green', marker='o', linestyle='--')
6.
7. # Adding labels (The Legend)
8. plt.title("My Gold Over Time")           # Title of the spell
9. plt.xlabel("Day of Adventure")           # X-axis label
10. plt.ylabel("Gold Coins")                 # Y-axis label
11. plt.grid(True)                           # Add a grid for better reading
12.
13. plt.show()

```

۵. جعبه ابزار نقاش: ۱۰ طلسم پرکاربرد

یک مصورساز داده (Data Visualizer) باید این ۱۰ ابزار را همیشه در کمربند خود داشته باشد.

۱. `plt.plot(x, y)` (خط زمان): رسم نقاط به هم پیوسته. عالی برای روندها.

```
1. plt.plot([1, 2, 3], [2, 4, 6])
```

۲. `plt.bar(x, height)` (ستون‌ها): رسم میله‌های عمودی. عالی برای مقایسه دسته‌ها.

```
1. plt.bar(['A', 'B'], [10, 20])
```

۳. `plt.scatter(x, y)` (پراکندگی): فقط نقاط را رسم می‌کند (بدون خط). عالی برای دیدن رابطه بین دو چیز (مثلاً رابطه «سن» و «قدرت»).

```
1. plt.scatter([20, 30, 40], [50, 45, 30])
```

۴. `plt.hist(x)` (توزیع): هیستوگرام. نشان می‌دهد که داده‌ها چقدر تکرار شده‌اند (مثلاً چند نفر نمره ۲۰ گرفتند، چند نفر ۱۵ و...).

```
1. scores = [10, 10, 20, 20, 20, 30]
2. plt.hist(scores, bins=3)
```

۵. `plt.pie(x)` (کیک جادویی): نمودار دایره‌ای. برای نشان دادن سهم هر بخش از کل (مثلاً چند درصد جادوگرند، چند درصد جنگجو).

```
1. plt.pie([40, 60], labels=['Mages', 'Warriors'])
```

۶. `plt.title("Name")` (نام‌گذاری): انتخاب نام برای کل نمودار.

```
1. plt.title("Name")
```

۷. `plt.xlabel()` و `plt.ylabel()` (راهنمای محورها): نوشتن توضیح برای محور افقی و عمودی.

```
1. plt.xlabel('Titles label')
2. plt.ylabel('Numbers label')
```

۸. `plt.figure(figsize=(10, 5))` (اندازه بوم): تغییر اندازه تصویر قبل از رسم. (۱۰ اینچ عرض، ۵ اینچ ارتفاع).

```
1. plt.figure(figsize=(8, 6))
```

۹. `plt.savefig("filename.png")` (ذخیره طلسم): به جای نشان دادن روی صفحه، نمودار را به صورت یک عکس ذخیره می‌کند.

```
1. plt.savefig("my_chart.png")
```

۱۰. `plt.legend()` (راهنمای نقشه): اگر چند خط در یک نمودار دارید، نشان می‌دهد کدام رنگ مربوط به کدام داده است.

```
1. plt.plot(x, y1, label="Gold")
2. plt.plot(x, y2, label="Silver")
3. plt.legend() # Shows the legend box
```

۶. پروژه: گزارش تصویری انجمن (The Visual Report) – 100 xp

ماجراجو، در فصل قبل ما فایل `adventurers.csv` را تحلیل کردیم و نخبگان را پیدا کردیم. اما رئیس انجمن، وقت خواندن جداول طولانی را ندارد. او یک گزارش تصویری می‌خواهد. یک

تصویر خوب، از هزار کلمه گویاتر است. امروز می‌خواهیم با استفاده از کتابخانه هنرمند پایتون یعنی **Matplotlib**، داده‌هایمان را نقاشی کنیم.

تصویر نهایی:

قبل از کدنویسی، تصور کن که قرار است چه تابلوهایی را به دیوار انجمن بیاویزیم:

۱. **تابلوی اول: کیکِ ارتش (Pie Chart)** رئیس می‌خواهد بداند ترکیب نیروهایش چطور است. آیا بیشتر سربازانش جادوگر (Mage) «هستند یا جنگجو (Warrior)؟ ما یک نمودار دایره‌ای (مثل یک پیتزا) می‌کشیم که هر قاچ آن نشان‌دهنده یک کلاس است.

۲. **تابلوی دوم: تالار افتخارات (Bar Chart)** رئیس می‌خواهد ۵ قهرمان برتر را ببیند. ما یک نمودار میله‌ای می‌کشیم که در آن، ۵ ستون بلند وجود دارد و ارتفاع هر ستون، نشان‌دهنده امتیاز آن قهرمان است.

نقشه و طرح اولیه

برای خلق این آثار هنری، ما به دو دستیار نیاز داریم:

۱. **دستیار تحلیلگر: (Pandas)** داده‌ها را می‌خواند و آماده می‌کند.
۲. **دستیار نقاش: (Matplotlib)** داده‌ها را می‌گیرد و روی بوم می‌کشد.

مرحله ۱: آماده‌سازی رنگ‌ها: (Data Prep)

- فایل `adventurers.csv` را می‌خوانیم.
- برای نمودار دایره‌ای: باید بشماریم از هر کلاس چند نفر داریم (با `value_counts()`).
- برای نمودار میله‌ای: باید لیست را مرتب کنیم و ۵ نفر اول را برداریم (با `sort_values` و `head`).

مرحله ۲: برپایی بوم: (The Canvas)

- قبل از هر نقاشی، باید بوم سفید را آماده کنیم. (`plt.figure`)

مرحله ۳: نقاشی و ذخیره: (Plot & Save)

- دستور رسم `plt.pie` یا `plt.bar` را صادر می‌کنیم.
- عنوان و برچسب می‌زنیم تا معلوم شود نمودار چیست.
- اثر هنری را در یک فایل عکس (`.png`) ذخیره می‌کنیم.

چالش‌های فکری

قبل از اینکه قلم‌مو را به دست بگیرید، به این سوالات فکر کنید:

چالش ۱: احضار نقاش کتابخانه رسم نمودار در پایتون `matplotlib.pyplot` نام دارد. معمولاً آن را با چه نام مستعاری وارد (`import`) می‌کنیم؟ (راهنمایی: `plt`)

چالش ۲: شمارش خودکار برای نمودار دایره‌ای، ما نیاز داریم بدانیم "چند تا Mage داریم؟". پانداس یک دستور جادویی دارد که مقادیر تکراری یک ستون را می‌شمارد. آن چیست؟ (راهنمایی: `df["Class"].value_counts()`)

چالش ۳: محورهای مختصات در نمودار میله‌ای (Bar Chart)، محور افقی (X) باید نام قهرمانان باشد و محور عمودی (Y) باید امتیازشان. دستور `plt.bar(x, y)` چه چیزی را اول می‌گیرد؟ (پاسخ: اول نام‌ها، دوم امتیازها)

کد نهایی پروژه (The Artist's Studio)

نکته مهم برای بچه‌های ژوپیترو کولب: برای اینکه نمودارها روی هم نیفتند و تمیز نمایش داده شوند، کد زیر را به دو سلول جداگانه تقسیم کنید (بخش ۱ در یک سلول، بخش ۲ در سلول دیگر). اما اگر فایل `py` می‌سازید، همه را یکجا بنویسید.

بخش ۱: آماده‌سازی و نمودار دایره‌ای (Pie Chart)

```
1. # visualize_guild.py
2. # The Visual Report Generator
3.
4. import pandas as pd
5. import matplotlib.pyplot as plt
6.
7. print("--- Summoning Data & Painter ---")
8.
9. # 1. Read the data (Ingredients)
10. # We assume 'adventurers.csv' exists from the previous chapter
11. df = pd.read_csv("adventurers.csv")
12.
13. # --- MISSION 1: The Class Distribution (Pie Chart) ---
14.
15. # Count how many adventurers are in each class
16. # This gives us something like: Warrior: 30, Mage: 25...
17. class_counts = df["Class"].value_counts()
18.
19. # Create a new canvas (Size: 8x8)
20. plt.figure(figsize=(8, 8))
21.
```



```

22. # Draw the Pie
23. # autopct='%1.1f%%' -> Shows the percentage on the chart
24. plt.pie(class_counts, labels=class_counts.index, autopct='%1.1f%%',
startangle=140)
25.
26. # Add Title
27. plt.title("Guild Class Distribution")
28.
29. # Save the spell result
30. plt.savefig("guild_classes.png")
31. print("Saved 'guild_classes.png'")
32.
33. # Show it to us
34. plt.show()

```

بخش ۲: نمودار میله‌ای نخبگان (Bar Chart)

```

1. # --- MISSION 2: The Top 5 Elites (Bar Chart) ---
2.
3. # Sort by score (Highest first) and take top 5
4. top_5 = df.sort_values(by="Score", ascending=False).head(5)
5.
6. # Create a new canvas (Size: 10 wide, 6 tall)
7. plt.figure(figsize=(10, 6))
8.
9. # Draw the Bars
10. # X axis = Names, Y axis = Scores
11. colors = ['gold', 'silver', 'brown', 'blue', 'green']
12. plt.bar(top_5["Name"], top_5["Score"], color=colors)
13.
14. # Decorate the map (Labels)
15. plt.title("Top 5 Guild Members")
16. plt.xlabel("Adventurer Name")
17. plt.ylabel("Score (0-100)")
18.
19. # Add a grid behind bars to read numbers easily
20. plt.grid(axis='y', linestyle='--', alpha=0.7)
21.
22. # Save the spell result
23. plt.savefig("top_5_elites.png")
24. print("Saved 'top_5_elites.png'")
25.
26. # Show it
27. plt.show()

```

اجراکن و تماشاکن! کد را اجرا کن.

۱. یک نمودار دایره‌ای رنگارنگ ظاهر می‌شود که نشان می‌دهد ارتش ما از چه کسانی تشکیل شده.

۲. (اگر فایل پایتون است پنجره را ببند تا بعدی بیاید). نمودار بعدی ظاهر می‌شود: ۵ ستون رنگی (طلایی، نقره‌ای...) که قدرتمندترین اعضا را نشان می‌دهد.

۳. به پوشه پروژه‌ات برو. دو عکس جدید png آنجا ذخیره شده است.

جشن پیروزی و تحلیل

تبریک می‌گویم! تو اکنون «سه‌گانه قدرت علم داده» را تکمیل کردی:

۱. جمع‌آوری و محاسبات (NumPy)

۲. تحلیل و مدیریت (Pandas)

۳. نمایش و هنر (Matplotlib)

حالا تو فقط یک کدنویس نیستی؛ تو یک «تحلیلگر داده» هستی که می‌تواند اطلاعات خام را به دانش قابل فهم تبدیل کند. این مهارتی است که در دنیای واقعی بسیار ارزشمند است. تو آماده‌ای تا مسیر نهایی خود را در دنیای بی‌کران برنامه‌نویسی انتخاب کنی.

🌟 ارتقاء لول: ۹۹ → ۹۵ 🌟

مهارت‌های جدید:

♦ کتابخانه Matplotlib

با نام باستانی: چشم بصیرت

توضیح: تبدیل اعداد خشک و بی‌روح به

تصاویری گویا و رنگارنگ.

♦ لقب جدید:

برنامه نویس ارشد

♦ سطح جدید: A

توضیح: تبریک به سطح

جدیدی رسید.

مأموریت‌های جانبی: تالار پیشگویان

بسیار عالی! رسیدیم به پایان بخش ششم. اینجا جایی است که ماجراجوی ما ابزارهای ساده را کنار گذاشته و با «ابزارهای صنعتی» و قدرتمند پایتون (Numpy, Pandas, Matplotlib) آشنا شده است. این ۴۰ مأموریت (۱۰ تا برای هر فصل) طوری طراحی شده‌اند که تو را برای ورود به دنیای واقعی «هوش مصنوعی» و «تحلیل داده» آماده کنند.

قانون تالار: ماجراجو! تو دیگر یک شاگرد نیستی که منتظر درس استاد باشی؛ تو اکنون وارد قلمروی **مهندسی داده** شده‌ای. در دنیای واقعی، هیچ برنامه‌نویسی همه دستورات pandas یا matplotlib را حفظ نیست. قدرت واقعی یک مهندس، در **توانایی جستجو و حل مسئله** است. برای انجام این ۴۰ مأموریت، باید طبق این ۳ اصل پیش بروی:

۱. **کتابخانه اصلی اینترنت است:** قبل از اینکه تسلیم شوی، باید یاد بگیری که پاسخ را در مستندات رسمی پیدا کنی. خواندن این سایت‌ها دشوار اما حیاتی است:

- سایت **Numpy**: برای درک ریاضیات آرایه‌ها.
- سایت **Pandas**: برای یادگیری متدهای تحلیل داده.
- سایت **Matplotlib**: برای دیدن گالری نمودارها و کد رسم آن‌ها.
- سایت **PyPI.org**: برای شناختن کتابخانه‌های جدیدی که نصب می‌کنی.

۲. **دستیار هوشمند را احضار کن:** گیر کردی؟ باگ خوردی؟ نترس! از **Gemini**, **ChatGPT** یا **DeepSeek** استفاده کن. اما نه برای اینکه کد آماده بگیری و کپی کنی؛ بلکه بپرس:

- "چرا این خط کد ارور میدهد؟"
- "بهترین روش برای فیلتر کردن داده‌ها در پاندا چیست؟"
- "تفاوت بین لیست و آرایه نامپای رو برام توضیح بده."

۳. **پاسخ‌نامه (کتابخانه سایه‌ها) آخرین سنگر است:** فقط زمانی به کدهای پایین صفحه نگاه کن که تمام راه‌ها را رفته باشی و کدت کار نکند، یا وقتی که کدت کار کرد و خواستی راه حل خودت را با راه حل ما مقایسه کنی.

مأموریت‌ها

فصل ۱۷: دروازه احضار (Pip & Venv)

۱. مأموریت: خلق جهان موازی (Create Venv) – 20 xp

- **وظیفه:** یک پوشه جدید بساز. یک محیط مجازی (venv) به نام `potion_lab` در آن بساز و فعالش کن. نشان بده که پرانتز (`potion_lab`) کنار ترمینال ظاهر شده.

۲. مأموریت: لیست مواد اولیه (Pip List) – 20 xp

- **وظیفه:** داخل محیط مجازی، دستور `pip list` را بزن. بین چه چیزهایی از قبل نصب است؟ (باید خیلی خلوت باشد).

۳. مأموریت: احضار رنگ‌ها (Install Colorama) – 20 xp

- **وظیفه:** کتابخانه `colorama` را نصب کن. کدی بنویس که متن "`Fire Spell`" را به رنگ قرمز و "`Ice Spell`" را به رنگ آبی چاپ کند.

۴. مأموریت: شکلک‌های جادویی (Install Emoji) – 20 xp

- **وظیفه:** کتابخانه `emoji` را نصب کن. برنامه‌ای بنویس که پیام "`Python is magic`" را چاپ کند (باید شکل مار واقعی  چاپ شود).

۵. مأموریت: نوار پیشرفت (Install TQDM) – 20 xp

- **وظیفه:** کتابخانه `tqdm` را نصب کن. یک حلقه بساز که ۱۰۰ بار بچرخد و با استفاده از `tqdm` نوار پیشرفت (Loading Bar) زیبا در ترمینال نشان دهد.

۶. مأموریت: هنر (Install Art) – 20 xp

- **وظیفه:** از کتابخانه `art` استفاده کن تا اسم خودت را با فونت `block` یا `random` به صورت بزرگ در ترمینال چاپ کنی.

۷. مأموریت: نسخه خاص (Specific Version) – 20 xp

- **وظیفه:** گاهی یک طلسم قدیمی نیاز داریم. سعی کن نسخه قدیمی `pandas` مثلاً نسخه ۱/۵/۰ را نصب کنی. `pip install pandas==1.5.0`

۸. مأموریت: نابودی طلسم (Uninstall) 20 xp –

- **وظیفه:** کتابخانه‌ای که در مأموریت ۷ نصب کردی را با دستور `pip uninstall` حذف کن تا آزمایشگاه تمیز شود.

۹. مأموریت: نسخه پیچی (Freeze) 20 xp –

- **وظیفه:** تمام کتابخانه‌هایی که تا الان نصب کردی را در یک فایل `requirements.txt` ذخیره کن.

۱۰. مأموریت: مهاجرت بزرگ (Install from Req) 20 xp –

- **وظیفه:** (تست نهایی) محیط مجازی فعلی را غیرفعال و حذف کن. یک محیط جدید بساز و با دستور `pip install -r requirements.txt` همه چیز را یکجا برگردان.

فصل ۱۸: جادوی (NumPy محاسبات عددی)

۱۱. مأموریت: سربازگیری (Array Creation) 20 xp –

- **وظیفه:** یک لیست پایتونی `[10, 20, 30]` را به یک آرایه NumPy تبدیل کن و نوع آن (type) را چاپ کن تا مطمئن شوی `ndarray` است.

۱۲. مأموریت: شمارش معکوس (Arange) 20 xp –

- **وظیفه:** با استفاده از `np.arange`، یک آرایه از اعداد ۵۰ تا ۱ (نزولی) بساز

۱۳. مأموریت: آرایش نظامی (Reshape) 20 xp –

- **وظیفه:** یک آرایه شامل اعداد ۰ تا ۱۱ بساز (۱۲ عدد). حالا شکل آن را به یک ماتریس ۴ سطری و ۳ ستونی تغییر بده.

۱۴. مأموریت: تاس‌ریز اعظم (Random Int) 20 xp –

- **وظیفه:** یک ماتریس ۵x۵ بساز که پر از اعداد تصادفی بین ۱ تا ۱۰ باشد.

۱۵. مأموریت: تقویت ارتش (Vectorized Ops) 20 xp –

- **وظیفه:** یک آرایه اعداد تصادفی داری. بدون استفاده از حلقه `for`، به قدرت همه سربازان ۱۰ واحد اضافه کن و همه را تقسیم بر ۲ کن.

۱۶. مأموریت: گزارش میدان (Statistics) – 20 xp

- **وظیفه:** یک آرایه بزرگ از نمرات بساز. با دستورات نامپای، «کمترین»، «بیشترین» و «میانگین» نمرات را چاپ کن.

۱۷. مأموریت: پیدا کردن جاسوس‌ها (Boolean Indexing) – 20 xp

- **وظیفه:** یک آرایه اعداد داری. کدی بنویس که فقط اعداد زوج را از داخل آن بیرون بکشد و در یک آرایه جدید بریزد.

۱۸. مأموریت: بوم نقاشی (Zeros & Ones) – 20 xp

- **وظیفه:** یک تصویر سیاه) ماتریس 10×10 پر از صفر (و یک تصویر سفید) ماتریس 10×10 پر از یک (بساز.

۱۹. مأموریت: فاصله گذاری (Linspace) – 20 xp

- **وظیفه:** می‌خواهیم بین عدد ۰ تا ۱۰۰، دقیقاً ۵ سنگ جادویی با فاصله‌های مساوی بچینیم. از `np.linspace` استفاده کن.

۲۰. مأموریت: ضرب جادویی (Dot Product) – 20 xp

- **وظیفه:** دو ماتریس 2×2 بساز. آن‌ها را با هم جمع کن (+) و سپس در هم ضرب ماتریسی کن (@) و تفاوت نتیجه را ببین.

فصل ۱۹: کتابخانه Pandas تحلیل داده

۲۱. مأموریت: ثبت نام دستی (Create DataFrame) – 20 xp

- **وظیفه:** یک دیکشنری شامل نام ۳ دوست، سن و غذای مورد علاقه‌شان بساز. آن را به یک `DataFrame` تبدیل و چاپ کن.

۲۲. مأموریت: بارگذاری اکسل (Read CSV) – 20 xp

- **وظیفه:** یک فایل CSV ساده (در نوت‌پد بساز `Name, Age`: سپس زیرش ۲ خط دیتا). آن را با `pandas` بخوان و نمایش بده.

۲۳. مأموریت: بازرسی سریع (Head & Tail) – 20 xp

- **وظیفه:** فرض کن دیتای تو ۱۰۰۰ سطر دارد. کدی بنویس که فقط ۵ سطر اول و ۵ سطر آخر جدول را نشان دهد.

۲۴. مأموریت: کارآگاه داده (Info & Describe) – 20 xp

- **وظیفه:** روی دیتافریم پروژه قبل، دستور (info) را بزن تا جنس ستون‌ها را ببینی. دستور (describe) را بزن تا آمار ستون‌های عددی را ببینی.

۲۵. مأموریت: ستون‌برداری (Select Column) – 20 xp

- **وظیفه:** از جدول ماجراجویان، فقط ستون "Name" را جدا کن و به صورت یک لیست چاپ کن.

۲۶. مأموریت: الک کردن ثروتمندان (Filter) – 20 xp

- **وظیفه:** جدولی از ماجراجویان با ستون "Gold" داری. کدی بنویس که فقط کسانی که سکه‌هایشان بیشتر از ۵۰۰ است را نشان دهد.

۲۷. مأموریت: رتبه‌بندی (Sorting) – 20 xp

- **وظیفه:** جدول را بر اساس "Age" از کوچکترین به بزرگترین مرتب کن.

۲۸. مأموریت: تغییر واقعیت (Add Column) – 20 xp

- **وظیفه:** یک ستون جدید به نام "Level_Up_Gold" بساز که مقدارش برابر با ستون "Gold" ضربدر ۱۰ باشد.

۲۹. مأموریت: سرشماری (Value Counts) – 20 xp

- **وظیفه:** ستونی به نام "City" داری. کدی بنویس که بگوید از هر شهر دقیقاً چند نفر در لیست حضور دارند.

۳۰. مأموریت: بایگانی (To CSV) – 20 xp

- **وظیفه:** دیتافریمی که تغییر دادی را در یک فایل جدید به نام final_report.csv ذخیره کن (بدون ذخیره‌index).

فصل ۲۰: نمودار جادویی (Matplotlib)

۳۱. مأموریت: خط زمان (Simple Plot) – 20 xp

- **وظیفه:** لیستی از روزهای هفته و دمای هوا بساز. یک نمودار خطی بکش که تغییرات دما را نشان دهد.

۳۲. مأموریت: زرادخانه (Bar Chart) – 20 xp

- **وظیفه:** لیستی از اسلحه‌ها و تعداد آن‌ها در انبار داری. یک نمودار میله‌ای برای مقایسه موجودی انبار بکش.

۳۳. مأموریت: سهم‌خواهی (Pie Chart) – 20 xp

- **وظیفه:** انجمن شامل ۴۰٪ انسان، ۳۰٪ الف و ۳۰٪ کوتوله است. یک نمودار دایره‌ای بکش و درصدها را روی آن نمایش بده.

۳۴. مأموریت: پراکندگی (Scatter Plot) – 20 xp

- **وظیفه:** لیستی از "قد" و "وزن" ۱۰ هیولا داری. نمودار پراکندگی (نقطه‌ای) بکش تا ببینی آیا رابطه‌ای بین قد و وزن آن‌ها وجود دارد؟

۳۵. مأموریت: توزیع نمرات (Histogram) – 20 xp

- **وظیفه:** یک لیست شامل ۱۰۰ نمره تصادفی (بین ۰ تا ۲۰) بساز. هیستوگرام آن را بکش تا فراوانی نمرات را ببینی.

۳۶. مأموریت: رنگ‌آمیزی (Colors & Styles) – 20 xp

- **وظیفه:** نمودار خطی مأموریت ۳۱ را دوباره بکش، اما این بار خط را "قرمز"، نقطه‌ها را "ستاره‌ای" و خط چین (--) کن.

۳۷. مأموریت: تابلو راهنما (Title & Labels)

- **وظیفه:** به نمودارت "عنوان اصلی"، "برچسب محور X" و "برچسب محور Y" اضافه کن.

۳۸. مأموریت: شبکه مختصات (Grid) – 20 xp

- **وظیفه:** به نمودار میله‌ای خود یک صفحه مشبک (Grid) اضافه کن.

۳۹. مأموریت: ذخیره اثر هنری (Save Fig) – 20 xp

- **وظیفه:** کدی بنویس که نمودار را نشان ندهد، بلکه مستقیماً آن را با کیفیت بالا در فایل my_masterpiece.png ذخیره کند.

۴۰. مأموریت: گزارش ترکیبی (Combined Report) – 20 xp

- **وظیفه:** دو نمودار خطی در یک قاب بکش: یکی رشد "سکه" و دیگری رشد "تجربه" در طول ۱۰ روز. برای هر کدام رنگ متفاوت و یک راهنما (Legend) بگذار.

کتابخانه ساینه (نتیجه مأموریت‌ها)

فصل ۱۷: دروازه احضار (Terminal Commands)

توجه: بعضی از دستورات باید در خط فرمان (CMD/Terminal) اجرا شوند، نه داخل فایل پایتون.

۱. مأموریت: خلق جهان موازی (Create Venv)

```
# Windows
python -m venv potion_lab
potion_lab\Scripts\activate

# Mac / Linux
python3 -m venv potion_lab
source potion_lab/bin/activate
```

۲. مأموریت: لیست مواد اولیه (Pip List)

```
pip list
```

۳. مأموریت: احضار رنگ‌ها (Colorama) نصب pip install colorama: کد پایتون:

```
1. from colorama import Fore, init
2.
3. # Initialize colorama
4. init(autoreset=True)
5.
6. print(Fore.RED + "Fire Spell")
7. print(Fore.BLUE + "Ice Spell")
```

۴. مأموریت: شکلک‌های جادویی (Emoji) نصب pip install emoji: کد پایتون:

```
1. import emoji
2. print(emoji.emojize("Python is magic :snake:"))
```

۵. مأموریت: نوار پیشرفت (TQDM) نصب pip install tqdm: کد پایتون:

```
1. from tqdm import tqdm
2. import time
3.
4. for i in tqdm(range(100)):
5.     time.sleep(0.05) # Simulate working on a spell
```

۶. مأموریت: هنر (Art) نصب pip install art: کد پایتون:

```
1. from art import tprint
2. tprint("WIZARD", font="block")
```

۷. مأموریت: نسخه خاص (Specific Version)

```
pip install pandas==1.5.0
```

۸. مأموریت: نابودی طلسم (Uninstall)

```
pip uninstall pandas
```

۹. مأموریت: نسخه پیچی (Freeze)

```
pip freeze > requirements.txt
```

۱۰. مأموریت: مهاجرت بزرگ (Install from Req)

```
# 1. Exit current lab
deactivate

# 2. Create new lab
python -m venv new_lab

# 3. Install everything
pip install -r requirements.txt
```

فصل ۱۸: جادوی NumPy محاسبات عددی

۱۱. مأموریت: سربازگیری (Array Creation)

```
1. import numpy as np
2.
3. py_list = [10, 20, 30]
4. np_arr = np.array(py_list)
5. print(type(np_arr))
```

۱۲. مأموریت: شمارش معکوس (Arange)

```
1. # Numbers from 50 down to 1
2. arr = np.arange(50, 0, -1)
3. print(arr)
```

۱۳. مأموریت: آرایش نظامی (Reshape)

```
1. # Create 0 to 11, then reshape to 4x3
2. arr_12 = np.arange(12)
3. matrix = arr_12.reshape(4, 3)
4. print(matrix)
```

۱۴. مأموریت: تاس‌ریز اعظم (Random Matrix)

```
1. # 5x5 matrix with random numbers between 1 and 10
2. dice_matrix = np.random.randint(1, 11, (5, 5))
3. print(dice_matrix)
```

۱۵. مأموریت: تقویت ارتش (Vectorized Ops)

```
1. power = np.array([10, 20, 30])
2. # Add 10 and divide by 2 (No loop needed!)
3. new_power = (power + 10) / 2
4. print(new_power)
```

۱۶. مأموریت: گزارش میدان (Statistics)

```
1. scores = np.array([15, 20, 10, 18])
2. print(f"Min: {np.min(scores)}")
3. print(f"Max: {np.max(scores)}")
4. print(f"Mean: {np.mean(scores)}")
```

۱۷. مأموریت: پیدا کردن جاسوس‌ها (Boolean Indexing)

```
1. nums = np.array([1, 2, 3, 4, 5, 6])
2. # Filter only even numbers
3. evens = nums[nums % 2 == 0]
4. print(evens)
```

۱۸. مأموریت: بوم نقاشی (Zeros & Ones)

```
1. black = np.zeros((10, 10))
2. white = np.ones((10, 10))
```

۱۹. مأموریت: فاصله گذاری (Linspace)

```
1. # 5 numbers evenly spaced between 0 and 100
2. stones = np.linspace(0, 100, 5)
3. print(stones)
```

۲۰. مأموریت: ضرب جادویی (Dot Product)

```
1. a = np.array([[1, 2], [3, 4]])
2. b = np.array([[5, 6], [7, 8]])
3.
4. sum_res = a + b
5. dot_res = a @ b # Matrix multiplication
6. print(dot_res)
```

فصل ۱۹: کتابخانه Pandas تحلیل داده

۲۱. مأموریت: ثبت نام دستی (Create DataFrame)

```
1. import pandas as pd
2.
3. data = {
4.     "Name": ["Ali", "Sara", "Reza"],
5.     "Age": [20, 22, 19],
6.     "Food": ["Pizza", "Pasta", "Burger"]
7. }
8. df = pd.DataFrame(data)
9. print(df)
```

۲۲. مأموریت: بارگذاری اکسل (Read CSV)

```
1. # Assuming you have a file named 'my_file.csv'
2. # df = pd.read_csv("my_file.csv")
3. # print(df)
```

۲۳. مأموریت: بازرسی سریع (Head & Tail)

```
1. # print(df.head(5)) # Top 5 rows
2. # print(df.tail(5)) # Bottom 5 rows
```

۲۴. مأموریت: کارآگاه داده (Info & Describe)

```
1. # df.info() # Check column types
2. # df.describe() # Check statistics
```

۲۵. مأموریت: ستون‌برداری (Select Column)

```
1. names = df["Name"]
2. print(names)
```

۲۶. مأموریت: الک کردن ثروتمندان (Filter)

```
1. # Show rows where Gold is greater than 500
2. # rich = df[df["Gold"] > 500]
3. # print(rich)
```

۲۷. مأموریت: رتبه‌بندی (Sorting)

```
1. # Sort by Age (Ascending)
2. # sorted_df = df.sort_values(by="Age")
```

۲۸. مأموریت: تغییر واقعیت (Add Column)

```
1. # Create new column based on math
2. # df["Level_Up_Gold"] = df["Gold"] * 10
```

۲۹. مأموریت: سرشماری (Value Counts)

```
1. # Count how many people are in each city
2. # print(df["City"].value_counts())
```

۳۰. مأموریت: بایگانی (To CSV)

```
1. # Save the modified dataframe to a new file
2. # df.to_csv("final_report.csv", index=False)
```

فصل ۲۰: نمودار جادویی (Matplotlib)

۳۱. مأموریت: خط زمان (Simple Plot)

```
1. import matplotlib.pyplot as plt
2.
3. days = ["Mon", "Tue", "Wed"]
4. temp = [20, 22, 19]
5.
6. plt.plot(days, temp)
7. plt.show()
```

۳۲. مأموریت: زرادخانه (Bar Chart)

```
1. weapons = ["Sword", "Bow", "Axe"]
2. counts = [10, 5, 2]
3.
4. plt.bar(weapons, counts)
5. plt.show()
```

۳۳. مأموریت: سهم‌خواهی (Pie Chart)

```
1. sizes = [40, 30, 30]
2. labels = ["Human", "Elf", "Dwarf"]
3.
4. plt.pie(sizes, labels=labels, autopct='%1.1f%%')
5. plt.show()
```

۳۴. مأموریت: پراکندگی (Scatter Plot)

```
1. heights = [150, 160, 170]
2. weights = [50, 60, 70]
3.
4. plt.scatter(heights, weights)
5. plt.show()
```

۳۵. مأموریت: توزیع نمرات (Histogram)

```
1. import numpy as np
2.
3. # Generate random scores
4. scores = np.random.randint(0, 21, 100)
5.
6. plt.hist(scores, bins=5)
7. plt.show()
```

۳۶. مأموریت: رنگ‌آمیزی (Colors & Styles)

```
1. # Red line, Star markers, Dashed line
2. plt.plot(days, temp, color='red', marker='*', linestyle='--')
3. plt.show()
```

۳۷. مأموریت: تابلو راهنما (Labels)

```
1. plt.plot(days, temp)
2. plt.title("Weather Report")
3. plt.xlabel("Days")
4. plt.ylabel("Temperature")
5. plt.show()
```

۳۸. مأموریت: شبکه مختصات (Grid)

```
1. plt.bar(weapons, counts)
2. plt.grid(True)
3. plt.show()
```

۳۹. مأموریت: ذخیره اثر هنری (Save)

```
1. plt.plot(days, temp)
2. plt.savefig("my_masterpiece.png")
3. print("Saved!")
```

۴۰. مأموریت: گزارش ترکیبی (Combined Report)

```
1. days_num = range(1, 11)
2. gold = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
3. xp = [5, 15, 25, 35, 45, 55, 65, 75, 85, 95]
4.
5. # Plot two lines on one chart
6. plt.plot(days_num, gold, color="gold", label="Gold")
7. plt.plot(days_num, xp, color="purple", label="XP")
8.
9. plt.legend() # Show legend box
10. plt.show()
```

بخش هفتم: ماجراجویی ادامه دارد...

فصل ۲۱: افق‌های بی‌پایان

(نقشه‌های گنج آینده)

مأموریت فصل: ماجراجویی با پایتون در اینجا تمام نمی‌شود؛ بلکه تازه شروع شده است! حالا که کوله پشتیت پر از طلسم‌های قدرتمنده، باید تصمیم بگیری که چه نوع جادوگری می‌خواهی باشی؟ در این فصل، مسیرهای تخصصی دنیای تکنولوژی را بررسی می‌کنیم و قراره انتخاب کنی مرحله بعدی برای تو چیه

نقشه راه:

- قدم‌های بعدی: توصیه‌های ضروری برای همه جادوگران.
- مسیر معماران دیجیتال: دنیای وب و اپلیکیشن‌ها (Web & App).
- مسیر پیشگویان داده: هوش مصنوعی و علم داده (AI & Data Science).
- مسیر محافظان و مدیران: امنیت و مدیریت سیستم (Security & SysAdmin).
- مسیر خالقان دنیاها: بازی‌سازی و واقعیت مجازی (Game & AR/VR).
- مسیر صنعتگران جادویی: اینترنت اشیا و رباتیک (IoT & Robotics).

۱. قدم‌های بعدی (The Essential Next Steps)

«قبل از انتخاب تخصص، باید شمشیر خود را تیزتر کنید.»
فرقی نمی‌کند کدام مسیر تخصصی را انتخاب کنید؛ بر اساس تجربیات بزرگان سه مهارت وجود دارد که همین الان میتونه شمارو به برنامه نویس بهتری تبدیل کنه:

۱. پایتون پیشرفته (Advanced Python)

- یادگیری عمیق‌تر OOP درک مفاهیمی مثل ارث‌بری.
- Async و Multi-threading اجرای همزمان چندین طلسم (پردازش همزمان) برای سرعت بخشیدن به برنامه‌های سنگین.
- مدیریت خطا و Testing یادگیری نوشتن تست‌های خودکار برای اینکه مطمئن شوید کدهایتان در شرایط سخت منفجر نمی‌شوند!

۲. ساختمان داده و الگوریتم:

- یادگیری اصول مهندسی برای نوشتن کدهای بهینه. (حل مسئله به صورتی که سرعت بالاتر و مصرف حافظه کمتر استفاده بشه)

۳. گیت (Git) جادوی زمان و همکاری:

- Git یادگیری ورژن کنترل (مثل قابلیت Save Point در بازی‌ها که به شما اجازه می‌دهد در زمان سفر کنید و به نسخه‌های قبلی کدتان برگردید).
- GitHub سرزمینی برای اشتراک‌گذاری طلسم‌ها با دیگر جادوگران و شرکت در پروژه‌های تیمی.

۲. مسیر معماران دیجیتال (Programming & Web)

برای کسانی که می‌خواهند زیرساخت‌های دنیای دیجیتال را بسازند. اگر دوست دارید سایت و برنامه‌هایی بسازید که از طریق مرورگرها و گوشی‌ها در دسترس میلیون‌ها نفر باشد، این مسیر شماست.

توسعه وب (Web Development):

- Backend: ساخت مغز متفکر: یادگیری فریم‌ورک‌های قدرتمند پایتونی مثل FastAPI یا جنگو.
- Frontend: آشنایی با ظاهر ماجرا: (HTML, CSS, JavaScript...React).

برنامه‌نویسی موبایل و دسکتاپ:

- برای ساخت اپلیکیشن‌های گوشی، زبان **Kotlin** (اندروید) یا فریم‌ورک **Flutter** با زبان **Dart** را پیشنهاد می‌دهم. برای ویندوز نیز **#C** یا **Electron** انتخاب‌های هوشمندانه‌ای هستند.

۳. مسیر پیشگویان داده (AI & Data)

«برای کسانی که می‌خواهند آینده را از روی داده‌ها بخوانند و به ماشین‌ها یادگیری بیاموزند.»
این مسیر برای کسانی است که عاشق ریاضیات، آمار و هوش مصنوعی هستند.

هوش مصنوعی (AI & Machine Learning):

- یادگیری ماشین: یاد دادن الگوها به کامپیوتر برای پیش‌بینی آینده.
- یادگیری عمیق (Deep Learning): ساخت شبکه‌های عصبی مصنوعی.
- بینایی ماشین (Computer Vision): بینایی دادن به کامپیوترها برای تشخیص چهره و اشیاء.
- NLP: پردازش زبان طبیعی؛ آموزش فهمیدن و صحبت کردن به زبان انسان‌ها مثل ChatGPT

۴. مسیر محافظان و مدیران سیستم (SysAdmin & Security)

«برای کسانی که می‌خواهند نظم را برقرار کنند و از قلعه‌ها محافظت کنند.»
اگر به شبکه، سرورها و امنیت علاقه دارید، این مسیر هیجان‌انگیز شماست.

مدیریت سیستم و DevOps:

- **Linux** تسلط بر سیستم عامل لینوکس که قلب تپنده اکثر سرورهای جهان است.
- **DevOps** هنر خودکارسازی فرآیندها؛ کسی که پل میان کدنویس‌ها و سرورهاست و وظیفه دارد کدها را به سلامت روی ابرهای دیجیتال (Cloud) مستقر کند.

امنیت سایبری (Cybersecurity) :

- **هک اخلاقی و تست نفوذ** : پیدا کردن سوراخ‌های قلعه قبل از اینکه دشمن آن‌ها را پیدا کند.
- **جرم‌شناسی دیجیتال** : ردیابی ردپای مهاجمان در دنیای صفر و یک.

بلاک‌چین (Blockchain):

- برای بلاک‌چین زبان سالی‌دیتی رو پیشنهاد میکنم.
- نوشتن قراردادهای هوشمند (Smart Contracts).
- توسعه برنامه‌های غیرمتمرکز (DApps).

۵. مسیر خالقان دنیاها (Game Dev & XR)

برای کسانی که رویاهایشان را به تصویر می‌کشند.

بازی‌سازی: (Game Dev)

- کار با موتورهای قدرتمندی مثل **Unity** (بسیار محبوب)، **Unreal Engine** (برای گرافیک‌های خیره‌کننده) یا **Godot** (سبک و متن‌باز)

واقعیت افزوده و مجازی: (AR/VR)

- خلق تجربه‌هایی که مرز بین واقعیت و خیال را از بین می‌برند. برای این کار، مدل‌سازی سه‌بعدی در **Blender** و برنامه‌نویسی در **Unity** کلید موفقیت شماست.

۶. مسیر صنعتگران جادویی (IoT & Robotics)

- برای کسانی که می‌خواهند به اشیاء بی‌جان، جان ببخشند. اینجا جایی است که کدها از صفحه مانیتور خارج شده و در دنیای واقعی حرکت می‌کنند.
- اینترنت اشیاء (IoT) اتصال اشیاء روزمره به اینترنت
- رباتیک و سخت‌افزار : برای کار با بازوهای رباتیک و سنسورهای دقیق، علاوه بر پایتون، یادگیری زبان C یا کار با پلتفرم **Arduino** به شما قدرت بی‌پایانی می‌دهد.

۷. مسیر طراحان دنیاها (Design & Graphic)

برای کسانی که به زیبایی و تجربه بصری اهمیت می‌دهند. یک جادوی قدرتمند اگر ظاهر زیبایی نداشته باشد، شاید هرگز دیده نشود.

- طراحی رابط کاربری (UI/UX) درک رفتار کاربر و طراحی ظاهر اپلیکیشن‌ها به شکلی که کار با آن‌ها لذت‌بخش باشد (با ابزارهایی مثل Figma)
- گرافیک و مدل‌سازی: خلق کاراکترها، لوگوها و محیط‌های سه‌بعدی که روح پروژه‌های شما هستند.

🌟 ارتقاء لول: ۱۰۰ → ۹۹ 🌟

وضعیت جدید:

◆ لقب جدید:

برنامه نویسنده افسانه‌ای

◆ سطح جدید: S

توضیح: تبریک به سطح

جدیدی رسید.

سخن پایانی: پایان آغاز

ماجرای عزیز، کتاب «ماجرایی با پایتون» در اینجا به پایان می‌رسد، اما داستان تو تازه شروع شده است. برنامه نویسی مثل جادوگری است؛ هرچه بیشتر تمرین کنی، طلسم‌های قدرتمندتر می‌شوند.

به یاد داشته باش:

- از اشتباه کردن نترس (ارورها مسیر بهتر شدن رو بهت نشون میدن).
- همیشه کنجکاو باش و یادگیری را متوقف نکن (هیچ حداکثری وجود نداره).
- و مهم‌تر از همه... ادامه بده (اتفاق خوب بلاخره میوفته)!

مأموریت بعدی: دنیای واقعی را بساز!

مأموریت جانبی: از پروژه‌هایی که زدی در شبکه‌های اجتماعی با هشتگ `#codewithmahdi` و `#pythonadventure` با ما اشتراک بگذار. با این کار ما می‌فهمیم که یک ماجراجو به سطح S رسیده و باعث خوشحالی ما میشه!

منابع

سفر ما در اینجا به پایان می‌رسد، اما اقیانوس دانش بی‌انتهاست. برای نگارش این کتاب و همچنین برای ادامه مسیر یادگیری شما، از منابع ارزشمند زیر استفاده شده است.

۱. (مستندات رسمی) این‌ها معتبرترین منابعی هستند که توسط خود خالقان این ابزارها نوشته شده‌اند:

- Python Official Documentation (docs.python.org) مرجع اصلی زبان پایتون.
- NumPy Documentation (numpy.org/doc) برای محاسبات عددی.
- Pandas Documentation (pandas.pydata.org/docs) برای تحلیل داده‌ها.
- Matplotlib Documentation (matplotlib.org/stable) برای رسم نمودارها.

۲. کتاب‌های جادویی (منابع آموزشی پیشنهادی) اگر دوست دارید با خواندن کتاب یاد بگیرید، این‌ها بهترین همراهان شما خواهند بود:

- Fluent Python اثر Luciano Ramalho (برای کسانی که می‌خواهند به سطح استادی برسند).
- Python Tricks: A Buffet of Awesome Python Features اثر Dan Bader مناسب برای یادگیری نکات حرفه‌ای و عمیق.

۳. اوراکل‌های آنلاین (جوامع و وب‌سایت‌ها) زمانی که در طلسمی گیر کردید، این‌ها مکان‌هایی هستند که پاسخ را در آن‌ها خواهید یافت:

- Stack Overflow بزرگترین انجمن پرسش و پاسخ برنامه‌نویسان.
- Real Python (realpython.com) مقالات و آموزش‌های عالی و عمیق.
- W3Schools & GeeksforGeeks برای دسترسی سریع به مثال‌ها و مفاهیم پایه.

۴. تقدیر و تشکر با تشکر از جامعه متن باز (Open Source) پایتون که این ابزارهای قدرتمند را به رایگان در اختیار تمام ماجراجویان جهان قرار داده اند.